# nwsaom

Stochastic actor-oriented models in Stata
(a package that builds on `nwcommands`)

User manual and reference

Thomas U. Grund
RWTH Aachen University

This manual describes `nwsaom` 1.5.x
(listings checked with `nwsaom` 1.5.14, 10 November 2026)

**How to cite.** This manual describes the series 1.5 of `nwsaom` (it holds for all versions 1.5.x); the listings were checked with version 1.5.14 (10 November 2026). It is the citable reference for the package. Please cite it as:

Grund, T. U. (2026). *nwsaom: Stochastic actor-oriented models in Stata. Manual.* Zenodo. https://doi.org/10.5281/zenodo.23623807

The DOI is the one of all versions of the record. Please also report the exact version of the program that produced your results, as printed by `nwsaom version` (this manual: 1.5.14).

# Acknowledgments

`nwsaom` follows the Siena project closely and exists because of it. The model is due to Tom Snijders [3, 4]. RSiena (Ripley, Snijders, Bóda, Vörös, Preciado and many contributors) [2] is the reference implementation on which the effects, the estimation algorithm, the defaults and the output of `nwsaom` are modelled. The Bayesian estimator is based on multiSiena (Snijders, Koskinen and Ripley; version 1.2.40) [5], which implements the method of Koskinen and Snijders [6]. The R source code and the manuals of these packages were the specification against which the Stata code was written and checked.

`nwsaom` is not affiliated with or endorsed by the Siena project, and its errors are the author's. It is meant for researchers who work in Stata or prefer it to R. It is not a replacement for RSiena, which remains the reference and the more complete program.

**Use of an AI coding assistant.** The software was developed with Claude Code (Anthropic), an AI coding assistant, working under the author's direction. It helped port algorithms from RSiena and multiSiena into Stata, Mata and C and helped write the tests. The author specified the scope, reviewed the code, and is responsible for the package. Its correctness rests on validation, not on the assistant: the results are compared numerically with RSiena, and the test suite reruns these comparisons.

# Contents

**Acknowledgments** **2**

**1 Introduction** **8**
1.1 What `nwsaom` is . . . 8
1.2 Relation to RSiena . . . 8
1.3 What is covered . . . 9
1.4 Installation and requirements . . . 10
1.5 How to read this manual . . . 12

**2 The model in brief** **14**
2.1 Data and the idea of the model . . . 14
2.2 The rate function . . . 14
2.3 The objective function and the ministep . . . 14
2.4 Behavior and several networks . . . 15
2.5 Estimation by the method of moments . . . 15
2.6 Conditional and unconditional estimation . . . 16
2.7 Convergence . . . 16
2.8 Non-directed and two-mode networks . . . 17

**3 Getting started** **18**
3.1 The data . . . 18
3.2 A first complete example . . . 19
3.3 A model with covariates . . . 21
3.4 Two waves . . . 21
3.5 A model with behavior co-evolution . . . 22
3.6 Goodness of fit . . . 22
3.7 Stored results . . . 23
3.8 A word on run time and reproducibility . . . 24

**4 Options reference** **25**
4.1 Syntax . . . 25
4.2 Observed waves . . . 25
4.3 Baseline network effects . . . 26
4.4 Covariate effects . . . 27
4.5 Structural network effects . . . 28
4.6 Interactions . . . 30
4.7 Behavior co-evolution . . . 30
4.8 Composition change, missing data and structural values . . . 31
4.9 Rates . . . 31
4.10 Non-directed relations . . . 32
4.11 Estimation control . . . 32
4.12 Several dependent networks: `nwsaom multinet` . . . 33

4.13 Postestimation commands . . . 34
4.14 Combinations that are not available . . . 34

**5 The effects** **36**
5.1 Naming and the option families . . . 36
5.2 Statistics, contributions and what "validated" means . . . 37
5.3 Effect families added in the later versions . . . 37
5.4 Effects that are native only . . . 37

**6 Network-behavior co-evolution** **38**
6.1 The model in brief . . . 38
6.2 A first model . . . 38
6.3 Shape effects . . . 39
6.4 Influence effects . . . 40
6.4.1 The two main effects: avalt and avsim . . . 40
6.4.2 Endowment and creation roles . . . 41
6.4.3 Network effects on behavior: beheff, behnet2 and friends . . . 41
6.5 Selection effects: the behavior shapes the network . . . 44
6.6 Several behaviors . . . 45
6.7 The rate of the behavior . . . 46
6.8 Estimation by maximum likelihood and Bayes . . . 46
6.9 Co-evolution with other features . . . 46
6.10 Estimation notes . . . 46
6.11 Limits . . . 47

**7 Non-directed, two-mode and several networks** **49**
7.1 Non-directed (symmetric) networks . . . 49
7.1.1 When the symmetric model is used . . . 49
7.1.2 Model types: symtype() . . . 50
7.1.3 Effects in a non-directed network . . . 51
7.1.4 Combinations . . . 51
7.1.5 Validation . . . 51
7.1.6 Limits . . . 51
7.2 Two-mode networks . . . 52
7.2.1 Setup . . . 52
7.2.2 A model . . . 53
7.2.3 Effects available in a two-mode network . . . 53
7.2.4 A co-evolving behavior of the actors . . . 54
7.2.5 Further options in a two-mode network . . . 55
7.2.6 Validation . . . 55
7.2.7 Limits . . . 55
7.3 Several networks: nwsaom multinet . . . 56
7.3.1 Within-network and cross-network effects . . . 56
7.3.2 Cross-network effects with a covariate . . . 57
7.3.3 Example . . . 57
7.3.4 Non-directed members . . . 58
7.3.5 Further cross-network effects . . . 59
7.3.6 A two-mode network as one of the two networks . . . 59
7.3.7 Starting values, fixed effects and tests . . . 60
7.3.8 Validation . . . 61
7.3.9 A co-evolving behavior . . . 61
7.3.10 Limits . . . 62

**8 Rates and composition change** **64**
8.1 Covariate-dependent rates . . . 64
8.2 Structural rate effects . . . 65
8.3 Rate effects in non-directed and two-mode networks . . . 66
8.4 Rate effects of the behavior . . . 67
8.5 Composition change . . . 67
8.5.1 Whole-period presence: present() . . . 68
8.5.2 Joiners and leavers at fractional times: joinat() and leaveat() . . . 69
8.5.3 What combines with composition change . . . 70
8.6 Limits . . . 70

**9 Estimation and diagnostics** **71**
9.1 The algorithm in brief . . . 71
9.1.1 Conditional and unconditional estimation . . . 72
9.2 Algorithm controls . . . 72
9.3 Judging convergence . . . 74
9.4 Errors, divergence and what to do . . . 74
9.5 Stored results . . . 75
9.6 Goodness of fit: `estat gof` . . . 76
9.7 Micro effects on macro structure: `estat mems` . . . 77
9.8 Maximum likelihood: `ml` . . . 78
9.9 Bayesian estimation: `nwsaom bayes` . . . 79
9.10 Run time . . . 80

**10 Simulation, tests and importance of effects** **82**
10.1 How the commands are organised . . . 82
10.2 Describing the data: `nwsaom describe` . . . 82
10.3 Holding effects fixed: `fix()` . . . 83
10.4 Simulation: `estat simulate` and `nwsaom simulate` . . . 84
10.5 Score-type tests: `estat score` . . . 85
10.6 Time heterogeneity: `estat timetest` . . . 86
10.7 Relative importance of effects: `estat importance` . . . 87
10.8 Several groups: `nwsaom meta` . . . 89
10.9 What is not available . . . 90

**11 Several groups: joint estimation** **91**
11.1 The idea . . . 91
11.2 The data layout . . . 91
11.3 The options `groups()` and `gwaves()` . . . 92
11.4 A first joint model . . . 93
11.5 Joint model or meta-analysis? . . . 95
11.6 Co-evolution of a behavior in several groups . . . 96
11.7 After the fit: time heterogeneity and score tests . . . 98
11.8 Agreement with RSiena . . . 100
11.9 What is not available with groups . . . 100

**12 Validation against RSiena** **103**
12.1 Method . . . 103
12.2 Regression tests that ship with the package . . . 104
12.3 Summary of what was validated . . . 104
12.4 Where results differ, or RSiena itself is degenerate . . . 106
12.4.1 Effects where RSiena crashes or throws . . . 106
12.4.2 Effects that are degenerate or collinear in both programs . . . 106

12.4.3 Open disagreements (not claimed as validated) . . . 107
12.4.4 Several groups . . . 107
12.4.5 Constants that differ by construction . . . 107
12.5 Reliability compared with RSiena . . . 108
12.6 How to reproduce . . . 108

**13 Missing data, structural zeros, and what is not implemented 109**
13.1 Missing data: `missnet()` and `missbeh()` . . . 109
13.2 Structural zeros and ones . . . 110
13.3 What RSiena has that `nwsaom` does not (yet) . . . 111
13.3.1 Effects not implemented, by reason . . . 112
13.3.2 Effects that RSiena cannot evaluate . . . 113
13.3.3 RSiena features beyond effects . . . 113
13.4 Row-level list of the effects that are not implemented . . . 114

**14 Technical notes 119**
14.1 The three layers . . . 119
14.2 The native plugin . . . 119
14.2.1 Source code of the plugin . . . 119
14.2.2 Building the plugin on your computer . . . 120
14.2.3 Rebuilding the Mata libraries . . . 120
14.2.4 Plugin versions . . . 120
14.2.5 Native-only effects . . . 120
14.2.6 Which engine ran . . . 121
14.2.7 Fixed limits of the plugin . . . 121
14.3 Threads and reproducibility . . . 121
14.4 Size limits . . . 122
14.4.1 What Stata limits . . . 122
14.4.2 What `nwsaom` costs . . . 122
14.4.3 The fallback warning . . . 122
14.5 Performance . . . 123
14.6 The size limit of an ado program . . . 123
14.7 File layout . . . 124

**A Effects reference 125**

**B RSiena definitions reproduced on purpose 164**

**C Effects with a Mata fallback 166**
C.1 Effects that run in Mata as well . . . 166
C.2 Effects that need the plugin . . . 166

**D Comparison with RSiena: the same estimator, and where the speed comes from 168**
D.1 The same algorithm . . . 168
D.2 The same simulation . . . 169
D.3 The same answers . . . 170
D.4 Where the speed comes from . . . 172
D.4.1 The amount of work . . . 172
D.4.2 Time for a fixed amount of work . . . 172
D.4.3 The cost of a Phase 2 step . . . 174
D.4.4 The simulation itself . . . 174
D.4.5 Several cores . . . 174

D.4.6 What was not measured . . . . . 176

# Chapter 1

# Introduction

## 1.1 What `nwsaom` is

`nwsaom` estimates stochastic actor-oriented models (SAOMs) in Stata. A SAOM is a statistical model for network panel data: the same set of actors is observed at two or more points in time, and the model describes how the ties between them change from one observation to the next as the outcome of the decisions of the individual actors. The model was introduced by Snijders and co-workers; the standard introduction is Snijders, van de Bunt and Steglich [1]. Chapter 2 summarises the model for readers who have not met it before.

`nwsaom` is a Stata package of its own that builds on the `nwcommands` package. It works on networks that were declared with `nwset` (or created by one of the other `nw` commands), it reads actor attributes from the ordinary Stata dataset, and it returns its results in `e()` like any other Stata estimation command. The heavy computation, the simulation of the network evolution, runs in a compiled C plugin that is called from Mata. No R installation or other external statistical software is used when you fit a model.

## 1.2 Relation to RSiena

The reference implementation of SAOMs is the R package RSiena [2], whose estimation function is `siena07` (called `siena` in the current RSiena manual). The other functions of RSiena that this manual refers to also have two names: `sienaGOF` is `test_gof`, `sienaTimeTest` is `test_time`, `sienaRI` is `interpret_size`, `siena08` is `meta_siena`, and `sienaBayes` of the package multiSiena is `multi_siena`. The manual uses the older names, which are the ones in the literature. `nwsaom` follows RSiena closely in its model, in its estimation algorithm, in the names of its effects and in the conventions that matter for the numbers (centring of covariates, the treatment of missing data, the scale of the rate parameters), so that RSiena's documentation and the published literature on SAOMs apply to `nwsaom` as well.

`nwsaom` is nevertheless an *independent reimplementation*. It was written in Stata/Mata and C, using RSiena's published manual and, where the manual was not specific enough, RSiena's open-source code as the methodological reference. During development, RSiena (version 1.6.14, the current release) was used to certify the results: for the effects that were validated, the observed statistics (the targets of the method of moments) are identical to those of `siena07` and the estimates agree within simulation noise. The validation is documented effect by effect in the validation chapter. `nwsaom` is not affiliated with, endorsed by or supported by the RSiena project.

Where the two programs differ in a way that matters to a user, the manual says so. The most important differences are these.

- Effects are chosen by Stata options (for example `transtrip`, `egox(alcohol1)`) and not by an effects table that is edited object by object.

- The goodness-of-fit test (`estat gof`) simulates new replicates at the fitted parameters instead of re-using networks stored during the fit; the test itself is the same Mahalanobis-distance test as in RSiena's `sienaGOF`.
- The option `gwesp()` takes the decay parameter on the scale used in the `ergm` literature, which is 1/100 of RSiena's parameter: RSiena's `gwespFF` with parameter 69 is `gwesp(.69)`.

## 1.3 What is covered

- **Directed one-mode networks**, with two or more waves. With three or more waves the effects are pooled over the periods between consecutive waves and each period has its own rate.
- **Non-directed (symmetric) networks**, with RSiena's model types for non-directed relations (`symtype()`).
- **Two-mode networks**, in which one set of nodes (the actors) chooses its ties to the other set (the events).
- **Behavior co-evolution**: a bounded integer behavior (an attitude, a count of cigarettes) evolves together with the network, so that selection and influence can be separated in one model; several behaviors are possible.
- **Several dependent networks** that evolve together on the same actors, with cross-network effects, through `nwsaom multinet`.
- **Several groups**: one model estimated jointly for several independent groups of actors (school classes, organizations) with the option `groups()`, RSiena's `sienaGroupCreate` and `siena07` (Chapter 11); or the separate fits combined afterwards with `nwsaom meta`.
- **Composition change**: actors that join or leave the network, either between waves (`present()`) or at fractional times inside a period (`joinat()`, `leaveat()`).
- **Missing data**: missing ties (`missnet()`), missing behavior values (`missbeh()`) and missing covariate values; and **structural zeros and ones** (`structural()`).
- **Rate covariates**: actor attributes that raise or lower an actor's rate of change (`ratecov()`), and structural rate effects that depend on an actor's degree.
- **Covariates**: constant and time-varying actor covariates, and dyadic covariates.
- **Interactions** between effects, and the **endowment and creation** split of effects.
- **Other estimators**: maximum likelihood (`ml`) and Bayesian estimation (`nwsaom bayes`) for a directed one-mode network, with or without one co-evolving behavior (Chapter 9).
- **Postestimation and tools**: goodness of fit (`estat gof`), micro effects on macro structure (`estat mems`), score-type tests (`estat score`), the time-heterogeneity test (`estat timetest`), the relative importance of effects (`estat importance`), simulation at chosen parameters (`nwsaom simulate`), a description of the data before a fit (`nwsaom describe`), and a meta-analysis of the same model fitted to several networks (`nwsaom meta`), the counterpart of RSiena's `siena08`.

The effects library follows RSiena's table of effects. Of the 1201 rows of the effects table of RSiena 1.6.6 (`getEffects()` over twelve data configurations), 1058 are implemented. (The counts refer to RSiena 1.6.6; the effects that RSiena 1.6.7 to 1.6.14 added or changed, for example `WXZX` and the two-mode `toU`, are implemented and compared with RSiena 1.6.14 but are not part of these counts, and the counts change with RSiena releases.) Chapter 5 and Appendix A give the details.

**What RSiena has that `nwsaom` does not.** `nwsaom` does not cover all of RSiena. Not implemented are the generalised method of moments estimator (31 statistics), the settings model (12 effects), a hierarchical Bayesian analysis of several groups, and the continuous (stochastic differential equation) behavior (80 rows, not yet implemented). Inside `nwsaom multinet` there is no composition change, no missing ties and no `estat gof`, `estat importance` or `estat simulate`. Maximum likelihood and Bayesian estimation do not cover non-directed or two-mode networks,

several networks, missing data or composition change. The joint estimation of several groups (`groups()`) does not yet cover maximum likelihood and Bayesian estimation, two-mode networks, several networks, structural zeros and ones, and time-varying dyadic covariates (Section 11.9). Chapter 13 lists every missing effect with its reason.

Binary ties only are supported. As RSiena, `nwsaom` judges a network by its tie values: every tie must have the value 1, 10 or 11. A network with other tie values (real shares, counts) stops the command with an error and is never silently dichotomised; a network that is flagged as valued by `nwcommands` but whose ties are all 1 is accepted. The values 10 and 11 are RSiena's codes for structural zeros and ones; `nwsaom` detects them in the wave networks (Section 13.2). All waves of a model must have the same number of nodes; the actors are identified by their position, so node $i$ must be the same actor in every wave.

## 1.4 Installation and requirements

`nwsaom` needs the `nwcommands` core package, which provides the network objects, `nwset` and the data-handling commands. Install the core first and then `nwsaom`:

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, all
. nwinstall, modelpkg(nwsaom)
```

The first line installs the core, `nwinstall, all` installs its help files, dialog boxes and menu, and `nwinstall, modelpkg(nwsaom)` installs `nwsaom` from its repository. To install the core and all four model packages (`nwsaom`, `nwergm`, `nwdynam`, `nwrem`) in one step, use

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, everything
```

`nwsaom` can also be installed by itself, once the core is there:

```
. net install nwsaom, from(https://raw.githubusercontent.com/thomas-u-grund/nwsaom/main
    /) replace
```

The core must be version 3.6.0 or later (the listings of this manual were checked with core 3.17.2). `nwsaom` checks this when it starts, and also whether the Mata libraries of the core and of `nwsaom` were built together, and says how to reinstall if they were not. (The Mata library of `nwsaom` contains the class definitions of the core that it was compiled against; a core of a different build would otherwise stop Mata with the error `class compiled at different times`.) The Mata libraries are indexed automatically, so that a fresh installation works in a new Stata session without further steps. Installing or updating `nwcommands` later never replaces `nwsaom`, because the core contains no file with the name of a `nwsaom` command. Alternatively, add a clone of the `nwsaom` repository to the ado-path with `adopath + "/path/to/nwsaom"`; the tests of the repository find the core through the environment variable `NWCORE`.

**How to cite.** The manual is the citable reference for the package; it is archived at Zenodo and has a DOI. Please cite it as: Grund, T. U. (2026). *nwsaom: Stochastic actor-oriented models in Stata. Manual.* Zenodo. https://doi.org/10.5281/zenodo.23623807. The DOI is the one of all versions of the record; this manual describes `nwsaom` 1.5.x. Please also report the exact version that produced your results, as printed by `nwsaom version` (this manual: 1.5.14; Section 4.13).

**Versions.** `nwsaom version` reports the installed release of `nwsaom`, the version of the `nwcommands` core, the native plugin in use, the version of RSiena that the results were compared with and the license. `nwversion` lists the core and all installed model packages. `net install`

`nwsaom` installs the state of the main branch. A specific release is installed from its tag, for example

```
. net install nwsaom, from(https://raw.githubusercontent.com/thomas-u-grund/nwsaom/v1
    .5.5/) replace
```

**What changed in the 1.5 series.** The points below are the changes since the previous archived manual (labelled 1.4). Results of models that were possible before are unchanged, except for the count of groups in `nwsaom meta` described below.

- *`uncentered(varlist)`:* raw values for the listed covariates only, RSiena's `coCovar(..., centered = FALSE)` for single covariates (Chapter 4); `nocenter` still applies to all covariates.
- *`groups()` with `netbeh()`:* network effects of the dependent behavior in a joint model of several groups (Section 11.9).
- *Large joint models:* models with `groups()`, about 800 or more actors and a per-actor behavior covariate such as `behfrom()` no longer stop with "too many options" (needs the plugin of 1.5.0 or later), and such models start in seconds instead of minutes.
- *`nwsaom meta`:* an effect held fixed with `fix()` in a group (standard error 0) is no longer counted for that group, as in `siena08`; before, it raised $n$ and the degrees of freedom of the tests of that effect (Section 10.8).
- *Rebuilding the Mata libraries* with the Stata that runs, `nwinstall, buildmata(nwsaom)` (nwcommands 3.14 or later; Section 14.2.3).
- *Data in memory* (1.5.1): a model that names no variable of the data no longer extends or re-sorts the data in memory.
- *Output* (1.4.1 to 1.4.3): `nwsaom describe`, `estat gof`, `estat score`, `estat timetest`, `estat importance`, `estat simulate` and `nwsaom meta` print Stata-style tables; the numbers and the `r()` results are unchanged.

**What changed in the 1.2 series.** This manual describes the behavior of version 1.2; the points below are the changes since the previous archived manual (labelled 1.1), in case you re-run old analyses. Results of models that were possible before are unchanged.

- *Joint estimation of several groups:* the options `groups()` and `gwaves()`, with the rates numbered as RSiena numbers the periods of the combined series, the stored results `e(rates_group)`, `e(rates_period)`, `e(groupn)`, `e(groupwaves)`, `e(ngroups)` and `e(groupvar)`, and `estat timetest` and `estat score` after such a fit (Chapter 11).
- *Two-mode networks:* `nwsaom describe` works for a two-mode network (the actors, events and attendances; before, it counted the cells within the modes and reported an undirected network of all nodes) and has the option `actormode()`; `estat simulate` and `nwsaom simulate` work after and for a two-mode model, and `generate()` saves two-mode networks (Section 10.2 and Section 10.4). A `theta()` matrix of the wrong shape gives a message.
- *`nwsaom meta, rates`* used standard errors that did not exist, so every rate had a missing standard error and was silently excluded; it now uses `e(rates_se)`. The result matrices `r(theta)` and `r(se)` now carry the names of the stored estimates (rows) and of the effects (columns).
- *A message:* the refusal of `reciprocity` for a non-directed network no longer prints a second line.
- *Speed* (version 1.1.1): estimation with the compiled plugin is faster, most for two-mode networks; the results are identical.

**What changed in the 1.1 series.** The points below are the changes between the manual labelled 1.0.3 and the manual labelled 1.1.

- *Tie values are judged by their values.* A network with any tie value other than 1 (real shares, counts, 10 or 11) stops `nwsaom`, `nwsaom multiplex` and `nwsaom multinet` with an error; before, a network of shares that had been saved as not valued was turned into ties silently. A network flagged as valued whose ties are all 1 is accepted.
- *A visible warning for the Mata fallback* (Section 14.4.3).
- *Installation and requirements:* `nwinstall` installs the package; the core must be version 3.6.0 or later; Stata 17 or later; the Mata libraries are built with Stata 17 (libraries of earlier releases were built with Stata 18 and could not be used in Stata 17); a fresh installation works without `mata mlib index`; a mismatch between the Mata libraries of the core and of `nwsaom` gives a clear message instead of the error r(3021).
- *Convergence safeguards* for rate effects and behavior rates (Chapter 9).
- *Messages and dialogs:* the messages about a plugin that is too old now say that a newer plugin is needed; `nwsaom version` prints "plugin build" for the plugin; every option and effect can be set from a dialog box (`db nwsaom`); the version is shown at the bottom of each help file.

**License.** `nwsaom` is free software under the GNU General Public License, version 3. It is derived from RSiena (Ripley, Snijders, Boda, Vörös and others; GPL-2 or GPL-3) and, for the Bayesian estimator, from the algorithm of the multiSiena package (Snijders, Koskinen and Ripley; GPL-3). The source code is public; the file `NOTICE.md` of the repository lists the sources and the version of RSiena that each part was compared with.

**Software.** `nwsaom` needs Stata 17 or later and was tested with Stata 17, 18 and 19 (Stata/BE is enough); older versions of Stata have not been tested. The Mata libraries are built with Stata 17, the oldest tested version, because a library built by a newer Stata is ignored by older ones. The listings of this manual were produced with Stata/BE 18. The edition of Stata determines how large a network can be when it is passed as a matrix (Section 14.4).

**The native plugin.** The simulator is the compiled plugin `saom_sim`. The repository contains builds for macOS (`saom_sim_macos.plugin`), for Linux (`saom_sim_unix.plugin`) and for Windows (`saom_sim_windows.plugin`). The macOS build is compiled on the development machine; the Linux and Windows builds are produced by continuous integration and can be older than the `.ado` files. The Windows build was tested: the full test suite of `nwsaom` passes on Windows (Stata 19 MP, 9 November 2026), and the examples of the Stata Journal article give the same numbers there as on macOS in every digit that Stata displays (they agree to at least 12 significant digits). The Linux build has not been tested on a Linux machine. A plugin that is older than a model needs is detected (the plugin reports its version), and the model is then either fitted by the slower Mata simulator or, for the effects that exist only in the plugin, refused with an error message. Whenever a fit runs on the Mata simulator, `nwsaom` prints one warning line before the estimation starts and says why (Section 14.4.3); you can also look at `e(engine)` afterwards.

## 1.5 How to read this manual

If you know SAOMs from RSiena, read Chapter 3 for the syntax of a first model and use Chapter 4 as a dictionary from RSiena concepts to `nwsaom` options. If you are new to SAOMs, read Chapter 2 first; it is short and uses only the equations that you need to interpret the output.

**Chapter 2**
: The model in brief: rate function, objective function, ministeps, the method of moments and its three phases, conditional and unconditional estimation, convergence.

**Chapter 3**
: How the data must look, a complete first analysis with real output, and the results that

`nwsaom` stores.

**Chapter 4**
Reference of all options of `nwsaom`, organised by topic.

**Chapter 5**
The effects: what each statistic counts, its RSiena name and the option that selects it.

**Later chapters**
Behavior co-evolution, the network types, rates and covariate data, estimation practice, validation against RSiena, and composition change and missing data.

**Chapter 11**
Several groups: joint estimation with `groups()`, when to use it instead of the meta-analysis, and what is not yet available with groups.

**Chapter 14**
Technical notes: the plugin, threads, size limits, performance and the file layout of the source.

**Typographic conventions.** Stata commands and options are set in `typewriter`. Output shown in listings was produced by the commands above it with the version of `nwsaom` described in this manual; the random-number seed is always given, so that the output can be reproduced. Simulation-based estimates change in the last digits when the model or the seed changes, and, for a given seed, they do not depend on the number of processor cores.

**Files named in this manual.** Paths such as `cscripts/`, `dev/`, `native/` and `replication/` are folders of the repository of `nwsaom` (`https://github.com/thomas-u-grund/nwsaom`). The folder `replication/` holds the replication files of the Stata Journal article on `nwsaom`; the scripts that Appendix D names (such as `R/15_fixed_sim.R` or `tools/build_tables.py`) are in it, and their names are relative to it.

# Chapter 2

# The model in brief

This chapter describes the stochastic actor-oriented model (SAOM) and its estimation in the terms that `nwsaom` uses. It is not a substitute for the literature [1, 2]; its purpose is to define the quantities that appear in the output: the rates, the coefficients, the standard errors and the convergence $t$-ratios.

## 2.1 Data and the idea of the model

The data are a network observed at waves $t_1 < t_2 < \cdots < t_W$ on a fixed set of $n$ actors. Let $x(t_m)$ be the adjacency matrix at wave $m$, with $x_{ij} = 1$ if actor $i$ has a tie to actor $j$. The observations are viewed as snapshots of a continuous-time Markov process $X(t)$ on the set of all networks: between two waves ties change one at a time, in the order of unobserved *ministeps*. Each ministep is the decision of one actor to change one of its own outgoing ties (or to leave things as they are). The process is simulated between waves $m$ and $m+1$ starting from the observed $x(t_m)$ over a time interval of length one; the *rate* (below) therefore has the meaning of the expected number of opportunities for change per actor between two waves, and each period $m = 1, \ldots, W-1$ has its own rate $\rho_m$.

## 2.2 The rate function

In each period every actor $i$ gets opportunities to make a ministep at a rate $\lambda_i$. In the basic model all actors share the rate of the period, $\lambda_i = \rho_m$. The waiting time until the next opportunity of anyone is exponentially distributed with parameter $\sum_i \lambda_i$, and the actor who gets the opportunity is $i$ with probability $\lambda_i / \sum_h \lambda_h$. With `ratecov()` the rate depends on actor covariates $z_{ih}$,

$$\lambda_i = \rho_m \exp\Big(\sum_h b_h z_{ih}\Big), \tag{2.1}$$

and the coefficients $b_h$ are estimated jointly with the other parameters (they are shared by all periods; each period keeps its own $\rho_m$). The structural rate effects (`outrate` and relatives) let the rate depend on the actor's current degree, so that it changes during the simulation.

## 2.3 The objective function and the ministep

When actor $i$ gets an opportunity, it chooses among the $n$ alternatives of changing the tie variable $x_{ij}$ for one $j \neq i$ (creating the tie if it does not exist, withdrawing it if it does) and of changing nothing. Write $x(i \rightsquigarrow j)$ for the network that results from the alternative $j$, with $x(i \rightsquigarrow i) = x$. The actor evaluates each alternative with an *objective function* (in RSiena's terminology the *evaluation function*),

$$f_i(\beta, x) = \sum_k \beta_k \, s_{ik}(x), \tag{2.2}$$

a weighted sum of effect statistics $s_{ik}$, and chooses with the multinomial-logit probability

$$P(i \text{ changes } x_{ij} \mid x) = \frac{\exp\big(f_i(\beta, x(i \rightsquigarrow j))\big)}{\sum_h \exp\big(f_i(\beta, x(i \rightsquigarrow h))\big)}. \tag{2.3}$$

The statistics $s_{ik}$ describe the local network around $i$. In `nwsaom` they are selected by options; for example the *outdegree* (density) effect has $s_i(x) = \sum_j x_{ij}$, the *reciprocity* effect $s_i(x) = \sum_j x_{ij}x_{ji}$ and the *transitive triplets* effect (`transtrip`) counts, in RSiena's definition, the triples $(i, j, h)$ with $x_{ij} = x_{ih} = x_{hj} = 1$. A positive coefficient means that an actor is, other things equal, more likely to choose alternatives that increase the statistic. The coefficients are log-odds in the sense of (2.3); the estimates for outdegree are typically negative because ties are sparse.

The model is *actor-oriented* and *myopic*: an actor evaluates the alternatives only through its own statistics $s_{ik}$, not through the effect that its choice would have on other actors' statistics. This is the property that distinguishes a SAOM from an exponential random graph model, which has no actors and no ministeps, but one probability distribution over whole networks.

**Endowment and creation.** The objective function can be split by the direction of the change. A *creation* effect contributes only when a tie is created, an *endowment* effect only when a tie is withdrawn (a tie, once it exists, is "endowed" with extra value). The options `endow()` and `creation()` select such effects; the statistics used in the estimation are, for the endowment role, minus the sum over the ties lost between two waves of the effect's tie statistic in the start network and, for the creation role, the sum over the ties gained of the tie statistic in the end network.

## 2.4 Behavior and several networks

A behavior variable $z_i$ with a bounded range of integer values can evolve together with the network. It has its own rate function and its own objective function. A behavior ministep has three alternatives, to decrease the value by one, to keep it, or to increase it by one (clamped at the range of the observed values), and they are chosen by the same multinomial-logit rule (2.3). Opportunities for network ministeps and behavior ministeps come from competing processes: one waiting time is drawn from the total rate summed over all variables, then the variable that acts is drawn in proportion to its share of the total rate, and then the actor. Selection (network effects that depend on the behavior) and influence (behavior effects that depend on the network) are thus estimated in one model. In a model with several dependent networks, `nwsaom multinet`, the network that changes is drawn in the same way.

## 2.5 Estimation by the method of moments

Let $\theta = (\beta, \rho)$ be the vector of all parameters and $S(X)$ the vector of the statistics that correspond to the parameters. The method of moments chooses $\theta$ such that the expected value of the statistics at the end of the simulated period, given the observed start, equals what was observed,

$$E_\theta\big[S(X(t_{m+1})) \mid X(t_m) = x(t_m)\big] = S(x(t_{m+1})), \tag{2.4}$$

summed over the $W - 1$ periods (the effect statistics are pooled over periods, each period keeping its own rate). The expectation cannot be computed in closed form, so it is replaced by simulation, and the equation is solved by stochastic approximation (Robbins–Monro). `nwsaom` follows the three phases of RSiena's `siena07`.

**Phase 1** $k_0$ simulations at the starting values (option `k0()`; default, as in RSiena, $\max(50, 7+3p)$ for $p$ estimated parameters) estimate the derivative matrix $D = \partial E_\theta S/\partial\theta$. It is estimated with the score-function method as the covariance of the statistics with the scores of the simulated ministeps (with finite differences in the rare case described in Section 9.2); the phase ends with a first step towards the solution.

**Phase 2** Iterations $\theta^{(N+1)} = \theta^{(N)} - a_N D^{-1}(S^{(N)} - s_{\text{obs}})$, where $S^{(N)}$ is the statistic of one simulated period and $s_{\text{obs}}$ the observed one. The gain $a_N$ is reduced over four subphases, starting with `firstg()` (default 0.2) and multiplied by 0.5 in each next subphase, as RSiena's defaults. As in RSiena, $S^{(N)}$ is corrected by its regression on its own score (Section 9.1).

**Phase 3** $k_3$ simulations at the final estimate (option `k3()`, default 1 000) estimate $D$ and the covariance matrix $\Sigma$ of the statistics again. The covariance matrix of the estimates is

$$V = D^{-1}\,\Sigma\,(D^{-1})', \tag{2.5}$$

which is `e(V)`; the standard errors in the table are its diagonal elements' square roots. The same simulations give the convergence statistics.

The first and the third phase are the part of the algorithm that is trivially parallel, and `nwsaom` runs their replicates on worker threads (Section 14.3).

## 2.6 Conditional and unconditional estimation

The rate parameters can be handled in two ways.

**Unconditional estimation.** Each rate $\rho_m$ is a parameter of the method of moments like the others. The statistic that corresponds to a rate is the *distance*, the number of dyads in which the network at the end of the unit-length period differs from the network at its start; its observed value is the number of dyads that differ between the two waves. The rates are estimated together with the effects, with standard errors that come from the same sandwich formula. Option `unconditional` requests this for a network-only model.

**Conditional estimation.** The rates are not Robbins–Monro parameters. Each period is simulated until the distance from the start equals the observed distance between the two waves, so that the number of changes is conditioned on, and the effect statistics are those of the network at that point. A period's rate estimate is the mean of the time the simulation needed, over the $k_3$ phase-3 replicates, and its reported standard error is the standard deviation of these times. There are no rate $t$-ratios. Conditional estimation is the default for a model with one dependent network and no behavior, as in RSiena.

Co-evolution models, models with several networks and network-only models in which composition change restricts which actors are present are always estimated unconditionally. A rate covariate coefficient stays a method-of-moments parameter under conditional estimation. The two estimators are consistent for the same model but differ in finite samples; in RSiena's own tests on the Glasgow data the estimates differed by up to 0.3 standard errors.

## 2.7 Convergence

After phase 3, for each parameter the *convergence $t$-ratio* is computed as the mean deviation $S_k^{(r)} - s_{\text{obs},k}$ over the $k_3$ simulated replicates divided by its standard deviation. If the estimate solves (2.4), the mean deviation is zero. As in RSiena, the fit is considered converged when every $t$-ratio is below 0.1 in absolute value and the *overall maximum convergence ratio* (RSiena's `max convergence ratio`, the largest convergence $t$-ratio over linear combinations of the parameters; stored in `e(tconv_max)`) is below 0.25. The ratios are printed below the coefficient table for all parameters, rates included, and stored in `e(tconv)`; `e(tratio)` holds those of the coefficients in `e(b)`.

These $t$-ratios are not the $z$-statistics of the table, which are the coefficients divided by their standard errors, and a large convergence ratio does not mean a significant effect: it means that

the algorithm did not reach the solution. The usual remedy is to run the model again, starting from the estimates (options `theta0()` and, for unconditional fits, `rate0()`); an example is in Chapter 3. A model that cannot be made to converge, or that stops with the message that the estimation diverged, is usually too rich for the data. Parameters that are not identified because there are too few instances of an effect in the data (isolate effects on small networks, endowment and creation effects of rare events) are a frequent cause.

## 2.8 Non-directed and two-mode networks

In a non-directed network every tie is symmetric ($x_{ij} = x_{ji}$), so that a tie is the joint property of two actors and the model has to say who decides. `nwsaom` offers RSiena's model types, selected with `symtype()`: in the *forcing* model (the default) an actor chooses a tie to create or end and imposes the change on the other actor; in the *confirmation* model a new tie also needs the other actor's consent; and in the pairwise models *force*, *agree* and *joint* a pair of actors meets and the tie changes if the initiator wants it, if both want a new tie or either wants to end one, or if their summed preferences favour the change. In a two-mode network the actors are the nodes of one mode, and they choose ties to the nodes of the other mode (the events), which do not act. See the chapter on network types.

# Chapter 3

# Getting started

This chapter takes you from data to a first fitted model, shows how to read the output and which results are stored. All commands were run with `nwsaom` 1.5.8 on the Glasgow data of RSiena's tutorial.

## 3.1 The data

**Networks are `nwset` objects.** `nwsaom` does not read the network from variables. It takes the *names* of networks that exist in the current `nwcommands` session. Each wave is one network, and a model is told which networks are its waves, in temporal order.

**Waves.** A model needs at least two waves. With exactly two waves you can write `wave1()` and `wave2()`; with two or more waves you write `waves()`, which takes the names in temporal order (`wave1()`/`wave2()` and `waves()` cannot be combined). With $W$ waves there are $W - 1$ periods. The effects are shared by all periods, so that the statistics of the periods are added up in the estimation, and every period has its own rate parameter (`e(rates)`, a $1 \times (W - 1)$ matrix, in place of the scalar `e(rate)` of a two-wave model).

**The actors.** All waves must describe the same set of actors, with the same number of nodes. The actors are identified by their position (node $i$ is the same actor in every wave), and the node attributes are read by row from the dataset.

**Covariates and behavior.** Actor attributes are ordinary variables in the dataset that holds the nodes. `samex(smoke1)` refers to the variable `smoke1` in the current dataset. A time-varying attribute or a behavior is given as one variable per wave, and the variables are listed in the temporal order of the waves, for example `behavior(alcohol1 alcohol2 alcohol3)`. Dyadic covariates and the indicators for missing data are Stata matrices; see the chapters on rates and data and on missing data.

**Binary ties.** The ties must be binary: every tie has the value 1, or 10 or 11 (RSiena's codes for structural zeros and ones, Section 13.2). A network with other tie values (real shares, counts) is rejected with an error; a network that is flagged as valued but whose ties are all 1 is accepted.

### The Glasgow data

The data of the examples are the three waves of friendship among 50 pupils of a Scottish school (RSiena's `s50`), with their alcohol use, smoking and sport. They are downloaded by `nwwebuse`:

```
. nwwebuse glasgow, nwclear
. nwset
```

```
(3 networks)
--------------------
      glasgow1
      glasgow2
      glasgow3
```

The option `nwclear` removes the networks that are in memory first. The dataset has one row per pupil and the variables `alcohol1`–`alcohol3` (1–5), `smoke1`–`smoke3` (1–3) and `sport1`–`sport3` (1–2).

### Your own data

A network is declared from an adjacency matrix with `nwset, mat()` or from an edge list with `nwfromedge` (see `help nwset` and `help nwfromedge`). The following sequence builds the Glasgow waves from three Stata matrices `A1`, `A2`, `A3` and a dataset with 50 rows, in the way that you would build them from your own data. The matrices come from the original networks with `nwtomata`; the dataset keeps the covariates:

```
. nwwebuse glasgow, nwclear
. forvalues w = 1/3 {
  2.     nwtomata glasgow`w', mat(M`w')
  3.     mata: st_matrix("A`w'", M`w')
  4. }
. keep smoke1 alcohol1 alcohol2 alcohol3
. save glasgow_cov, replace
. nwclear
. use glasgow_cov, clear
. forvalues w = 1/3 {
  2.     nwset, mat(A`w') directed name(w`w')
  3. }
. nwset
(3 networks)
--------------------
      w1
      w2
      w3
. nwsaom, waves(w1 w2 w3) outdegree reciprocity transtrip samex(smoke1) seed(1)
```

The last command fits the model; its result is in Section 3.2 (with the same effects plus the covariate it gives `samex_smoke1` = 0.133 with standard error 0.147).

## 3.2 A first complete example

The classical first model has three effects: the outdegree (density) effect, the reciprocity effect and the transitive triplets effect. The first effect is required in every model.

```
. nwwebuse glasgow, nwclear
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip seed(1)

SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
Actors: 50                             Periods: 2

------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.675306   .1130571   -23.66   0.000    -2.896894   -2.453718
 reciprocity |   2.453519   .1841138    13.33   0.000     2.092662    2.814375
```

```
   transtrip |   .6151182    .075624     8.13   0.000     .4668978    .7633385
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):

-----------------------------------
             |   period1    period2
-------------+---------------------
        rate |    6.4432     5.2473
          se |    1.0683     0.8841
-----------------------------------
Overall maximum convergence ratio: 0.123 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

The fit takes a second or two.

**Reading the output.**

- The first line says that the model is estimated by the method of moments and, for a network-only model, conditionally (Section 2.7). `Periods: 2` are the periods between the three waves. With a two-wave model the line shows the estimated rate instead.
- The coefficient table contains the parameters of the objective function with the standard errors from phase 3, the $z$-ratios and the confidence intervals. The label in the upper left corner shortens the name of the dependent network. A negative outdegree coefficient is typical: ties are rare, and an actor creates one only if the other effects make it worthwhile. The positive reciprocity coefficient says that actors prefer to reciprocate ties; the positive transitive triplets coefficient that they prefer to befriend friends of friends.
- The rate parameters are listed per period below the table. Under conditional estimation the rate is the mean simulated time that the process needs, at rate 1 for every actor, to reproduce the observed amount of change, and the "se" is the standard deviation of that time. Under unconditional estimation the rates are estimated like the other parameters.
- The convergence $t$-ratios are discussed next.

**Checking convergence.** All $t$-ratios should be smaller than 0.1 in absolute value, and the overall maximum convergence ratio smaller than 0.25. In the example the overall ratio is 0.123 and all three individual ratios are below 0.1, so both criteria are met. When they are not, RSiena's advice is to run the estimation again from the current estimates. In `nwsaom` that is done with `theta0()`, which takes the starting values of the coefficients in the order of `e(b)`. An example with covariates, where the starting values are taken from the first fit:

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip samex(
    smoke1) egox(alcohol1) altx(alcohol1) simx(alcohol1) seed(2)
. matrix b = e(b)
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip samex(
    smoke1) egox(alcohol1) altx(alcohol1) simx(alcohol1) theta0(‘=b[1,1]’ ‘=b[1,2]’ ‘=b
    [1,3]’ ‘=b[1,4]’ ‘=b[1,5]’ ‘=b[1,6]’ ‘=b[1,7]’) seed(3)
```

The first of these two fits has the overall maximum convergence ratio 0.112 and all individual ratios below 0.1; the second, started from its estimates, gives 0.103. The coefficients of the two runs differ by much less than a standard error, which is the kind of agreement that you want to see. Another seed gives different digits but should give the same picture. The unconditional fit has the option `unconditional`, and a restart must then also give the rates with `rate0()` (an example is in the help file).

## 3.3 A model with covariates

Covariates are selected by options named after RSiena's effects. The first of the two fits above gives (with `detail`, which adds the lines below the table and does not change the estimates):

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip samex(
    smoke1) egox(alcohol1) altx(alcohol1) simx(alcohol1) seed(2) detail
...
------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.779783   .1599143   -17.38   0.000    -3.093209   -2.466357
 reciprocity |   2.422549   .1902843    12.73   0.000     2.049599    2.795499
samex_smoke1 |   .1141395   .1806006     0.63   0.527    -.2398312    .4681102
altx_alcoh~1 |  -.0332311   .0760984    -0.44   0.662    -.1823812    .1159189
egox_alcoh~1 |   .0600181   .0834505     0.72   0.472    -.1035418     .223578
   transtrip |   .6238508    .076142     8.19   0.000     .4746151    .7730864
simx_alcoh~1 |   .5879409   .3007171     1.96   0.051    -.0014538    1.177336
------------------------------------------------------------------------------
...
RSiena effect names:
  samex_smoke1                  same smoke1
  altx_alcohol1                 alcohol1 alter
  egox_alcohol1                 alcohol1 ego
  simx_alcohol1                 alcohol1 similarity
Covariates centred by their means, as RSiena's coCovar(centered = TRUE):
  alcohol1                          2.880
```

Note three things. First, the coefficients are named *effect_variable* in the spelling of the option; the effect types come in a fixed order, which is not the order of the options on the command line (`matrix list e(b)` shows it). Second, covariates are centred by their means by default, as in RSiena, so that the outdegree coefficient is RSiena's; the means are listed under the table (with `detail`) and stored in `e(covmeans)`; `nocenter` switches centring off. Third, the lines "RSiena effect names" give the names that RSiena uses for the effects, which are stored in `e(rsiena_labels)`. A model with all effects of the table in the chapter on effects is written in the same way.

## 3.4 Two waves

Two waves can be given as `wave1()` and `wave2()`. The rate is then a scalar:

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity seed(1)

SAOM (Method of Moments, conditional), waves: glasgow1 -> glasgow2
Actors: 50                          Estimated rate:  5.552 (0.856)

------------------------------------------------------------------------------
glasgow1_t~2 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |   -2.26413    .127996   -17.69   0.000    -2.514997   -2.013262
 reciprocity |    2.50883   .2328995    10.77   0.000     2.052355    2.965304
------------------------------------------------------------------------------
Overall maximum convergence ratio: 0.178 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

## 3.5 A model with behavior co-evolution

A behavior is added with `behavior()`, with one variable per wave. At least the effect `linear` is required with it. The network side then has the effect `behsim` (similarity of the behavior as a selection effect), and the behavior side has the effects `linear`, `quadratic` and `avalt` (average alter, an influence effect). Co-evolution models are always estimated unconditionally.

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behsim
    behavior(alcohol1 alcohol2 alcohol3) linear quadratic avalt seed(12345)

SAOM co-evolution (Method of Moments), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2
Behavior: alcohol1 alcohol2 alcohol3

------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |   -2.76126   .1460683   -18.90   0.000    -3.047548   -2.474971
 reciprocity |    2.35587   .1982928    11.88   0.000     1.967223    2.744517
   transtrip |    .621357   .0803867     7.73   0.000      .463802    .7789121
      behsim |   1.460717   .6218014     2.35   0.019      .242009    2.679426
  beh_linear |   .3948453   .2135712     1.85   0.064    -.0237465    .8134372
beh_quadra~c |  -.6002813    .320045    -1.88   0.061    -1.227558    .0269954
   beh_avalt |   1.305018   .7780128     1.68   0.093    -.2198586    2.829896
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):
-----------------------------------
             |   period1    period2
-------------+---------------------
network_rate |    6.4944     5.1666
          se |    1.2965     0.8975
behavior_r~e |    1.3126     1.8015
          se |    0.3471     0.4619
-----------------------------------
...
Overall maximum convergence ratio: 0.138 (criterion: below 0.25; met)
```

The behavior coefficients carry the prefix `beh_`; the network coefficients come first in `e(b)` (their number is `e(p_net)`, here 4). The rates of the behavior are in `e(rates_beh)`. The co-evolution is the subject of its own chapter.

## 3.6 Goodness of fit

After a fit, `estat gof` compares the observed network with networks simulated at the estimates, for the distributions of the outdegrees, of the indegrees and of the geodesic distances (the same three statistics as RSiena's defaults), with a Mahalanobis-distance test for each:

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip seed(1)
. estat gof
Goodness of fit: Mahalanobis distance test
50 simulations at the estimates, 2 periods pooled

-----------------------------------------------------
           Statistic |   Mahalanobis  P-value      Fit
---------------------+-------------------------------
           Outdegree |        13.714    0.100       ok
            Indegree |         8.669    0.560       ok
```

```
            Geodesic |        36.428    0.000 rejected
-----------------------------------------------------
rejected: P < 0.05 (one-tailed); the model does not reproduce this statistic
```

(In the output the plots are announced as well; the lines about them are omitted here.) A small $p$-value is evidence against the model for that statistic: here the three-effect model does not reproduce the distribution of the geodesic distances. The $p$-values are returned in `r(p_outdegree)`, `r(p_indegree)` and `r(p_geodesic)`. Goodness of fit has its own chapter.

## 3.7 Stored results

The most used results of the three-wave fit above are these (`ereturn list` shows all of them):

```
. matrix list e(b)
e(b)[1,3]
      outdegree  reciprocity    transtrip
y1   -2.6753063    2.4535188    .61511818
. display e(tconv_max)
.1227256
. matrix list e(rates)
e(rates)[1,2]
        period1    period2
rate  6.4432009  5.2472905
```

| Result | Content |
|---|---|
| `e(b)`, `e(V)` | Coefficients and their covariance matrix; only the parameters of the objective function (the rates are not included). In a co-evolution fit the network coefficients come first, then the behavior coefficients (`beh_`), `e(p_net)` is the number of the network ones. |
| `e(tratio)`, `e(tconv)`, `e(tconv_max)` | Convergence $t$-ratios of the coefficients, of all parameters including the rates, and the overall maximum convergence ratio. |
| `e(rate)`, `e(rate_se)` | Rate and its standard error in a two-wave fit (`wave1()`/`wave2()`). |
| `e(rates)`, `e(rates_se)`, `e(rate_tratios)` | The same per period, $1 \times (W-1)$ matrices, in a fit with `waves()`. |
| `e(rates_beh)` | Behavior rates per period (co-evolution). |
| `e(ratecoefs)` | Covariate-rate coefficients of `ratecov()` (with standard errors, $t$-ratios and the flag whether a coefficient was kept fixed). |
| `e(covmeans)` | Means used to centre the covariates. |
| `e(rsiena_labels)` | RSiena's names of the coefficients, in the order of `e(b)`, separated by `\|`. |
| `e(conditional)` | 1 for conditional, 0 for unconditional estimation. |
| `e(engine)` | `native` or `mata`: which simulator ran (Chapter 14). |
| `e(waves)` | The names of the waves. |
| `e(nodes)`, `e(nwaves)` | Number of actors and of waves. |
| `e(ngroups)`, `e(groupn)`, `e(groupwaves)`, `e(groupvar)` | With `groups()`: the number of groups, the number of actors and of waves of each group (matrices $1\times$ groups) and the name of the group variable; `e(N)` is the total number of actors. |
| `e(rates_group)`, `e(rates_period)` | With `groups()`: for each column of `e(rates)` (and `e(rates_beh)`) the group and RSiena's number of the period; the columns are named `period1`, `period2`, `period4`, .... Chapter 11. |

Because `e(b)` and `e(V)` are in place, ordinary Stata postestimation works on them, for example `test` and `lincom` for Wald tests of the coefficients:

```
. test reciprocity = transtrip
```

```
 ( 1)  reciprocity - transtrip = 0
           chi2(  1) =   74.84
         Prob > chi2 =    0.0000
```

## 3.8 A word on run time and reproducibility

The examples of this chapter run in one to a few seconds each. The estimates depend on the random-number seed, which you set with `seed()`; for a given seed, the result is the same with any number of threads (`cores()`). Always report the seed together with the model, and judge a model by the convergence ratios and not by the digits of a single run.

# Chapter 4

# Options reference

This chapter lists the options of `nwsaom` by topic. The definition of each effect (what is counted, its RSiena name and its validation) is in Chapter 5 and Appendix A; here you find which option selects it, which arguments it takes and what it can be combined with. The same information is in the help files `nwsaom.sthlp` and `nwsaom_remarks.sthlp`. Options marked *native* exist only in the C plugin: with a plugin that is too old, or with the Mata engine, they are refused with an error message (Chapter 14).

## 4.1 Syntax

```
nwsaom, waves(netlist)
        outdegree [reciprocity]  |  outdegreeendow outdegreecreation [...]
        [ endow(effects) creation(effects) ]
        [ covariate, structural, interaction and alias options ]
        [ behavior(...) and its effects ]
        [ composition change and missing data options ]
        [ groups(varname) gwaves(spec) ]
        [ ratecov(varlist) ratecovcoef(numlist) nocenter uncentered(varlist) ]
        [ symmetric symtype(string) ]
        [ unconditional rate0(numlist) theta0(numlist) k0(#) k3(#) firstg(#) seed(#)
    cores(#) ]

nwsaom multinet, net1(namelist) net2(namelist) [ eff1() eff2() cross1() cross2() ... ]
estat gof | estat mems
```

Every model needs the waves and the baseline effect `outdegree`. Everything else is optional. Options that take a *varlist* are repeated per variable: one effect with its own coefficient per variable, named *effect_variable* in the spelling you used (for example `samex(smoke1 sport1)` gives `samex_smoke1` and `samex_sport1`). In `e(b)`, and thus in `theta0()`, the effect types appear in a fixed order and the variables within a type in the order listed; `matrix list e(b)` shows it.

## 4.2 Observed waves

| Option | Meaning |
|---|---|
| `wave1(netname)`, `wave2(netname)` | The two waves of a model with exactly two waves. Both are required together; not combinable with `waves()`. |

| Option | Meaning |
|---|---|
| `waves(netlist)` | Two or more waves in temporal order; a netlist may use patterns such as `glasgow*` or `w1-w3`, which expand in the order in which the networks were created. The $W-1$ periods are chained into one pooled fit with one rate per period. |
| `tvcov(groups)` | Time-varying covariates (RSiena's `varCovar`). `tvcov(a1 a2 a3 \| s1 s2 s3)` declares covariates with one variable per wave; `\|` separates covariates. Period $p$ uses the value of wave $p$ (the value of the last wave is not used). A covariate is named after its first variable without trailing digits (here `a` and `s`) and used under that name in `nodeocov()`, `nodeicov()`, `nodecov()`, `nodematch()`, `simcov()` or their aliases, for example `tvcov(a1 a2 a3) egox(a) simx(a)`. Centred by the mean over all periods. No missing values. Possible with `behavior()` (network covariate effects only) and in two-mode networks; not with `interact()`, `symmetric` or `estat gof`. |
| `groups(`*varname*`)` | Joint estimation of several groups (RSiena's `sienaGroupCreate` and `siena07`): the observations are the actors of all groups, *varname* gives the group of each actor, and the wave networks have no ties between groups. One set of effect parameters for all groups; every period of every group has its own rate (also of a behavior). Needs `waves()`. Chapter 11. |
| `gwaves(`*spec*`)` | With `groups()`: the waves of the groups when they differ, `gwaves(2: 1 2 \| 3: 2 3)` (group value, colon, positions in `waves()`); a group not named has all waves. |
| `actormode(#)` | Two-mode networks: the mode whose nodes are the actors, 1 or 2; the default 2 is the rows of the affiliation matrix given to `nwset, bipartite`. |

## 4.3 Baseline network effects

| Option | Meaning |
|---|---|
| `outdegree` | The density effect, evaluation-function role. Required in every model unless the endowment/creation form is used. |
| `reciprocity` | Reciprocated ties. |
| `endow(effects)`, `creation(effects)` | Endowment and creation roles of effects (RSiena effect types `endow` and `creation`). The list may contain `outdegree reciprocity transties cycle4 gwesp balance transtrip cycle3 transmedtrip transrectrip inpop outpop inact outactsqrt indegpopularity outpopularity outactivity inactivity` for the network and `linear quadratic avalt avsim` for a behavior. An effect named in both lists is split into its two roles and replaces the plain effect; naming it in only one list, or also giving the plain effect, is an error. For `gwesp` the decay is taken from `gwesp(#)`. |

| Option | Meaning |
|---|---|
| `endow(eff:var)`, `creation(eff:var)` | Covariate effects are split per variable: `endow(altx:alc)` `creation(altx:alc)`, where the effect is `egox`, `altx`, `samex`, `simx` or `nodecov`. Coefficients `altxendow_alc` and `altxcreation_alc`. No missing values yet. |
| `outdegreeendow`, `outdegreecreation` | The long forms for the outdegree effect. They are currently *refused*: the two statistics (lost and gained ties) add up to the distance statistic of the rate, so with the rate estimated the model is not identified (RSiena reports a singular covariance matrix for it as well). |
| `reciprocityendow`, `reciprocitycreation` | Reciprocity split, used together with `outdegree`. |
| `transtiesendow`, `transtiescreation` | `transties` split. Not yet with `symmetric`. |

The pairs `outdegreeendow`/`outdegreecreation`, `reciprocityendow`/..., `transtiesendow`/... and `linearendow`/... are the same thing as the `endow()`/`creation()` lists and still work. Endowment and creation effects work with `waves()`, `present()`, `missnet()` and `behavior()`; with `structural()` they are rejected.

## 4.4 Covariate effects

Covariates are read from the current dataset by row. The options below take a varlist.

| Option (alias) | Meaning |
|---|---|
| `nodeocov()` (`egox()`) | Ego (sender) covariate effect, RSiena's `egoX`. |
| `nodeicov()` (`altx()`) | Alter (receiver) covariate effect, `altX`. |
| `nodematch()` (`samex()`) | Homophily on a categorical attribute, exact match, `sameX`. |
| `simcov()` (`simx()`) | Similarity, $1 - \lvert x_i - x_j \rvert$/range minus its mean over all pairs, `simX`. |
| `nodecov()` | Sum of the sender's and the receiver's own value (continuous covariate). |
| `absdiffx()`, `egoxaltx()`, `diffx()`, `higher()`, `egosqx()`, `altsqx()`, `samexrecip()`, `simrecipx()` | RSiena's `absDiffX`, `egoXaltX`, `diffX`, `higher`, `egoSqX`, `altSqX`, `sameXRecip`, `simRecipX`. |
| `egoplusaltsqx()`, `diffsqx()`, `egodiffx()`, `althigheregox()`, `unequalx()`, `egolthresholdx()`, `egorthresholdx()`, `altlthresholdx()`, `altrthresholdx()`, `degabsdiffx()`, `degposdiffx()`, `degnegdiffx()`, `covthreshold(#)` | Covariate contrast and threshold effects. The four threshold effects compare the centred covariate with `covthreshold(#)` (default 0). Directed relations, no missing values. Native. |

| Option (alias) | Meaning |
|---|---|
| `dyadcov(matnames)`, `dyadcovrecip(matnames)` | Dyadic covariate (RSiena's `X` and `XRecip`). Each name is a Stata $n \times n$ matrix in the node order of the waves; it is centred by the mean of its non-missing off-diagonal cells (not with `nocenter`), missing cells count as 0 after centring. One coefficient `dyadcov_`*name* per matrix. Not with `interact()`. |
| `tvdyadcov(stem ...)`, `tvdyadcovrecip(stem ...)` | Changing dyadic covariates: matrices *stem*1 ... *stem*$(W-1)$, one per period. |
| `nocenter` | Use the raw values of the covariates of `egox()`, `altx()`, `nodecov()` and `ratecov()`. By default they are centred by their means, as RSiena's `coCovar(centered = TRUE)`; missing values are set to the mean. The means are returned in `e(covmeans)`. `samex()` does not depend on centring; `simx()` is centred by its similarity mean either way. |
| `uncentered(varlist)` | Use the raw values of the listed covariates of `egox()`, `altx()`, `samex()`, `nodecov()` and related effects, and centre the others as usual: RSiena's `coCovar(..., centered = FALSE)` for single covariates. Centring changes only the outdegree parameter (by the covariate's coefficient times its mean), so this option reproduces a script that sets `centered = FALSE` for one covariate, such as the log of the group size in RSiena's multi-group examples. |

Missing covariate values: for `absdiffx()`, `egoxaltx()`, `altx()`, `egox()`, `samex()` and `simx()` a missing value is imputed with the mean when actors decide, but the statistic skips the ties whose relevant actor has a missing value, as in RSiena.

## 4.5 Structural network effects

| Option | Meaning |
|---|---|
| `indegpopularity`, `outpopularity`, `outactivity`, `inactivity` | Square-root and squared degree effects (`inPopSqrt`, `outPopSqrt`, `outAct`, `inActSqrt`). |
| `inpop`, `outpop`, `inact`, `outactsqrt` | RSiena's `inPop`, `outPop`, `inAct`, `outActSqrt`. `outpop` and `inact` have the same total statistic and cannot be used together. |
| `inpopc`, `outpopc`, `outactc`, `inactc`, `degplus`, `degplusc`, `avdeg(#)`, `transtriads` | Centred and combined degree effects, the average-degree effect and the triangle count; `degplus`, `degplusc` and `transtriads` for non-directed relations only. Native. |
| `outtrunc(#)`, `outmore(#)`, `outthreshold(#)`, `outinv(#)`, `outsqinv(#)` | Functions of the outdegree with a parameter; one term per value given. |
| `transtrip`, `transties`, `transmedtrip`, `transrectrip`, `cycle3`, `cycle4`, `balance`, `sqrtcycle4`, `between` | Triadic and cyclic effects. `balance` has no parameter (its constant is computed from the data). |

| Option | Meaning |
|---|---|
| `gwesp(#)` | Geometrically weighted edgewise shared partners, the direct decay value (RSiena's parameter divided by 100). |
| `gwespfb(#)`, `gwespbf(#)`, `gwespbb(#)`, `gwesprr(#)`, `gwdspff(#)`, `gwdspfb(#)` | The other weighted shared-partner versions, with the same decay scale. |
| `nbrdist2`, `dblnbrdist2` | Actors at distance two (once, at least twice). |
| `recipop`, `recipopsqrt`, `reciact`, `reciactsqrt` | Reciprocal-degree effects. |
| `isolatenet`, `outiso`, `antiiso`, `antiiniso`, `antiiniso2`, `inplus3`, `isolatepop` | Isolate effects. They are weakly identified on small or sparse networks; `antiiso` and `isolatepop` can destabilise the estimation there. |
| `outoutass`, `ininass`, `outinass`, `inoutass` | Degree assortativity, default parameterisation. |
| `ownx(effect[:#] ...)` | Further own-network effects without a covariate (native), among them `jin jout transtrip1 transtrip2 transrectrip2 transtripfr transtripfe transtripee`, the `outpopmore`, `outpopsqrtmore`, `outpopthreshold` families, the ego degree effects, the assortativity effects with parameter 2 and the primary-setting effects. |
| `netx(effect:var ...)`, `xparam(#)` | Further covariate network effects (native), for example `crossxoutact`, `homxoutact2`, `transtripx`, `altdist2`, `sameinxcycle4`, `samexv:v1:v2`; `xparam()` is the parameter of some of them. |
| `samextranstrip()`, `diffxtranstrip()`, `simxtranstrip()`, `homxtranstrip()`, `samexoutact()`, `diffxoutact()`, `samexreciact()`, `diffxreciact()`, `homxoutact()`, `altxoutact()`, `inpopx()`, `outpopx()`, `inactx()`, `outactx()`, `samexinpop()`, `diffxinpop()` | Covariate-by-triad, popularity and activity effects. One numeric covariate per name, directed one-mode networks, no missing values. Native. |
| `dyadx(effect:matrix ...)` | Network effects of a constant dyadic covariate (`wwx`, `cywwx`, `inwwx`, `outwwx`, `wxx`, `xwx`, `xwx1`, `xwx2`, `xxw`). Matrix used raw. Native. |
| `tvdyadx(effect:stem ...)` | The same with changing dyadic covariates. |

The structural effects that are meaningless in a non-directed network (`reciprocity`, `cycle3`, `inactivity`, `outpopularity`, `ininass`, `inoutass`, `outoutass`, `antiiso`, `isolatepop`, `transrectrip`, `transtrip`) are rejected when the relation is non-directed. In a two-mode network the available effects are `outdegree`, `indegpopularity`, `outactivity`, `cycle4`, `outiso`, `antiiniso`, `antiiniso2`, `inplus3`, `gwdspfb()`, `avdeg()`, `outtrunc()`/`outmore()`/`outthreshold()`, `ownx()`, and the covariate effects `nodeocov()` (on the actors), `nodeicov()` (on the events), `egosqx()`, `altsqx()` and some `netx()` effects.

## 4.6 Interactions

| Option | Meaning |
|---|---|
| `interact(e1#e2[#e3] ...)` | Two- or three-way interaction of effects already in the model (RSiena's `includeInteraction`), with its own coefficient. Several interactions may be listed. RSiena's rule applies: effects are ego effects, dyadic effects or other effects, and two effects need an ego effect or two dyadic effects, three effects two ego effects or only ego and dyadic effects. A covariate effect for several variables is named by its coefficient name, for example `interact(samex_smoke1#egox_alcohol1)`; with one variable the effect type alone works. Coefficients are named `interact_`*A*`_`*B* (`ix_`*A*`_`*B* if longer than 32 characters). Behavior interactions are not supported. |

## 4.7 Behavior co-evolution

| Option | Meaning |
|---|---|
| `behavior(varlist)` | A bounded integer behavior, one variable per wave in temporal order. Several behaviors: `behavior(a1 a2 a3 : linear avalt \| s1 s2 s3 : linear)`, separated by `\|`, each optionally followed by `:` and its own effects. Coefficients are named `beh_`*name*`_`*effect*. |
| `linear` | Linear shape effect, the behavior's baseline; required with `behavior()` unless `linearendow`/`linearcreation` are given. (The latter are currently refused, for the same reason as `outdegreeendow`.) |
| `quadratic`, `avalt`, `avsim` | Quadratic shape; average-alter influence; average-similarity influence. Each also in an endowment/creation form (`quadraticendow` `quadraticcreation`, `avaltendow` `avaltcreation`, `avsimendow` `avsimcreation`). |
| `behsim`, `behegox`, `behaltx`, `behegoplusalt` | Selection effects on the network side that use the current behavior (similarity, ego, alter, ego plus alter). Network coefficients; require `behavior()`. |
| `beheff(tokens)` | Behavior effects of network position, alters and similarity (the tokens are listed in the help file); some are native only. |
| `behfrom(varlist)` | Covariate main effects on the behavior (`effFrom`); at most 8 covariates in total with `behx()`. |
| `behx(family:var ...)` | Behavior effects of the network times a covariate; *var* may also be `beh2`, `beh3`, ..., another behavior. Native. |
| `behavxalt()`, `behavsamexalt()`, `behtotxalt()`, `behtotsamexalt()` | Covariate-moderated influence (`avXAlt`, `avSameXAlt`, `totXAlt`, `totSameXAlt`). |
| `behw(family:matrix ...)`, `tvbehw(family:stem ...)`, `behwparam(#)` | Behavior effects of the network times a constant or changing dyadic covariate (native). |

| Option | Meaning |
|---|---|
| `behshape(effect[:#] ...)`, `behnet2(effect[:#] ...)` | Further shape effects and further behavior effects of the network (native). |
| `netbeh(effects)` | Network effects with the dependent behavior as covariate (native). First behavior only; coefficients `bv_`*effect*. |
| `behtheta0(numlist)` | Starting values of the behavior-side coefficients. |

## 4.8 Composition change, missing data and structural values

| Option | Meaning |
|---|---|
| `present(varlist)` | Actors present at each wave: one 0/1 variable per wave. An actor can act in a period if it is present at the wave at which the period starts. |
| `joinat(varname)`, `leaveat(varname)` | Joiners and leavers at fractional times within a period, exactly as RSiena's `sienaCompositionChange()` option 1; time scale wave 1 = 1, last wave = $W$. A missing value means from the first wave, respectively to the last. One interval per actor. |
| `missnet(matlist)` | One 0/1 $n \times n$ matrix per wave marking missing dyads. |
| `missbeh(varlist)` | One 0/1 variable per wave marking missing behavior values; requires `behavior()`. |
| `structural(matname)` | Deprecated way to mark structural zeros and ones: one 0/1 matrix of dyads fixed in all periods, converted to the codes of every wave. Replaced by the codes 10 and 11 in the wave networks, which need no option (Section 13.2). |

Composition change implies unconditional estimation. Data preparation (coding of absent actors' ties and of the missing values) is the user's responsibility; see the chapter on composition change and missing data.

## 4.9 Rates

| Option | Meaning |
|---|---|
| `ratecov(varlist)` | Covariates that change the rate: actor $i$'s rate is the period rate times $\exp(\sum b_k x_k[i])$. One coefficient per variable, shared by all periods. Works with `waves()`, `symmetric` and two-mode networks (with changing covariates and with the behavior as the covariate). Not with co-evolution of a one-mode network (use `behratecov()` for the rate of the behavior). |
| `ratecovcoef(numlist)` | Starting values of these coefficients, one per variable or one for all. Default 0. |

| Option | Meaning |
|---|---|
| `outrate`, `inrate`, `reciprate` and their `inv`/`log` versions (`outrateinv`, `outratelog`, ...) | Structural rate effects: the rate depends on the actor's current out-, in- or reciprocated degree (or its inverse or logarithm). Each adds a covariate-rate coefficient. Directed one-mode relations, plain waves only; non-directed and two-mode networks have `outrate`, `outrateinv`, `outratelog`. Native. |
| `behratecov(varlist)`, `behrate(list)`, `behratecoef(numlist)` | Rate of the behavior in a co-evolution model: covariates (constant, changing, or the behavior itself) and rate effects of the network and of the exposure to the behavior of the alters (Section 8.4). |

## 4.10 Non-directed relations

| Option | Meaning |
|---|---|
| `symmetric` | Model a non-directed relation. It is not needed when the waves are declared undirected (`nwset, undirected`) or when all directed waves happen to be tie-symmetric: `nwsaom` then models a non-directed relation automatically, as RSiena does. A tie-symmetric directed wave stays directed with `reciprocity` or with effects for which the non-directed model does not apply. With directed waves that are not tie-symmetric `symmetric` is an error: it does not symmetrise data. Combinable with `waves()`, `present()`, `missnet()`, `ratecov()` (network only) and co-evolution (native plugin required). |
| `symtype(string)` | The model type, by name or by RSiena's `modelType` number: `forcing` (2, the default), `confirmation` (3), `force` (4), `agree` (5), `joint` (6). |

## 4.11 Estimation control

| Option | Meaning |
|---|---|
| `ml`, `mloptions(`*`string`*`)` | Maximum likelihood instead of the method of moments, for one directed one-mode network with or without one behavior; controls `n1()`, `nsub()`, `nstart2()`, `n3()`, `mult()`, `gain()`, `seed()` (Section 9.8). |
| `unconditional` | Unconditional method of moments for a network-only model; the rates become method-of-moments parameters. The default for a network-only model is conditional estimation. Co-evolution, `nwsaom multinet` and `nwsaom multiplex` models are always unconditional. |
| `rate0(numlist)` | Starting value(s) of the rate(s) of an unconditional network-only fit: one value or one per period. Default: RSiena's closed-form value from the data. Not used under conditional estimation. |

| Option | Meaning |
|---|---|
| `theta0(numlist)` | Starting values of the coefficients, one per effect in the order of `e(b)` (the error message lists the order if the argument is missing or has the wrong length). Default all zero, except that `outdegree` of a network-only model starts at RSiena's data-derived value. `nwsaom refit` restarts the last fit from its estimates (Chapter 9). |
| `k0(#)` | Number of phase-1 replicates; default, as in RSiena, $\max(50, 7 + 3p)$ for $p$ estimated parameters. |
| `k3(#)` | Number of phase-3 replicates, default 1 000. |
| `firstg(#)` | Initial gain of phase 2, default 0.2. |
| `seed(#)` | Random-number seed. |
| `cores(#)` | Threads of the native simulator; 0 (default) is all physical cores, 1 is single-threaded. Results for a given seed do not depend on it. |

## 4.12 Several dependent networks: `nwsaom multinet`

```
nwsaom multinet, net1(namelist) net2(namelist) [eff1(effects) eff2(effects)
        cross1(effects) cross2(effects) sym1 sym2 actormode(#) tvcov(groups)
        behavior(namelist) beheff1(effects) beheff2(effects) beheff12(effects)
        beheff21(effects) behint(interactions) missbeh(namelist)
        theta0(numlist) fix(namelist) k0(#) k3(#) firstg(#) seed(#)]
```

Two networks on the same actors evolve together (RSiena's two dependent network variables); estimation is unconditional, with a rate per network and period. Native only.

- `net1()` and `net2()` list the waves of the two networks; the number of waves is the same, at least two. One of the networks may be two-mode (detected from the data); `actormode()` says which mode acts.
- `eff1()` and `eff2()` give the within-network effects of each network; default `outdegree reciprocity`. Possible are `outdegree reciprocity transtrip cycle3 transmedtrip cycle4 indegpop outact` and, with a variable, `egox:`*var*, `altx:`*var*, `samex:`*var*, `simx:`*var*, and `netbeh:`*effect* for the behavior as covariate.
- `cross1()` lists the cross-network effects of network 1 that depend on network 2, `cross2()` those of network 2 on network 1 (the basic ones are `crprod crprodrecip crprodmutual from frommutual fromany to toback torecip clxwx clxwx1 clxwx2 mixedinxw`; there are many further families, with a covariate, a changing covariate (`tvcov()`) or a dyadic covariate matrix; a parameter follows a colon, for example `from:2`). The cross-network statistics read the other network at the start of the period, the simulation its current state, as RSiena does.
- `sym1` and `sym2` declare the first, respectively second network non-directed.
- `behavior()` adds one behavior; `beheff1()` and `beheff2()` give its effects driven by the first and by the second network, `beheff12()` and `beheff21()` the effects of both networks, `behint()` interactions of behavior effects and `missbeh()` flags missing behavior values (Section 7.3.9).
- `theta0()` and `fix()` give the starting values and hold effects fixed; `estat score` and `estat timetest` are available afterwards.

## 4.13 Postestimation commands

**estat gof**
: Goodness of fit with RSiena's Mahalanobis-distance test for the outdegree, indegree and geodesic distributions. Options and the returned `r()` results are in `help nwsaom_estat`.

**estat mems**
: Micro effects on macro structure sensitivity analysis (Duxbury); `help nwsaom_estat`.

**estat score, estat timetest, estat importance, estat simulate**
: Score-type tests, the test of time heterogeneity (with `fixrates` as RSiena's `sienaTimeTest`), relative importance of effects and simulation from the fit; Chapter 10. After a fit with `groups()` only `estat score` and `estat timetest` are available (the latter adds a test per group and per period; Section 11.7).

**nwsaom bayes**
: Bayesian estimation after a fit; Section 9.9.

**nwsaom describe**
: Description of the waves before a fit; Section 10.2.

**nwsaom meta**
: Combines the same model fitted to several networks, the counterpart of RSiena's `siena08`; Section 10.8 and `help nwsaom_meta`. The joint estimation of the groups is the option `groups()`.

**nwsaom version**
: Displays the installed release of `nwsaom`, the version of the `nwcommands` core, the native plugin in use, the version of RSiena that the results were compared with and the license. It returns `r(version)`, `r(date)`, `r(core)` and `r(license)`; after estimation by the method of moments, `e(version)` holds the release that produced the results. Quote it when reporting an analysis. `nwversion` lists the core and all installed model packages. A specific release is installed from its tag: `net install nwsaom, from(https://raw.githubusercontent.com/thomas-u-grund/nwsaom/v1.5.1/) replace`.

## 4.14 Combinations that are not available

The following list collects the restrictions that the program enforces and that the help file states; a combination that is not allowed stops with an error message and never gives a silently different model.

- `wave1()`/`wave2()` together with `waves()`; `outdegree` together with `outdegreeendow`/`outdegreecreation`, or any plain effect together with its own endowment/creation split.
- `structural()` (deprecated, converted to the codes 10 and 11): no longer restricted; see Section 13.2. The codes 10 and 11: not yet with `groups()`, `ml`, `nwsaom multinet`; the postestimation commands know them (Section 13.2).
- `ratecov()`: not with co-evolution; with `symmetric` not together with `present()`, `joinat()` or `leaveat()`, and not with co-evolution.
- `tvcov()`: no missing values; not with `interact()`, `symmetric` or `estat gof`.
- `groups()`: not with `ml`, `nwsaom bayes`, two-mode networks, `nwsaom multinet`/`multiplex`, several behaviors, `structural()` or the structural codes 10 and 11, or the time-varying dyadic covariates; `gwaves()` needs `groups()` (Section 11.9).
- Several behaviors: not with `symmetric`, `ratecov()`, `present()`, `missnet()`/`missbeh()`, `structural()`, network endowment/creation, or `estat gof`/`estat mems`.
- `joinat()`/`leaveat()`: not with `behsim` or endowment/creation effects in co-evolution models; not yet with `endow()`/`creation()`; with `symmetric` not with `ratecov()`.
- Networks with tie values other than 1 (valued networks with real shares or counts) and networks carrying `nwset`-style temporal metadata are rejected; a network flagged as valued

whose ties are all 1 is accepted.

# Chapter 5

# The effects

`nwsaom` follows RSiena's effect library. Of the 1201 rows of the effects table of RSiena 1.6.6 (`getEffects()` over twelve data configurations), 1058 are implemented, 1 is partly implemented and the other 142 are not implemented yet (80 of them belong to continuous dependent variables). (The counts refer to RSiena 1.6.6; the effects that RSiena 1.6.7 to 1.6.14 added or changed, for example `WXZX` and the two-mode `toU`, are implemented and compared with RSiena 1.6.14 but are not part of these counts, and the counts change with RSiena releases.) Appendix A lists every implemented effect with the RSiena short name, the network type and the `nwsaom` option that selects it; Chapter 13 lists what is missing.

## 5.1 Naming and the option families

RSiena names an effect by a short name (`transTrip`, `egoX`, `avAlt`) and the covariate or network it interacts with. `nwsaom` selects effects by Stata options, grouped by what the effect needs:

**Structural effects** switches such as `outdegree`, `reciprocity`, `transtrip`, `cycle3`, `indegpopularity`, `outactivity`, `gwesp`; effects with a parameter use `outtrunc(#)`, `gwespfb(#)` etc., and the additional own-network effects of the later versions are collected in `ownx(effect[:#] ...)`.

**Covariate effects** `nodeocov()`/`egox()`, `nodeicov()`/`altx()`, `nodematch()`/`samex()`, `simcov()`/`simx()`, `egosqx()`, `altsqx()`, `absdiffx()`, ..., and `netx(effect:var)` for the further covariate effects. Each takes a variable list. A changing covariate (RSiena `varCovar`) is declared with `tvcov()` and then used by its stem name.

**Dyadic covariates** `dyadcov()`, `dyadcovrecip()`, `dyadx(effect:matrix)`, and with changing matrices `tvdyadcov()`, `tvdyadcovrecip()`, `tvdyadx()`.

**Behavior effects** `behavior(vars : shape effects)` with `beheff()`, `behx()`, `behw()`, `tvbehw()`, `behshape()`, `behnet2()`; and the selection effects of the dependent behavior on the network: `behsim`, `behegox`, `behaltx`, `netbeh()`.

**Several networks** `nwsaom multinet` with `cross1()`/`cross2()` (Chapter 7).

**Rate effects** `ratecov()`, `outrate`, `inrate`, ... (Chapter 8).

Where RSiena offers a parameter (the RSiena "internal effect parameter") the option takes it after a colon, for example `ownx(outinass:2)` or `behnet2(totgwdspffalt:69)`; the defaults are RSiena's.

## 5.2 Statistics, contributions and what "validated" means

For every effect RSiena defines a *contribution* (used when the simulation evaluates a ministep) and a *tie or ego statistic* (used for the target of the method of moments). `nwsaom` implements both from the RSiena source, including the cases in which the two differ, and checks them in two ways: the observed statistic is compared with the target that `siena07` reports for the same data, and the estimates are compared over at least two seeds (the estimates of weakly identified effects agree only within their standard errors). The column "Validated" in Appendix A says `siena07` when this comparison was made. Every implemented effect has been compared this way; for a few effects the comparison rests on the statistic alone, because the estimates are degenerate or RSiena gives no usable result for them (the notes in Appendix A say which). Where RSiena's definition has a feature that a reader might not expect (a tie enters a statistic twice, a covariate is read at another node's index, ...) it is reproduced on purpose (Appendix B).

## 5.3 Effect families added in the later versions

Besides the effects of the single directed network, the library now covers the following families; each is described where its network type is treated.

- **Rate effects** (Chapter 8): the structural rate effects `outrate`, `outrateinv` and `outratelog` for non-directed and two-mode networks; `ratecov()` for two-mode networks, with changing covariates and with the behavior as the covariate; and the rate effects of the behavior in a co-evolution model (`behratecov()`, `behrate()`).
- **Two-mode networks** (Section 7.2): `dyadcov()` with an actors-by-events matrix, `interact()`, and `netbeh(crossxoutact avgroupegox totgroupegox)` for the behavior of the actors.
- **Several networks** (Section 7.3): a two-mode network as one of the two, the 21 two-mode cross effects of RSiena and their counterparts for a one-mode network that depends on a two-mode one, `toany`, `gwespmix`, `betweenpop`, `from.w.ind`, the dyadic-covariate cross effects `toU`, `sharedToU` and `avAltU`, changing covariates in the cross covariate effects, and cross effects for non-directed pairs; with a behavior the behavior effects of both networks, covariate, shape and network effects of the behavior, the effects `netbeh:`, endowment and creation of behavior effects, missing behavior values and interactions of behavior effects.
- **Effects of RSiena 1.6.14**: `toU` (whose two networks were exchanged in 1.6.12), `toU` and `WXX` for two-mode networks, and `WXZX`.

Effects that RSiena cannot evaluate are not offered (Section 13.3.2).

## 5.4 Effects that are native only

Not every effect can be simulated without the compiled plugin; Appendix C lists the effects that also run in Mata. All new effects, in particular those of the families above, exist only in the plugin, which is part of the package. For all of these `nwsaom` stops with an explicit message if the plugin is missing or too old.

# Chapter 6

# Network-behavior co-evolution

A network-only model treats the actors' attributes as fixed. In many applications the attributes change too, and they change *because of* the network: adolescents drink more when their friends drink (influence), and they choose friends who drink like they do (selection). `nwsaom` models both processes at once by adding a second dependent variable, an ordinal *behavior*, that evolves between the same observed waves as the network. This is RSiena's network-behavior co-evolution model [7]. Throughout the chapter the data are the 50 pupils of the Glasgow friendship study that ship with the package (`nwwebuse glasgow`): three friendship waves `glasgow1`–`glasgow3`, an alcohol variable `alcohol1`–`alcohol3` (five categories), smoking `smoke1`–`smoke3` (three categories) and sport `sport1`–`sport3`. These are the same data as RSiena's `s501`–`s503`, `s50a`, `s50s`; the node order differs, the results do not.

## 6.1 The model in brief

A behavior is a bounded integer variable, given as one Stata variable per wave. Between two waves the simulated process has two kinds of ministeps:

- a *network ministep*: an actor reconsiders one outgoing tie, exactly as in Chapter 2;
- a *behavior ministep*: an actor changes its own behavior value by $-1$, 0 or $+1$ (clamped to the observed minimum and maximum). The three alternatives are chosen with the same multinomial-logit rule as a network ministep, now over three alternatives instead of $n$.

One pooled waiting time is drawn from the sum of the network rate and the behavior rate. Which variable acts is chosen in proportion to its share of that total, and then the actor is chosen uniformly. This is RSiena's construction (as in its `EpochSimulation.cpp`). Every period has its own network rate and its own behavior rate.

Co-evolution models are always estimated by *unconditional* Method of Moments, as in RSiena: both kinds of rates are estimated jointly with the effects, and they come with standard errors. The statistic of a network rate is the number of dyads in which the simulated end network differs from the start network; the statistic of a behavior rate is the sum of the absolute changes of the behavior. Statistics that link the two variables are *lagged* as in RSiena: network effects that read the behavior (`behsim`, `behegox`, ...) use the behavior at the *start* of the period, behavior effects that read the network (`avalt`, ...) use the network at the start of the period. The ministep change statistics always use the current simulated state.

## 6.2 A first model

The behavior is declared with `behavior()`, which takes the wave variables in temporal order. At least one baseline behavior effect is required, as `outdegree` is on the network side: `linear` (RSiena's `linear` shape).

```
nwwebuse glasgow, nwclear
```

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behsim ///
    behavior(alcohol1 alcohol2 alcohol3) linear quadratic avalt seed(12345) k3(300)
```

```
SAOM co-evolution (Method of Moments), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2
Behavior: alcohol1 alcohol2 alcohol3
------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|
-------------+----------------------------------------------------------------
   outdegree |   -2.76126   .1590258   -17.36   0.000
 reciprocity |    2.35587   .2176339    10.82   0.000
   transtrip |     .621357   .0943404     6.59   0.000
      behsim |   1.460717   .7549667     1.93   0.053
  beh_linear |   .3948453   .2061589     1.92   0.055
beh_quadra~c |  -.6002813    .339811    -1.77   0.077
   beh_avalt |   1.305018   .8184937     1.59   0.111
------------------------------------------------------------------------------
             |   period1    period2
network_rate |    6.4944     5.1666
behavior_r~e |    1.3126     1.8015
Overall maximum convergence ratio: 0.166 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

(Rate standard errors are omitted here; `e(rates)`, `e(rates_beh)` and `e(tconv)` hold them and the convergence $t$-ratios.) This fit has converged by the two criteria (the overall ratio is 0.166 and all $t$-ratios are below 0.1). The first four coefficients belong to the network side, the last three to the behavior side. Network coefficients keep their plain names; behavior coefficients start with `beh_`. The network effects use the `waves()` and `seed()` options exactly as in Chapter 3.
A comparison with RSiena 1.6.14 on this model, both programs with seed 12345 and the default 1,000 simulations of Phase 3 (estimate, standard error in brackets). With `k3(1000)` `nwsaom` gives the same estimates as the run above with `k3(300)`; only the standard errors, which come from Phase 3, differ:

| effect | RSiena | `nwsaom` |
|---|---:|---:|
| friendship rate, period 1 / 2 | 6.46 / 5.09 | 6.49 / 5.17 |
| outdegree | −2.753 (.148) | −2.761 (.146) |
| reciprocity | 2.354 (.200) | 2.356 (.198) |
| transitive triplets | 0.617 (.081) | 0.621 (.080) |
| drinking similarity (`behsim`) | 1.452 (.604) | 1.461 (.622) |
| drinking rate, period 1 / 2 | 1.33 / 1.75 | 1.31 / 1.80 |
| linear shape | 0.392 (.265) | 0.395 (.214) |
| quadratic shape | −0.581 (.378) | −0.600 (.320) |
| average alter | 1.267 (.902) | 1.305 (.778) |

The quantities returned by the fit that are specific to co-evolution are `e(rates_beh)` and `e(rates_beh_se)` (the behavior rates; with several behaviors one row per behavior), `e(has_behavior)`, and `e(engine)`, which says whether the native plugin or the Mata engine did the simulations.

## 6.3 Shape effects

Shape effects describe the behavior's own distribution, independent of the network. They are the behavior-side analogue of `outdegree`.

| option | RSiena effect | what it does |
|---|---|---|
| `linear` | `linear` | Baseline tendency to higher values. Ministep statistic: the raw change ($\pm 1$); observed statistic: sum of the current values. Required unless `behshape()` replaces the shape effects. |
| `quadratic` | `quad` | Quadratic shape, as in RSiena: the statistic sums the squared centered values $(z - \bar{z})^2$, and the ministep uses its change $(2(z - \bar{z}) + d)\, d$. |
| `behshape(quad_cc)`, `behshape(quad_nc)` | `quad` variants (centred / non-centred) | Alternatives to `quadratic`; use them *instead of* it, never together. |
| `behshape(threshold:#)` | `threshold` | Indicator for values at or above a threshold. |
| `behshape(simallnear:#)`, `behshape(simallfar:#)` | `simAllNear`, `simAllFar` | Similarity to *all* actors within / beyond a distance (defaults 2 and 4). |
| `behshape(avgroup[:#])`, `behshape(totgroup[:#])` | `avGroup`, `totGroup` | Group-average and group-total effects (default parameter 1). |
| `behshape(constant)` | `constant` | Coded but not estimable: its statistic is minus the behavior rate statistic, so the Jacobian is singular. |

Coefficients of `behshape()` effects are named `beh_`*effect*`[(#)]`. The estimates agree with `siena07` on the s50 data (for example `quad_cc`, linear 1.791 / quad −0.448 / avAlt 1.057 against 1.790 / −0.450 / 1.071).

## 6.4 Influence effects

Influence effects make an actor's behavior depend on its network neighbours.

### 6.4.1 The two main effects: avalt and avsim

`avalt` (RSiena's `avAlt`) pulls an actor's value towards the average value of its out-neighbours: the ministep contribution is $d\,(\bar{z}_{\text{alters}} - \bar{z})$ (zero for an actor without out-ties), with $\bar{z}$ the overall behavior mean. `avsim` (RSiena's `avSim`) is the alternative parameterisation: the actor maximises its average *similarity* $1 - |z_i - z_j|/\text{range}$ to its neighbours, net of a data-derived centring constant (the similarity mean, computed from the observed data and never supplied by the user). One RSiena convention is reproduced: this constant is exactly 0 when the pooled behavior has zero variance. The two effects are alternatives; the examples below use them separately.

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    behegox behaltx behavior(alcohol1 alcohol2 alcohol3) linear quadratic avsim ///
    seed(1) k3(300)
```

```
    outdegree |  -2.646096   .1242861   -21.29   0.000
  reciprocity |    2.42636   .2069539    11.72   0.000
    transtrip |    .608684   .0899488     6.77   0.000
      behegox |   .0441969   .0930487     0.47   0.635
      behaltx |  -.0434219   .0867788    -0.50   0.617
   beh_linear |   .3838098   .1792664     2.14   0.032
```

```
beh_quadratic |  -.0537883   .1159717    -0.46   0.643
    beh_avsim |   4.085601   2.393857     1.71   0.088
(output omitted)
Overall maximum convergence ratio: 0.258 (criterion: below 0.25; not met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    outdegree (0.114) transtrip (0.102) beh_linear (0.126) rate_beh_period2 (0.118)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the overall ratio is 0.258 and four $t$-ratios are above 0.1); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains.

### 6.4.2 Endowment and creation roles

Every shape and influence effect has the three RSiena roles: evaluation (the default, direction-blind), endowment (counts only when the value decreases) and creation (counts only when it increases). The split is requested with the pair `quadraticendow`/`quadraticcreation`, `avaltendow`/`avaltcreation` or `avsimendow`/`avsimcreation` (equivalently `endow(avalt) creation(avalt)`); both members must be given and the plain effect must be absent, because all three together are exactly collinear. Splits are weakly identified in general:

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behsim ///
    behavior(alcohol1 alcohol2 alcohol3) linear quadratic avaltendow avaltcreation ///
    seed(1) k3(300)
```

```
 beh_avalt_endow |   -2.19999   9.337483    -0.24   0.814
beh_avalt_creation|   3.119126   12.78062     0.24   0.807
```

Here the fit converges (maximum convergence ratio 0.190, below the 0.25 guideline) but the two roles have standard errors of 9 and 13; the data do not separate them. Such fits can also stop with the `thetaBound` error or a near-singular covariance matrix; both mean "use the plain effect". The split of `linear` (`linearendow`/`linearcreation`) is refused: the sums of increases and decreases add to the behavior-rate statistic, so with an estimated behavior rate the model is not identified (RSiena also reports a singular covariance matrix).

### 6.4.3 Network effects on behavior: beheff, behnet2 and friends

RSiena offers many further behavior effects that depend on the network. All are native-only.

| option | what it adds |
|---|---|
| `beheff(tokens)` | Ego behavior × position or alter summaries. *Position:* `indeg outdeg recipdeg isolate outisolate`. *Alter summaries:* `totalt avaltpop totaltpop maxalt minalt avrecalt totrecalt avinalt totinalt popalt totpopalt`. *Similarity families:* `totsim avatthigher avattlower totatthigher totattlower avsimpopalt totsimpopalt avsimpopego avinsim totinsim avinsimpopalt totinsimpopalt avsimrecip totsimrecip avsimrecpop totsimrecpop`. *Distance two:* `avaltdist2 totaltdist2 avtaltdist2 totaaltdist2 avinaltdist2 totinaltdist2 avtinaltdist2 totainaltdist2`. The tokens are the lower-cased RSiena short names (`totalt` = `totAlt`, `avaltpop` = `avAltPop`, ...). Coefficients `beh_`*token*. |
| `behnet2(effect[:#])` | Further effects: `indegavgroup indegtotgroup avaltcc totaltcc avinaltdist2nc totinaltdist2nc avtinaltdist2nc totainaltdist2nc avinsimdist2 totinsimdist2 totgwdspffalt totgwdspfbalt totgwdspffaltnc totgwdspfbaltnc varalt avsimvaralt behdensetriads simdensetriads`. The number after a colon is RSiena's effect parameter (group effects: centring value, default 1; gwdsp: weight, default 69; dense triads: 5 or 6). |
| `behfrom(varlist)` | Covariate main effect on the behavior (RSiena `effFrom`): behavior tends to be higher where the covariate is higher; contribution is the change times the centred covariate. Constant or `tvcov()` covariates (stem name), first behavior, at most 8; coefficient `beh_from_`*var*. |
| `behavxalt()`, `behavsamexalt()`, `behtotxalt()`, `behtotsamexalt()` | Covariate-moderated influence (RSiena `avXAlt`, `avSameXAlt`, `totXAlt`, `totSameXAlt`). |
| `behx(family:var)` | Behavior effects of the network × a covariate, 32 families (`avxinalt totxinalt avxrecalt totxrecalt avsamexinalt totsamexinalt avsamexrecalt totsamexrecalt avaltsamex totaltsamex avrecaltsamex totrecaltsamex avaltaltx avsimaltx totsimaltx minxalt maxxalt degabscontrx degposcontrx degnegcontrx` and the distance-two and ego families `avxaltdist2 ... totaltegox`). *var* is a numeric variable, a `tvcov()` stem, or `beh2`, `beh3`, ... (another co-evolving behavior, Section 6.6). Coefficients `beh_`*family*`_`*var*. |

| option | what it adds |
|---|---|
| `behw(family: matrix)` | Behavior effects of the network × a dyadic covariate $W$: `avaltw totaltw avinaltw totinaltw avwalt totwalt avwinalt totwinalt avsimw totsimw`. *matrix* is a Stata $n \times n$ matrix, centred like `dyadcov()`; at most 8 matrices. `behwparam(1\|2)` chooses RSiena's parameter of `avAltW`, `avInAltW`, `avSimW` (2, the default, divides by the sum of the weights; 1 by the number of alters; names then carry `_p1`). |
| `tvbehw(family: stem)` | The same ten effects with a changing dyadic covariate; matrices *stem*1 … *stem*($W-1$), one per period. May be mixed with `behw()`. |

For all of these the observed statistics equal `siena07`'s targets and the estimates agree within simulation noise, with a handful of exceptions that are unstable or degenerate in `siena07` as well (for example `avAltW` and `avSimW` with parameter 2 when the weights nearly sum to zero). Combining many weakly identified effects makes the standard errors explode; the next fit is shown only to illustrate the syntax:

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behsim ///
    behavior(alcohol1 alcohol2 alcohol3) linear quadratic avalt ///
    beheff(totalt indeg) behfrom(sport1) seed(1) k3(300)
```

```
   beh_linear |   .0709788   .4118153     0.17   0.863
beh_quadratic |   -.550236   .2640077    -2.08   0.037
    beh_avalt |   .1543022   2.264936     0.07   0.946
   beh_totalt |   .3680405   .7861524     0.47   0.640
    beh_indeg |   .1284015   .1684863     0.76   0.446
beh_from_sport1|   .2422216   .4565276     0.53   0.596
(output omitted)
Overall maximum convergence ratio: 0.237 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

(`avalt` and `totalt` are nearly collinear in these data; the standard error of `beh_avalt` grows from 0.82 in the first model to 2.3.) This fit has converged by the two criteria (the overall ratio is 0.237).

An example with a dyadic covariate: the first wave of the friendship network used as the weight matrix $W$.

```
nwtomata glasgow1, mat(M1)
mata: st_matrix("DM", M1)
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity ///
    behavior(alcohol1 alcohol2 alcohol3 : linear) behw(avsimw:DM) seed(1) k3(300)
```

```
    outdegree |   -2.35188   .1182981   -19.88   0.000
  reciprocity |   2.813118   .1978835    14.22   0.000
   beh_linear |   .3589825   .1613146     2.23   0.026
beh_avsimw_DM |   3.454459   1.390661     2.48   0.013
```

(This is only a demonstration of the syntax: using the network itself as a dyadic covariate is not a sensible substantive model.)

## 6.5 Selection effects: the behavior shapes the network

Selection effects are *network* effects whose covariate is the dependent behavior. Unlike `simcov()` and `nodeocov()`, which read a fixed covariate, they read the behavior as it changes during the simulation. In the ministeps they use the live value; in the observed and simulated statistics the start-of-period value (lagging, see above). Their coefficients are listed among the network coefficients, after the other network effects.

| option | RSiena effect | what it does |
|---|---|---|
| `behsim` | `simX` (dependent behavior) | Actors prefer ties to others with similar behavior: tie contribution $1 - \lvert z_i - z_j \rvert$/range − simMean, with range and similarity mean the behavior's own (the same constants `avsim` uses). The centring matches RSiena, so the outdegree coefficient is comparable to RSiena's. |
| `behegox` | `egoX` | Sender's current behavior minus the overall behavior mean ("alcohol ego"). |
| `behaltx` | `altX` | Receiver's behavior minus the mean ("alcohol alter"). |
| `behegoplusalt` | `egoPlusAltX` | Sum of both. |
| `netbeh(effects)` | the covariate effects with the dependent behavior as covariate | Any of `samex diffx higher egosqx altsqx samexrecip absdiffx egoxaltx samextranstrip diffxtranstrip homxtranstrip samexoutact diffxoutact samexreciact diffxreciact homxoutact altxoutact inpopx outpopx inactx outactx samexinpop diffxinpop egoplusaltsqx diffsqx egodiffx althigheregox unequalx egolthresholdx egorthresholdx altlthresholdx altrthresholdx degabsdiffx degposdiffx degnegdiffx`, plus the similarity versions `simdist2 simegodist2 simegoindist2`. The behavior is centred by its overall mean in the simulation and taken at the start of the period in the statistic. Coefficients `bv_`*effect*. |
| `netx(effect:var)` with `netbeh()` | distance-two and covariate-closure effects | The `netx()` effects of Chapter 5 also work with the behavior as covariate when given via `netbeh()` (all except `homxoutact2`). |

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    netbeh(samex egosqx) behavior(alcohol1 alcohol2 alcohol3) linear quadratic avalt
    ///
    seed(1) k3(300)
```

```
    outdegree |  -3.014934   .2136443   -14.11   0.000
  reciprocity |   2.392201   .2273052    10.52   0.000
    transtrip |   .6092567   .0887855     6.86   0.000
     bv_samex |   .6691935   .3507814     1.91   0.056
    bv_egosqx |   .1210067   .0791059     1.53   0.126
   beh_linear |   .4043015   .2666101     1.52   0.129
beh_quadratic |  -.5861862   .3763233    -1.56   0.119
```

```
    beh_avalt |   1.310731   .9561009     1.37   0.170
(output omitted)
Overall maximum convergence ratio: 0.218 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    beh_linear (-0.138)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the overall ratio is 0.218, but one $t$-ratio is above 0.1); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains.
According to the validation runs, 32 `netbeh()` estimates agree with `siena07` on the s50 data; `degNegDiffX` is degenerate in both programs.

## 6.6 Several behaviors

Any number of behaviors can co-evolve with the network. In `behavior()` the behaviors are separated by `|`, and each can carry its own effect list after a colon; without a colon the global effect options apply to every behavior (one behavior with a colon is identical to the global form).

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    behavior(alcohol1 alcohol2 alcohol3 : linear quadratic avalt behsim | ///
             smoke1 smoke2 smoke3 : linear quadratic avalt) seed(1) k3(300)
```

```
SAOM co-evolution of the network and 2 behaviors (Method of Moments)
            outdegree |  -2.754282   .2418467   -11.39   0.000
          reciprocity |   2.360833   .2803256     8.42   0.000
            transtrip |   .6139462   .0867132     7.08   0.000
               behsim |   1.442482   .8507705     1.70   0.090
   beh_alcohol_linear |   .3899792   .2499822     1.56   0.119
beh_alcohol_quadratic |  -.5778837   .3922876    -1.47   0.141
    beh_alcohol_avalt |   1.258185   1.327251     0.95   0.343
     beh_smoke_linear |  -1.892423   .9046728    -2.09   0.036
  beh_smoke_quadratic |   1.984255   1.363699     1.46   0.146
      beh_smoke_avalt |    1.77957   1.008014     1.77   0.077
(output omitted)
Overall maximum convergence ratio: 0.268 (criterion: below 0.25; not met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for behsim
    (-0.100) beh_alcohol_linear (-0.105) rate_smoke_period1 (0.142)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the overall ratio is 0.268 and three $t$-ratios are above 0.1); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains. `behsim` is written inside the group of the *first* behavior. (A global `behsim` given outside the groups together with several behaviors was accepted without error but produced no `behsim` coefficient and the same estimates as the model without it, so always put it inside the first group.)
Coefficients are named `beh_`*name*`_`*effect*, with *name* the first variable without trailing digits. `e(rates_beh)` has one row per behavior (values of the model above):

```
                period1    period2
rate_alcohol  1.3233     1.7980
  rate_smoke  3.0471     2.7616
```

Validation against RSiena 1.6.6 (alcohol and smoking, `s501`–`s503`) gives, for example, alcohol linear 0.42 (RSiena 0.43), smoking linear −1.91 to −1.94 (−1.92 to −1.94), smoking avAlt 1.92–1.97 (2.01–2.06), with the network and behavior rates agreeing.
The behaviors interact through the network, and in one further way: another behavior can be the *covariate* of the behavior effects that take one, by writing `beh2`, `beh3`, ... in place of a variable name in `behfrom()`, `behx()`, `behavxalt()` and relatives (the statistics read its start-of-period values, the ministeps its current ones):

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    behavior(alcohol1 alcohol2 alcohol3 : linear quadratic avalt | ///
             smoke1 smoke2 smoke3 : linear quadratic avalt) behfrom(beh2) seed(1) k3
    (300)
```

```
beh_alcohol_from_beh2 |  -.6804636   1.111647    -0.61   0.540
(output omitted)
Overall maximum convergence ratio: 0.249 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

(smoking as a main effect on drinking; not significant in these data.) This fit has converged by the two criteria (the overall ratio is 0.249).

## 6.7 The rate of the behavior

The rate of the behavior can depend on actor covariates, on the behavior itself and on the network position and exposure of the actor: `behratecov()` (RSiena's `RateX` for the behavior), `behrate()` (`outrate inrate reciprate avexposure totexposure susceptavin infectin infectout ...`) and `behratecoef()` for starting values. The options, a complete example and the stored results are in Section 8.4.

## 6.8 Estimation by maximum likelihood and Bayes

A fit with one co-evolving behavior can be re-estimated by maximum likelihood (`ml`) or sampled with `nwsaom bayes`; the ministeps of the behavior (the actor changes the behavior by −1, 0 or +1) and their rates are then part of the augmented data. On a Glasgow co-evolution model the maximum likelihood estimates are within 0.1 standard errors of RSiena's. See Chapter 9.

## 6.9 Co-evolution with other features

- **Non-directed networks.** A symmetric network can co-evolve with a behavior; see Chapter 7. It requires the native plugin with the threaded batch path.
- **Two-mode networks.** Actors of a two-mode network may carry a behavior; see Chapter 7.
- **Composition change.** `joinat()`/`leaveat()` work with one or several behaviors (not with `behsim` or endowment/creation effects), Chapter 8.
- **Time-varying covariates.** `tvcov()` works with `behavior()` for the network covariate effects; `behfrom()`, `behx()` and `behavxalt()` accept a `tvcov()` stem.
- **Starting values.** `behtheta0()` gives starting values for the behavior-side parameters.

## 6.10 Estimation notes

- With few actors, `avalt` can make the joint Robbins–Monro fit diverge: a stronger pull makes the ministep more deterministic, which makes the pull look stronger. This is a

small-sample identification problem (the Jacobian is well conditioned and the simulator unbiased at the true parameter), resolved by using more actors or more behavior activity. A divergence stops with an explicit `thetaBound` error (return code 498), as in RSiena.
- Two of the examples above have a maximum convergence ratio between 0.25 and 0.27; RSiena's guideline is 0.25. Rerun with another `seed()` or a larger `k3()` when a fit is above the limit and the reported $t$-ratios of individual parameters exceed 0.1.
- Random or weakly varying behaviors are usually not identified; the test suite therefore accepts return codes 0 and 498 for synthetic co-evolution data.

## 6.11 Limits

The following are things RSiena can do (or that this chapter would naturally want) and `nwsaom` cannot.

- **Behavior type.** Behaviors are bounded integers, one Stata variable per wave. There is no continuous behavior.
- **Linear endowment/creation is refused** (not identified with an estimated behavior rate; RSiena also yields a singular covariance matrix). Network `outdegreeendow` with `outdegreecreation` is refused too, with or without a behavior. The other endow/creation splits of behavior effects are allowed but weakly identified.
- **Several behaviors.** The following are not combinable with several behaviors: `symmetric`, `present()` (use `joinat()`/`leaveat()` with integer times), `ratecov()`, `missnet()`, `structural()`, network `endow()`/`creation()`, and `estat gof`/`estat mems`. `behsim` refers to the first behavior only (`behsim` inside the group of the second behavior is rejected with return code 198; a global `behsim` with several behaviors is silently ignored, see Section 6.6). The same first-behavior restriction holds for `behegox`, `behaltx`, `behegoplusalt` and `netbeh()`.
- **Cross-behavior effects** exist only through the covariate slot (`beh2` in `behfrom()`, `behx()`, `behavxalt()` ...). No other effects between two dependent behaviors are offered.
- **`ratecov()` with `behavior()`** is refused for the rate of the network ("ratecov() is not yet supported for co-evolution models"), and so are the structural rate effects of the network. The rate of the behavior can depend on covariates and on the network through `behratecov()` and `behrate()` (Section 6.7).
- **`missbeh()` with several behaviors** is not available in the form the design note describes: a call `missbeh(a1 a2 a3 | b1 b2 b3)` is rejected. Note also that `missbeh()` variables are 0/1 flags, not Stata missing values.
- **Native plugin only.** Almost everything on this page after `avsim` (`beheff()` tokens beyond the first 20, `behnet2()`, `behx()`, `behw()`, `tvbehw()`, `netbeh()`, `behshape()`, `behavxalt()` and relatives) needs the C plugin. The Linux and Windows plugins can lag behind the macOS one.
- **Degenerate or unestimable effects.** `behshape(constant)` cannot be estimated (it is confounded with the behavior rate, and RSiena reports a standard error of NA). The five double-degree behavior effects `ffdeg bbdeg fbdeg frdeg brdeg` together are not identified, in RSiena as well. For a number of effects `siena07` itself is unstable or degenerate (`beheff()`: `outisolate totaltpop maxalt avaltpop avattlower totattlower totsim`; `behw()` with parameter 2; some `netbeh()` effects), so no reference estimate exists. In a non-directed network `degabsdiffx` and `degposdiffx` with the

behavior as covariate agree with `siena07` in their statistics but are badly determined in both programs, and `higher` with the behavior as covariate equals the density statistic and is not identified (`siena07` gives a standard error of NA, `nwsaom` stops).

- **Symmetric co-evolution** is only partly validated against RSiena, because RSiena's own fit of the model is unstable on the test data. The agreement is by simulation from known parameters.
- **Behavior inside `nwsaom multinet`** (several networks) is available with limits; see Section 7.3.9.
- **Inference.** The default is the method of moments. Maximum likelihood (`ml`) and Bayesian estimation (`nwsaom bayes`) are available for a network with one co-evolving behavior (Sections 9.8 and 9.9), but not for several behaviors, non-directed or two-mode networks, missing data, composition change or rate covariates, and not together with `groups()`. Several groups with one or several co-evolving behaviors are estimated by the method of moments (Section 11.6). Continuous (stochastic differential equation) behavior is not yet implemented.

# Chapter 7

# Non-directed, two-mode and several networks

The default model of the previous chapters is a single *directed, one-mode, binary* network: an actor controls its own outgoing ties. This chapter covers the three other network situations that RSiena supports and that `nwsaom` implements: non-directed relations, two-mode (affiliation) networks, and several dependent networks that evolve together. Valued (weighted) networks and signed networks are not supported: a valued wave is rejected with an explicit error and never silently coerced.

## 7.1 Non-directed (symmetric) networks

In a non-directed relation $x_{ij} = x_{ji}$ always holds. A tie change therefore involves two actors, and RSiena has several *model types* for who decides.

### 7.1.1 When the symmetric model is used

Like RSiena, `nwsaom` models a relation as non-directed automatically when all waves are declared undirected (`nwset, ... undirected`, or `nwsym`) or when they are directed networks that are all tie-symmetric. The option `symmetric` requests it explicitly and is an error for directed waves that are not tie-symmetric: `nwsaom` never symmetrizes data for you. Waves that mix undirected networks with non-symmetric directed ones are refused. A tie-symmetric directed network stays directed (with a note) when `reciprocity` is requested or when the non-directed model does not apply (network `outdegreeendow`/`reciprocityendow` effects).
For the examples we symmetrise the Glasgow friendship waves (a tie exists if either pupil named the other) and declare them undirected:

```
nwwebuse glasgow, nwclear
forvalues w = 1/3 {
    nwtomata glasgow`w', mat(G`w')
    mata: st_matrix("__S`w'", ((G`w' + G`w'') :> 0))
    nwset, mat(__S`w') undirected name(sg`w') nooutput
}
nwsaom, waves(sg1 sg2 sg3) outdegree gwesp(0.69) nodematch(smoke1) seed(1) k3(300)
```

```
SAOM (Method of Moments, conditional), waves: sg1 sg2 sg3
Actors: 50                              Periods: 2
       outdegree |  -2.358639   .1837347   -12.84   0.000
nodematch_smoke1 |   .2021306   .1783414     1.13   0.257
      gwesp_0.69 |   1.607131   .1672707     9.61   0.000
        rate     |    2.4764     2.0350
```

No `symmetric` was needed. The engine is native (`e(engine)` is `native`); symmetric models require 100% native term coverage.

### 7.1.2 Model types: symtype()

`symtype()` chooses how a tie changes. It takes the name or RSiena's `modelType` number.

| `symtype()` | `modelType` | RSiena code | mechanism |
|---|---|---|---|
| `forcing` (default) | 2 | AFORCE | An actor, at the per-actor rate, chooses one tie to create or end (or none) exactly as in the directed model and imposes the change. |
| `confirmation` | 3 | AAGREE | As forcing, but a new tie is only made if the alter confirms it, with the logistic probability of the alter's utility of the new tie. Ending a tie is unilateral. |
| `force` | 4 | BFORCE | Pairwise: a random pair meets; the tie changes if the *initiator* wants it. |
| `agree` | 5 | BAGREE | Pairwise: a new tie needs both to want it; either can end a tie. The alter probability is $\sigma(-\lvert u\rvert)$ for utility $u$, exactly as RSiena. |
| `joint` | 6 | BJOINT | Pairwise: the tie changes if the *summed* utilities of the two favour the change. |

Before 2026-10-01 the default was `joint` and types 2 and 3 did not exist; old scripts that relied on the default now fit a different model. The five types on the same data (conditional estimation, density and `nodematch(smoke1)`, `seed(1)`):

| `symtype()` | outdegree | nodematch(smoke1) | rate, period 1 | rate, period 2 |
|---|---|---|---|---|
| forcing | −1.561 | 0.234 | 1.998 | 1.748 |
| confirmation | −0.883 | 0.165 | 4.136 | 3.600 |
| force | −2.894 | 0.390 | 0.320 | 0.275 |
| agree | −1.307 | 0.243 | 0.366 | 0.314 |
| joint | −1.450 | 0.199 | 0.320 | 0.273 |

The coefficients are not comparable across model types, because a different mechanism generates the same data. The substantive conclusion about `nodematch` should be checked under more than one type when the choice is not obvious from theory.

**Scale of the rate.** For `forcing` and `confirmation` the rate is the per-actor rate, as in the directed model. For the three pairwise types, RSiena gives each actor the basic rate $\lambda$ and each pair the rate $\lambda^2$, while `nwsaom` simulates a pairwise ministep as an actor with opportunity rate $\rho$ picking an alter among the other $n-1$ actors. The processes are identical with $\lambda^2 = \rho/(n-1)$, and `e(rate)` is reported on RSiena's scale (`e(rate_actor)` keeps $\rho$). In unconditional estimation `e(rate)` $= \lambda = \sqrt{\rho/(n-1)}$; in conditional estimation it is the mean time to reach the observed distance at basic rate 1, which is on the scale of $\lambda^2$ (on these data 0.32 and 0.57 for `joint`; $0.57^2 = 0.32$). `rate0()` is on RSiena's scale too. RSiena's default start for the rate of the pairwise types is on the per-actor scale (5.6 on the Glasgow data) and, from it, unconditional estimation often fails; the references in the help file start at 0.5, so give `rate0(0.5)` if an unconditional pairwise fit stops with "Unlikely to terminate".

### 7.1.3 Effects in a non-directed network

Several directed-network effects are meaningless when every tie is symmetric and are rejected (`reciprocity` under `symmetric` stops with "reciprocity is not meaningful under symmetric", return code 198): `reciprocity`, `cycle3`, `inactivity`, `outpopularity`, `ininass`, `inoutass`, `outoutass`, `antiiso`, `isolatepop`, `transrectrip` and `transtrip`. These are constants, exact duplicates of another effect, or effects RSiena does not offer for a non-directed variable. Available: `outdegree`, `indegpopularity`, `outactivity`, `cycle4`, `isolatenet`, `outiso`, `antiiniso`, `antiiniso2`, `inplus3`, `outinass`, `gwesp()`, `transties`, `balance`, and every covariate effect (`nodecov()`, `nodeicov()`, `nodeocov()`, `nodematch()`, `simcov()` and their RSiena-spelling aliases). In addition the following exist for non-directed relations: `degplus`, `degplusc` and `transtriads`; the `ownx()` and `netx()` families, including the primary-setting effects; dyadic covariates and the `dyadx()` effects; the behavior-as-covariate effects of Chapter 6; and time-varying covariates (`tvcov()` with `symmetric` ran in a test for this manual, although the `tvcov()` entry of the help file lists `symmetric` among the exclusions).

### 7.1.4 Combinations

`waves()`, `present()`, `missnet()` and `ratecov()` (network-only) combine with `symmetric`. A symmetric network can co-evolve with a behavior (native plugin with the batch path required):

```
nwsaom, waves(sg1 sg2 sg3) outdegree behavior(alcohol1 alcohol2 alcohol3) ///
    linear avalt symtype(joint) seed(1) k3(300)
```

```
   outdegree |  -1.327199   .0831625   -15.96   0.000
  beh_linear |   .3430058   .1375989     2.49   0.013
   beh_avalt |   .2599666   .2286222     1.14   0.255
network_rate |    0.5688     0.5253
```

(maximum convergence ratio 0.165).

### 7.1.5 Validation

For `forcing`, glasgow waves 1–2 symmetrised, density and density plus `nodematch`, conditional and unconditional, five seeds each: every estimate, including the rate, lies within 0.06 RSiena standard errors of RSiena 1.6.6. For the pairwise types (density, `nodematch`, `ratecov`) all estimates including the rates agree within 0.1 standard errors (mean 0.04), except one conditional covariate-rate coefficient (`force`, 0.13). The coevolution of a symmetric network with a behavior could be checked against RSiena only partly, because RSiena's own estimation of that model is unstable (three of four seeds stop at `thetaBound`); validation then relies on a simulation study in which RSiena simulated 40 data sets from known parameters and `nwsaom` re-estimated them (network rate 0.612 for a true 0.6, behavior rate 1.60 for 1.5, outdegree $-1.44$ for $-1.4$, linear median 0.37 for 0.3, avAlt median 1.29 for 1.5, a few replicates blowing up on which RSiena fails too).

### 7.1.6 Limits

- **Several behaviors** with a non-directed network are refused (return code 198).
- **Mata engine.** Symmetric co-evolution needs the native batch path; there is no Mata fallback for it.
- **Co-evolution with a symmetric network** is only partly cross-validated against RSiena (see above); the avAlt coefficient and the rate scale of that combination are validated by simulation, not by an RSiena fit.

- **The confirmation step for the no-change option.** RSiena's `confirmation` model also computes the confirmation step for the no-change option, which changes nothing but adds to the scores behind the derivative estimate; `nwsaom` leaves it out.
- **No automatic symmetrisation.** Data are never symmetrised by the estimator.
- **Rejected effects** listed above; `degplus`, `degplusc` and `transtriads` are only available for non-directed relations, `outpopc` and `inactc` only for directed ones.
- **Structural zeros and ones** (the codes 10 and 11 in the wave networks, and `structural()`) are combinable with `symmetric` when the codes are given for both directions.
- **Degenerate effects.** `degabsdiffx` and `degposdiffx` with the behavior as covariate in a non-directed network are available and equal `siena07`'s statistics; their estimates have large standard errors in both programs. `higher` with a behavior or a changing covariate equals the density statistic in a non-directed network and is not identified (`siena07` reports a standard error of NA, `nwsaom` stops).

## 7.2 Two-mode networks

A two-mode (bipartite) network relates two kinds of nodes, for example persons and the clubs they join. In RSiena's SAOM only the actors of one mode act: an actor changes one tie $x_{ij}$ to an *event* $j$ of the other mode, or does nothing; events never act.

### 7.2.1 Setup

The network is declared with `nwset, mat(`$A$`) bipartite`. By default the *rows* of the affiliation matrix are the actors (stored by nwcommands as mode "2"); `actormode(1|2)` swaps the roles. Covariates are variables on the *node* rows of the current dataset (one row per node after `nwset`); the variable `_nwmode` tells which mode a row belongs to, and a covariate is read only on the rows of its own mode, centred within that mode. A fully runnable data set (simulated; 40 actors by 20 events, three waves) is:

```
clear
nwclear
mata:
mata set matastrict off
void tm_data() {
    na = 40
    ne = 20
    rseed(2026)
    pop = 0.04 :+ 0.25 * runiform(1, ne):^2
    X = (runiform(na, ne) :< pop)
    for (w=1; w<=3; w++) {
        st_matrix("__T" + strofreal(w), X)
        drop = (runiform(na, ne) :< 0.25) :* X
        add  = (runiform(na, ne) :< 0.5 * pop) :* (1 :- X)
        X = X - drop + add
    }
    st_matrix("__CA", rnormal(na, 1, 0, 1))
    st_matrix("__CB", rnormal(ne, 1, 0, 1))
}
tm_data()
mata: CAm = st_matrix("__CA"); CBm = st_matrix("__CB")
end
```

```
forvalues w = 1/3 {
    nwset, mat(__T‘w’) bipartite name(tm‘w’) nooutput
}
gen double age = .
gen double size = .
mata: md = st_sdata(., "_nwmode"); ia = selectindex(md :== "2"); ie = selectindex(md
    :== "1")
mata: st_store(ia, "age", CAm); st_store(ie, "size", CBm)
```

(`age` is an actor covariate, `size` an event covariate.)

### 7.2.2 A model

```
nwsaom, waves(tm1 tm2 tm3) outdegree cycle4 nodeocov(age) nodeicov(size) seed(1) k3
    (300)
```

```
(two-mode network: 40 actors (mode 2) choose among 20 events)
SAOM (Method of Moments, conditional), waves: tm1 tm2 tm3
Two-mode network: 40 actors choose among 20 events
    outdegree |  -.8967505   .1182357    -7.58   0.000
nodeicov_size |  -.1309193   .0747607    -1.75   0.080
 nodeocov_age |   .0138817   .0908917     0.15   0.879
       cycle4 |   .0549947   .0336124     1.64   0.102
        rate  |    2.0916     2.0732
```

`e(twomode)` is 1, and `e(n_actors)` and `e(n_events)` give the mode sizes (40 and 20). The header line “Actors: 60” counts all nodes. With `actormode(1)` the roles swap (20 actors choose among 40 events). In this implementation the network is embedded in a directed one-mode graph on all nodes in which only actor-to-event dyads are free (all others are structural zeros), so the one-mode sampler, plugin and statistics are used unchanged, and the rate is a per-actor rate. Several waves (`waves()`) are supported.

### 7.2.3 Effects available in a two-mode network

| option | RSiena effect | what it does |
|---|---|---|
| `outdegree` | `density` | Baseline. |
| `indegpopularity` | `inPopSqrt` | Events with higher degree attract more actors. |
| `outactivity` | `outAct` | Actors with higher degree make further ties. |
| `cycle4` | `cycle4` | Four-cycles (two actors sharing two events). |
| `outiso`, `antiiniso`, `antiiniso2`, `inplus3` | `outIso`, `antiInIso`, `antiInIso2`, `in3Plus` | Isolate and threshold-degree effects. |
| `gwdspfb()`, `avdeg()`, `outtrunc()`, `outmore()`, `outthreshold()` | `gwdspFB`, `avDeg`, `outTrunc`, `outMore`, `outThreshold` | Further structural effects. |
| `ownx()` ego-degree effects and `outinass` | `outInAss` etc. | Own-network effects without a covariate. |

| option | RSiena effect | what it does |
|---|---|---|
| `nodeocov(var)` | `egoX` | Actor covariate, a value per actor. |
| `nodeicov(var)` | `altX` | Event covariate, a value per event. |
| `egosqx(var)`, `altsqx(var)` | `egoSqX`, `altSqX` | Squared covariate effects, actor side / event side. |
| `netx(`*effect*`:`*var*`)` with *effect* one of `altindist2` `totindist2` `simegoindist2` `sameegoindist2` `crossxoutact` `samexcycle4` `avgroupegox` `totgroupegox` | `altInDist2`, `totInDist2`, `simEgoInDist2`, `sameEgoInDist2`, `crossXOutAct`, `sameXCycle4`, `avGroupEgoX`, `totGroupEgoX` | Covariate effects on the *actor* mode. |
| `netx(sameinxcycle4:`*var*`)`, `netx(homxoutact2:`*var*`)` | `sameInXCycle4`, `homXOutAct2` | Covariate effects on the *event* mode. `homxoutact2` needs `xparam(3)` for RSiena's default. |
| `tvcov()` covariates | `varCovar` | Work in all of these; `avgroupegox`/`totgroupegox` only with `tvcov()`, as in RSiena. |

Coefficients carry the option spelling plus the variable (for example `nodeicov_size`, `egosqx_age`).

### 7.2.4 A co-evolving behavior of the actors

An actor behavior may co-evolve with a two-mode network. The events carry no behavior; internally they are generated as *absent actors* (never acting, but remaining among the alters), and their behavior rows are replaced by the actors' mean, so the mean, the range and the centred shape statistics are those of the actors. The network effects that read the behavior are `behegox` and `netbeh(egosqx altindist2 totindist2 simegoindist2 sameegoindist2 samexcycle4 crossxoutact avgroupegox totgroupegox)`; these nine are the only `netbeh()` effects of a two-mode network (other tokens used to run on dummy values and are now refused). The events mirror the behavior of the actors, as in RSiena, so there must be at least as many actors as events for the last three.

```
gen double z1 = 0
gen double z2 = 0
gen double z3 = 0
mata: st_store(ia, ("z1","z2","z3"), BAm)     // behavior values of the 40 actors only
nwsaom, waves(tm1 tm2 tm3) outdegree behavior(z1 z2 z3) linear behegox seed(1) k3(300)
```

(`BAm` is an actor-by-wave matrix of integer values 0–3 built in the same Mata block as above.)

```
   outdegree |  -.7255881   .0910187    -7.97   0.000
     behegox |    .079329   .0891253     0.89   0.373
  beh_linear |  -.0497332   .2748901    -0.18   0.856
network_rate |    2.0487     2.0639
behavior_rate|    0.3717     0.7549
```

(This listing comes from an earlier version of `nwsaom` and was not rerun for this edition, because the code that builds its data is not shown; the numbers of a run with the current version differ within simulation error.) **Gotcha:** the behavior variable must have non-missing values on the

event rows (set them to 0 as above). The same model with Stata missing values (`.`) on the event rows stopped with the `thetaBound` divergence error (return code 498), whereas filling the rows with 0 converged.

### 7.2.5 Further options in a two-mode network

- `dyadcov()` takes a matrix with one row per actor and one column per event; it is centred by the mean of all its cells (RSiena's bipartite dyadic covariate). `dyadcovrecip()` does not exist in a two-mode network.
- `interact()` is allowed (for example `egox` with `altx`). The interactions of `egoX` with the density and of the density with `altX` are degenerate in RSiena itself.
- The rate may depend on covariates and on the degree: `ratecov()` with constant and changing covariates (the names of `tvcov()`) and with the behavior of the actors as the covariate (this needs `behavior()` and `waves()`), and the structural rate effects `outrate`, `outrateinv` and `outratelog` (Chapter 8). `inRateInv` and `inRateLog` do not exist for two-mode networks, because the receivers of the ties are not the actors.
- A two-mode network can be one of the two networks of `nwsaom multinet` (Section 7.3).
- Speed: the four-cycle statistic is accumulated for the whole row of alternatives in one sweep, which made two-mode fits with `cycle4` 2.5 times as fast as before.

### 7.2.6 Validation

On a simulated $40 \times 20$, three-wave network (simulated from known parameters with RSiena itself, since RSiena ships no bipartite example data) the observed statistics equal RSiena's targets (density 529, cycle4 12990, inPopSqrt 2399.29, outAct 4359, egoX 103.142, altX 71.484) and the estimates agree (density $-2.37$ to $-2.50$ against $-2.42$ to $-2.53$; cycle4 0.094 against 0.09–0.096; inPopSqrt 0.57 against 0.57–0.59; rates 4.49 and 3.03 against 4.52 and 3.05). Covariate effects were validated in the same way on a second simulated data set. A two-mode co-evolution with behavior has the base model and six effects checked against `siena07` (rates 4.31/3.44 against 4.32/3.42).

### 7.2.7 Limits

- **Effects that do not exist.** `reciprocity`, triadic effects, `balance`, `gwesp` (`reciprocity` stops with return code 198). These have no meaning in a bipartite graph; the RSiena effect list for bipartite variables does not contain them either. Likewise `nodecov()` is refused in a two-mode network (return code 198); use `nodeocov()`/`nodeicov()`.
- **Not available in a two-mode network** (return code 198): `endow()`/`creation()`, `missnet()`, a behavior of the events, and a two-mode member of `nwsaom multinet` together with a behavior. `present()` with all actors present was accepted in a test run; the help file does not list composition change for two-mode networks, so treat it as untested.
- **Effects RSiena cannot evaluate either.** `inPop_ego`, `sameXOutAct` and `diffXOutAct` abort or throw in `siena07` in a two-mode network and are not offered; `avGroupEgoX`/`totGroupEgoX` are not offered with a constant covariate (RSiena reports them as not meaningful); `inRateInv` and `inRateLog` stop `siena07`; `altInDist2` estimates diverge in `siena07` too (the statistic equals RSiena's).
- **RSiena definitions reproduced.** `crossXOutAct` reads the actor covariate at the event's index within its mode for the alter, which needs at least as many actors as events;

`avGroupEgoX` averages over the first $m$ actors ($m$ = number of events). Both behave as in RSiena and may surprise.

- **Behavior effects.** With a co-evolving behavior, only `behegox` and the nine `netbeh()` effects above are allowed. Behaviors of events do not exist.
- **Only binary affiliation matrices**, as everywhere in `nwsaom`.
- **Missing values in event rows** of a behavior variable (see the gotcha above).
- **Wire size.** The structural list of a two-mode network is sent to the plugin as an $n^2$ list, which limits the size of very large bipartite networks (a known limit); the rate start values use $n$ = all nodes.
- **Validation base.** No empirical bipartite example exists in RSiena; all validation uses simulated data.

## 7.3 Several networks: nwsaom multinet

Two networks on the same actors can evolve together, each with its own rate, and the evolution of one may depend on the state of the other (RSiena: two dependent network variables). Each network is directed, non-directed or, for one of them, two-mode (Section 7.3.6). This is a separate subcommand with its own syntax.

```
nwsaom multinet, net1(namelist) net2(namelist) [eff1(effects) eff2(effects)
    cross1(effects) cross2(effects) sym1 sym2 actormode(#) tvcov(groups)
    behavior(namelist) beheff1(effects) beheff2(effects) beheff12(effects)
    beheff21(effects) behint(interactions) missbeh(namelist)
    theta0(numlist) fix(namelist) k0(#) k3(#) firstg(#) seed(#)]
```

`net1()` and `net2()` list the waves of the first and second network (same number, at least two; all networks have the same actor set). Estimation is unconditional Method of Moments; every network has a rate per period, estimated jointly with the effects. At every step the network is drawn in proportion to its total rate, then the actor uniformly, then the alter by the network's own objective function (RSiena's `EpochSimulation`). The engine runs in the plugin only.

### 7.3.1 Within-network and cross-network effects

`eff1()` and `eff2()` list the within-network effects (default `outdegree reciprocity`; with `sym1`/`sym2` the default for that network is `outdegree` only): `outdegree reciprocity transtrip cycle3 transmedtrip cycle4 indegpop outact`, and with a variable `egox:`*var*, `altx:`*var*, `samex:`*var*, `simx:`*var*.
`cross1()` lists the effects of network 1 that depend on network 2; `cross2()` the reverse. A cross-network statistic reads the other network at the *start* of the period, the simulation its current state, as RSiena.

| family | options (lower-case RSiena names) |
|---|---|
| Basic cross effects | `crprod crprodrecip crprodmutual from frommutual fromany to toback torecip clxwx clxwx1 clxwx2 mixedinxw` (RSiena `crprod`, `crprodRecip`, `crprodMutual`, `from`, `fromMutual`, `fromAny`, `to`, `toBack`, `toRecip`, `cl.XWX`, `cl.XWX1`, `cl.XWX2`, `mixedInXW`). |

| family | options (lower-case RSiena names) |
|---|---|
| Degree families | `inpopintn inactintn crprodinactintn outpopintn outactintn divoutegointn divinegointn divoutaltintn divinaltintn avdegintn absoutdiffintn both outoutactintn outoutavintn dist2outinactintn outoutdist2actintn outoutdist2avintn ndist2actintn doubleoutact` |
| Closure and mixed triads | `closure cyclosure sharedin wwxee wwxfr wxxfe wxxer xwxer xwxfe jinmix joutmix gwespffmix gwespbbmix gwespbfmix gwespfbmix gwesprrmix inpopoutw sharedto avalt2mtie avalt2mtot doubleinpop` |

A number after a colon is RSiena's effect parameter, for example `from:2`.

### 7.3.2 Cross-network effects with a covariate

Cross effects that involve a covariate of the actors are written *effect*`:`*var*`[:`#`]`, where *var* is a variable in the current dataset and the optional number is RSiena's parameter, for example `cross1(covnetnet:alcohol1)`. The 34 effects are:

```
samewwclosure diffwwclosure jumpwwclosure jumpfrom jumpsharedin samewxclosure
diffwxclosure jumpwxclosure homwxclosure samexwclosure diffxwclosure
covnetnet homcovnetnet contrastcovnetnet alldifcovnetnet covnetnetin
homcovnetnetin contrastcovnetnetin alldifcovnetnetin inpopintnx inactintnx
outpopintnx outactintnx samexinpopintn samexinactintn samexoutpopintn
samexoutactintn altdist2w totdist2w simdist2w altindist2w totindist2w
simegodist2w simegoindist2w
```

(RSiena `sameWWClosure`, `covNetNet`, `inPopIntnX`, `altDist2W`, ...).

**Constants.** The covariate is centred by its mean (RSiena's `coCovar` convention), which matters for the degree-weighted and distance-two families; the similarity versions use $(x - \bar{x})$/range with the similarity mean as the parameter. The cross-degree effects use two network constants, taken from the data over all waves: the average degree of both networks and the average squared in-degree of the other network (as RSiena's `averageInDegree`). No user-supplied constants are needed. Some RSiena definitions are reproduced: `alldifcovnetnetin` is the same function as `contrastcovnetnetin`; `DifferentCovariateOutStar` tests the tie alter$\to k$ while `Same CovariateOutStar` tests $k \to$ alter; the average-sameX versions divide by the number of actors with the ego's covariate value.

### 7.3.3 Example

For the second network we need a relation on the same 50 pupils. We derive an "advice" network from friendship (each friendship tie is kept with probability 0.6, plus a few random additions), fixed across waves so that it changes when friendship does. This is only a demonstration device, not a substantive claim.

```
nwwebuse glasgow, nwclear
mata: rseed(99); keep = (runiform(50, 50) :< 0.6); add = (runiform(50, 50) :< 0.02)
forvalues w = 1/3 {
    nwtomata glasgow`w', mat(G`w')
    mata: B = ((G`w' :* keep) + add) :> 0; B = B - diag(diag(B)); st_matrix("__B`w'", B
    )
    nwset, mat(__B`w') directed name(advice`w') nooutput
```

```
}
nwsaom multinet, net1(glasgow1 glasgow2 glasgow3) net2(advice1 advice2 advice3) ///
    eff1(outdegree reciprocity transtrip egox:alcohol1) eff2(outdegree reciprocity) ///
    cross1(crprod covnetnet:alcohol1) cross2(from) seed(1) k3(300)
```

```
Several co-evolving networks (Method of Moments), 3 waves, 50 actors
            net1_outdegree |  -2.615272   .1180239   -22.16   0.000
          net1_reciprocity |   2.524563   .2301626    10.97   0.000
            net1_transtrip |   .6382761   .0973272     6.56   0.000
        net1_egox_alcohol1 |   .0074951   .0746293     0.10   0.920
             net1_x_crprod |   -.573532   .3979841    -1.44   0.150
net1_xc_covnetnet_alcohol1 |   .2316152   .4222422     0.55   0.583
            net2_outdegree |  -1.876933   .1193721   -15.72   0.000
          net2_reciprocity |   1.049762   .2890008     3.63   0.000
               net2_x_from |   .1985325    .318445     0.62   0.533
Rates (estimated):

---------------------------------
             |  period1   period2
-------------+-------------------
   rate_net1 |    5.703     4.761
   rate_net2 |    1.729     1.739
---------------------------------
Overall maximum convergence ratio: 0.341 (criterion: below 0.25; not met)
```

This fit has not converged by the two criteria (the overall ratio is 0.341); before it is reported it would be restarted from its estimates with `theta0()`, as Chapter 9 explains. Coefficients are named `net1_`/`net2_` plus the effect; cross effects add `x_` (`net1_x_crprod`) or, with a covariate, `xc_`*effect_var*. `e(rates)` has four rows (rate and standard error for each network). On these artificial data `crprod` is not significant, as expected, and because the advice network is a thinned copy of the friendship network a model with `crprod` in *both* directions diverged (return code 498) in a trial run: when one network is nearly a subset of the other, the cross effects are close to unidentified.

### 7.3.4 Non-directed members

`sym1` / `sym2` declares the first / second network non-directed (tie-symmetric or undirected `nwset` waves; the actor's choice changes the tie in both directions, the forcing model, RSiena's `AFORCE`, `symtype` 4 in the plugin). The rate, the distance (two arcs per tie) and the cross statistics are those of the directed engine, and the default within-network effect is `outdegree` only.

```
nwsaom multinet, net1(sg1 sg2 sg3) net2(advice1 advice2 advice3) sym1 ///
    cross1(crprod from) seed(1) k3(300)
```

```
  net1_outdegree |  -1.561466   .1349579   -11.57   0.000
   net1_x_crprod |   .1239251   .4684387     0.26   0.791
     net1_x_from |   .7701941   .2699889     2.85   0.004
  net2_outdegree |  -1.843321   .0950933   -19.38   0.000
net2_reciprocity |   1.120022   .2648287     4.23   0.000
Overall maximum convergence ratio: 0.284 (criterion: below 0.25; not met)
```

This fit has not converged by the two criteria (the overall ratio is 0.284); before it is reported it would be restarted from its estimates with `theta0()`, as Chapter 9 explains. Twenty cross effects were validated against `siena07` for a non-directed first network with a directed second one (for example `crprod from fromany to closure clxwx mixedinxw joutmix outpopintn`

`outactintn avdegintn doubleoutact sharedto inpopoutw`).

**Earlier subcommand.** The older `nwsaom multiplex` (two directed networks, exactly two waves each, `outdegree`, `reciprocity` and the cross effect `crprod`/`crprodb` only) still exists but is deprecated and prints a note; use `nwsaom multinet`, which generalises it and will replace it.

### 7.3.5 Further cross-network effects

Besides the families above, the following cross-network effects exist: `toany`, `betweenpop` and `gwespmix` (equal to `gwespfbmix`); RSiena's `from.w.ind` as `fromwindoo` (the other network for both the agreement and the in-degree weights), `fromwindow` (agreement in the other network, weights from the own) and `fromwindwo` (agreement in the own, weights from the other). With a *dyadic covariate*, written *effect*`:`*matrix*`[:#]` with a Stata matrix, there are `tou`, `sharedtou` and `avaltutie` (RSiena's `toU`, `sharedToU`, `avAltU.2M.tie`), for example `cross1(sharedtou:W:1)`. As in the current RSiena, `tou` counts for the ego $i$ and the alter $j$ the actors $q$ with a tie $i \to q$ in the *other* network and a tie $q \to j$ in the own network, each weighted by the covariate value for $(i, q)$. A covariate that changes between waves is declared with `tvcov(a1 a2 a3)`, one variable per period, and used by its stem in the covariate cross effects: `cross1(samewxclosure:a)`; this does not apply to the three similarity effects.
The cross effects of non-directed pairs are available, too (`toany gwespmix outoutactintn outoutavintn both avalt2mtie fromwind... betweenpop`); a model with a non-directed member was checked against `siena07` with these effects.

### 7.3.6 A two-mode network as one of the two networks

One of the two networks may be two-mode (declared with `nwset, bipartite`; the network type is detected from the data). Its actors are the nodes of the mode `actormode()` (default 2, the rows of the affiliation matrix); they choose among the events, at a rate of their own in every period, together with the one-mode network, which must have the same actors (the same number; at least 12 nodes in all). The effects of the two-mode network itself are `outdegree indegpop outact cycle4` (default `outdegree`), those of the one-mode network the usual ones. The cross-network effects are those of RSiena for such a pair:

**Two-mode network depending on the one-mode network** `inactintn outactintn divoutegointn divinegointn divinaltintn avdegintn outoutactintn outoutavintn dist2outinactintn outoutdist2actintn avalt2mtot to toback torecip sharedto avalt2mtie inpopoutw ndist2actintn toany joutmix gwespfbmix`, and with a covariate of the actors `samewxclosure:`*var*, `diffwxclosure`, `inactintnx`, `outactintnx`, `samexinactintn`, `samexoutactintn` and the distance-two effects `altdist2w totdist2w simdist2w` (these with at most as many events as actors).

**One-mode network depending on the two-mode network** `outpopintn outactintn divoutegointn divoutaltintn avdegintn outoutactintn outoutavintn dist2outinactintn absoutdiffintn from fromany sharedto avalt2mtie inpopoutw ndist2actintn joutmix gwespmix`; with a covariate of the events (a variable that is missing for the actors) `altdist2w totdist2w simdist2w` and, with at least as many events as actors, `outactintnx outpopintnx`.

**With a dyadic covariate** `tou sharedtou avaltutie` (actors by actors for a one-mode other network, actors by events for a two-mode one) and `fromwindoo`.

`avalt2mtie` needs the parameter 1 or 2 (give it as `avalt2mtie:2`). In RSiena some effects index both networks by the same node number, although the alters of a two-mode network are events: `divinaltintn`, `joutmix` and `gwespfbmix` of a two-mode network, and `outoutactintn` and

outoutavintn of a one-mode network that depends on a two-mode one. nwsaom copies this and, as RSiena does, they are available only if the events do not outnumber the actors. The example uses the simulated $40 \times 20$ data of Section 7.2 and a one-mode friendship network among the 40 actors that was generated for the example (the first network is the two-mode one).

```
. nwsaom multinet, net1(tm1 tm2 tm3) net2(fr1 fr2 fr3) eff1(outdegree) eff2(outdegree
    reciprocity) cross1(to) seed(1) k0(100) k3(1000)
Several co-evolving networks (Method of Moments), 3 waves, 40 actors and 20 events (
    network 1 is two-mode)
------------------------------------------------------------------------------
                  | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
------------------+-----------------------------------------------------------
   net1_outdegree |  -.8501347   .1316513    -6.46   0.000    -1.108167   -.5921028
        net1_x_to |   .2510078   .1819668     1.38   0.168    -.1056406    .6076562
   net2_outdegree |  -1.146603   .0953186   -12.03   0.000    -1.333424   -.9597825
 net2_reciprocity |   .3832283   .2964688     1.29   0.196      -.19784    .9642965
------------------------------------------------------------------------------
Rates (estimated):

---------------------------------
             |  period1   period2
-------------+-------------------
   rate_net1 |    2.039     2.068
   rate_net2 |    1.940     1.965
     se_net1 |    0.285     0.281
     se_net2 |    0.269     0.245
---------------------------------
Overall maximum convergence ratio: 0.098
```

(This listing comes from an earlier version of nwsaom and was not rerun for this edition, because the code that builds its data is not shown; the numbers of a run with the current version differ within simulation error.) Nothing in the cross effect to (actors who are tied to the same friends choose the same events) is expected in these artificial data, and the estimate is not significant.

### 7.3.7 Starting values, fixed effects and tests

theta0() gives the starting values of all effects in the order of the coefficient table (net1_*, then net2_*, then the behavior effects), and fix() holds the named effects (the coefficient names, or _all) at those values, as in a single-network fit. After a multinet fit estat score and estat timetest are available (Chapter 10); a fixed cross effect is shown as omitted:

```
. nwsaom multinet, net1(glasgow1 glasgow2 glasgow3) net2(advice1 advice2 advice3) eff1(
    outdegree reciprocity transtrip) eff2(outdegree reciprocity) cross1(crprod) cross2(
    from) theta0(-2.6 2.5 0.6 0 -1.9 1 0) fix(net1_x_crprod) seed(1) k3(300)
  net1_outdegree |  -2.644865   .1151705   -22.96   0.000
net1_reciprocity |    2.41315   .1910847    12.63   0.000
  net1_transtrip |   .6087724   .0837007     7.27   0.000
   net1_x_crprod |           0  (omitted)
  net2_outdegree |  -1.879335    .117141   -16.04   0.000
net2_reciprocity |    1.04886   .2888188     3.63   0.000
     net2_x_from |   .2055701    .346817     0.59   0.553
(output omitted)
Overall maximum convergence ratio: 0.255 (criterion: below 0.25; not met)
```

This fit has not converged by the overall criterion (the ratio is 0.255); before it is reported it would be restarted from its estimates with theta0(), keeping fix(), as Chapter 9 explains. estat gof, estat importance and estat simulate are not available after a multinet fit.

### 7.3.8 Validation

Observed statistics and estimates of the cross effects agree with `siena07` on RSiena's s50 friendship data with a second network built from it: the 13 basic cross effects (for example `crprod` and `mixedInXW` 1.258 against 1.258; rates 6.1/4.8 and 4.36/4.14 against 6.2/4.8 and 4.36/4.16), and, for the 34 covariate effects, all targets equal and estimates within about 0.05 for the similarity effects. The families in the degree and closure rows were validated against `siena07` as well.

### 7.3.9 A co-evolving behavior

`behavior(`*`b1 b2 ...`*`)` adds one behavior variable (one variable per wave, as in plain `nwsaom`) that evolves together with the two networks, with its own rate in every period. `beheff1()` lists the behavior effects of the behavior itself and of the first network on it (`linear quadratic avalt avsim` and the other tokens of the behavior effects of `nwsaom`; the default is `linear quadratic avalt`); `beheff2()` lists behavior effects driven by the *second* network, shown as `beh2_`*effect*.

```
. nwsaom multinet, net1(glasgow1 glasgow2 glasgow3) net2(nb1 nb2 nb3) behavior(smoke1
    smoke2 smoke3) beheff1(linear quadratic avalt) beheff2(avalt) k0(100) k3(1000) seed
    (1)
  net1_outdegree |  -2.357408   .0979547   -24.07   0.000
net1_reciprocity |   2.820887   .1781864    15.83   0.000
  net2_outdegree |    .220287   .1533902     1.44   0.151
net2_reciprocity |   -.444934   .2799107    -1.59   0.112
      beh_linear |  -1.878621   .3780448    -4.97   0.000
   beh_quadratic |   2.006742   .4216608     4.76   0.000
       beh_avalt |   1.774456    1.18047     1.50   0.133
      beh2_avalt |  -.1019861   1.064681    -0.10   0.924
Rates (estimated):

---------------------------------
             |  period1   period2
-------------+-------------------
   rate_net1 |    5.799     4.521
   rate_net2 |    1.893     2.139
    rate_beh |    2.782     2.645
---------------------------------
Overall maximum convergence ratio: 0.209
```

(This listing comes from an earlier version of `nwsaom` and was not rerun for this edition, because the code that builds its data is not shown; the numbers of a run with the current version differ within simulation error.) (`nb1 nb2 nb3` is a second, randomly generated network in this example.) The behavior can also be a covariate of the network effects, with `behsim`, `behegox`, `behaltx` and `behegoplusalt` in `eff1()`/`eff2()`, and of the cross-network effects, written *effect*`:behavior` in `cross1()`/`cross2()`; any of the covariate cross-network effects listed above works this way, for example `cross1(samewwclosure:behavior covnetnet:behavior)`. In the plugin the behavior statistics are the end behavior on the *starting* network of the period, as in RSiena.
All of these were compared with `siena07`: the observed statistics are equal (RSiena counts a reciprocated dyad twice and centres the linear statistic of a behavior, which shifts its target by a constant but changes nothing else), and the coefficients and rates agree within simulation noise (at most 0.06 standard errors in the main models), for shape and `avalt` effects of either network, `behsim`, `behegox`, `behaltx`, and all 34 covariate cross-network effects with the behavior as covariate.

**Behavior effects of both networks.** `beheff12()` and `beheff21()` list effects that use the first network as RSiena's first network and the second as its second (and, for `beheff21()`, the networks exchanged). For the actor $i$ with the centred behavior $z_i$ they add $z_i$ times a count over $i$'s alters: `ffdeg` (alters that $i$ nominates in both networks), `bbdeg` (in-alters in both), `fbdeg` (out-alter in the first, in-alter in the second), `bfdeg` (in-alter in the first, out-alter in the second), `frdeg` and `brdeg` (out- and in-alters of the first network that are reciprocated in the second), `indegmixedpop` and `outdegmixedpop`. A suffix `:2` (`ffdeg:2`) subtracts the degree in the first network (RSiena's parameter 2). Coefficients are named `beh12_`*effect* and `beh21_`*effect*. All five double-degree effects `ffdeg`, `bbdeg`, `fbdeg`, `frdeg`, `brdeg` together are not identified, in RSiena as well.

**Covariate, shape and network effects of the behavior.** In `beheff1()` and `beheff2()` a token *family*`:`*varname* names a covariate effect of the behavior (a numeric variable that is constant over time; at most eight different ones): `from` (the main effect of the covariate), `avxalt`, `avsamexalt`, `totxalt`, `totsamexalt`, and the families of `behx()`. The shape effects `quad_cc quad_nc threshold:# simallnear:# simallfar:# avgroup totgroup` and the further network effects of `behnet2()` are written in the same way. In `beheff1()` the first network drives the effect, in `beheff2()` the second. RSiena's `constant` effect is confounded with the behavior rate (RSiena returns a standard error of NA), so a fit that includes it stops.

**The behavior as a covariate of the network effects.** In `eff1()` and `eff2()`, `netbeh:`*effect* adds the effect of the centred behavior on tie formation in that network, for example `eff2(outdegree reciprocity netbeh:absdiffx netbeh:avgroupegox)`. 54 effects of this kind can be used (`samex diffx higher egosqx altsqx absdiffx egoxaltx samextranstrip ... avgroupegox totgroupegox`); 53 of them gave the same estimates as `siena07`, and `netbeh(indties)` differs from `siena07` and is refused. The threshold effects take their threshold after a colon.

**Endowment, creation, interactions and missing values.** `linearendow quadraticendow avaltendow avsimendow` in `beheff1()` and `beheff2()` replace the evaluation effect by an endowment and a creation effect (coefficients `beh_`*effect*`_endow` and `_creation`). `behint(`*effect1*`#`*effect2*`)` adds the product of two or three behavior effects of the model (RSiena's `includeInteraction` for behavior effects), each written by its coefficient name, for example `behint(beh_quadratic#beh_avalt)`; the components must be `linear`, `quadratic`, `avalt` or `avsim`. `missbeh(m1 m2 ...)` names one 0/1 variable per wave that flags missing behavior values; they are replaced by the actor's previous observation, else the next, else the most frequent value of the wave, and do not enter the statistics.

**A caution on collinear effects.** A model with `avalt`, `avsim` and `linear` together, or with `avalt` and `totalt` of a sparse network, has enormous standard errors in both `nwsaom` and RSiena on the Glasgow data. The estimates of these effects are correlated at $\pm 0.95$ to $\pm 1.00$: with a behavior of three categories and 50 actors, most of whom are in the lowest category, the statistics are nearly linear functions of each other and the data can identify only a combination of the coefficients. Use one influence effect at a time.

### 7.3.10 Limits

- **Exactly two networks.** There is no third dependent network, only `net1()` and `net2()`.
- **One behavior.** Exactly one behavior can be added. A two-mode member together with a behavior is not available (return code 198).

- **Not available inside multinet:** composition change (`present()`, `joinat()`, `leaveat()`), missing ties, `behw()` and `tvbehw()`, `ratecov()`, `endow()`/`creation()` of network effects, `interact()`, `structural()`, `netbeh(indties)`, and `estat gof`, `estat importance` and `estat simulate`.
- **Estimation.** Only `k0()`, `k3()`, `firstg()`, `theta0()`, `fix()` and `seed()` control estimation; conditional estimation does not exist (unconditional only, as in RSiena).
- **Refused or not offered.** `toU` for a one-mode network that depends on a two-mode one is rejected by RSiena; `avAlt.2M.tie` with RSiena's default parameter 3 throws, so `nwsaom` offers the parameters 1 and 2; `outOutDist2AvIntn` has no implementation in RSiena; `sameWXClosure` for a two-mode network with a behavior is not available.
- **Same-actor sets only.** The two networks (or the actors of the two-mode network) must have the same number of nodes.

# Chapter 8

# Rates and composition change

In a SAOM the *rate* says how often actors get an opportunity to change their ties (or their behavior). By default every actor shares one constant rate per period. This chapter describes how to let the rate depend on actor attributes or on the actor's current network position, and how to handle actors who join or leave the study between waves (composition change). Structural zeros and ones, and missing data, are treated in Chapter 13.
All examples use the Glasgow data of Chapter 6 (`nwwebuse glasgow`, three waves, 50 pupils) unless stated otherwise.

## 8.1 Covariate-dependent rates

`ratecov(varlist)` lets node covariates speed some actors up and slow others down: the rate of actor $i$ in a period becomes

$$\lambda_i = \lambda \exp\Big(\sum_k b_k \, x_k[i]\Big),$$

with one coefficient $b_k$ per variable (RSiena's `RateX` effects). A positive coefficient means that actors with higher covariate values get more opportunities to act. The coefficient is estimated jointly with all other parameters by the same Robbins–Monro procedure. By default the covariates are centred by their means (RSiena's `coCovar(centered = TRUE)`); `nocenter` uses raw values, and the means are shown below the table with `detail` and stored in `e(covmeans)`.

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    ratecov(alcohol1) seed(1) k3(300)
```

```
SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
   outdegree |  -2.667728   .1334985   -19.98   0.000
 reciprocity |   2.450087   .1907086    12.85   0.000
   transtrip |   .6133418   .0731855     8.38   0.000
        rate |    6.2694     5.2364
Covariate-rate coefficient (alcohol1):    0.3867 (se 0.1095)
Overall maximum convergence ratio: 0.129 (criterion: below 0.25; met)
```

Pupils who drink more get significantly more opportunities to revise their friendships ($\exp(0.39) \approx 1.5$ per category). The coefficients are returned in the 1×K matrix `e(ratecoefs)` (columns named by the variables), with `e(ratecoefs_se)`, `e(ratecoefs_tratio)` and `e(ratecoefs_fixed)`; with one variable the scalars `e(ratecoef)`, `e(ratecoef_se)`, `e(ratecoef_tratio)` and `e(ratecoef_fixed)` exist as well.

| option | RSiena effect | what it does |
|---|---|---|
| `ratecov(varlist)` | `RateX` | One covariate-rate coefficient per variable. With `waves()` the coefficient is one value shared by all periods (RSiena's `RateX` has no period index); each period keeps its own basic rate. |
| `ratecovcoef(numlist)` | starting values | Starting values for the coefficients, one per variable or one for all (default 0). |
| `nocenter` | `centered = FALSE` | Use raw covariate values in `ratecov()`, `nodecov()`, `egox()` and `altx()`. |

If the data do not identify a coefficient (non-positive derivative estimate and the plain fit diverges), it is held fixed at its starting value, as RSiena does; `e(ratecoef_fixed)` says whether this happened. With `waves()` the rates are as in RSiena's: on `s501`–`s503` with the alcohol covariate, `RateX` 0.355 against RSiena's 0.357, density −2.38 against −2.375, rates 5.7/4.4 against 5.67/4.47. A ratecov effect can be combined with `symmetric`. In a non-directed network the rate of actor $i$ is $\lambda e^{bx_i}$ and the alter is drawn in proportion to the same rates; the conversion to RSiena's scale (Chapter 7) uses all $n$ actors. Example, again with the symmetrised Glasgow waves `sg1`–`sg3`:

```
nwsaom, waves(sg1 sg2 sg3) outdegree gwesp(0.69) ratecov(smoke1) seed(1) k3(300)
```

```
   outdegree |  -2.239343   .1400216   -15.99   0.000
  gwesp_0.69 |   1.613057   .1654806     9.75   0.000
        rate |    2.3533     1.8719
Covariate-rate coefficient (smoke1):    0.5888 (se 0.1651)
```

For the symmetrised glasgow waves, agreement with RSiena holds for `joint` (RateX 0.153 against 0.151, density −1.316 against −1.312, rates 0.307/0.264 against 0.305/0.263) and, for all three pairwise types, agreement within 0.1 standard errors.

## 8.2 Structural rate effects

A rate may also depend on the actor's current network position. The structural rate effects (RSiena's `outRate`, `inRate`, `recipRate` and their `Inv`/`Log` versions) multiply the period rate by $\exp(\sum_k b_k f_k)$, where $f$ is a function of the actor's *current* out-degree, in-degree or number of reciprocated ties $d$ in the simulated network. The rate therefore changes with every tie change in the simulation, which makes the opportunity process depend on the network itself.

| option | RSiena effect | $f(d)$ |
|---|---|---|
| `outrate` | `outRate` | out-degree $d$ |
| `inrate` | `inRate` | in-degree $d$ |
| `reciprate` | `recipRate` | number of reciprocated ties $d$ |
| `outrateinv`, `inrateinv`, `reciprateinv` | `outRateInv`, ... | $1/(d+1)$ of the respective degree |
| `outratelog`, `inratelog`, `recipratelog` | `outRateLog`, ... | $\log(d+1)$ of the respective degree |

Each adds one covariate-rate coefficient to `e(ratecoefs)`, handled like `ratecov()` coefficients. The observed statistic is the sum, over the dyads that differ between the waves, of the degree

function of the tie sender at the start of the period.

```
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip outrate ///
    seed(1) k3(300)
```

```
   outdegree |  -2.530166    .146247   -17.30   0.000
 reciprocity |   2.414198   .2010697    12.01   0.000
   transtrip |   .5727413   .1004406     5.70   0.000
        rate |    3.6811     3.0169
Covariate-rate coefficient (outrate):    0.2127 (se 0.1023)
Overall maximum convergence ratio: 0.151 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    transtrip (0.134)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the $t$-ratio of `transtrip` is 0.134); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains. Pupils with more friends are slightly faster; the effect is just significant at the 5% level ($t = 2.1$). A trial with two structural rate effects at once (`inrateinv` and `outratelog`) stopped with "a simulated period did not reach the observed distance within 1000000 ministeps" (return code 498); structural rate effects are strongly dependent and one at a time is the safe starting point. Validation (s50, conditional estimation, two `siena07` seeds): all nine effects agree, for example `outRate` rates 2.81/2.15 against 2.79/2.10 and coefficient 0.297 against 0.300, `inRate` 0.370 against 0.379/0.377, `outRateInv` $-1.096$ against $-1.12/-1.14$.

## 8.3 Rate effects in non-directed and two-mode networks

The structural rate effects `outrate`, `outrateinv` and `outratelog` are also available for non-directed and two-mode networks, where they are the only degree versions, as in RSiena. In a two-mode network `ratecov()` is available as well, with constant covariates, with changing covariates (the stems declared in `tvcov()` are written inside `ratecov()`) and with the behavior of the actors as the covariate (this needs `behavior()` and `waves()`). The simulated $40 \times 20$ data of Section 7.2:

```
. nwsaom, waves(tm1 tm2 tm3) outdegree cycle4 ratecov(age) outrate seed(1) k0(100) k3
    (1000)
   outdegree |  -.9831879    .140031    -7.02   0.000    -1.257644   -.7087321
      cycle4 |   .0823168   .0345298     2.38   0.017     .0146396    .1499941
Rate parameters (estimated, one per inter-wave period):
        rate |    3.0559     3.3033
          se |    0.4330     0.4677
Covariate-rate coefficient (age):    0.1933 (se 0.1069)
Covariate-rate coefficient (outrate):   -0.1589 (se 0.1262)
Overall maximum convergence ratio: 0.098 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

The effects `inRateInv` and `inRateLog` do not exist for two-mode networks (`siena07` stops, because the receivers of the ties are not the actors), and the pairwise non-directed model types are refused with rate effects and a behavior (`siena07` fails there).

## 8.4 Rate effects of the behavior

In a co-evolution model the rate of the behavior can depend on actor characteristics as the rate of the network does. With `behavior()` and `waves()`, the rate of actor $i$ in a period is the period's behavior rate times $\exp(\sum_k b_k x_k[i])$. `behratecov(varlist)` takes constant covariates, changing covariates declared in `tvcov()` and the name of the behavior itself (its current value; RSiena's `RateX`). `behrate(list)` takes the rate effects of the position in the network and of the exposure to the behavior of the alters: `outrate inrate reciprate` (out-degree, in-degree, reciprocated degree), `totexposure` and `avexposure` (sum and mean of the alters' behavior values), `susceptavin` (mean exposure times the in-degree), `infectin` and `infectout` (sum of the alters' values times their in- or out-degree; `infectdeg` in a non-directed network), `avtinexposuredist2 totinexposuredist2 totainexposuredist2 anyinexposuredist2` (exposure through the in-neighbours of the alters), and `susceptavcovar(`*var*`)` and `infectcovar(`*var*`)` (the same with a covariate of the actors, which may be a changing covariate). A threshold, RSiena's internal effect parameter, is set with *effect*`=#`: with $\# > 0$ the exposure counts only if at least # alters have a behavior value of 0.5 or more, with $\# < 0$ it is also capped at $|\#|$. A non-directed network has `outrate avexposure totexposure infectdeg` and the two covariate effects. `behratecoef(numlist)` gives starting values.

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behavior(
    smoke1 smoke2 smoke3 : linear quadratic avalt) behratecov(alcohol1) behrate(outrate
    ) k0(100) k3(1000) seed(1)
Rate parameters (estimated, one per inter-wave period):
-----------------------------------
             |   period1    period2
-------------+---------------------
network_rate |    6.5112     5.3081
          se |    1.4385     0.9347
behavior_r~e |    1.2519     1.2129
          se |    1.3106     1.0362
-----------------------------------
Rate effect of the behavior (alcohol1):    0.4195 (se 0.4656)
Rate effect of the behavior (outrate):    0.3039 (se 0.2800)
Overall maximum convergence ratio: 0.180 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    rate_beh_period1 (0.105)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the $t$-ratio of the behavior rate of period 1 is 0.105); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains. The coefficients are returned in `e(behratecoefs)`, `e(behratecoefs_se)` and `e(behratecoefs_tratio)`, the observed statistics in `e(rc_target)`. The options cannot be combined with `present()`, `joinat()`, `missnet()`, `missbeh()`, several behaviors, the structural rate effects of the network, a two-mode network or the pairwise non-directed model types, and no `estat` command is available afterwards. They need the native plugin. `RateX` of a continuous behavior is not planned, and `susceptAvCovar` and `infectCovar` with the behavior itself as the covariate do not exist in RSiena.

## 8.5 Composition change

Actors may join or leave the network during the study. RSiena handles this with the "joiners and leavers" method (Huisman and Snijders 2003; Section 5.3.3 of the RSiena manual): the actor set is fixed over the whole study, and each actor is marked as present or absent. `nwsaom` offers two ways.

### 8.5.1 Whole-period presence: present()

`present(varlist)` takes one 0/1 variable per wave, in temporal order (the same convention as `behavior()`): 1 marks an actor as present (eligible to act and to be chosen as an alter) at that wave. An actor can act in an inter-wave *period* if it is present at the wave at which the period starts, as in RSiena's `sienaCompositionChange()` for joins and leaves at an observation. A joiner who first appears at a wave therefore acts from the next period on, and a leaver who is present up to a wave still acts in the period that starts there. This is the common case (a pupil enrolled between survey waves).
Composition change requires *unconditional* estimation in RSiena (manual, Section 7.12.1) and `nwsaom` uses it automatically, even without `unconditional`: rates are estimated with the other parameters, only present actors get opportunities to act, and a rate's score counts only them. The fit runs in the native backend at about the speed of the same model without composition change.
Example. We let pupils 1–5 join after wave 1 and pupils 46–50 leave after wave 2. For the absent waves the ties of absent actors are coded 0:

```
nwwebuse glasgow, nwclear
forvalues w = 1/3 {
    nwtomata glasgow`w', mat(G`w')
}
mata:
for (w=1; w<=3; w++) {
    Z = (w==1 ? G1 : (w==2 ? G2 : G3))
    ab = J(50, 1, 0)
    if (w == 1) ab[1::5] = J(5, 1, 1)
    if (w == 3) ab[46::50] = J(5, 1, 1)
    out = selectindex(ab :== 1)
    Z[out, .] = J(rows(out), 50, 0)
    Z[., out] = J(50, rows(out), 0)
    st_matrix("__C" + strofreal(w), Z)
}
end
forvalues w = 1/3 {
    nwset, mat(__C`w') directed name(cc`w') nooutput
}
gen byte p1 = !inrange(_n, 1, 5)
gen byte p2 = 1
gen byte p3 = !inrange(_n, 46, 50)
nwsaom, waves(cc1 cc2 cc3) outdegree reciprocity transtrip present(p1 p2 p3) ///
    seed(1) k3(300)
```

```
SAOM (Method of Moments, unconditional), waves: cc1 cc2 cc3
   outdegree |   -2.65185   .1311795   -20.22   0.000
 reciprocity |   2.557583   .2301005    11.12   0.000
   transtrip |   .5803097   .1036101     5.60   0.000
        rate |    9.1350     4.2694
Overall maximum convergence ratio: 0.159 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    outdegree (0.120) reciprocity (0.126) transtrip (0.138)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (all three $t$-ratios are above 0.1); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains. Rates are unconditional here and therefore not on the scale of the conditional rates of the earlier examples.

**How to prepare the data.** `nwsaom` does not derive the data of absent actors for you. The ties an absent actor has in the data are used as given and counted in the distance, so code them 0 for the waves in which the actor is absent, as in the example above, where the leavers' ties are coded 0 at the wave they have left. Code consistently: an absent actor's row and column that look like activity the actor can never explain are a real, avoidable way to trigger the `thetaBound` error. An absent actor stays among the possible alters of an active actor, as in RSiena.

### 8.5.2 Joiners and leavers at fractional times: joinat() and leaveat()

RSiena's `sienaCompositionChange()` (option 1) lets an actor join or leave at any time inside a period. `joinat(varname)` and `leaveat(varname)` give each actor's joining and leaving time on RSiena's time scale: wave 1 is time 1 and the last wave is time $W$. An actor is active from `joinat()` to `leaveat()`; a missing value means from the first wave / to the last wave; one interval per actor, and every interval must contain a wave. The rules, from RSiena's source:

- inactive actors have rate 0 but stay among the alters of an active actor;
- a joiner takes on the ties it had in the start network; a leaver loses all its ties;
- a tie made to an inactive actor is removed again at the end of a period that has join or leave events;
- the rate parameter's compensator is the time-integral of the number of active actors.

```
gen double jn = .
gen double lv = .
replace jn = 1.5 in 1/5
replace lv = 2.5 in 46/50
nwsaom, waves(cc1 cc2 cc3) outdegree reciprocity transtrip joinat(jn) leaveat(lv) ///
    seed(1) k3(300)
```

```
SAOM (Method of Moments, unconditional), waves: cc1 cc2 cc3
   outdegree |  -2.666646   .1244786   -21.42   0.000
 reciprocity |   2.540217   .2121802    11.97   0.000
   transtrip |   .5889974   .0746351     7.89   0.000
        rate |    8.3560     5.6453
Overall maximum convergence ratio: 0.232 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    outdegree (0.129) reciprocity (0.170) transtrip (0.175)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (all three $t$-ratios are above 0.1); before it is reported it would be restarted from its estimates with `nwsaom refit`, as Chapter 9 explains. (Pupils 1–5 join in the middle of period 1, pupils 46–50 leave in the middle of period 2; their ties in the data are coded 0 for the waves they are absent. Note that actor 1–5 are marked absent at wave 1 and present at wave 2, so `joinat(1.5)` is consistent.) `present()` is the special case of whole-period presence.

**Validation.** Five single-event scenarios at fixed parameters give the same deviations from the targets as `siena07`; on s50 with 10 joiners/leavers the estimates (density, reciprocity, transitive triplets, both rates, and a rate covariate) lie within noise of RSiena over three seeds. With one or several co-evolving behaviors (via `joinat()`) the rates and shape effects agree with `siena07` as well (network rates 6.46/5.10 against 6.48/5.10, alcohol rates 0.99/1.32 against 0.98/1.32, smoking rates 2.85/2.47 against 2.88/2.49).

### 8.5.3 What combines with composition change

| combined with | `present()` | `joinat()/leaveat()` |
|---|---|---|
| `waves()` | yes | yes |
| one co-evolving behavior | yes | yes (not with `behsim`) |
| several behaviors | no (error 198) | yes (not with `behsim`) |
| `symmetric` | yes | yes (not with `ratecov()`) |
| `ratecov()` | yes | yes |
| `missnet()` | — | — |
| `tvcov()` | — | yes |
| `endow()/creation()` | not stated | no (error 198) |
| structural rate effects | no (error 198) | — |

A dash means the combination is not determined.

## 8.6 Limits

- **Rate effects.** `ratecov()`, the nine structural rate effects of directed networks, the degree versions for non-directed and two-mode networks, and the rate effects of the behavior are implemented (Sections 8.3 and 8.4). There is no period-specific covariate-rate coefficient (as in RSiena, `RateX` has no period index).
- **`ratecov()` with co-evolution** for the network rate of a one-mode network is refused ("ratecov() is not yet supported for co-evolution models"); the rate of the behavior is modelled with `behratecov()`.
- **Structural rate effects** of directed networks are restricted to plain waves: no `behavior()`, `present()`, `missnet()`, `symmetric` (use the non-directed versions above) and no `endow()/creation()`. They run in the plugin only. Combining several structural rate effects can fail (see above).
- **Composition change.**
  - RSiena's composition-change options 2–4 (other treatments of the data of absent actors) and several intervals per actor are not offered: only RSiena option 1 with one interval per actor.
  - The "structural zeros/ones" alternative documented in RSiena's manual is not a substitute here; joiners and leavers was chosen instead.
  - `present()` cannot be used with several behaviors; `joinat()/leaveat()` cannot be used with `behsim` or endowment/creation effects, and not (yet) with `endow()/creation()`.
  - With `symmetric`, `joinat()/leaveat()` cannot be combined with `ratecov()`.
  - Conditional estimation is not possible with composition change.
  - The user must code absent actors' tie data consistently (see above); `nwsaom` neither derives nor checks it.
- **Multinet and two-mode.** `nwsaom multinet` has no composition change and no missing ties; two-mode networks with composition change are untested.

# Chapter 9

# Estimation and diagnostics

This chapter explains what `nwsaom` does when it fits a model, which options steer the algorithm, how to decide whether a fit has converged, what the error messages mean, and which results are available afterwards.

## 9.1 The algorithm in brief

Coefficients are estimated by the method of moments with Robbins–Monro stochastic approximation, which is also the default of RSiena's `siena07`. It is not maximum likelihood and not a Bayesian method; these two estimators are available as the option `ml` and the subcommand `nwsaom bayes` and are described at the end of this chapter (Sections 9.8 and 9.9). The estimation runs in three phases.

**Phase 1** estimates the Jacobian, that is, the sensitivity of each expected statistic to each parameter, from `k0()` independent simulated replicates at the starting values (score-function derivative estimator), and ends with a partial quasi-Newton step. The default length is RSiena's, $\max(50, 7 + 3p)$ replicates for $p$ estimated parameters, so a larger model gets a longer phase 1. Phase 1 is lengthened, and if necessary run with finite differences, when a derivative is not positive (Section 9.2).

**Phase 2** performs the Robbins–Monro updates in RSiena's default of four sub-phases with diminishing gain (`nsub=4`, `firstg()` default 0.2, `reduceg=0.5`, RSiena's per-sub-phase minimum and maximum iteration schedule, `diagonalize=0.2`). Three further rules are RSiena's defaults as well. First, each simulated statistic is corrected by its regression on its own score, with the coefficients estimated from the phase-1 simulations (RSiena's `dolby = TRUE`); a score has expectation zero, so the correction removes noise from the steps without changing what is estimated. Second, a sub-phase whose steps oscillate is run again: when, after at least 50 iterations, the autocorrelation of the deviations of a free parameter falls below $-0.8$, the sub-phase ends and starts again from the current values with the same gain, at most four times. Third, a step that would take a rate below zero sets it to half of its running average. Parameters fixed with `fix()` or at their starting values (Section 9.2) take no part in the autocorrelation rule or in the truncation of large steps.

**Phase 3** simulates `k3()` further replicates at the estimates. They give the covariance matrix `e(V)` (RSiena's sandwich formula), the convergence $t$-ratios, and the overall maximum convergence ratio.

With the native (C) engine, phases 1 and 3 run all their simulations in one call on worker threads. Phase 2 is sequential by construction (each step needs the previous one); only the periods within a step run in parallel, and only when the network is large enough for threads to pay off.

### 9.1.1 Conditional and unconditional estimation

The rate parameters (how often an actor gets the opportunity to change a tie, one per period) are estimated in one of RSiena's two ways, and `nwsaom` chooses RSiena's default for each kind of model.

- **Conditional** (RSiena `cond = TRUE`) is the default for a network-only model. The rates are not Robbins–Monro parameters. Each period is simulated at rate 1 until the distance between the simulated network and the network at the start of the period reaches the observed distance between the period's two waves; the effect statistics are those of the network at that point. A period's rate is the mean of the simulated times over the `k3()` phase-3 replicates, and `e(rate_se)`/`e(rates_se)` are the standard deviations of those times, as in RSiena. There is no rate $t$-ratio under conditional estimation. A `ratecov()` coefficient remains a method-of-moments parameter.
- **Unconditional** (RSiena `cond = FALSE`) is requested with `unconditional`. It is always used for co-evolution models and for `nwsaom multiplex` models (two dependent variables), and for network-only models with composition change. Every rate is then a method-of-moments parameter estimated jointly with the effects, with a standard error from the sandwich formula and a convergence $t$-ratio.

The two estimators are consistent for the same model but differ in finite samples. On RSiena's `s50` data RSiena's own conditional and unconditional estimates differ by up to 0.3 standard errors (outdegree more negative and reciprocity larger under conditional estimation).

## 9.2 Algorithm controls

| Option | Meaning |
|---|---|
| `k0(#)` | Phase-1 replicate count; default $\max(50, 7 + 3p)$ for $p$ estimated parameters, as in RSiena. |
| `k3(#)` | Phase-3 replicate count (diagnostics and covariance matrix); default 1,000. |
| `firstg(#)` | Phase-2 starting gain; default 0.2, as in RSiena. |
| `seed(#)` | Seeds the random-number generator. |
| `theta0(numlist)` | Starting values of the effect parameters, in the order of `e(b)`. |
| `rate0(numlist)` | Starting value(s) of the network rate(s) of an unconditional network-only fit: one value, or one per period. Default is RSiena's closed-form starting value. |
| `unconditional` | Unconditional instead of conditional estimation (network-only models). |
| `cores(#)` | Threads of the native simulator; default 0 = all physical cores, `cores(1)` = single thread. |

**Replicates.** Larger `k0()` stabilises the Jacobian; larger `k3()` reduces the noise in the standard errors and in the convergence $t$-ratios. In tests, `k0()` mattered for sparse effects: for the transitive-reciprocated-triplet endowment/creation split on `s50`, `k0(200)` turned two diverging seeds into converged fits that agree with RSiena. See the next paragraph for the safeguard that now does this automatically.

**Non-positive derivatives.** If the phase-1 derivative of a free parameter, that is, of its own statistic with respect to it, is not positive, the parameter cannot be estimated from these simulations. `nwsaom` then follows RSiena's rule (`CalculateDerivative` in `R/phase1.r`), for network-only and co-evolution fits alike. While phase 1 has fewer than 200 replicates, its length is doubled and phase 1 is run again. If the derivative is still not positive, phase 1 is run once more, at the same

length, with finite differences instead of the score function: each simulation at the current values is repeated with each free parameter increased by a small step $\varepsilon$, with the same random numbers (common random numbers), and the derivative is the mean of the difference quotients. As in RSiena, $\varepsilon$ is 0.1 for an effect and a tenth of the value for a rate; if after the first 10 simulations a step has changed its own statistic in fewer than 5 of them, that step is enlarged (by a factor of 10, or $\sqrt{10}$ when it changed it 3 or 4 times; 3 or 2 for a rate; kept between 0.01 and 10) and phase 1 starts again, at most four times. No score correction is applied in phase 2 after a phase 1 with finite differences, as in RSiena, and phase 3 uses the score function again. The output says when finite differences were used. Only a parameter whose derivative is still not positive after that is kept fixed at its starting value, with standard error 0, and a note names it. A typical case is an effect whose statistic never changes, for example the ego effect of a covariate that is constant within the data. This is loud, not silent: look for the note, because a fixed parameter is not an estimate.

**Seeds and threads.** Every simulation draws from its own random stream derived from the seed and the position of the simulation. A given `seed()` therefore gives identical results with `cores(1)` and with any other number of threads. Results for different seeds differ by simulation noise; when comparing two specifications, fit each with several seeds.

**Starting values and restarts.** `theta0()` takes one value per requested effect, in the order shown by `matrix list e(b)` (outdegree first, then the effects in the order in which their options appear in the help file; if the count is wrong the error message lists the order). Without `theta0()` all parameters start at zero, except that a network-only model starts `outdegree` at RSiena's data-derived value. To continue a fit that has not converged, restart from its estimates with `nwsaom refit` (new in 1.5.12; RSiena's `prevAns`):

```
nwwebuse glasgow, nwclear
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip seed(1)
nwsaom refit, seed(2)
```

`nwsaom refit` reruns the command stored in `e(cmdline)` with `theta0()` and `behtheta0()` set to `e(b)`, `rate0()` to the rates of an unconditional network-only fit and `ratecovcoef()` to `e(ratecoefs)`; `k3()`, `seed()`, `cores()` and `firstg()` given to it replace those of the stored command. `maxruns(#)` repeats the refit until both convergence criteria below are met, at most # times, and `e(refit_runs)` and `e(refit_converged)` report what happened. Since 1.5.13 it also works after `nwsaom multinet`, where `theta0()` gets all coefficients of both networks and the behavior. By hand, the same restart is

```
matrix b = e(b)
nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip ///
    theta0(‘=b[1,1]’ ‘=b[1,2]’ ‘=b[1,3]’) seed(2)
```

and an unconditional fit also passes the rates, from `e(rates)` (or `e(rate)` for the two-wave path), through `rate0()`. A start far from the data, for example twice the closed-form rate, can make the phase-1 derivative estimates unusable and the fit diverge. As in RSiena, a parameter whose absolute value exceeds 50 in phase 2 (`thetaBound`) stops the fit with a message and the error r(498); this holds for every free parameter, the rates and the rate-effect coefficients included, and it stops a co-evolution fit with a behavior rate effect that the data do not support within seconds instead of after hours. A simulated period that needs more than $\max(10^6, 500n)$ ministeps is abandoned in the same way.

**Which engine ran.** If the compiled plugin cannot be used (the plugin file is missing for the platform, it is older than the model needs, or the model exceeds a limit of the native engine),

`nwsaom` falls back to the Mata engine, and says so: one warning line is printed before the estimation starts, and a note follows the table (Section 14.4.3). The results are the same, but the fit is orders of magnitude slower. `e(engine)` (`native` or `mata`) and `e(engine_why)` give the engine and the reason. Many effects exist only natively (Appendix C lists those that also run in Mata); a missing or too-old plugin then gives an explicit error rather than a fallback (Chapter 5).

## 9.3 Judging convergence

After the coefficient table, `nwsaom` prints the rate table and the convergence $t$-ratios. Each $t$-ratio is the phase-3 mean deviation between simulated and target statistic divided by its standard deviation, one per parameter, rates included (under conditional estimation the rates have none). As in RSiena a fit is considered converged when

- every $|t|$ is below 0.1, and
- the overall maximum convergence ratio `e(tconv_max)` is below 0.25.

If either condition fails, run the model again from the estimates with `nwsaom refit`, as shown above. If after one or two restarts the overall ratio stays near 0.25 while the estimates hardly change from one restart to the next, the remaining excess is mostly noise in the phase-3 estimate of the ratio: restart once more with a longer phase 3, for example `k3(3000)`, rather than restarting again and again with the default length. The $t$-ratios of the coefficients in `e(b)` are in `e(tratio)`; all of them, with the rates, are in `e(tconv)`.

A real run on the Glasgow data (RSiena's `s50`, three waves) illustrates the output. All three $t$-ratios are below 0.1 in absolute value, and the overall ratio of 0.123 is below 0.25 (the two criteria are separate, and the detail listing is shown):

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip seed(1)
    detail
SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2
   outdegree |  -2.675306   .1130571   -23.66   0.000
 reciprocity |   2.453519   .1841138    13.33   0.000
   transtrip |   .6151182    .075624     8.13   0.000
Rate parameters (estimated, one per inter-wave period):
        rate |    6.4432     5.2473
          se |    1.0683     0.8841
Convergence t-ratios (criterion: every |t| below 0.1; the overall maximum ratio below
    0.25 is a separate criterion):
  outdegree                -0.005      reciprocity              -0.031
  transtrip                -0.076
Overall maximum convergence ratio:  0.123
```

(Columns trimmed. The digits depend on the seed.) By the two criteria above this fit has converged. Without `detail`, `nwsaom` prints one line per criterion and names the parameters whose $t$-ratio is too large; when a criterion is not met, restarting from the estimates is the next step.

## 9.4 Errors, divergence and what to do

`nwsaom` stops loudly rather than return numbers it cannot support. The return codes below are those the code sets.

**rc 498, "diverged during phase 2"** a coefficient exceeded `thetaBound` (50, RSiena's default) after a Robbins–Monro step. RSiena halts under the identical condition (`R/phase2.r`). It usually signals a genuine identification problem of the model with these data, not a software defect: a

weakly identified or saturated effect, too many effects for the amount of change, or a start too far from the solution. Remedies: a narrower specification, `theta0()` from a simpler fit, a larger `k0()`.

**rc 498, "did not reach the observed distance within 1000000 ministeps"** under conditional estimation a simulated period that does not reach the observed distance within a million ministeps is abandoned (RSiena: "Unlikely to terminate this epoch"). The estimates have probably diverged, for example because a phase-1 derivative was estimated from too few simulations. Try the default `k0()`, other `theta0()`, or a narrower model.

**rc 505, "phase-3 covariance matrix contains missing values"** `theta` stayed within the bound, but the phase-3 Jacobian was too close to singular to invert. This is another symptom of weak identification. Typical causes found in testing are listed below.

**rc 198** the combination of options is refused (for example `outdegreeendow` with `outdegreecreation` and an estimated rate, or an effect that the model type does not offer). The message names the reason; nothing is silently coerced.

**rc 301** Stata's own "last estimates not found": a postestimation command such as `estat gof` was issued without a preceding `nwsaom` fit in memory.

`estat gof` of a non-directed fit also stops with rc 498 if the native simulator is unavailable.

**Known sources of rc 498 and 505.**

- **Sparse effects.** The transitive-reciprocated-triplet endowment/creation split diverged on some seeds in early versions. After the per-period Jacobian (RSiena's `derivativeFromScoresAndDeviations`, summed over periods) and the phase-1 doubling were adopted, 1 of 50 seeds still diverged, compared with 3 of 200 seeds (1.5%) for RSiena's `siena07` on the same model. Divergences of this size are expected for sparse effects in both programs.
- **Collinear effects.** The linear shape effect split into endowment and creation together with the behavior rate, or the outdegree endowment and creation together with the network rate, is not identified, because the two statistics add up to the rate's distance statistic. These combinations are refused (rc 198), as RSiena reports a singular covariance matrix for them. Likewise a constant behavior effect equals minus the behavior-rate statistic, so under unconditional estimation its row is collinear with the behavior rate and the fit ends in rc 505.
- **Covariate effects that reproduce the outdegree.** In a two-mode network, `sameXInPop` or `homXOutAct` with a continuous covariate reproduce the outdegree statistic; `siena07` fits them with arbitrary estimates, `nwsaom` stops with rc 505.
- **Synthetic or random behavior data.** Random behavior is not identifiable; tests on such data accept rc 0 or rc 498.
- **Non-directed co-evolution.** RSiena itself is unstable for small non-directed co-evolution examples (three of four seeds aborted at `thetaBound` in one test). See Chapter 12.

## 9.5 Stored results

After a successful fit `nwsaom` stores, among others, the following in `e()`. The authoritative list is the section Stored results of the `nwsaom` help file.

| Result | Content |
|---|---|
| `e(b)`, `e(V)` | Coefficients and covariance matrix of the effect parameters (the rates are not included). In a co-evolution fit the network part comes first (`e(p_net)` columns), then the behavior part, prefixed `beh_`. |
| `e(tratio)` | 1× nparam convergence $t$-ratios of the coefficients. |
| `e(tconv)`, `e(tconv_max)` | Convergence $t$-ratios of all parameters including rates; RSiena's overall maximum ratio (below 0.25 is good). |
| `e(rate)`, `e(rate_se)`, `e(rate_tratio)` | Network rate, standard error and $t$-ratio (two-wave path; no $t$-ratio under conditional estimation). |
| `e(rates)`, `e(rates_se)`, `e(rate_tratios)` | The same per period, $1 \times (\text{waves} - 1)$, for `waves()` (with `groups()`: one column per period of each group). |
| `e(rate_beh)`, `e(rates_beh)`, ... | Behavior rates of a co-evolution fit, with standard errors and $t$-ratios. |
| `e(ratecoefs)` | `ratecov()` fits: rate-covariate coefficients, with standard errors, $t$-ratios and a fixed indicator. |
| `e(rsiena_labels)` | RSiena's name of each coefficient, in the order of `e(b)`, separated by `\|`. |
| `e(conditional)`, `e(centered)` | 1 for conditional estimation; 1 if covariates were centred. |
| `e(engine)`, `e(engine_why)` | `native` or `mata`, and why Mata. |
| `e(N)`, `e(nodes)`, `e(nwaves)` | Number of actors and waves. |
| `e(ngroups)`, `e(groupn)`, `e(groupwaves)`, `e(groupvar)` | With `groups()`: number of groups, actors and waves per group, name of the group variable (`e(N)` is the total number of actors). |
| `e(rates_group)`, `e(rates_period)` | With `groups()`: the group and RSiena's period number of each column of `e(rates)`; `e(rates)` has one column per period of each group, named `period1`, `period2`, `period4`, ... (Chapter 11). |
| `e(waves)`, `e(behavior)`, `e(symtype)` | Names of the waves and behaviors; model type of a non-directed fit. |
| `e(estat_cmd)` | Postestimation dispatch (`nwsaom_estat`). |

Stored estimates can be combined across networks with `nwsaom meta`, the counterpart of RSiena's `siena08` (Section 10.8); several groups can also be estimated jointly with `groups()` (Chapter 11).

## 9.6 Goodness of fit: `estat gof`

`estat gof` implements the methodology of RSiena's `sienaGOF()` and `plot.sienaGOF()` (Lospinoso and Snijders 2019). For each auxiliary statistic (a user-chosen summary of the network, not a model term) it simulates `nsim()` fresh replicates at the fitted coefficients and tests whether the observed wave is an outlier relative to the simulated distribution.

**The test.** The test is a port of RSiena's `applyTest()`. The covariance matrix of the simulated statistic vectors is inverted by a Moore–Penrose pseudoinverse (necessary because the category proportions of a distribution are collinear, for example they sum to one); each simulated vector's Mahalanobis distance from the simulated mean is computed; and the reported $p$-value is the proportion of simulated distances at or beyond the observed distance, a permutation-style value, not a chi-square one. A small $p$ (below 0.05, RSiena's convention) is evidence *against* the model on that statistic; agreement is evidence for the model, not proof. The $p$-value is one-tailed by default, as RSiena's `twoTailed=FALSE`; `twotailed` switches it.

**Statistics.** The default statistics are the outdegree, indegree and geodesic-distance distributions, plus the behavior distribution whenever the fit used `behavior()`. The triad census (16 categories; raw counts, as RSiena's `TriadCensus()`) is available with `stats(triad)` and is never added by default. `maxdeg()` (default 15) and `maxdist()` (default 6) set the highest exact category before the overflow bin.

**Pooling and `join(off)`.** With three or more waves the statistic vector is pooled (summed) across periods before a single test, RSiena's `join=TRUE`. `join(off)` runs a separate test and violin per period (RSiena's `join=FALSE`); the stored results then carry the suffix `_p#`.

**Difference from RSiena.** RSiena reuses networks stored during the fit. `nwsaom` stores none, so `estat gof` runs fresh simulations from each period's observed start wave. The Mahalanobis test does not depend on how the reference draws were generated, only on their being independent draws at the fitted parameters, so the construction is unchanged. Co-evolution fits simulate network and behavior jointly for every replicate, regardless of the requested statistics. GOF simulates endowment/creation, `structural()`, `ratecov()`, `present()` and non-directed model types in the native plugin when it covers the model. `estat gof` is refused for fits with several co-evolving behaviors.

**The violin plot.** Each statistic is also drawn as a violin plot built from `kdensity` and `twoway rarea, horizontal`: the kernel-density shape per category, a thin box for the interquartile range, dashed lines at the 2.5th and 97.5th percentile, and the observed value as a red line, with the $p$-value as the $x$-axis title (as RSiena). Graphs are named `gof_outdegree`, `gof_indegree`, `gof_geodesic`, `gof_triad` and `gof_behavior`; `name()` changes the stub, and with `join(off)` each period gets its own graph (`gof_outdegree_p1`, ...).

```
. estat gof, nsim(30) seed(1)
Goodness of fit: Mahalanobis distance test
30 simulations at the estimates, 2 periods pooled

------------------------------------------------------
           Statistic |   Mahalanobis  P-value      Fit
---------------------+--------------------------------
           Outdegree |        23.919    0.067       ok
            Indegree |        14.906    0.133       ok
            Geodesic |        23.311    0.000 rejected
------------------------------------------------------
rejected: P < 0.05 (one-tailed); the model does not reproduce this statistic
```

This three-effect model on the Glasgow data is deliberately minimal; the output shows how a poorly fitting specification is reported, and that it is a reason to extend the model, not a failure of the command. `nsim()` defaults to 50 (minimum 5); `nsim(30)` was used here only to be quick.

**Stored results.** For each statistic `estat gof` stores `r(p_`*stat*`)` (the $p$-value) and `r(mhd_`*stat*`)` (the observed Mahalanobis distance); with `join(off)` the names carry `_p1`, `_p2`, ....

## 9.7 Micro effects on macro structure: `estat mems`

`estat mems` is a port of the `MEMS()` function (SAOM version) of Duxbury's R package `netmediate`. It asks how much a chosen micro-level effect (one coefficient of the fitted model, named as a column of `e(b)`) changes a macro-level network summary of your choice. You supply the summary as a Stata program that takes a network name and returns `r(stat)`.

```
program myDensity, rclass
    args netname
    nwsummarize 'netname', matonly
    return scalar stat = r(density)
end
estat mems, effect(reciprocity) macro(myDensity) nsim(500) seed(42)
```

The algorithm draws `nsim()` coefficient vectors (default 500) from a multivariate normal whose sample mean and covariance equal `e(b)` and `e(V)` exactly (achieved by a Cholesky whitening, not by `MASS`'s SVD, so the random numbers differ from R's). For each multiplier in `interval()` (default `0 1`: "as if the effect did not operate" versus "at its fitted strength") the effect's coefficient is scaled, a network is simulated forward from the fitted model's observed start wave, and the macro statistic is evaluated. The paired difference between the last and first multiplier gives the MEMS estimate, its Monte Carlo standard deviation, a 95% percentile interval and a Monte Carlo $p$-value (the proportion of draws whose sign is opposite to the mean effect, not doubled). It stores `r(mems)`, `r(mems_sd)`, `r(mems_lb)`, `r(mems_ub)`, `r(mems_p)` and `r(propchange)` (the difference as a proportion of the macro value at the last multiplier; a point estimate only, as in `netmediate`).

**Scope.** Version 1 handles exactly two waves (`wave1()`/`wave2()`) and network-only fits; co-evolution, `waves()`, `symmetric` and multiplex fits are rejected with a message. The VanderWeele E-value sensitivity bound of `netmediate` is not implemented. The theta draw is shared across a row's interval values, but each simulation draws fresh randomness (no common random numbers).

## 9.8 Maximum likelihood: `ml`

With the option `ml`, `nwsaom` first runs the method of moments, whose estimates are the starting values, and then estimates the model by maximum likelihood, as `siena07` does with `maxlike = TRUE` (Snijders, Koskinen and Schweinberger 2010). The unobserved sequence of ministeps between two waves (who was selected, which tie was considered, including the ministeps without a change) is part of the data that are augmented. It is simulated by Metropolis–Hastings steps: inserting and deleting a ministep that changes nothing, permuting ministeps, and inserting or deleting a pair of ministeps that cancel. The complete-data score of the effects and of the rates is averaged by a Robbins–Monro algorithm, with the same three phases as the method of moments. The estimates and the covariance matrix replace `e(b)` and `e(V)`; the standard errors come from the mean complete-data information minus the covariance matrix of the scores. The rates are estimated along and stored in `e(rates)`; `e(rates_ml)` adds the convergence $t$-ratio of every rate, and `e(tconv_max_ml)` is the largest $t$-ratio of all scores (below 0.1 is good).

The estimator covers one directed one-mode network, with or without one co-evolving behavior (its ministeps and its rates are part of the augmented data), and no endowment or creation effects, rate covariates, structural zeros, missing data or composition change. Non-directed and two-mode networks and several networks are not covered. The controls are given in `mloptions()`: `n1(#)` iterations of phase 1 (default 50), `nsub(#)` subphases of phase 2 (4) with `nstart2(#)` iterations in the first (100, then 2.52 times as many in each further subphase), `n3(#)` iterations of phase 3 (1 000), `mult(#)` Metropolis–Hastings steps per iteration as a multiple of the number of tie changes (5), `gain(#)` the initial gain (0.2) and `seed(#)`. The same estimates are available after an `nwsaom` fit as `nwsaom_ml`.

```
. nwwebuse glasgow, nwclear
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip k3(500) seed
    (1) ml mloptions(n3(4000) nstart2(200) seed(2))
Maximum likelihood estimates (Robbins-Monro on the score of the augmented data)
------------------------------------------------------------------------------
glasgow1_t~2 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
```

```
-------------+----------------------------------------------------------------
   outdegree |  -2.508867   .1283755   -19.54   0.000    -2.760478   -2.257256
 reciprocity |   1.954275   .2177942     8.97   0.000     1.527406    2.381143
   transtrip |   .6850565   .0905747     7.56   0.000     .5075335    .8625796
------------------------------------------------------------------------------
Rate parameters (one per inter-wave period):

------------------------
             |   period1
-------------+----------
        rate |    6.4168
          se |    0.9363
       tconv |   -0.0593
------------------------
Overall maximum t-ratio of the scores: 0.096 (good; below 0.1)
```

The method-of-moments fit of the same model, which is printed first and used as the starting point, has −2.53, 2.15 and 0.59. With the default controls this example takes about half a minute on an Apple M5 Pro and ends with a largest $t$-ratio of 0.22, which the output flags as acceptable; the longer phases above (a little over a minute) bring it below 0.1, and the output tells you to run longer when this happens. Estimates from different seeds differ by simulation noise. Against `siena07` with `maxlike = TRUE` the estimates of this model are −2.65, 2.22 to 2.24 and 0.69 to 0.70 in `nwsaom` (several runs) and −2.66 and −2.63, 2.22 and 2.19, 0.71 and 0.70 in two runs of RSiena. With a co-evolving behavior the estimates are within 0.1 standard errors of RSiena's.

## 9.9 Bayesian estimation: `nwsaom bayes`

`nwsaom bayes` samples the posterior distribution of the model that was just fitted, by data augmentation. It is run *after* an `nwsaom` fit for a directed one-mode network, with or without one co-evolving behavior, and takes the network waves, the effects and the covariates from that fit. The estimates of the fit are the starting values and its covariance matrix is the proposal distribution of the effects. The algorithm and the names of its controls follow `multi_siena` (`sienaBayes`) of the R package multiSiena (Snijders, Koskinen and Ripley) for one group. One batch consists of

1. Metropolis–Hastings steps on the sequences of ministeps of all periods, with the moves of RSiena's maximum likelihood chain (insertion and deletion of a ministep without change, permutation of consecutive ministeps, insertion or deletion of a pair of cancelling ministeps combined with a permutation);
2. one Metropolis step for the rates, with a normal prior around the rates of the fit (standard deviation 1);
3. `nsampconst()` random-walk Metropolis steps for the effects, with the covariance matrix of the fit and, by default, a flat prior.

The proposal scale factors are tuned in rounds of `nimprovemh()` batches before and after the warming phase, as in `improveMH` of multiSiena, towards the acceptance probability `targetmh()` (0.25). The schedule is: pre-warming (`nprewarm()`, default 50), tuning, warming (`nwarm()`, 50), tuning and the main run (`nmain()`, 250 iterations, one draw saved per iteration with `nrunmh()` (20) batches in between). `mult()` (5) sets the number of Metropolis–Hastings steps on the sequences as a multiple of the observed number of tie changes. The priors are set with `priormeaneta()` and `priorsigeta()` (the effects) and `ratepriormean()` and `ratepriorsd()` (the rates); `gibbsrates` draws the rates from their gamma full conditional instead.

The following example uses a short run to keep the time low (about 25 seconds on an Apple M5 Pro, together with the fit); the draws are strongly autocorrelated, as the effective sample sizes (ESS) show.

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip k3(500) seed
    (1)
. nwsaom bayes, nprewarm(20) nwarm(20) nmain(100) nimprovemh(40) seed(1)
Bayesian SAOM (data augmentation)               Number of actors  =       50
Waves: glasgow1 glasgow2                        Saved draws       =      100
Pre-warm 20, warm 20, main 100 x 20             Mean chain length =    306.2
Acceptance rates: effects 0.497, rates 0.246
Chain steps accepted: diagonal 0.751, permute 0.755
                      insert-permute 0.410, delete-permute 0.370
------------------------------------------------------------------------------
      Effect |       Mean  Std. dev.    [95% cred. interval]     Median   ESS
-------------+----------------------------------------------------------------
   outdegree |    -2.5229     0.1446     -2.7498     -2.2554    -2.5264    46
 reciprocity |     2.0143     0.2430      1.5666      2.4366     2.0243    36
   transtrip |     0.6618     0.0988      0.4623      0.8475     0.6545    77
rate_period1 |     6.0569     0.6129      4.9734      7.2384     6.0627    65
------------------------------------------------------------------------------
```

The result replaces the estimates of the fit (`estimates store` first to keep them): `e(b)` and `e(V)` are the posterior means and covariance matrix, `e(rates)` the posterior mean and standard deviation of the rates, `e(interval)` the interval limits and median, `e(ess)` the effective sample sizes, and `e(post_frame)` the name of the frame that holds the draws (one row per main iteration). The acceptance rates of all steps are in `e(accept_effects)`, `e(accept_rates)` and the `e(accept_`*step*`)` results, the tuned scale factors in `e(scale_effects)`.

**Validation.** On a case with four actors the expected numbers of ministeps of the sampler are 16.249 and 32.385 against the exact values 16.258 and 32.344 from complete enumeration. On the Glasgow data (waves 1 to 3) with outdegree, reciprocity and transitive triplets the posterior means are −2.656, 2.256 and 0.686 (posterior standard deviations 0.109, 0.175, 0.069), against the maximum likelihood estimates of `siena07` −2.661, 2.224, 0.713 (standard errors 0.139, 0.181, 0.083) and −2.632, 2.187, 0.702 in a second run. With the alcohol covariates the posterior means are within one maximum-likelihood standard error. A run with the default controls takes about five minutes and gives effective sample sizes of 67 to 152 out of 250 draws; for inference increase `nmain()` or `nrunmh()`.

**Differences from `multi_siena` and limits.** Only one group is supported: there is no group-level mean and covariance matrix, no randomly varying effects and no hierarchical prior (with one group `multi_siena` itself keeps the rate prior fixed, which is the default here). The starting values and the proposal covariance come from the `nwsaom` fit, where `multi_siena` first runs a short method-of-moments estimation of its own. The Metropolis–Hastings moves on the sequences were written from the published description, not from the code, and the length of a permuted block is kept fixed in the main run. Not available: several behaviors, several networks or groups, missing data, composition change, rate covariates, endowment and creation effects, and non-directed and two-mode networks. With these limits the posterior of the effects under a flat prior is close to the maximum likelihood estimate and its standard error.

## 9.10 Run time

With the native engine, fit time grows roughly with the cube of the number of actors above about 1,000 actors. For a plain model (outdegree, reciprocity, transitive triplets; three waves; about ten ties per actor; `k0(30) k3(200)`) it takes 1.5 s for 250 actors, 2.6 s for 500, 10 s for 1,000, 67 s for 2,000 and 520 s for 4,000; a full-length run costs several times more. Timings on one 18-core machine: the `s50` three-wave model `outdegree reciprocity transtrip` 0.33 s, and the

`s50` three-wave co-evolution model 0.63 s. Beyond roughly 10,000 actors the model itself (every ministep considers all possible alters) is the limit. Stata's own limits matter for matrices passed to `nwsaom` (`dyadcov()`, `missnet()`): 800×800 in Stata/BE, 11,000×11,000 in Stata/SE.

# Chapter 10

# Simulation, tests and importance of effects

This chapter describes the commands that describe the data before a fit and work with a fitted model or simulate one: the description of the data, holding effects fixed, simulating a model at chosen parameters, the score-type test for effects that were left out of a model, the test for differences between periods, and the relative importance of the effects. All of them follow the corresponding procedures of RSiena; the comparison is in Chapter 12.

## 10.1 How the commands are organised

A property of the data is detected from the data: a two-mode network or a non-directed network is recognised from the way the network was declared and needs no option. An option changes how a model is specified or fitted (`behavior()`, `present()`, `fix()`). A different job is a subcommand (`nwsaom multinet`, `nwsaom simulate`) or a postestimation command (`estat`). `nwsaom multiplex` is deprecated and forwards to `nwsaom multinet`.

## 10.2 Describing the data: `nwsaom describe`

Before a fit, `nwsaom describe` prints what RSiena prints before an estimation: for every wave the number of ties, the density and the average degree; for every period the numbers of ties that are kept, created and dissolved, the Jaccard index and the Hamming distance; and, with `behavior()`, the mean and standard deviation of the behavior in every wave and the numbers of actors whose behavior goes up, goes down or stays the same.

```
. nwwebuse glasgow, nwclear
. nwsaom describe, waves(glasgow1 glasgow2 glasgow3) behavior(alcohol1 alcohol2
    alcohol3)
Network data                          Number of actors  =        50
Directed                              Number of waves   =         3

------------------------------------------------------------------
        Wave |       Ties     Density   Avg. degree
-------------+----------------------------------------------------
           1 |        113      0.0461         2.260
           2 |        116      0.0473         2.320
           3 |        122      0.0498         2.440
------------------------------------------------------------------

------------------------------------------------------------------
      Period |    Kept   Created  Dissolved   Jaccard   Hamming
```

```
------------+-----------------------------------------------------
       1->2 |      57        59         56     0.331       115
       2->3 |      66        56         50     0.384       106
-------------------------------------------------------------------


-------------------------------------------------------------------
   Behavior |        Mean   Std. dev.     Missing
------------+------------------------------------------------------
     Wave 1 |       2.880       1.118           0
     Wave 2 |       3.100       1.199           0
     Wave 3 |       3.360       0.985           0
-------------------------------------------------------------------


-------------------------------------------------------------------
     Change |          Up        Down        Same
------------+------------------------------------------------------
       1->2 |          16           6          28
       2->3 |          18           9          23
-------------------------------------------------------------------
```

A period with a Jaccard index below about 0.3 has few stable ties and carries little information about the change process; here the two periods are at 0.33 and 0.38. The results are in `r(periods)`, `r(waves)` and `r(behchange)`. For a two-mode network the ties are the attendances (the cells of the actors-by-events matrix), the density is taken over those cells and the average degree is the mean number of events per actor; the option `actormode(#)` names the mode of the actors (default 2, as in `nwsaom`), and `r(n_actors)` and `r(n_events)` hold the numbers of actors and events. The description of a stacked data set with groups (Chapter 11) is that of the combined network; describe the groups one by one by making a network of each.

## 10.3 Holding effects fixed: `fix()`

`fix(`*effects*`)` keeps the named effects at their `theta0()` values. They are not estimated and have no standard error; this is RSiena's `fix = TRUE`. `fix(_all)` fixes every effect. The names are those of the effects in the call; the effects of a co-evolving behavior are named by their token (`fix(avalt)`) or by the label in the coefficient table (`beh_avalt`).

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip theta0(-2 1
    0) fix(reciprocity) k3(300) seed(1)
   outdegree |  -2.311036   .1114178   -20.74   0.000
 reciprocity |          1          .        .       .
   transtrip |   .7747333   .0996184     7.78   0.000
(output omitted)
Overall maximum convergence ratio: 0.188 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    outdegree (0.132) reciprocity (-1.656) transtrip (0.185)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

The fixed coefficient equals its start value and its variance is 0. The overall maximum convergence ratio leaves out the fixed parameter, but the line of the $t$-ratios checks the $t$-ratios of all parameters, the fixed one included, so it lists `reciprocity` ($-1.656$); a large $t$-ratio of a fixed parameter is expected and is what the score test below examines. The $t$-ratios of the two free parameters, `outdegree` (0.132) and `transtrip` (0.185), are above 0.1 as well, so this fit has not converged by the two criteria; before it is reported it would be restarted from its estimates with `theta0()`, as Chapter 9 explains. Fixing is used for the score test below, for fitting a model with a known part, and for the simulation-only mode.

## 10.4 Simulation: `estat simulate` and `nwsaom simulate`

`estat simulate` simulates the fitted model forward. Every period starts at the observed start wave of the period and runs for the period's length at the fitted coefficients and rates, the same simulation that `estat gof` uses; the default is 100 replications.

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip k3(200) seed
    (1)
. estat simulate, nsim(20) seed(5)
Simulation from the fitted model: 20 replication(s) of 1 period(s)

------------------------------------
   Statistic |       Sim       Obs
-------------+----------------------
Period 1     |
        Ties |     112.4       116
      Mutual |      34.0        35
  Transitive |      77.8        88
------------------------------------
Sim is the mean over the replications; Obs is the observed end wave.
```

Options: `theta(`*matname*`)` replaces the coefficients (a row vector in the order of `e(b)`), `rates()` and `ratesbeh()` replace the rates, `generate(`*stub*`)` saves the simulated networks as networks named *stub*`_`*s*`_p`*period* (and the behaviors as variables of the same names). The result `r(sim)` has one row per replication and period (number of ties, mutual dyads, transitive triples, mean behavior), `r(obs)` the same for the observed end waves.

**Simulation-only mode.** `nwsaom simulate` simulates at coefficients you choose, without any estimation (RSiena's `siena07` with `simOnly = TRUE`). The model is specified exactly as in `nwsaom`; `theta()` gives one coefficient per network effect in the order of the coefficient table of a fit of the same model (the order in which `nwsaom` lists the effects, which is not always the order in which you typed them), `behtheta()` one per behavior effect, `rates()` the rate of each period.

```
. nwsaom simulate, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip
    theta(-2.3 2 0.5) rates(5) nsim(30) seed(3)
------------------------------------
   Statistic |       Sim       Obs
-------------+----------------------
Period 1     |
        Ties |     127.1       116
      Mutual |      36.3        35
  Transitive |      72.0        88
------------------------------------
Sim is the mean over the replications; Obs is the observed end wave.
```

Internally the model is set up by a short call of `nwsaom` with all coefficients fixed, so every data option of `nwsaom` is understood. Both commands work for a two-mode model: the simulation runs on the actors-by-events network, the table shows the number of ties (attendances) only (the columns for mutual dyads and transitive triples are missing), and `generate()` saves two-mode networks (one row per actor, one column per event). A `theta()` matrix of the wrong shape stops with a message that says what was expected. After a fit with `groups()`, `estat simulate` is not available.

## 10.5 Score-type tests: `estat score`

The score-type test of Schweinberger (2012) asks, for effects that are not in the model, whether adding them would improve the fit, using only the restricted model. Fit the restricted model with the effects to be tested in the model but held fixed at the restricted value (usually 0), then call `estat score`:

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip cycle3 theta0
    (-2.3 2 0 0) fix(transtrip cycle3) k0(100) k3(1000) seed(1)
. estat score

Score-type test of fixed effects
H0: each effect below equals its fixed value

------------------------------------------------------------------
      Effect |  Fixed at      chi2   df  Prob > chi2  One-sided z
-------------+----------------------------------------------------
   transtrip |    0.0000    81.920    1       0.0000        9.051
      cycle3 |    0.0000    67.902    1       0.0000        8.240
-------------+----------------------------------------------------
       Joint |              82.061    2       0.0000
------------------------------------------------------------------

One-step estimates (restricted estimates plus the one-step correction)
------------------------------------------------------------------
      Effect |    Estimate
-------------+----------------------------------------------------
   outdegree |     -2.6717
 reciprocity |      2.0383
   transtrip |      1.3303
      cycle3 |      0.2607
------------------------------------------------------------------
```

The joint statistic $c$ is chi-squared with as many degrees of freedom as tested effects. The one-sided variate is a normal variate with the sign changed, as in RSiena, so that it is positive when the effect is expected to be positive. The one-step estimates give the value each coefficient would approximately take if the tested effects were freed, which is a quick way to see the size of the effects before fitting the larger model. `tested(`*effects*`)` restricts the test to some of the fixed effects; the others stay fixed and are treated as redundant. The test works for network effects and for the effects of a co-evolving behavior.

**After `nwsaom multinet`.** The restricted model of a multinet fit is specified with `theta0()` and `fix()` of that command, with the coefficient names of the table (for example `fix(net1_x_crprod)`); the rates of all networks and of the behavior are estimated along. The example fixes the cross-network effect `crprod` at 0 (the data are those of Section 7.3).

```
. nwsaom multinet, net1(glasgow1 glasgow2 glasgow3) net2(advice1 advice2 advice3) eff1(
    outdegree reciprocity transtrip) eff2(outdegree reciprocity) cross1(crprod) cross2(
    from) theta0(-2.6 2.5 0.6 0 -1.9 1 0) fix(net1_x_crprod) seed(1) k3(300)
. estat score

Score-type test of fixed effects
H0: each effect below equals its fixed value

------------------------------------------------------------------
      Effect |  Fixed at      chi2   df  Prob > chi2  One-sided z
-------------+----------------------------------------------------
```

```
net1_x_crp~d |    0.0000     2.323    1       0.1275       -1.524
------------------------------------------------------------------

One-step estimates (restricted estimates plus the one-step correction)
------------------------------------------------------------------
      Effect |    Estimate
-------------+----------------------------------------------------
net1_outde~e |     -2.6173
net1_recip~y |      2.5168
net1_trans~p |      0.6207
net1_x_crp~d |     -0.4489
net2_outde~e |     -1.9142
net2_recip~y |      1.1544
 net2_x_from |      0.2009
       rate1 |      5.5497
       rate2 |      5.0184
       rate3 |      1.8038
       rate4 |      1.7646
------------------------------------------------------------------
```

On the artificial advice network `crprod` has no evidence in its favour, as in the fit of Section 7.3. The rates are listed as `rate1`, `rate2` (the first network in periods 1 and 2) and `rate3`, `rate4` (the second network).

## 10.6 Time heterogeneity: `estat timetest`

`estat timetest` asks whether the effects of a model with at least three waves have the same size in all periods (RSiena's `sienaTimeTest`). For each effect the model is extended by deviations for the periods 2 to $P$, and the hypothesis that all deviations are 0 is tested by a score-type test, from the simulations of the fit in memory; no second fit is needed.

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip k0(100) k3
    (1000) seed(1)
. estat timetest
Test of time heterogeneity (score-type test)
H0: the parameter of each effect is the same in all periods

------------------------------------------------------------
      Effect |          chi2        df        Prob > chi2
-------------+----------------------------------------------
   outdegree |         0.074         1             0.7860
 reciprocity |         0.963         1             0.3264
   transtrip |         0.439         1             0.5075
-------------+----------------------------------------------
       Joint |         3.567         3             0.3122
------------------------------------------------------------

Period-specific deviations (one-step estimates) and separate tests
------------------------------------------------------------
      Effect |  Period   Deviation      chi2  Prob > chi2
-------------+----------------------------------------------
   outdegree |       2     -0.3527     0.074       0.7860
 reciprocity |       2      0.6927     0.963       0.3264
   transtrip |       2      0.0779     0.439       0.5075
------------------------------------------------------------
```

With $P$ periods there are $P-1$ deviations per effect. In this example there is no evidence that

the effects change between the two periods. The tests are most informative when the deviation of an effect is the question: the deviation is reported as the one-step estimate of the difference between the period and the first period. `effects(`*names*`)` restricts the test to some effects.

**Treatment of the rates.** By default the rate parameters are treated as estimated, so that the test equals the score test of a model that adds one fixed period dummy to the fit (what RSiena's `score.Test` gives after such a fit). The option `fixrates` treats the rates as known, which is what RSiena's `sienaTimeTest` does. The two agree closely except for effects whose statistic is strongly correlated with the rate, typically the outdegree effect of a network whose changes are close to random; there the default gives the larger value. In the example above they are identical to four decimals; `estat timetest, fixrates` gives the same table.

**After `nwsaom multinet`.** The test is also available for the effects of all networks and of the behavior of a multinet fit.

```
. estat timetest
Test of time heterogeneity (score-type test)
H0: the parameter of each effect is the same in all periods

------------------------------------------------------------
      Effect |          chi2        df        Prob > chi2
-------------+----------------------------------------------
net1_outde~e |         0.077         1              0.7808
net1_recip~y |         0.998         1              0.3177
net1_trans~p |         0.396         1              0.5291
net2_outde~e |         1.487         1              0.2227
net2_recip~y |         3.592         1              0.0580
 net2_x_from |         1.501         1              0.2205
-------------+----------------------------------------------
       Joint |         7.105         6              0.3113
------------------------------------------------------------
```

(Run after the multinet fit above with `crprod` fixed. The output continues with the period-specific deviations.)

## 10.7 Relative importance of effects: `estat importance`

`estat importance` computes the relative importance of the effects of a network model (RSiena's `sienaRI`, `interpret_size`). At the start network of each period the contribution of every effect to every possible tie change of every actor is computed. The importance of an effect for an actor is the $L^1$ distance between the actor's choice probabilities with the fitted coefficients and with the effect's coefficient set to 0; the relative importance is its share among all effects, averaged over the actors. The command also reports $\sigma$ (the within-actor standard deviation of the effect's contributions), the semi-standardised estimate (the coefficient times $\sigma$, which compares effects on a common scale) and the certainty $R_H$ of the actors' choices.

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip cycle3 k0
    (100) k3(500) seed(1)
. estat importance
------------------------------------------------------------------------------
      Effect | Estimate  Std. err.    Sigma  Semi-std.     RI p1     RI p2
-------------+----------------------------------------------------------------
   outdegree |  -2.7041     0.1146   0.4384    -1.1854    0.4309    0.5015
 reciprocity |   2.5066     0.2064   0.2140     0.5364    0.3763    0.3085
   transtrip |   0.6853     0.1514   0.6514     0.4464    0.1741    0.1727
```

```
      cycle3 |  -0.1348     0.2977   0.3374    -0.0455    0.0187    0.0173
-------------+------------------------------------------------------------
   Certainty |                                              0.2674    0.3548
------------------------------------------------------------------------------
```

The command is available after fits of a directed or non-directed one-mode network or of a two-mode network, with or without one co-evolving behavior, without endowment and creation effects, `structural()`, `present()` or missing-data options; RSiena has the same restrictions. It needs the native plugin. All sizes agree with `sienaRI` to four decimals.

**Non-directed and two-mode networks.** For a non-directed network the importance is that of the forcing model, as RSiena notes. For a two-mode network the actors choose among the events. With the simulated $40 \times 20$ data of Section 7.2:

```
. nwsaom, waves(tm1 tm2 tm3) outdegree cycle4 nodeocov(age) nodeicov(size) seed(1) k3
    (300)
. estat importance
------------------------------------------------------------------------------
      Effect | Estimate  Std. err.    Sigma  Semi-std.     RI p1     RI p2
-------------+----------------------------------------------------------------
   outdegree |  -0.8968     0.1182   0.6843    -0.6136    0.7242    0.6962
nodeicov_s~e |  -0.1309     0.0748   1.2020    -0.1574    0.1678    0.1533
nodeocov_age |   0.0139     0.0909   0.7709     0.0107    0.0136    0.0129
      cycle4 |   0.0550     0.0336   2.6018     0.1431    0.0943    0.1376
-------------+----------------------------------------------------------------
   Certainty |                                              0.0771    0.0732
------------------------------------------------------------------------------
```

**Co-evolution.** After a co-evolution fit a second table gives the behavior effects. The options of an actor are to change the behavior by $-1$, 0 or $+1$; the importance of a behavior effect is again the $L^1$ distance of these choice probabilities with and without the effect. The results are `r(ri_beh)`, `r(importance_beh)`, `r(sigma_beh)`, `r(sigma_period_beh)` and `r(certainty_beh)`.

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip behavior(
    smoke1 smoke2 smoke3 : linear quadratic avalt) k0(100) k3(500) seed(1)
. estat importance
Network effects
------------------------------------------------------------------------------
      Effect | Estimate  Std. err.    Sigma  Semi-std.     RI p1     RI p2
-------------+----------------------------------------------------------------
   outdegree |  -2.6446     0.1408   0.4384    -1.1593    0.4444    0.5151
 reciprocity |   2.4235     0.2075   0.2140     0.5186    0.3895    0.3204
   transtrip |   0.6070     0.0942   0.6514     0.3954    0.1661    0.1646
-------------+----------------------------------------------------------------
   Certainty |                                              0.2604    0.3396
------------------------------------------------------------------------------

Behavior effects (the options are changing by -1, 0, +1)
------------------------------------------------------------------------------
      Effect | Estimate  Std. err.    Sigma  Semi-std.     RI p1     RI p2
-------------+----------------------------------------------------------------
  beh_linear |  -1.9183     0.3811   0.7517    -1.4419    0.6001    0.5875
beh_quadra~c |   2.0247     0.4437   0.6712     1.3590    0.1808    0.2095
   beh_avalt |   1.9822     1.4992   0.4392     0.8707    0.2191    0.2030
-------------+----------------------------------------------------------------
   Certainty |                                              0.5814    0.5745
```

```
------------------------------------------------------------------------------
```

## 10.8 Several groups: `nwsaom meta`

This section is about the meta-analysis of separate fits. To estimate one model for all groups at once, use the option `groups()` (Chapter 11); Section 11.5 compares the two with the same data.

When the same model is fitted to several networks of the same kind, for example school classes, the group estimates can be combined, which is what RSiena's `siena08` does. `nwsaom meta` takes the stored estimates (`estimates store`) of the same model fitted to each group, with the same effects in the same order. (The name `nwsaom_meta`, used by earlier versions, still works.) For every effect it reports the combined mean and its standard error (an iteratively reweighted least squares estimate with a random effect for the group, Snijders and Baerveldt 2003), the estimated standard deviation between groups, a heterogeneity test $Q$, the maximum likelihood estimates of the mean and the between-group standard deviation with profile-likelihood intervals, and Fisher's combination of the one-sided tests.

The following example runs with the data of the package. It makes four groups by simulating the model four times from the same start network at known coefficients (Section 10.4), fits each group on its own, and combines the fits. The combined estimates should be close to the values used for the simulation ($-2.3$, 2 and 0.5).

```
. nwwebuse glasgow, nwclear
. nwsaom simulate, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip
    theta(-2.3 2 0.5) rates(5) nsim(4) seed(7) generate(g)
. forvalues s = 1/4 {
.     quietly nwsaom, wave1(glasgow1) wave2(g_‘s’_p1) outdegree reciprocity transtrip
    k3(500) seed(‘s’)
.     estimates store grp‘s’
. }
. nwsaom meta grp1 grp2 grp3 grp4
Meta-analysis of separate fits                  Number of fits    =         4

Estimates with a standard error of 10 or more are excluded;
tests of the means use a t distribution with N-1 d.f.
IWLS estimates (Snijders and Baerveldt 2003)
      Effect |   n      Mean  Std. err.       t   P>|t|      SD       Q     P>Q
-------------+------------------------------------------------------------------
   outdegree |   4   -2.2509     0.0579  -38.87   0.000  0.1158    2.59   0.460
 reciprocity |   4    1.8694     0.0737   25.37   0.000  0.1473    1.18   0.759
   transtrip |   4    0.4758     0.0114   41.81   0.000  0.0228    0.14   0.987
```

The combined means recover the coefficients, and no effect differs between the groups (`P>Q` between 0.46 and 0.99), as it should for groups simulated from the same model. In a real analysis a small `P>Q` for an effect means that the groups differ in that effect by more than their standard errors allow, and the estimated between-group standard deviation shows by how much. The full set of results is returned in `r(meta)`; the option `rates` combines the rate parameters as well, and `bound(#)` excludes groups with a standard error of at least # (default 10), as RSiena does. An effect that a group fitted with `fix()` has a standard error of 0 and is left out for that group, as `siena08` leaves it out (version 1.4 and earlier counted such a group in $n$ and in the degrees of freedom of the tests of that effect, although it did not enter the weights; the estimates and the heterogeneity statistics of the other effects are unchanged). The option `rates` uses the standard errors of the rates, `e(rates_se)`, of each fit (version 1.1 took them from a row that does not exist, so that every rate was dropped without a message); the rows are named `rate_period1`, `rate_period2`, .... The matrices `r(theta)` and `r(se)` (one row per stored estimate, one column

per effect) carry the names of the stored estimates and of the effects. The results reproduce RSiena's functions to five decimals (Chapter 12).

## 10.9 What is not available

After `nwsaom multinet`, `estat score` and `estat timetest` are available, but `estat gof`, `estat importance` and `estat simulate` are not. The same holds after a fit with `groups()` (Chapter 11), where `estat timetest` also reports a test per group and per period. The generalised method of moments of RSiena is not implemented (Chapter 13). Maximum likelihood and Bayesian estimation are described in Chapter 9.

# Chapter 11

# Several groups: joint estimation

Many network panels consist of several independent groups of actors: school classes, teams, villages, organizations. Each group is a separate network with its own actors, observed at the same occasions (or at different ones). RSiena offers two ways to analyze such data: fit one model to all groups at once (`sienaGroupCreate` and `siena07`), or fit each group separately and combine the estimates (`siena08`). `nwsaom` offers both. This chapter is about the first, the option `groups()`; the second is `nwsaom meta` (Section 10.8), and Section 11.5 says when to use which. All outputs below were produced with `nwsaom` 1.5.7 in Stata/BE 18; the data and do-files are described at the end of the chapter.

## 11.1 The idea

In a joint analysis the groups share *one set of effect parameters*: the model assumes that actors in every group choose their ties by the same rule (the same weights for reciprocity, transitive closure, and so on). What is not shared is the speed of change. Every group, and every period of every group, has its own rate parameter, because classes differ in how much they change and the time between waves may differ. The statistics of all periods of all groups are added up when the coefficients are estimated, exactly as the periods of one group are in a model with three or more waves (Chapter 3). Two things follow.

- Small groups, each too small to estimate a model on its own, together can. The standard errors of the joint model reflect the pooled information of all groups.
- The joint model does *not* estimate how much the groups differ. If you want to know whether the groups behave differently, or if you suspect they do, use the meta-analysis, or let selected effects differ by a group indicator (Section 11.5).

## 11.2 The data layout

The groups are stacked in one data set. The observations (rows) are the actors of all groups, one variable says which group an actor belongs to, and each wave of the network is one nwcommands network on *all* actors that has no tie between two different groups, that is, a block-diagonal adjacency matrix. A covariate is an ordinary variable of the data set; a dyadic covariate is a matrix that is block-diagonal like the networks. A tie between two groups stops the command with the number of such ties. The groups may have different numbers of actors (at least three each); they must have the same variables and the same network type. Figure 11.1 sketches the layout for two groups.

The four school classes used below come as eight csv files (an adjacency matrix per class and wave). The following lines read them, remove the (few) ties of an actor to itself, and build the two stacked wave networks `w1` and `w2` and the group variable `class`:

| data set (rows = actors) | | | wave network $W_t$ (block-diagonal) | |
|---|---|---|---|---|
| group | alc1 | x | | |
| 1 | 3 | 0.5 | | |
| 1 | 2 | 1.2 | | $A_g$: ties among the actors |
| 1 | 4 | 0.7 | $\begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}$ | of group $g$ at wave $t$; |
| 2 | 1 | 0.3 | | the zero blocks: no ties |
| 2 | 5 | 1.1 | | between groups. |
| 2 | 3 | 0.9 | | |
| 2 | 2 | 0.1 | | |

Figure 11.1: The layout of a stacked data set for two groups.

```
mata: B1 = J(0, 0, .); B2 = J(0, 0, .)
local sizes ""
forvalues s = 1/4 {
    forvalues w = 1/2 {
        quietly import delimited using "school`s'_wave`w'.csv", clear
        local n = _N
        quietly mkmat v1-v`n', matrix(X)
        matrix X = X - diag(vecdiag(X))
        mata: B`w' = blockdiag(B`w', st_matrix("X"))
    }
    local sizes "`sizes' `n'"
}
clear
nwclear
mata: st_matrix("G1", B1); st_matrix("G2", B2)
quietly set obs `=rowsof(G1)'
quietly generate byte class = .
* ... class = 1 for the first 45 rows, 2 for the next 37, 3 for 33, 4 for 36
nwset, mat(G1) directed name(w1) nooutput
nwset, mat(G2) directed name(w2) nooutput
```

The same can be done from stacked edge lists with `nwfromedge`. The data are the classes N3401, N3403, N3404 and N3406 of the Dutch Social Behavior Data Set of Chris Baerveldt, which ship with RSiena (wave 1 and the second wave HN3401 ... HN3406); they have 45, 37, 33 and 36 pupils (Houtzager and Baerveldt 1999; Snijders and Baerveldt 2003).

## 11.3 The options `groups()` and `gwaves()`

| Option | Meaning |
|---|---|
| `groups(`*varname*`)` | Joint estimation of several groups. *varname* gives the group of each actor (a variable with at least two values; one value is refused, an ordinary fit is the one-group case). Needs `waves()`, the compiled plugin, and the stacked layout above. |
| `gwaves(`*spec*`)` | Only with `groups()`. The waves of the groups when they differ: `gwaves(2: 1 2 \| 3: 2 3)` says that group 2 was observed at the first two entries of `waves()` and group 3 at the last two (group value, colon, indices into `waves()`; at least two, increasing). A group not named has all waves. |

**How the rates are numbered.** Each period of each group has a rate. RSiena numbers the periods of the combined series, and the step from the last wave of one group to the first wave of the next is not a period; `nwsaom` uses the same numbers, so that its rate rows can be matched to RSiena's output. Four groups with two waves each have the periods 1, 3, 5 and 7 (periods 2, 4 and 6 are the switches between classes): `period1 period3 period5 period7`. Three groups with three waves each give `period1 period2 period4 period5 period7 period8`. The matrices `e(rates_group)` and `e(rates_period)` give, for every rate column, the group (in the order of the values of the group variable) and RSiena's period number.

**What is shared and what is not.**

- *Shared:* all effect parameters (including those of a behavior), the form of the model.
- *Not shared:* the rate of every period of every group (also the rate of a behavior). There is no option for a common rate across groups, as in RSiena; a rate covariate (`ratecov()`) with a group indicator lets the rates depend on the group.
- *Computed for each group:* the constants that RSiena keeps per group. These are the similarity mean of a covariate (`simx()`, with the range of the covariate over all groups), the balance mean (`balance`), the mean, similarity mean and variance of a dependent behavior, and the centering of a dyadic covariate (by the mean of its off-diagonal cells within the group). An actor covariate is centered at its mean over *all* actors of all groups, as RSiena does, so a variable that is constant within a group (group size, a group indicator) can be used with `egox()`.
- To let an effect *differ* between groups, interact it with a group indicator (`interact()`); this is the advice of the RSiena manual too.

Conditional estimation (the default for a network without behavior or composition change) and `unconditional` both work. Composition change (`present()`), missing data (`missnet()`, `missbeh()`) and changing covariates (`tvcov()`) are given for all actors; a group uses the waves it has.

## 11.4 A first joint model

The model of Chapter 3, `outdegree reciprocity transtrip`, for the four classes at once. The only new option is `groups(class)`.

```
. tabulate class
      class |      Freq.     Percent        Cum.
------------+-----------------------------------
          1 |         45       29.80       29.80
          2 |         37       24.50       54.30
          3 |         33       21.85       76.16
          4 |         36       23.84      100.00
------------+-----------------------------------
      Total |        151      100.00

. nwsaom describe, waves(w1 w2)
Network data                          Number of actors  =       151
Directed                              Number of waves   =         2

------------------------------------------------------------------
        Wave |       Ties     Density   Avg. degree
-------------+----------------------------------------------------
           1 |        246      0.0109         1.629
           2 |        335      0.0148         2.219
------------------------------------------------------------------
```

```
------------------------------------------------------------------
      Period |    Kept   Created  Dissolved   Jaccard   Hamming
-------------+----------------------------------------------------
        1->2 |     139       196        107     0.314       303
------------------------------------------------------------------

. nwsaom, waves(w1 w2) groups(class) outdegree reciprocity transtrip seed(5)

SAOM (Method of Moments, conditional), waves: w1 w2
Actors: 151                           Periods: 4
Groups: 4 (variable class), actors per group: 45, 37, 33, 36; waves per group: 2, 2, 2,
     2

------------------------------------------------------------------------------
    w1_to_w2 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.246167   .0820687   -27.37   0.000    -2.407018   -2.085315
 reciprocity |   1.862864   .1569308    11.87   0.000     1.555285    2.170443
   transtrip |   .4655751   .0541992     8.59   0.000     .3593466    .5718035
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):

------------------------------------------------------
             |   period1    period3    period5    period7
-------------+----------------------------------------
        rate |    6.4989     5.3821     2.2893     8.2490
          se |    0.9714     1.4169     0.5299     2.1306
------------------------------------------------------
Overall maximum convergence ratio: 0.017 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

`nwsaom` says in the header that there are four groups with 45, 37, 33 and 36 actors and two waves each. There is one coefficient table: the weights of the three effects are common to all 151 pupils. Ties in these classes are reciprocated and closed beyond what density alone gives (reciprocity 1.86, transitive triplets 0.47, both far from zero relative to their standard errors). There are *four* rates, one for each class and its single period: pupils in class 4 (rate 8.2) had about three and a half times as many opportunities to change a tie between the two waves as pupils in class 3 (2.3), the rates of classes 1 and 2 lie in between (6.5 and 5.4). The rates are named `period1`, `period3`, `period5` and `period7` because RSiena numbers the switches (2, 4, 6) as periods that are skipped. The stored matrices say which column belongs to which group:

```
. display "groups: " e(ngroups) "   actors: " e(N)
groups: 4   actors: 151
. matrix list e(groupn)
e(groupn)[1,4]
    c1  c2  c3  c4
r1  45  37  33  36
. matrix list e(rates_group)
e(rates_group)[1,4]
    period1  period3  period5  period7
r1        1        2        3        4
. matrix list e(rates_period)
e(rates_period)[1,4]
    period1  period3  period5  period7
r1        1        3        5        7
```

## 11.5 Joint model or meta-analysis?

The same four classes fitted one by one give four sets of estimates. `nwsaom meta` combines them (Section 10.8). The listing shows the separate fits, the meta-analysis (abridged to its first block), and the joint fit of the previous section next to them.

```
. forvalues s = 1/4 {
.     quietly nwsaom, wave1(school‘s’_w1) wave2(school‘s’_w2) outdegree reciprocity
    transtrip seed(5)
.     estimates store school‘s’
. }
. estimates table school1 school2 school3 school4, b(%7.3f) se(%7.3f)
-----------------------------------------------------
    Variable | school1   school2   school3   school4
-------------+---------------------------------------
   outdegree |  -2.274    -2.343    -3.331    -2.011
             |   0.122     0.214     0.428     0.141
 reciprocity |   1.571     2.611     2.266     1.554
             |   0.238     0.385     0.675     0.302
   transtrip |   0.469     1.133     1.043     0.494
             |   0.066     0.284     0.291     0.110
-----------------------------------------------------
                                       Legend: b/se
. nwsaom meta school1 school2 school3 school4
IWLS estimates (Snijders and Baerveldt 2003)
------------------------------------------------------------------------------
      Effect |  N      Mean Std. err.        t   P>|t|      SD       Q     P>Q
-------------+----------------------------------------------------------------
   outdegree |  4   -2.3922    0.2552    -9.37   0.003  0.5104    9.44   0.024
 reciprocity |  4    1.8994    0.2698     7.04   0.006  0.5397    6.51   0.089
   transtrip |  4    0.6963    0.1711     4.07   0.027  0.3422    8.53   0.036
------------------------------------------------------------------------------
. matrix list cmp, format(%7.3f)
cmp[3,3]
            outdegree  reciprocity    transtrip
avg_classes    -2.490        2.001        0.785
  meta_IWLS    -2.392        1.899        0.696
      joint    -2.246        1.863        0.466
```

The three rows are the simple average of the four class estimates, the random-effects mean of the meta-analysis, and the joint estimate. They agree on the picture (negative density effect, strong reciprocity, closure) and differ in detail. The standard errors tell the other part of the story: those of the joint model (0.08, 0.16, 0.05) are much smaller than those of the mean in the meta-analysis (0.26, 0.27, 0.17), because the joint model assumes that all classes have the same parameters, while the meta-analysis treats the parameters as varying between classes (estimated between-class standard deviations of 0.51, 0.54 and 0.34) and does not take the sameness for granted. The heterogeneity test $Q$ says that the classes do differ in outdegree and in transitive closure ($p = 0.024$ and 0.036). When such a test is clearly significant, the joint model's small standard errors are too optimistic.

**When to use which.**

- Use **joint estimation** when you believe that the groups follow the same process (the same parameters) and want one estimate with its standard error, when the groups are too small to be fitted one by one, or when you want a model that goes beyond the effects of single groups (a behavior co-evolving in all groups, Section 11.6).
- Use the **meta-analysis** when the question is how much the groups differ, when you want to

describe the population of groups (a mean effect and a between-group standard deviation), or when the groups differ in features that are not captured by the model. Each group must be large enough for its own fit to converge.

- You can do both: the joint model is the first thing to try, the heterogeneity test of the meta-analysis checks it, and the joint model can then be extended by letting some effects differ. A joint model in which reciprocity may differ for class 2 adds an indicator of the class and its interaction with reciprocity:

```
. generate byte c2 = (class == 2)
. nwsaom, waves(w1 w2) groups(class) outdegree reciprocity transtrip egox(c2) interact(
    egox_c2#reciprocity) seed(5)
------------------------------------------------------------------------------------------

                   w1_to_w2 | Coefficient  Std. err.      z    P>|z|     [95% conf.
    interval]
----------------------------+-------------------------------------------------------------

                  outdegree |  -2.235427   .0821592   -27.21   0.000    -2.396456
    -2.074398
               reciprocity |   1.864939   .1691134    11.03   0.000     1.533482
    2.196395
                   egox_c2 |   .1400424   .1937284     0.72   0.470    -.2396584
    .5197431
                 transtrip |   .4968187    .057962     8.57   0.000     .3832153
    .6104222
interact_egox_c2_reciprocity |   1.100587   .4208535     2.62   0.009     .2757298
    1.925445
------------------------------------------------------------------------------------------

(output omitted)
Overall maximum convergence ratio: 0.193 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    reciprocity (-0.115)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the $t$-ratio of `reciprocity` is $-0.115$); before it is reported it would be restarted from its estimates with `theta0()`, as Chapter 9 explains. The interaction coefficient (1.10, $z = 2.6$) is the difference of the reciprocity effect between class 2 and the other classes: pupils in class 2 reciprocate more than the others. (The main effect of the indicator, `egox_c2`, is the difference in the tendency to send ties.)

## 11.6 Co-evolution of a behavior in several groups

A behavior is declared as usual with `behavior()`; its effects are shared by all groups, and its rate (like the rate of the network) is estimated per period of each group. The example uses the 50 pupils of the Teenage Friends and Lifestyle Study (the data `s501`–`s503` and `s50a` of RSiena; Michell and Amos 1997; Pearson and West 2003) *cut into three groups* of 15, 20 and 15 pupils (pupils 1–15, 16–35, 36–50, ties between the groups removed). This is a stand-in for three classes, not a real multi-group study; the data are the ones against which the implementation was validated. The behavior is alcohol use (1 to 5) at the three waves (`alc1 alc2 alc3`). The network effects `behegox` and `behaltx` are selection (a pupil's alcohol use affecting the ties he or she sends and receives), `avalt` is influence (the average alcohol use of the friends), `linear` and `quadratic` are the shape.

```
. nwsaom, waves(g1 g2 g3) groups(group) outdegree reciprocity transtrip ///
```

```
>     behavior(alc1 alc2 alc3) linear quadratic avalt behegox behaltx seed(1)

SAOM co-evolution (Method of Moments), waves: g1 g2 g3
Actors: 50                              Periods: 6
Behavior: alc1 alc2 alc3
Groups: 3 (variable group), actors per group: 15, 20, 15; waves per group: 3, 3, 3

------------------------------------------------------------------------------
    g1_to_g3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |   -2.97652   .2708628   -10.99   0.000    -3.507401   -2.445639
 reciprocity |   2.932225   .4010968     7.31   0.000      2.14609     3.71836
   transtrip |   .6524343   .1433909     4.55   0.000     .3713933    .9334754
     behegox |   .0226293   .1657629     0.14   0.891      -.30226    .3475187
     behaltx |   .0072251   .1742447     0.04   0.967    -.3342883    .3487385
  beh_linear |   .3868227    .177321     2.18   0.029     .0392798    .7343655
beh_quadra~c |  -.2763856   .1344151    -2.06   0.040    -.5398343   -.0129369
   beh_avalt |   .5175959   .3764368     1.37   0.169    -.2202067    1.255399
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):

------------------------------------------------------------------------------
             |   period1    period2    period4    period5    period7    period8
-------------+----------------------------------------------------------------
network_rate |    3.6424     1.9581     5.6745     2.3518     1.1541     2.3119
          se |    1.2967     0.9412     1.6757     0.6873     0.4610     1.0138
behavior_r~e |    1.2842     1.4431     1.8853     2.0285     0.6628     1.4247
          se |    0.7216     0.6579     0.6827     0.8332     0.3364     0.6035
------------------------------------------------------------------------------
Overall maximum convergence ratio: 0.211 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for
    rate_period2 (0.101)
Convergence is good only if both criteria are met. Refit starting from these estimates
    with theta0(); see nwsaom.
```

This fit has not converged by the two criteria (the $t$-ratio of the rate of period 2 is 0.101); before it is reported it would be restarted from its estimates with `theta0()`, as Chapter 9 explains. Six periods, six network rates and six behavior rates: groups 1, 2 and 3 have the periods 1–2, 4–5 and 7–8. The effects are shared. The estimated influence of the friends' alcohol use (`beh_avalt` 0.52, $z = 1.4$) and the selection effects are not distinguishable from zero in these small groups.

**Groups observed at different waves.** The groups need not have been observed at the same occasions. Suppose that group 1 was observed at all three waves, group 2 only at the first two and group 3 only at the last two. The option `gwaves()` gives the waves of the groups by their position in `waves()`:

```
. nwsaom, waves(g1 g2 g3) groups(group) gwaves(2: 1 2 | 3: 2 3) outdegree reciprocity
    transtrip seed(1)

SAOM (Method of Moments, conditional), waves: g1 g2 g3
Actors: 50                              Periods: 4
Groups: 3 (variable group), actors per group: 15, 20, 15; waves per group: 3, 2, 2

------------------------------------------------------------------------------
    g1_to_g3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.676822   .2586404   -10.35   0.000    -3.183748   -2.169896
 reciprocity |   2.617165   .4142432     6.32   0.000     1.805263    3.429067
```

```
   transtrip |   .5477705   .1367602     4.01   0.000     .2797255    .8158156
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):

------------------------------------------------------
             |   period1    period2    period4    period6
-------------+----------------------------------------
        rate |    3.1403     1.6238     5.2689     1.9980
          se |    1.0861     0.6607     1.3756     0.7814
------------------------------------------------------
Overall maximum convergence ratio: 0.064 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met

. matrix list e(groupwaves)
e(groupwaves)[1,3]
    c1  c2  c3
r1   3   2   2
. matrix list e(rates_period)
e(rates_period)[1,4]
    period1  period2  period4  period6
r1        1        2        4        6
```

Four periods remain (two in group 1, one each in groups 2 and 3), numbered 1, 2, 4 and 6 because the period numbers of the combined series are kept: period 3 is the switch from group 1 to group 2, period 5 the switch from group 2 to group 3. The data of the actors must of course exist at the waves used: for group 3 the first wave is not used, and its values are ignored.

## 11.7 After the fit: time heterogeneity and score tests

`estat timetest` and `estat score` work after a joint fit (a model without covariate effects is fitted unconditionally here; `estat timetest, fixrates` gives RSiena's `sienaTimeTest`). After a fit with groups the periods are the periods of all groups, so the test of time heterogeneity answers two questions: whether the effects are the same in all periods, and whether they are the same in all groups, because the periods of the same group are the periods of one class. The output has two more tables than for one group: a test per group and a test per period.

```
. nwsaom, waves(g1 g2 g3) groups(group) outdegree reciprocity transtrip egox(alc) altx(
    alc) unconditional seed(1)
(output omitted: outdegree -2.986, reciprocity 2.940, altx_alc 0.009, egox_alc -0.052,
    transtrip 0.655; six rates)
. estat timetest, fixrates

Test of time heterogeneity (score-type test)
H0: the parameter of each effect is the same in all periods

------------------------------------------------------
      Effect |          chi2        df        Prob > chi2
-------------+----------------------------------------
   outdegree |         3.081         5             0.6875
 reciprocity |         2.857         5             0.7220
    altx_alc |        10.791         5             0.0557
    egox_alc |         8.193         5             0.1459
   transtrip |        10.335         5             0.0663
-------------+----------------------------------------
       Joint |        36.338        25             0.0666
------------------------------------------------------
```

```
H0: the parameters of the periods of a group are the same as in the other groups
-------------------------------------------------------
       Group |          chi2        df        Prob > chi2
-------------+-----------------------------------------
           1 |         2.512         5             0.7747
           2 |        14.204        10             0.1639
           3 |        19.699        10             0.0322
-------------------------------------------------------

Test of each period (all effects)
-------------------------------------------------------
      Period |          chi2        df        Prob > chi2
-------------+-----------------------------------------
           1 |         2.512         5             0.7747
           2 |         6.014         5             0.3049
           3 |         9.393         5             0.0944
           4 |         6.753         5             0.2397
           5 |        16.287         5             0.0061
           6 |         4.428         5             0.4896
-------------------------------------------------------
```

(The table of the period-specific deviations follows in the output and is omitted here.) The joint test is borderline ($p = 0.067$); the effects of the alter's alcohol use (`altx_alc`) and of closure (`transtrip`) vary most, group 3 is the group that differs from the others ($p = 0.032$), and period 5, which is the first period of group 3, is the period where the model fits worst ($p = 0.006$). The periods are numbered 1 to 6 here (not with RSiena's numbers of the rates). The test per group follows RSiena (for the first group it is the test of the sum of the period dummies of each effect). The returned matrices are `r(group)` and `r(period)`. In RSiena 1.6.14 the test printed under the name *group test* is the per-period test.

The score test works as in one group (Chapter 10): fit the restricted model with the effects held fixed at zero, then ask if the fixed effects are needed.

```
. nwsaom, waves(g1 g2 g3) groups(group) outdegree reciprocity transtrip egox(alc) altx(
    alc) fix(egox(alc) altx(alc)) unconditional seed(1)
(output omitted)
. estat score

Score-type test of fixed effects
H0: each effect below equals its fixed value

------------------------------------------------------------------
      Effect |  Fixed at      chi2   df  Prob > chi2  One-sided z
-------------+----------------------------------------------------
    altx_alc |    0.0000     0.023    1       0.8782       -0.153
    egox_alc |    0.0000     0.131    1       0.7178       -0.361
-------------+----------------------------------------------------
       Joint |               0.156    2       0.9248
------------------------------------------------------------------
```

The alcohol effects are not needed in the network model ($p = 0.92$). In the fit of the restricted model the program prints that the convergence criteria are not met for the two fixed effects: their t-ratios are exactly the quantity the score test examines, not a failure of the estimation.

## 11.8 Agreement with RSiena

The examples were fitted in RSiena 1.6.14 (R 4.6.0) with `sienaGroupCreate` and `siena07`, the same data and effects, ten runs of each program with seeds 1 to 10. Table 11.2 gives the means over the ten runs. The two programs agree to within the simulation error of a run: the largest difference of an effect is 0.010 (average alter in example 3a), the largest of a rate 0.029 (example 3b, period 1), and the standard deviation of the estimates over seeds is about 0.002–0.02 for the effects; the standard errors agree as well.

Table 11.2: Joint estimates of `nwsaom` and RSiena 1.6.14, means over ten runs each.

| Example | Parameter | `nwsaom` | RSiena | Difference |
|---|---|---|---|---|
| 1 (four classes) | outdegree | −2.246 | −2.248 | 0.002 |
| | reciprocity | 1.866 | 1.870 | −0.004 |
| | transitive triplets | 0.465 | 0.464 | 0.001 |
| | rates (periods 1, 3, 5, 7) | 6.50, 5.37, 2.30, 8.16 | 6.51, 5.38, 2.31, 8.16 | $\leq 0.010$ |
| 3a (co-evolution) | outdegree | −2.981 | −2.982 | 0.001 |
| | reciprocity | 2.935 | 2.936 | −0.001 |
| | transitive triplets | 0.655 | 0.657 | −0.002 |
| | behavior ego (`behegox`) | 0.028 | 0.026 | 0.002 |
| | behavior alter (`behaltx`) | 0.007 | 0.006 | 0.000 |
| | linear shape | 0.386 | 0.384 | 0.002 |
| | quadratic shape | −0.276 | −0.273 | −0.003 |
| | average alter | 0.523 | 0.512 | 0.010 |
| | network rates (6) | 1.12 to 5.69 | 1.13 to 5.70 | $\leq 0.016$ |
| | behavior rates (6) | 0.65 to 2.02 | 0.65 to 2.02 | $\leq 0.014$ |
| 3b (waves 3, 2, 2) | outdegree | −2.672 | −2.677 | 0.005 |
| | reciprocity | 2.611 | 2.615 | −0.004 |
| | transitive triplets | 0.553 | 0.556 | −0.003 |
| | rates (periods 1, 2, 4, 6) | 3.15, 1.62, 5.27, 2.01 | 3.18, 1.63, 5.28, 2.02 | $\leq 0.029$ |

For example 3b RSiena 1.6.14 itself stops with an error for a group object whose groups have different numbers of waves, although its manual allows them. The reference is therefore the equivalent object in which the first group is split into two groups of the same actors (waves 1–2 and 2–3) and the others are as before; without covariates and without effects that use group constants the model is the same, period by period.

Table 11.3: Tests after the joint fit (example 4): $c$ is the chi-squared statistic, means over ten runs.

| Test | `nwsaom` | RSiena | Difference |
|---|---|---|---|
| Joint time test (25 d.f.) | 37.54 | 38.67 | −1.13 |
| Effect: outdegree / reciprocity (5 d.f.) | 3.19 / 2.97 | 3.19 / 3.01 | 0.00 / −0.04 |
| Effect: closure / altx / egox (5 d.f.) | 11.24 / 10.13 / 8.26 | 11.60 / 9.91 / 8.28 | −0.36 / 0.22 / −0.02 |
| Periods 1 to 6 (5 d.f.) | 2.3, 5.6, 8.8, 7.0, 17.4, 4.6 | 2.3, 5.5, 9.0, 6.6, 18.3, 4.5 | $\leq 0.86$ |
| Score test of altx and egox (2 d.f.) | 0.17 | 0.18 | −0.01 |

Beyond these examples the implementation was compared with RSiena for a changing covariate, a rate covariate, an interaction, a dyadic covariate, joiners (composition change), missing ties, and a co-evolving behavior with average-alter, average-similarity, egoX/altX/simX and rate-covariate effects, in conditional and unconditional estimation (Chapter 12).

## 11.9 What is not available with groups

The following stop with a message when combined with `groups()`:

- maximum likelihood (`ml`) and Bayesian estimation (`nwsaom bayes`), which run in separate parts of the program that do not yet know the group chain;
- two-mode networks and `nwsaom multinet` and `nwsaom multiplex` (several dependent networks);
- `estat` after a fit with several behaviors in `behavior()` (as in a single group);
- `structural()` and the structural codes 10 and 11 in the wave networks (see Section 13.2);
- `tvdyadcov()` and the other time-varying dyadic covariates;
- `estat gof`, `estat simulate` and `estat importance` after a fit with `ratecov()` together with a behavior or with `behratecov()`/`behrate()` (`estat importance` also after `present()`).

Available with groups since version 1.3: `estat gof`, `estat simulate` (and `nwsaom simulate`) and `estat importance`, each for one group (`group(#)`) or for all groups in turn, `estat gof` also with the p-values of the groups combined (inverse-normal method); the centered degree effects (`inpopc`, `outpopc`, `outactc`, `inactc`), `ownx()` effects and structural equivalence, whose data constants (average degrees, `structmean`) are, as in RSiena, those of each group; `joinat()`/`leaveat()` (times on the scale of the waves of the actor's group); `behrate()`; and non-directed networks (`symmetric`, `symtype()`). Since version 1.4 several co-evolving behaviors can be given in `behavior()` (`behavior(a1 a2 a3 : linear quadratic avalt | b1 b2 b3 : linear quadratic avalt)`): every behavior has its own constants in every group (overall mean, similarity mean) and its own rate in every period of every group. Since version 1.5 `netbeh()` works with groups: the network effects of the dependent behavior as the covariate (for example `netbeh(egoxaltx)`, RSiena's `egoXaltX` of the behavior), with the behavior centred by the mean of its own group, as RSiena does for the data objects of a joint model. Also since version 1.5, joint models with many actors (about 800 or more) and a per-actor behavior covariate such as `behfrom()` run (the model description is handed to the plugin in pieces; this needs the plugin of 1.5.0 or later) and start in seconds: a model of 15 groups and 810 actors starts after about 3 seconds instead of about 4.5 minutes. Constraints on effects (`higher`, `disjoint`, `atLeastOne`) are not supported with groups in RSiena either.

**Validation since version 1.4.** Two co-evolving behaviors (alcohol and smoking) in the three groups of the examples above were compared with RSiena 1.6.14, ten runs of each program: the effects of the network and of both behaviors and all eighteen rates (a network rate and two behavior rates in each of six periods) agree within the simulation error.

**Validation since version 1.5.** `groups()` with `netbeh()` was compared with `siena07` on five classes of the Dutch school data (Baerveldt; Snijders's multi-group example): the largest difference of an estimate is 0.08 of its standard error.

**Validation since version 1.3.** The per-group constants, `joinat()`/`leaveat()`, `behrate()`, non-directed networks (model types 2, 3 and 6) and the postestimation commands were compared with RSiena 1.6.14 (ten runs of each program; `sienaGOF` for `estat gof`). For model type 6 several of RSienaś own runs did not converge, so that comparison is coarse.

**Missing behavior values and leavers.** With groups, a model with missing behavior values together with the selection effects `behegox` or `behaltx` agrees with RSiena: as in RSiena, a tie does not count in the statistic of these effects when the covariate value was missing at the start of the period (checked with ten seeds on three groups with 20 missing values; every estimate within 0.04 standard errors). The comparison of composition change in groups used actors who join; actors who leave (together with a behavior) were not compared with RSiena.

**Files.** The data files, the do-file and the log of these examples are on the website, in the tutorial *Several groups at once* (https://nwcommands.org/nwsaom/).

# Chapter 12

# Validation against RSiena

`nwsaom` is a re-implementation, not a wrapper: the simulation, the estimation and the statistics were written by reading RSiena's documentation and its C++ and R sources. A re-implementation is only worth using if it can be checked against the original. This chapter describes how that was done, what the checks showed, and where `nwsaom` and RSiena disagree or where RSiena itself fails. All reference results were produced with RSiena 1.6.6 (the installed version; the source tree used for reading code is the later 1.6.13, which adds one effect not present in 1.6.6). Appendix D adds the comparison of the whole estimator: the algorithm and its settings, the simulation at fixed parameters, the estimates over seeds on 27 models, and where the difference in run time comes from.

## 12.1 Method

**Data.** The reference data are RSiena's own `s50` data (three waves of friendship among 50 pupils, `s501`–`s503`, with the behaviors alcohol `s50a` and smoking `s50s`), which ship with RSiena and are available in `nwsaom` as the `glasgow` network (`nwwebuse glasgow`; the node order differs, the results do not). Because RSiena ships no two-mode data, the two-mode checks use a $40 \times 20$ three-wave data set simulated from known parameters with RSiena itself; non-directed checks use a symmetrised version of `s50`.

**Step 1: statistics.** For each effect the statistic that `nwsaom` computes on the observed waves is compared with the target that `siena07` reports for the same model. These must be *identical* (not merely close); a mismatch means that the definition of the effect differs. For many effects the statistic was also compared between the two internal engines of `nwsaom` (the Mata reference code and the native C plugin).

**Step 2: estimates.** The same model is fitted by `siena07` and by `nwsaom` with several seeds each (three to five, or more where an effect is noisy), restarting from the estimates while the convergence ratio is too large. The estimates must agree within the seed-to-seed noise of the two programs, typically stated as a fraction of RSiena's standard error. Rates are compared in the same way. This step checks the simulation, that is, the contribution of an effect to the choice probabilities.

**Step 3: simulation studies.** Where RSiena's own estimation is unstable, as for non-directed co-evolution, replicates were simulated by RSiena from known parameters and re-estimated with `nwsaom`, to check recovery of the truth rather than agreement with a failing reference.

**Step 4: other ports.** For the multi-group meta-analysis (`nwsaom meta`, RSiena's `siena08`) the Mata port was compared with RSiena's own `iwlsm`, `maxlik` and `confint` functions on made-up estimates.

A good agreement does not make an effect *correct* in every setting: the checks cover one or two data sets per effect family, and the tolerance is seed noise. The "validated" flag in the effects tables (Appendix) means that steps 1 and 2 were passed for at least one data set, unless the table says otherwise.

## 12.2 Regression tests that ship with the package

Two test files in `cscripts/` embed RSiena reference numbers and run with the package: `test_nwsaom_rsiena.do` (network-only models) and `test_nwsaom_coev_rsiena.do` (co-evolution). Their reference values are means over five RSiena seeds of `siena07` runs on `s50` (restarted with `prevAns` while the convergence ratio exceeded 0.25), and the tests require every estimate, the rates included, to lie within 0.25 RSiena standard errors of the reference, standard errors within a factor of 1.5, and the overall convergence ratio below 0.3. The five-seed validation behind them found at most 0.06 standard errors for network-only models (Chapter 9). The script `cscripts/run_nwsaom_tests.sh` runs a `quick` tier (about a minute) or the `full` tier (about 35 minutes sequentially, including these RSiena tests).

## 12.3 Summary of what was validated

Table 12.1 lists the families that were checked, with one or two numbers as examples. "Target" is the observed statistic, "est." the estimates in the order given, with `nwsaom` first. All agree within seed noise unless stated in the section on differences below.

Table 12.1: Families validated against `siena07`, with example numbers.

| Family | Example result |
|---|---|
| Estimation algorithm, network-only | Five models on `s50`, five seeds each: every parameter, rates included, within 0.07 RSiena s.e. of RSiena's default (conditional) output; within 0.06 of `cond = FALSE` with `unconditional`. Conditional and unconditional RSiena estimates differ by up to 0.3 s.e. from each other. |
| Estimation, co-evolution | Joint rate estimation: within 0.05 s.e. of RSiena's default. Real-data outdegree −2.760 in both programs after the per-period Jacobian change. |
| Endowment and creation, network | gwesp targets −67.98/80.71, cycle4 −19.75/32.5; balance −155.469/185.886, estimates density −1.73, reciprocity 2.33, endow −0.11, creation 0.72 within seed noise; transTrip −74/94, cycle3 −17.333/31, transMedTrip −74/93, transRecTrip −41/61. |
| Balance with missing ties | Targets −114.016/128.649 against RSiena's −114.02/128.65; rates 5.97/5.02 against 5.93–6.02/5.05–5.10; endow 0.11 against 0.13–0.15, creation 0.57 against 0.55–0.57. |
| Missing covariate values | Eight of 50 covariate values missing: density, reciprocity, altX, egoX and simX agree; simX 0.79 against RSiena's 0.78 (it was 1.02 before missing values were handled). |
| Composition change (joiners, leavers) | Estimates within noise over three seeds, with and without a rate covariate, for fractional join/leave times. |
| Covariate effects, one-mode | absDiffX and egoXaltX: targets 216 and 134.387; estimates −0.141 to −0.146 against RSiena's −0.145 to −0.149. |
| Covariate × triad, popularity, activity | Port of `calculateContribution()` and `tieStatistic()` including their special cases; targets and estimates equal. |
| Structural rate effects | Nine effects, two seeds each: outRate coefficient 0.297 against 0.300, outdegree −2.180 against −2.176; targets 594, 592, 340, 70.63, 272.67, 78.20, 261.51, 106.27, 185.94. |

| Family | Example result |
|---|---|
| Reciprocated-degree effects | Four targets (532, 335.803, 505, 319.113); estimates e.g. $-2.19/3.22/-0.238$ against $-2.19/3.23/-0.243$. |
| Own-network effects (`ownx()`) | 19 effects agree (statistics 42.19, 44.252, 225, 184, 266, 206... equal the targets). |
| Dyadic covariates | Targets and estimates, including missing cells. |
| Two-mode networks | Statistics equal siena07's targets on the $40 \times 20$ data (22 statistics for the extra effects); estimates and rates agree. With a co-evolving behavior: rates 4.31/3.44 against 4.32/3.42, simEgoInDist2 $-1.13$ against $-1.135$. |
| Non-directed networks | 31 constant-covariate and 7 changing-covariate effects, e.g. egoSqX $-0.231/0.230$ against $-0.223/0.219$, sameXOutAct $0.177/-0.071$ against $0.165/-0.073$. |
| Several behaviors | 15 parameters (alcohol and smoking, outdegree, reciprocity, transTrip, linear, quadratic, avAlt) within noise over three seeds; weakest: smoking avAlt 1.92–1.97 against 2.01–2.06 (s.e. about 0.5). |
| Behavior effects | quad_cc: linear 1.791, quadratic $-0.448$, avAlt 1.057 against $1.790/-0.450/1.071$; avXAlt, totXAlt, avSameXAlt and effFrom agree; 30 behavior-as-covariate network effects in symmetric networks agree. |
| Behavior as covariate, selection | netbeh(samex): $-2.53/2.79/0.556$ against $-2.52/2.78/0.554$. |
| Several dependent networks (multinet) | Targets equal, estimates agree within seed noise, with the exceptions listed below; 20 effects validated for non-directed members. |
| Symmetric co-evolution | Simulation study, 40 replicates from known parameters: network rate 0.612 (true 0.6), behavior rate 1.60 (1.5), outdegree $-1.44$ ($-1.4$), linear median 0.37 (0.3), avAlt median 1.29 (1.5). |
| Multi-group meta-analysis (`nwsaom meta`) | 17 numbers identical to five decimals with RSiena's functions. |
| Joint estimation of several groups (`groups()`) | RSiena 1.6.14 (`sienaGroupCreate`, `siena07`), ten runs of each program, conditional and unconditional: the estimates, standard errors and rates agree within the simulation error for density, reciprocity, transitive triplets, 3-cycles, egoX, altX, simX, balance, a changing covariate, a rate covariate, an interaction, a dyadic covariate, composition change, missing ties and a co-evolving behavior (linear and quadratic shape, average alter, average similarity, egoX/altX/simX of the behavior, a behavior rate covariate). Largest difference of an effect in the examples of Chapter 11: 0.008 (four classes: 0.002). Time test: joint $c$ 37.9 against 38.7, by effect and period within 0.5; score test 0.17 against 0.18. |
| Rate effects of behaviors and two-mode networks | Structural rate effects of two-mode and non-directed networks, `ratecov()` with changing covariates and with the behavior as covariate, and the rate effects of a co-evolving behavior (`behrate()`, `behratecov()`): observed statistics equal `siena07`'s and estimates within seed noise. |
| Two-mode member in `nwsaom multinet` | The 21 two-mode cross effects of RSiena and their counterparts for a one-mode network that depends on a two-mode one: targets equal, estimates within the Monte Carlo error of both programs. |
| Behavior in `nwsaom multinet` | Behavior effects of both networks, covariate and shape effects, endowment and creation of behavior effects, interactions of behavior effects, missing behavior values, and `netbeh:` effects (53 of 54 equal `siena07`'s estimates). |
| `estat importance` | Agrees with `sienaRI` to four decimals for one-mode, non-directed (forcing model) and two-mode networks and for the behavior effects of co-evolution fits. |
| `estat score` after `nwsaom multinet` | Cross-network effect fixed at 0: $\chi^2$ 0.14 to 0.25 against RSiena's 0.19 to 0.25; behavior `avAlt` fixed: 11.4 to 14.0 against 11.9 to 13.5. |
| `estat timetest` | With `fixrates` equal to `sienaTimeTest`; by default equal to the score test of a fitted fixed time dummy. After `nwsaom multinet` the joint test and the tests of all effects agree in this way; the separate test of the second network's outdegree differs between the two variants (default 1.1 to 1.3, with `fixrates` 0.2 to 0.35) because that effect is strongly correlated with the rate. |

| Family | Example result |
|---|---|
| Maximum likelihood (`ml`) | Glasgow waves 1 and 2, outdegree, reciprocity and transitive triplets: −2.65, 2.22 to 2.24, 0.69 to 0.70 against RSiena's −2.66 and −2.63, 2.22 and 2.19, 0.71 and 0.70. With a co-evolving behavior the estimates are within 0.1 standard errors. |
| Bayesian estimation (`nwsaom bayes`) | Exact enumeration on a four-actor case: expected numbers of ministeps 16.249 and 32.385 against the exact 16.258 and 32.344. Glasgow, outdegree, reciprocity and transitive triplets: posterior means −2.656, 2.256, 0.686 (standard deviations 0.109, 0.175, 0.069) against `siena07` maximum likelihood −2.661, 2.224, 0.713 (standard errors 0.139, 0.181, 0.083) and −2.632, 2.187, 0.702; with alcohol covariates the posterior means are within one maximum-likelihood standard error. |

## 12.4 Where results differ, or RSiena itself is degenerate

The following cases are the honest edge of the validation. They fall into four kinds.

### 12.4.1 Effects where RSiena crashes or throws

- `sameXOutAct` and `diffXOutAct` in a *two-mode* network: `siena07` aborts the R process. They are not offered for two-mode networks.
- `avGroupEgoX` and `totGroupEgoX` with a constant covariate in a two-mode network: RSiena throws an exception. They are available with a changing covariate.
- `inPop_ego` for bipartite data throws a C++ exception in `siena07`; it is not offered.
- `avAlt.2M.tie` with its default parameter (3) throws an error in R; with parameter 1 or 2 it works. For a non-directed network with a second network `siena07` throws.
- Several fits of non-directed *co-evolution*: RSiena's own fit of the symmetrised `s50` with alcohol was unstable (three of four seeds aborted at `thetaBound`; the one that finished took 206 s, fixed avAlt at 0 and reported a network rate of 5.6, where `nwsaom` gave 0.55). This is why the simulation study above was done instead.

### 12.4.2 Effects that are degenerate or collinear in both programs

- `higher` in a non-directed network reproduces the number of ties, so it is collinear with the density, in both programs.
- `outTrunc` and `outThreshold` are degenerate in both in a non-directed network.
- `inPopX` and `outActX` in a two-mode network sit on a flat ridge in both programs; `altInDist2` estimates diverge in both (the statistic is equal).
- The `avAltW` and `avSimW` versions with parameter 2, and the distance-2 and interaction-covariate versions of the behavior effects, are degenerate in `siena07` too. The observed statistics are equal; the estimates are not informative.
- `avAlt.2M.tot` (multinet) is poorly identified (s.e. about 1.7): −0.3 against −0.8, the target is 16.98 and the simulated statistic at RSiena's estimate is 17.6 for `nwsaom` against 18.1.
- A constant behavior effect equals minus the behavior-rate statistic; under unconditional estimation its row is collinear with the behavior rate. `siena07` prints arbitrary numbers; `nwsaom` stops with rc 505, and the effect is not claimed.

- `sameXInPop` and `homXOutAct` with a continuous covariate in a two-mode network reproduce the outdegree statistic: `siena07` fits them with arbitrary estimates, `nwsaom` stops with rc 505.
- The `inAct` endowment/creation split is nearly unidentified in RSiena too (estimates ±34, one seed errors); `nwsaom` stops with rc 498 on the runaway.

### 12.4.3 Open disagreements (not claimed as validated)

- `degAbsDiffX` and `degPosDiffX` in a non-directed network *with the dependent behavior as covariate*: the observed statistics equal `siena07`'s (814 and 812) and the behavior parameters agree. The estimates of the two effects themselves are badly determined in both programs (standard errors of 0.9 to 3.4), so that they differ between seeds by more than the standard error; only the statistics are claimed.
- `outMore(2)` in a *non-directed* network: the observed statistic equals `siena07`'s (578), but `siena07` reaches its `thetaBound` on the test data, so the estimates cannot be compared.
- Cycle4 endowment/creation estimates differ from RSiena by less than 0.1 with a standard error of 0.55 (six seeds each), judged to be noise.
- `estat gof`: RSiena reuses networks stored during the fit; `nwsaom` simulates afresh (Chapter 9). The p-value of the correlation diagnostic in `nwsaom meta` uses the $t$ approximation, RSiena the exact (Edgeworth) value.
- The simulated statistic of a few endowment/creation effects uses the union of the dyads missing at either wave, where RSiena treats start-missing and end-missing dyads differently. The observed target is exact; the simulated statistic differs for a handful of ties. This is a stated approximation.

### 12.4.4 Several groups

- RSiena 1.6.14 stops with an error (`INTEGER()` applied to a non-list in `initializeFRAN`) for a group object whose groups have different numbers of waves, although its manual allows this. `gwaves()` was therefore checked against the equivalent object of two-wave groups (uncentered covariates, no effects with group constants).
- With `groups()`, missing behavior values together with the selection effects `behegox` or `behaltx` agree with RSiena (since 1.2.0, as in RSiena a tie does not count in the statistic when its covariate value was missing at the start of the period; checked again for 1.5.12 with ten seeds on three groups with 20 missing values: every estimate within 0.04 standard errors). The comparison of composition change in groups used joiners; leavers (with a behavior) were not compared and are not claimed.
- Constraints (`higher`, `disjoint`, `atLeastOne`) are not supported by RSiena with groups, and `nwsaom` does not offer them.

### 12.4.5 Constants that differ by construction

In a two-mode network the raw statistics of `outInv` and `outSqInv` differ from RSiena's by a constant, because the events count as outdegree-0 nodes in the sum. The constant cancels against the simulation, and the estimates agree.

## 12.5 Reliability compared with RSiena

Robustness is part of validation. RSiena fails on a share of seeds for sparse effects; `nwsaom` does as well. For the transitive reciprocated triplet endowment/creation split on `s50`, RSiena failed 3 of 200 seeds (seeds 51–250), and `nwsaom` 1 of 50 after the per-period Jacobian and phase-1 doubling were adopted (Chapter 9). The cause of the earlier, larger failure rate was found by reading `derivativeFromScoresAndDeviations()`: RSiena sums the covariance of each *period's* scores and deviations, where `nwsaom` had taken one covariance of the period sums, adding noisy cross-period terms to the Jacobian.

## 12.6 How to reproduce

The scripts that produced these comparisons (named `dev/saom_rsiena_*.R` and `.do`, one pair per effect family, plus `dev/saom_sim_*` for the simulation studies) are *not* part of the repository. They are not part of the installed package, and some depend on RSiena installed locally. What is public is the recipe, and it is short.

1. Install RSiena 1.6.14. Build the data with `sienaDataCreate()` from `s501`–`s503`, `s50a`, `s50s`; add covariates as `coCovar` or `varCovar`.

2. Fit the model with `siena07(sienaAlgorithmCreate(projname = NULL, seed = s, cond = FALSE))` for several seeds `s`, with `batch = TRUE`; restart with `prevAns` while `tconv.max` exceeds 0.25. Keep the targets (`ans$targets`), the estimates, the standard errors and the rates.

3. Fit the corresponding `nwsaom` model on `nwwebuse glasgow` (and, for behaviors, the alcohol or smoking variables), with several seeds. Use `unconditional` where the RSiena run used `cond = FALSE`.

4. Compare: the statistics of the observed waves must agree exactly with the targets; estimates, standard errors and rates must agree within seed noise and a fraction of RSiena's standard error.

5. For a family where you have an RSiena fit that fails, compare instead by simulation: simulate data sets from known parameters with RSiena, then re-estimate each with `nwsaom` and check recovery of the true values.

The two regression tests named above reproduce steps 2–4 for network-only and co-evolution models without needing R, because the RSiena means are stored in the test files. The tier runner is `cscripts/run_nwsaom_tests.sh` (`quick` or `full`). Before testing a rebuilt plugin, put its directory first on the `adopath`: a stale plugin in Stata's PLUS folder shadows the repository build.

# Chapter 13

# Missing data, structural zeros, and what is not implemented

The first part of this chapter explains how `nwsaom` treats data that are unknown (missing ties and behavior values) or fixed by design (structural zeros and ones). The second part is an honest list of what RSiena offers and `nwsaom` does not.

## 13.1 Missing data: `missnet()` and `missbeh()`

Missing data here means dyads or actors whose true value at a wave is *unknown*. This is RSiena's regular missing-data mechanism and it is different from composition change (actors who are not part of the network at a wave, `present()` or `joinat()`/`leaveat()`): a missing tie is uncertainty about an otherwise active actor. The two can be used together.

**Specifying missingness.**

- `missnet(matlist)` takes one 0/1 $n \times n$ *matrix* name per wave, in the order of `wave1()`/`wave2()` or `waves()`; a 1 marks a dyad missing at that wave. It is a raw Stata matrix, not an `nwset` network. Build it with `matrix input` or `mkmat` from your own indicator. With two waves, `missnet(m1 m2)`.
- `missbeh(varlist)` takes one 0/1 *variable* per wave (the same one-variable-per-wave convention as `present()`), marking the actors whose behavior value is missing at that wave. It requires `behavior()`.
- Both are optional and independent: use either or both.

```
nwwebuse glasgow, nwclear
* the first five actors did not report their ties at wave 2
matrix m1 = J(50,50,0)
matrix m2 = J(50,50,0)
forvalues i = 1/5 {
    forvalues j = 1/50 {
        if `i' != `j' matrix m2[`i',`j'] = 1
    }
}
nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity transtrip ///
    missnet(m1 m2) seed(1)
```

(With `nwsaom` 1.5.4 this example converged, with an overall convergence ratio of 0.091.) Because the missing indicators are Stata matrices, they are limited by Stata's matrix size: 800×800 in Stata/BE, 11,000×11,000 in Stata/SE.

**How missing values are handled.** `nwsaom` follows RSiena's two steps.

1. **Imputation.** Each missing dyad is filled in by carrying forward the last observed value (0 if never observed up to that wave). A missing behavior value is filled in from the previous observation, else the next observation, else the mode of that wave (cross-sectional). The imputed values take part in the simulation completely normally; unlike `present()`, missingness does not restrict who can act.

2. **Masking.** A dyad or actor missing at *either* end wave of a period is excluded from the observed target statistic *and* from every simulated statistic of that period, so the moment condition is not biased by the exclusion. The same applies to the distance that conditional estimation simulates to, and to the distance statistic of the rate under unconditional estimation. The existing effects needed no special-casing: their statistic functions are reused unchanged.

Missing data are handled in the native engine, when the model is otherwise native-eligible; the masked statistic matches the Mata engine to machine precision.

**A disclosed approximation.** The statistic of `avalt` and `avsim` for an actor depends on the *current* values of the actor's alters, which differ across simulated replicates. For these effects the masking is only an approximation (it is exact for `linear` and `quadratic`). It is empirically better than not masking, but may be less precise than an idealised per-effect masking under heavy missingness combined with `avalt` or `avsim`.

**Missing covariate values.** Covariates of `nodematch()`, `nodecov()`, `nodeicov()`, `nodeocov()`, `simcov()` and their aliases follow RSiena. A missing value is imputed with the covariate mean when actors decide their ministeps, but the *statistic* (target and every simulated statistic) skips a tie whose relevant actor is missing: the alter for `altx()`, the ego for `egox()`, either for `samex()` and `simx()`. With the default centring a missing `altx()`, `egox()` or `nodecov()` value becomes 0 and skipping changes nothing; under `nocenter` and for `samex()`/`simx()` it does, and the range and similarity mean of `simx()` use the observed values only. On RSiena's `s501`/`s502` with 8 of 50 covariate values missing, density, reciprocity, altX, egoX and simX agree with RSiena 1.6.6 within sampling noise (simX 0.79 against 0.78). An `interact()` naming a `samex()` or `simx()` effect whose variable has missing values is rejected.

## 13.2 Structural zeros and ones

A structural dyad is one whose tie value is known and cannot change, for example a legally mandated reporting tie, a physically impossible tie, or a dyad that you wish to hold fixed for a counterfactual. It is a simpler idea than missing data: the value is not unknown. `nwsaom` follows RSiena: the values are part of the wave networks. In a wave network a tie with the value 10 is a structural zero and a tie with the value 11 a structural one (a 1 is an ordinary tie, a 0 no tie). `nwsaom` detects the codes; there is no option. With `nwset, mat()` the codes are the cell values of the matrix, with `nwfromedge` give the value variable the values 10 and 11. Codes on the diagonal are ignored; every other tie value than 1, 10 and 11 stops the command.

The rules are RSiena's, checked against RSiena 1.6.14 on the data of the examples:

- The codes belong to a wave; the structural dyads may differ from wave to wave. A dyad that is structural at the *start* of a period keeps its start value during the whole period (the value of its code: 11 is a tie, 10 no tie) and is not among the alternatives of the actors. A dyad that is structural at the *end* of a period only is free during the period and gets its end value before the statistics are computed.

- The observed statistics of a period are those of the end network with the dyads that were structural at the start of the period set to their start value. The observed distance between two waves, which is the target of the rate and of the conditional estimation, leaves out the dyads that are structural at either end of the period and the dyads that are missing at either end. The conditional estimation stops, as RSiena's, when the number of changes of the dyads that are not missing reaches that distance, so with structural dyads that change from wave to wave the unconditional estimation (option `unconditional`) is usually the better choice.
- An actor with no free alternative among the active actors has the rate 0; the other actors cannot choose a structural dyad.
- A dyad that is missing at a wave (`missnet()`) takes the code of the previous wave. Composition change (`present()`, `joinat()`, `leaveat()`), covariates, interactions, rate covariates and the structural rate effects (computed over the actors who can act), network endowment and creation (RSiena advises against them when the structural dyads change from wave to wave) and two-mode networks (the codes are cells of the actors-by-events matrix) work as without codes.

The following example makes the ties from pupil 1 to the pupils 11 to 20 structural zeros in both waves:

```
nwtomata glasgow1, mat(G1)
nwtomata glasgow2, mat(G2)
forvalues j = 11/20 {
    mata: G1[1,‘j’] = 10; G2[1,‘j’] = 10
}
mata: st_matrix("g1m", G1); st_matrix("g2m", G2)
nwset, mat(g1m) directed name(g1)
nwset, mat(g2m) directed name(g2)
nwsaom, wave1(g1) wave2(g2) outdegree reciprocity transtrip seed(1)
```

**Scope.** One directed one-mode network or one two-mode network, any number of waves, the method of moments. Co-evolving behaviors (one or several, also with rate covariates) are possible. In a non-directed relation (`symmetric`) the codes must be given for both directions of a dyad with the same value; all model types work. `nwsaom multinet` takes the codes in its one-mode directed networks (not with a two-mode or non-directed member or with `behavior()`). Not yet with `groups()`, `nwsaom multiplex`, `ml` and `nwsaom bayes`; `nwsaom describe` counts the codes and leaves them out of the distance; `estat gof`, `mems`, `simulate`, `importance`, `score` and `timetest` work after such a fit (observed network with the start-structural dyads at their start value, simulated networks with the end-structural dyads at their end value, as RSiena's gof). The code runs in the native plugin only (there is no Mata implementation).

**The older option `structural(matname)`.** It takes one 0/1 matrix of dyads that are fixed in all periods and need the same observed value at every wave. It is deprecated: the matrix is converted to the codes of every wave (with a note), so it works with `waves()` and the other options; it will be removed in a later major version.

## 13.3 What RSiena has that `nwsaom` does not (yet)

RSiena's effects table, as returned by `getEffects()` for twelve data configurations of RSiena 1.6.6, has 1201 rows (a row is one RSiena short name for one variable type and interaction context). The effects that RSiena 1.6.7 to 1.6.14 added or changed (for example `WXZX` and the two-mode

toU) are implemented and compared with RSiena 1.6.14 but are not part of these counts; the counts change with RSiena releases. Every row was compared with `nwsaom` by reading the code and the documentation, and by running models against `siena07` where they were implemented. Of these rows:

- 1058 are implemented, 1 is partly implemented;
- 62 are *not implemented*;
- 80 are not implemented either because they belong to continuous (SDE) behavior and the effects that involve a continuous dependent variable, a model that is not yet available.

The counts are by *rows*, not by how often an effect is used: the effects most often used in applied papers are largely covered, and the missing rows are concentrated in a few groups. The groups follow; the row-level list is at the end of the chapter.

### 13.3.1 Effects not implemented, by reason

The 62 rows that are not implemented are of five kinds.

**The generalised method of moments (31 rows).** RSiena's GMoM estimator uses its own statistics (the effects named like `egoX_gmm`, `avAlt_gmm`, `crprod_gmm`, and related placeholders). `nwsaom` implements the standard method of moments, and maximum likelihood and Bayesian estimation, but not the generalised method of moments.

**The settings model (12 rows).** The `setting*` effects (`settingSizeAct`, `settingOppAct` and relatives) belong to RSiena's settings model, a separate model family with setting steps and a rate for each setting; `nwsaom` has the standard tie-change model only. There is also nothing to validate against by estimation: in `siena07` eight of the twelve effects have a statistic that is constant under simulation (the estimate stays 0 and has no standard error), and the four variants with the suffix D have a zero derivative and are fixed at 0.

**Effects that RSiena cannot evaluate or does not offer (15 rows).** `siena07` stops or returns nothing useful for these combinations, so there is nothing to compare with and `nwsaom` refuses them with an error message (Table below and Section 13.3.2).

**Placeholders (2 rows).** `minXAlt` and `maxXAlt` in a non-directed network with unspecified variables are entries of RSiena's effects table that `siena07` does not offer.

**Two further effects.** `sameWXClosure` for a two-mode network together with a co-evolving behavior inside `nwsaom multinet` (a two-mode member and a behavior cannot be combined there), and `RateX` for a continuous behavior variable (continuous behavior is not yet implemented).

In addition 80 rows are not implemented: continuous (stochastic differential equation) dependent variables and all effects that involve them.

### 13.3.2 Effects that RSiena cannot evaluate

| Effect | What `siena07` does |
|---|---|
| `inPop_ego`, two-mode network | stops with a C++ exception |
| `sameXOutAct`, `diffXOutAct`, two-mode network, with a constant covariate, a changing covariate or a behavior | the R process aborts |
| `avGroupEgoX`, `totGroupEgoX`, two-mode network, constant covariate | reports the effect as not meaningful (available with a changing covariate) |
| `homXOutAct2` with a behavior | reports the effect as not implemented for behavioral variables |
| `inRateInv`, `inRateLog`, two-mode network | stops with an error (the receivers of ties are not the actors) |
| `susceptAvCovar`, `infectCovar` with a behavior as covariate | no such effect exists (the covariate is a constant or changing covariate) |
| `outOutDist2AvIntn` | listed in the effects table, but there is no implementation |
| `toU` for a one-mode network that depends on a two-mode one | rejected |
| `avAlt.2M.tie` with RSiena's default parameter 3 | throws an error; `nwsaom` offers the parameters 1 and 2 |

Further, `higher` in a non-directed network equals the density statistic and is not identified: RSiena returns a standard error of NA and `nwsaom` stops.

### 13.3.3 RSiena features beyond effects

Besides effects, RSiena provides estimation and analysis tools. This is how they compare.

**Method of moments (`siena07`).** Implemented, conditional and unconditional, with the same phase structure (Chapter 9).

**Maximum likelihood and Bayesian estimation (`siena07 with maxlike = TRUE`, `multi_siena`).** Available for a directed one-mode network, with or without one co-evolving behavior: the option `ml` and the subcommand `nwsaom bayes` (Chapter 9). Not available for non-directed and two-mode networks, several networks, missing data, composition change, rate covariates and several groups. The hierarchical Bayesian analysis of several groups (`multi_siena` with several groups) and Bayesian estimation of a behavior alone are not implemented.

**Multi-group analysis (`siena08`, `sienaGroupCreate`).** Both ways are available. The meta-analysis of separate fits is `nwsaom meta` (random-effects pooling by robust IWLS, heterogeneity tests, Fisher combination, maximum-likelihood mean and between-group sd with profile intervals). RSiena's joint multi-group *estimation* (one model fitted to several groups at once) is the option `groups()` (Chapter 11), by the method of moments, conditional and unconditional. Not yet implemented with groups: maximum likelihood and Bayesian estimation, `estat gof`, `estat simulate` and `estat importance` (`estat score` and `estat timetest` work), two-mode and non-directed networks, `nwsaom multinet` and several behaviors, the effects that use a constant pooled over the data (the centered degree effects `inpopc`, `outpopc`, `outactc`, `inactc`, `degplus`, `degplusc`, `avdeg`, `transtriads`, structural equivalence), `behrate()`, `structural()`, `joinat()` and `leaveat()`. The hierarchical Bayesian analysis of several groups is not implemented. With groups, leavers were not compared with RSiena (Section 12.4).

**Goodness of fit (`sienaGOF`).** Available as `estat gof` (Chapter 9), with the differences listed there, and not for fits with several co-evolving behaviors.

**Composition change.** Available, including fractional join/leave times (`joinat()`/`leaveat()`) as in `sienaCompositionChange()` (also with `groups()`, where the times are on the scale of the waves of the actorś group). Composition change and missing ties are not available inside `nwsaom multinet`.

**Continuous behavior.** Not implemented (out of scope).

**Simulation only, time-heterogeneity tests, score-type tests, effect importance.** Available: `nwsaom simulate`, `estat timetest`, `estat score` and `estat importance` (Chapter 10). After `nwsaom multinet` the tests are available, but not `estat gof`, `estat importance` and `estat simulate`.

**Fixing and testing individual parameters.** Available: `fix()` holds an effect at its `theta0()` value (RSiena's `fix`), and `estat score` tests it, also after `nwsaom multinet`.

**The generalised method of moments.** Not implemented (31 statistics).

**The settings model.** Not implemented (12 effects).

**Model types for non-directed relations.** Offered by name or RSiena's modelType number (`symtype()`: forcing, confirmation, force, agree, joint; modelType 2, 3, 4, 5, 6).

## 13.4 Row-level list of the effects that are not implemented

The tables below list every RSiena effects-table row that is *not implemented* (one table per reason), in the order of RSiena's effects table; the final entry is the out-of-scope group. A row names an RSiena short name together with the variable type and the interaction context (another network, a constant or changing covariate, a dyadic covariate, a behavior variable). Rows that are implemented are listed in the appendix on effects.

### GMoM statistics (the generalised method of moments estimator is not implemented) (31 rows)

| RSiena name | Network / context | Note |
|---|---|---|
| `newrecip` | gmm (one-mode network) | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `persistrecip` | gmm (one-mode network) | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `realrecip` | gmm (one-mode network) | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `realtrans` | gmm (one-mode network) | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `agreetrans` | gmm (one-mode network) | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (one-mode network) + constant covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |

| RSiena name | Network / context | Note |
|---|---|---|
| `simX_gmm` | gmm (one-mode network) + constant covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (one-mode network) + changing covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `simX_gmm` | gmm (one-mode network) + changing covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (one-mode network) + a behavior variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `simX_gmm` | gmm (one-mode network) + a behavior variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `avSim_gmm` | gmm (behavior) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `avAlt_gmm` | gmm (behavior) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `totAlt_gmm` | gmm (behavior) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `maxAlt_gmm` | gmm (behavior) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `minAlt_gmm` | gmm (behavior) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `crprod_gmm` | gmm (one-mode network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `from_gmm` | gmm (one-mode network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `to_gmm` | gmm (one-mode network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `to_gmm` | gmm (two-mode network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (one-mode network) + continuous variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `simX_gmm` | gmm (one-mode network) + continuous variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (non-directed network) + constant covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `simX_gmm` | gmm (non-directed network) + constant covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (non-directed network) + changing covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `simX_gmm` | gmm (non-directed network) + changing covariate | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| `egoX_gmm` | gmm (non-directed network) + a behavior variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |

| RSiena name | Network / context | Note |
|---|---|---|
| simX_gmm | gmm (non-directed network) + a behavior variable | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| from_gmm | gmm (non-directed network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| to_gmm | gmm (non-directed network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |
| crprod_gmm | gmm (non-directed network) + another network | GMoM statistic: nwsaom estimates by Method of Moments only (no GMoM estimator). |

## Effects that RSiena cannot evaluate or does not offer (15 rows)

| RSiena name | Network / context | Note |
|---|---|---|
| inPop_ego | two-mode network | Not implemented: siena07 itself stops with a C++ exception for this effect in a two-mode network, so there is nothing to compare with. |
| sameXOutAct | two-mode network + constant covariate | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| diffXOutAct | two-mode network + constant covariate | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| avGroupEgoX | two-mode network + constant covariate | Not implemented: RSiena reports the effect as not meaningful for a constant covariate. |
| totGroupEgoX | two-mode network + constant covariate | Not implemented: RSiena reports the effect as not meaningful for a constant covariate. |
| sameXOutAct | two-mode network + changing covariate | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| diffXOutAct | two-mode network + changing covariate | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| homXOutAct2 | one-mode network + a behavior variable | Not implemented: RSiena reports the effect as not implemented for a behavior variable. |
| sameXOutAct | two-mode network + a behavior variable | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| diffXOutAct | two-mode network + a behavior variable | Not implemented: siena07 crashes (R aborts) when it evaluates this effect in a two-mode network, so nwsaom refuses it. |
| homXOutAct2 | non-directed network + a behavior variable | Not implemented: RSiena reports the effect as not implemented for a behavior variable. |
| susceptAvCovar | rate (behavior) + another network + a behavior variable | Not implemented: RSiena has no such effect (the covariate can only be a constant or a changing covariate). |
| infectCovar | rate (behavior) + another network + a behavior variable | Not implemented: RSiena has no such effect (the covariate can only be a constant or a changing covariate). |
| inRateInv | rate (two-mode network) | Not implemented: siena07 stops with an error for this effect in a two-mode network (the receivers of ties are not the actors). |
| inRateLog | rate (two-mode network) | Not implemented: siena07 stops with an error for this effect in a two-mode network (the receivers of ties are not the actors). |

## RSiena's settings model (a different simulation procedure) (12 rows)

| RSiena name | Network / context | Note |
|---|---|---|
| `settingSizeAct` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingSizeActSqrt` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingSizeActLog` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingSizeActInv` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppAct` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppActSqrt` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppActLog` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingLogCreationAct` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppActD` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppActSqrtD` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingOppActLogD` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |
| `settingLogCreationAct D` | one-mode network | Not implemented: RSiena's settings model is a separate model family (setting steps with rates per setting). In siena07 the effect has no estimate to compare wi... |

## Placeholder rows of RSiena's effects table (2 rows)

| RSiena name | Network / context | Note |
|---|---|---|
| `minXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | Placeholder entry of the RSiena effects table; the effect is not offered by siena07 for these variables. |

| RSiena name | Network / context | Note |
|---|---|---|
| `maxXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | Placeholder entry of the RSiena effects table; the effect is not offered by siena07 for these variables. |

### Cross-network effects that also involve a behavior variable (needs behavior inside nwsaom multinet) (1 row)

| RSiena name | Network / context | Note |
|---|---|---|
| `sameWXClosure` | two-mode network + another network + a behavior variable | Not implemented in nwsaom multinet: a two-mode member together with a co-evolving behavior. |

### Rate effects of continuous behavior (not planned) (1 row)

| RSiena name | Network / context | Note |
|---|---|---|
| `RateX` | rate (one-mode network) + continuous variable | Not implemented: continuous (stochastic differential equation) behavior variables are not planned. |

### Deliberately out of scope (80 rows)

Continuous (SDE) dependent variables and their effects are not part of nwsaom (80 rows of the RSiena effects table).

# Chapter 14

# Technical notes

This chapter is for users who fit large models, who run `nwsaom` on several platforms, or who want to read or extend the source. It describes the three layers of the program, the native plugin and how to tell which engine ran, the threads, the size limits of Stata and of `nwsaom`, the run times to expect, and the limit on the size of an ado program that shapes the layout of the source files.

## 14.1 The three layers

`nwsaom` consists of three layers.

1. The **ado layer** (`nwsaom.ado` and a few helper files) parses the options, checks the data and the combinations of options, builds the model description, calls the estimator and posts the results in `e()`.
2. The **Mata core** (`unw_saom.do`, compiled into the library `lnwcommands.mlib`) contains the model description, the observed statistics, the Robbins–Monro estimator for all phases, the calculation of the covariance matrix and a reference simulator, written in Mata, for the core effects.
3. The **native plugin** (`native/saom_sim.c`, compiled for each platform into `lib/plugins/<platform>/saom_sim_<platform>.plugin`) simulates the ministeps. Almost all of the computing time of a fit is spent in it.

The Mata layer talks to the plugin through Stata frames. The calls are made once per batch of simulations and not once per ministep: in phases 1 and 3 all replicates of a fit are run in one call, on worker threads. The ministep sampler itself is the multinomial-logit rule of Chapter 2: for the acting actor the contribution of every alternative to the objective function is computed, the alternatives are given the choice probabilities, and one is drawn.

## 14.2 The native plugin

### 14.2.1 Source code of the plugin

The C source of the plugin is part of the package. `net install` installs the compiled plugins only; the command

```
. net get nwsaom, from(https://raw.githubusercontent.com/thomas-u-grund/nwsaom/main)
```

copies the source files, the build rules and a short build note into the current directory. The same files are in the folder `native/` of the repository (https://github.com/thomas-u-grund/nwsaom). `nwsaom` is free software under the GNU General Public License, version 3; the plugin interface files in `native/spi/` are StataCorp's Stata Plugin Interface, redistributed unchanged. The plugin is built with `make macos` (macOS, universal binary) or `make unix` in `native/`, and with the Visual Studio command line for Windows.

### 14.2.2 Building the plugin on your computer

If the plugin of your platform is missing, older than the `.ado` files, or does not work on your computer (for example a Linux system with an older C library), compile it there:

```
. nwinstall, buildplugin(nwsaom)
```

The command copies the source from the repository, compiles it with the compiler of your computer, and installs the plugin next to the one that `net install` put into the PLUS directory; an existing file is kept with the extension `.prev`. You need a C compiler and `make` (on macOS the command line tools of Xcode, `xcode-select -install`; on Linux `gcc` and `make`). Restart Stata afterwards, because a plugin that is already loaded is not replaced in a running session. The plugin depends on the operating system and the processor, not on the version or the flavour of Stata (BE, SE, MP): a plugin built on your computer works in every Stata installed there. The routine was tested on macOS; on Windows it stops with a message, and the build note that comes with the source gives the command for the Visual Studio command line. `buildfrom()` reads the source from a local copy of the repository, and `release()` from a tagged release.

### 14.2.3 Rebuilding the Mata libraries

The Mata libraries `lnwsaom1.mlib` and `lnwsaom2.mlib` are built with Stata 17. A library built by an older Stata loads in every later Stata; one built by a newer Stata is ignored by an older one (r(3499), "too new to be run by this version of Stata"). If the library does not load in your Stata, build it with that Stata (nwcommands 3.14 or later):

```
. nwinstall, buildmata(nwsaom)
```

The command fetches the Mata source from the repository, compiles it, installs the library and keeps the old file with the extension `.prev`. The library of `nwsaom` is compiled against the core library, so if the core library does not load either, use `nwinstall, buildmata(all)`, which rebuilds the core first and then the installed model packages. The build clears the data and the Mata objects in memory, so save your data first, and restart Stata afterwards.

### 14.2.4 Plugin versions

The plugin and the Mata layer communicate through a versioned interface. The plugin reports its version, and the Mata layer checks it before it uses a feature that needs a newer plugin. New effects, and new ways of combining existing ones, make the plugin newer.

A plugin that is older than the `.ado` files cannot run models that need a newer one. This happens most often on Linux and Windows: those two binaries are built by continuous integration (a workflow on GitHub), whereas the macOS binary is compiled on the development machine, so that the three can differ for a while after a commit. If you install from a repository checkout, `nwsaom` looks for the plugin first in `lib/plugins/<platform>/` next to the `nwsaom.ado` that is being run and only then with `findfile`. After a `net install` there is no `lib/plugins/` next to `nwsaom.ado`, and the copy in the PLUS directory is used.

### 14.2.5 Native-only effects

Not all effects are implemented twice. The Mata layer contains a simulator for the older core of the model, which is used if no plugin is available for the platform. The effects added later, for the most part the large families of RSiena effects that were ported after the core was finished, exist only in the plugin: the plugin computes the simulation and the observed statistic, and there is no Mata twin. The Mata simulator is far too slow for the long simulations that estimation needs, so new effects are added to the plugin only. In the help file such effects are marked *native plugin only*; Appendix C lists the effects that do work without the plugin.

If you ask for a native-only effect and the native engine cannot be used, `nwsaom` stops with an error and does not fall back. With the Mata engine forced by the debugging switch described below, the request

```
. nwsaom, wave1(glasgow1) wave2(glasgow2) outdegree reciprocity egoplusaltsqx(alcohol1)
    seed(1)
```

gives

```
nwsaom: egoplusaltsqx needs the native plugin (a newer plugin version); there is no
   Mata implementation of this effect.
```

and the return code is 198. The reverse also holds: a model that consists of effects with a Mata implementation runs without a plugin, correctly but slowly.

Two consequences follow for the user. First, with a plugin that is too old the same model is either slower (Mata) or refused (native-only), and the message says which. Second, to use a recent effect on Linux or Windows you may have to wait for, or build, a current plugin; the C source and the build rules are in the repository.

### 14.2.6 Which engine ran

After a fit, `e(engine)` is `native` or `mata`. If it is `mata`, a note is printed after the table and `e(engine_why)` gives the reason. Look at it after every large fit. The Mata engine gives statistically equivalent results (the random streams of the two engines are not the same, so that the digits differ), but it is slower by orders of magnitude. As an illustration, a two-wave Glasgow model with outdegree and reciprocity and `k0(10) k3(10)` took about 30 seconds in the Mata engine, where the native engine needs a small fraction of a second for the same model.

For tests, setting the global macro `NWSAOM_NONATIVE` to 1 before the call forces the Mata engine (and `e(engine_why)` then gives the message of a missing plugin); setting `NWSAOM_TRACE` to 4 prints a line with the version of the plugin that was found and whether the native and the batch paths are used. These are development switches.

### 14.2.7 Fixed limits of the plugin

The plugin has fixed capacities: at most 256 effects in a model, 1 024 covariate arrays (a dyadic covariate on $n$ actors uses $n$ of them, and its usable limit is quoted as 1 023), 64 behavior effects, 8 co-evolving behaviors and 16 rate covariates. A model that exceeds one of them runs in the Mata engine without an error if the effects allow it, which is a trap discussed in Section 14.4.

## 14.3 Threads and reproducibility

The plugin is multi-threaded. In phase 1 and phase 3 all replicates of all periods are run in one call by a pool of worker threads. Phase 2 is sequential by construction, because each Robbins–Monro step needs the previous one; there only the periods of one step are run in parallel, and only when the network is large enough for threads to pay off.

**The number of threads.** `cores(#)` sets it; the default 0 uses all physical cores and `cores(1)` runs single-threaded.

**Reproducibility.** Every simulated period draws from its own random stream, which is derived from the seed and from its position (replicate and period), and sums over the periods are formed after the threads have joined, in a fixed order. A given `seed()` therefore gives identical results whatever the number of threads. On the two-wave example with `cores(1)` and `cores(4)`: the

coefficients were identical to all printed digits (the outdegree coefficient in both runs −2.2380415). The results change if the model, the seed or the version of the program changes.

## 14.4 Size limits

### 14.4.1 What Stata limits

Everything that is handed to `nwsaom` as a Stata matrix is limited by the edition of Stata:

| | Stata/BE | Stata/SE | Stata/MP |
|---|---|---|---|
| Largest matrix | 800 × 800 | 11 000 × 11 000 | 65 534 × 65 534 |
| Variables | 2 048 | 32 767 | 120 000 |

The consequences for networks are these. A network that is declared from a matrix (`nwset, mat()`), and every matrix given to `dyadcov()`, `dyadcovrecip()`, `dyadx()`, `missnet()` or `structural()`, is limited by the matrix size: 800 actors in Stata/BE. Larger networks are brought in as edge lists (`nwfromedge`); nothing dense is needed for the network itself. A dyadic covariate also travels to the plugin as one variable per actor, so that the native engine handles it up to about 1 000 actors (1 023 covariate arrays), and Stata/BE with its 2 048 variables stops at about 2 000 actors in any case. Mata itself has no such limits; the networks are kept as sparse adjacency lists, and only dense by-products scale with $n^2$.

### 14.4.2 What `nwsaom` costs

Every simulated ministep considers all $n$ possible alters, and the number of ministeps per period grows with $n$ (the rate times $n$), so that the time of a fit grows roughly with $n^3$ above about 1 000 actors. For a plain model (outdegree, reciprocity, transitive triplets; three waves; about ten ties per actor; native engine; `k0(30) k3(200)`) the time was measured on one machine as follows.

| Actors | Seconds |
|---|---|
| 250 | 1.5 |
| 500 | 2.6 |
| 1 000 | 10 |
| 2 000 | 67 |
| 4 000 | 520 |

A full-length run, with the default `k3()` of 1 000 and more phase-2 iterations, costs several times more. One thousand actors is routine; a few thousand actors take minutes to tens of minutes; beyond roughly 10 000 actors the model itself, in which each ministep looks at all alters, is the limit and not the software. A dyadic covariate adds to this (at $n = 300$ about 4 seconds instead of 2, at $n = 700$ about 12 seconds in the measurement quoted).

### 14.4.3 The fallback warning

**If the compiled plugin cannot be used, the fit runs in Mata, and `nwsaom` says so before it starts.** Mata is slower by orders of magnitude; the results are the same. The command prints one line, once per fit, before the long computation:

```
(Warning: the compiled plugin is not used for this fit (the plugin file
saom_sim_macos.plugin was not found; reinstall the package with net install
nwsaom, replace); the Mata fallback is much slower, possibly hours. See Check which
    engine ran.)
```

The reason in the brackets is one of four: the plugin file for the platform was not found; the installed plugin is older than the model needs; the model contains an effect or an option that the plugin does not cover (it is outside the capacity of the plugin); or the global macro `NWSAOM_NONATIVE` is set to 1, a testing switch that forces Mata. Like all output the line is suppressed by `quietly`; in that case look at `e(engine)` after the fit. The warning exists because a fit that silently ran in Mata took more than an hour where the plugin takes seconds: until 22 October 2026 the test for "too many covariate arrays" used 127 instead of the plugin's real limit of 1 023, so that a dyadic covariate on 129 or more actors ran in Mata. That is fixed. Effects that exist only in the plugin are refused with an error instead of a fallback.

## 14.5 Performance

The following timings were measured on one 18-core machine with the native engine, and are given as orders of magnitude; Appendix D compares the run times with RSiena's.

| Model | Seconds |
|---|---|
| Glasgow, 3 waves, outdegree, reciprocity, transitive triplets | 0.33 |
| Glasgow, 2 waves, outdegree, reciprocity | 0.23 |
| Glasgow, 3 waves, co-evolution with alcohol | 0.63 |
| Synthetic, 500 actors, 3 waves, co-evolution | 18 |

For comparison, the larger Glasgow model of the validation (with `gwespFF`, `transRecTrip`, `inPopSqrt`, `outAct`, covariate effects and a rate covariate) took `nwsaom` about 1 second against 5–7 seconds for `siena07`. The examples in Chapter 3 took one to a few seconds each on the development machine, including the start of Stata.

Some hints. Use the default number of cores. Shorten exploratory runs with `k0()` and `k3()` (as the example fragments of Chapter 4 do) and use the defaults for the final run, because the standard errors and the convergence ratios come from phase 3. Models with many weakly identified effects take more iterations, not less.

## 14.6 The size limit of an ado program

Stata refuses to load a single ado program larger than about 141 000 bytes of code (comments and blank lines are not counted). The limit is a property of Stata and not of the file size: a program that is only a few lines above it fails to load with the message "system limit exceeded – see manual (error occurred while loading nwsaom.ado)", also when the same file loads in an empty session. The main program `nwsaom` is close to this cap, and the layout of the source is shaped by it.

If you modify the ado code, remember the rules that were learnt:

- Move blocks of code into separate programs in *separate ado-files*. The programs of one ado-file all count towards Stata's total adosize of 1 000 KB while loaded, and a program that another ado-file calls must have its own file (a program defined inside `nwsaom.ado` can be evicted when the adosize fills up).
- Mata functions defined in an ado-file are private to that file; helpers that several ado-files share belong in `unw_saom.do`.
- A subprogram receives the locals of its caller only through options. Results come back in `r()`.
- Do not name a new option as an extension of an existing one (`netx0` next to `netx`): Stata's `syntax` routes the shorter name to the option declared first. Option names that are a digit followed by letters are rejected altogether, which is why the options `dblnbrdist2` and `inplus3` and the arguments of `nwsaom multiplex` look as they do.

## 14.7 File layout

| File | Content |
|---|---|
| `nwsaom.ado` | The command: parsing, checks, model description, estimation dispatch, results. |
| `_nwsaom_nx.ado` | The native-only network options `netx()`, `ownx()`, `dyadx()`, `tvdyadcov()`, `tvdyadcovrecip()`, `tvdyadx()`, moved out of `nwsaom.ado` because of the size limit. |
| `nwsaom_multinet.ado` | `nwsaom multinet`: several dependent networks. |
| `nwsaom_multiplex.ado` | `nwsaom multiplex`: the earlier two-network subcommand with outdegree, reciprocity and the cross-network product. |
| `_nwsaom_postengine.ado`, `_nwsaom_rsnotes.ado`, `_nwsaom_tconvtable.ado` | Posting of `e(engine)`, the notes with RSiena's effect names, and the printing of the convergence table. |
| `nwsaom_estat.ado`, `nwsaom_meta.ado` | `estat gof` and `estat mems`; the meta-analysis. |
| `unw_saom.do` | The Mata core: model, estimator, reference simulator, interface to the plugin. |
| `native/saom_sim.c` | The C plugin source; `native/Makefile` builds it. |
| `lib/plugins/<platform>/` | The compiled plugins for `macos`, `unix` and `windows`. |
| `nwsaom.sthlp`, `nwsaom_remarks.sthlp` | The help file and its remarks. The help file was split in two because Stata's help Viewer fails on a very long file. |
| `cscripts/test_nwsaom_*.do` | The regression tests, one file per feature, and `cscripts/run_nwsaom_tests.sh`, which runs them (`quick` or `full`) and accepts the directory of a freshly built plugin. |

**Testing.** The tests are run in two modes: in *development mode*, in which the Mata sources are loaded directly (`do unw_core.do`, `do unw_ergm.do`, `do unw_saom.do`, and `nwclear` — never `clear all`, which would delete the loaded Mata functions) and in *production mode* against the compiled library and plugin. The native simulator is compared with the Mata simulator at fixed parameters for statistical equivalence of the simulated statistics (the two do not share a random stream), and the estimates are compared with RSiena as described in the validation chapter.

**A stale plugin.** A plugin from an earlier installation in the PLUS directory can shadow the plugin of a freshly built checkout in some set-ups and silently run an old binary. If a new effect is refused as unknown to the plugin, or the version shown by `NWSAOM_TRACE` is older than expected, check which plugin file is used.

# Appendix A

# Effects reference

All implemented effects, grouped as in RSiena's effects table. "Network / context" says which kind of dependent variable and which kind of interaction the row is for. The option column is what you type; `var`, `stem` and `matrix` stand for a variable, the stem of a changing covariate and a Stata matrix.

### A. One-mode, non-directed and two-mode network: density and reciprocity

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `density` | one-mode network | siena07 | `outdegree` |
| `recip` | one-mode network | siena07 | `reciprocity` |
| `density` | two-mode network | siena07 | `outdegree` |
| `density` | non-directed network | siena07 | `outdegree` |

### B. Network structural effects: triadic and distance-two effects

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `transTrip` | one-mode network | siena07 | `transtrip` |
| `transTrip1` | one-mode network | siena07 | `ownx(transtrip1)` |
| `transTrip2` | one-mode network | siena07 | `ownx(transtrip2)` |
| `transMedTrip` | one-mode network | siena07 | `transmedtrip` |
| `transRecTrip` | one-mode network | siena07 | `transrectrip` |
| `transRecTrip2` | one-mode network | siena07 | `ownx(transrectrip2)` |
| `cycle3` | one-mode network | siena07 | `cycle3` |
| `transTies` | one-mode network | siena07 | `transties` |
| `between` | one-mode network | siena07 | `between` |
| `balance` | one-mode network | siena07 | `balance` |
| `nbrDist2` | one-mode network | siena07 | `nbrdist2` |
| `nbrDist2twice` | one-mode network | siena07 | `dblnbrdist2` |
| `denseTriads` | one-mode network | siena07 | `ownx(densetriads:5)` |
| `gwespFF` | one-mode network | siena07 | `gwesp(#)` |
| `gwespBB` | one-mode network | siena07 | `gwespbb(#)` |
| `gwespFB` | one-mode network | siena07 | `gwespfb(#)` |
| `gwespBF` | one-mode network | siena07 | `gwespbf(#)` |
| `gwespRR` | one-mode network | siena07 | `gwesprr(#)` |
| `gwdspFF` | one-mode network | siena07 | `gwdspff(#)` |
| `gwdspFB` | one-mode network | siena07 | `gwdspfb(#)` |
| `sharedPop` | one-mode network | siena07 | `cycle4 / sqrtcycle4` |
| `cycle4` | one-mode network | siena07 | `cycle4 / sqrtcycle4` |
| `inStructEq` | one-mode network | siena07 | `ownx(instructeq)` |
| `Jin` | one-mode network | siena07 | `ownx(jin)` |
| `Jout` | one-mode network | siena07 | `ownx(jout)` |
| `transTrip.FR` | one-mode network | siena07 | `ownx(transtripfr)` |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| transTrip.FE | one-mode network | siena07 | ownx(transtripfe) |
| transTrip.EE | one-mode network | siena07 | ownx(transtripee) |
| cycle4 | two-mode network | siena07 | cycle4 |
| gwdspFB | two-mode network | siena07 | gwdspfb(0.69) |
| transTriads | non-directed network | siena07 | transtriads |
| transTies | non-directed network | siena07 | transties |
| between | non-directed network | siena07 | between |
| balance | non-directed network | siena07 | balance |
| nbrDist2 | non-directed network | siena07 | nbrdist2 |
| nbrDist2twice | non-directed network | siena07 | dblnbrdist2 |
| gwesp | non-directed network | siena07 | gwesp(#) |
| cycle4 | non-directed network | siena07 | cycle4 |
| Jout | non-directed network | siena07 | ownx(jout) |

## C. Network structural effects: degree effects (popularity, activity, assortativity, reciprocal degree)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| inPop | one-mode network | siena07 | inpop |
| inPop.c | one-mode network | siena07 | inpopc |
| inPopSqrt | one-mode network | siena07 | indegpopularity |
| inPop_dya | one-mode network | siena07 | ownx(inpopdya) |
| outPop | one-mode network | siena07 | outpop |
| outPop.c | one-mode network | siena07 | outpopc |
| outPopSqrt | one-mode network | siena07 | outpopularity |
| outPopMore | one-mode network | siena07 | ownx(outpopmore:#) |
| outPopSqrtMore | one-mode network | siena07 | ownx(outpopsqrtmore:#) |
| outPopThreshold | one-mode network | siena07 | ownx(outpopthreshold:#) |
| reciPop | one-mode network | siena07 | recipop |
| reciPopSqrt | one-mode network | siena07 | recipopsqrt |
| inActSqrt | one-mode network | siena07 | inactivity |
| outAct | one-mode network | siena07 | outactivity |
| outAct.c | one-mode network | siena07 | outactc |
| outActSqrt | one-mode network | siena07 | outactsqrt |
| inAct | one-mode network | siena07 | inact |
| inAct.c | one-mode network | siena07 | inactc |
| reciAct | one-mode network | siena07 | reciact / reciactsqrt |
| avDeg | one-mode network | siena07 | avdeg(c) |
| outOutAss | one-mode network | siena07 | ownx(outoutass) |
| outInAss | one-mode network | siena07 | ownx(outinass) |
| inOutAss | one-mode network | siena07 | ownx(inoutass) |
| inInAss | one-mode network | siena07 | ownx(ininass) |
| outAct_ego | one-mode network | siena07 | ownx(outactego) |
| outActMore_ego | one-mode network | siena07 | ownx(outactmoreego:#) |
| outActSqrtMore_ego | one-mode network | siena07 | ownx(outactsqrtmoreego:#) |
| outMore_ego | one-mode network | siena07 | ownx(outmoreego:#) |
| divOut_ego | one-mode network | siena07 | ownx(divoutego) |
| inAct_ego | one-mode network | siena07 | ownx(inactego) |
| divIn_ego | one-mode network | siena07 | ownx(divinego) |
| reciAct_ego | one-mode network | siena07 | ownx(reciactego) |
| inPop | two-mode network | siena07 | inpop |
| inPop.c | two-mode network | siena07 | inpopc |
| inPopSqrt | two-mode network | siena07 | indegpopularity |
| outAct | two-mode network | siena07 | outactivity |
| outAct.c | two-mode network | siena07 | outactc |
| outActSqrt | two-mode network | siena07 | outactsqrt |
| avDeg | two-mode network | siena07 | avdeg(2) |
| outInAss | two-mode network | siena07 | ownx(outinass:2) |
| outAct_ego | two-mode network | siena07 | ownx(outactego) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| divOut_ego | two-mode network | siena07 | ownx(divoutego:1) |
| outActMore_ego | two-mode network | siena07 | ownx(outactmoreego:0) |
| outActSqrtMore_ego | two-mode network | siena07 | ownx(outactsqrtmoreego:0) |
| outMore_ego | two-mode network | siena07 | ownx(outmoreego:0) |
| inPop | non-directed network | siena07 | inpop |
| inPop.c | non-directed network | siena07 | inpopc |
| inPopSqrt | non-directed network | siena07 | indegpopularity |
| inPop_dya | non-directed network | siena07 | ownx(inpopdya) |
| outAct | non-directed network | siena07 | outactivity |
| outAct.c | non-directed network | siena07 | outactc |
| outActSqrt | non-directed network | siena07 | outactsqrt |
| degPlus | non-directed network | siena07 | degplus |
| degPlus.c | non-directed network | siena07 | degplusc |
| outInAss | non-directed network | siena07 | ownx(outinass:2) |
| divOut_ego | non-directed network | siena07 | ownx(divoutego:1) |
| avDeg | non-directed network | siena07 | avdeg(c) |

## D. Network structural effects: functions of the outdegree

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| outTrunc | one-mode network | siena07 | outtrunc(#) |
| outTrunc2 | one-mode network | siena07 | outtrunc(#), parameter value of RSiena outTrunc2 |
| outMore | one-mode network | siena07 | outmore(#) |
| outMore2 | one-mode network | siena07 | outmore(#), parameter value of RSiena outMore2 |
| outMore3 | one-mode network | siena07 | outmore(#), parameter value of RSiena outMore3 |
| outThreshold | one-mode network | siena07 | outthreshold(#) |
| outThreshold2 | one-mode network | siena07 | outthreshold(#), parameter value of RSiena outThreshold2 |
| outInv | one-mode network | siena07 | outinv(#) |
| outSqInv | one-mode network | siena07 | outsqinv(#) |
| outTrunc | two-mode network | siena07 | outtrunc(c) |
| outTrunc2 | two-mode network | siena07 | outtrunc(5) |
| outMore | two-mode network | siena07 | outmore(c) |
| outMore2 | two-mode network | siena07 | outmore(1) |
| outMore3 | two-mode network | siena07 | outmore(1) |
| outThreshold | two-mode network | siena07 | outthreshold(c) |
| outThreshold2 | two-mode network | siena07 | outthreshold(1) |
| outInv | two-mode network | siena07 | outinv(c) |
| outSqInv | two-mode network | siena07 | outsqinv(c) |
| outTrunc | non-directed network | siena07 | outtrunc(c) |
| outTrunc2 | non-directed network | siena07 | outtrunc(5) |
| outMore | non-directed network | siena07 | outmore(c) |
| outMore2 | non-directed network | siena07 | outmore(1) |
| outMore3 | non-directed network | siena07 | outmore(1) |
| outThreshold | non-directed network | siena07 | outthreshold(c) |
| outThreshold2 | non-directed network | siena07 | outthreshold(1) |
| outInv | non-directed network | siena07 | outinv(c) |
| outSqInv | non-directed network | siena07 | outsqinv(c) |

## E. Network structural effects: isolates and degree thresholds

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| outIso | one-mode network | siena07 | outiso |
| inIsDegree | one-mode network | siena07 | ownx(inisdegree) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| isolateNet | one-mode network | siena07 | isolatenet |
| antiIso | one-mode network | siena07 | antiiso |
| antiInIso | one-mode network | siena07 | antiiniso |
| antiInIso2 | one-mode network | siena07 | antiiniso2 |
| in2Plus | one-mode network | siena07 | antiiniso2 |
| in3Plus | one-mode network | siena07 | inplus3 |
| isolatePop | one-mode network | siena07 | isolatepop |
| outIso | two-mode network | siena07 | outiso |
| antiInIso | two-mode network | siena07 | antiiniso |
| antiInIso2 | two-mode network | siena07 | antiiniso2 |
| in2Plus | two-mode network | siena07 | antiiniso2 |
| in3Plus | two-mode network | siena07 | inplus3 |
| outIso | non-directed network | siena07 | outiso |
| isolateNet | non-directed network | siena07 | isolatenet |
| antiInIso | non-directed network | siena07 | antiiniso |
| antiInIso2 | non-directed network | siena07 | antiiniso2 |

## F. Network structural effects: compression, primary-setting and other

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| nonPCompress | one-mode network | siena07 | ownx(nonpcompress) |
| primCompress | one-mode network | siena07 | ownx(primcompress) |
| primary | one-mode network | siena07 | ownx(primary) |
| primDegAct | one-mode network | siena07 | ownx(primdegact) |
| primDegActLog | one-mode network | siena07 | ownx(primdegactlog) |
| primDegActSqrt | one-mode network | siena07 | ownx(primdegactsqrt) |
| primDegActInv | one-mode network | siena07 | ownx(primdegactinv) |
| primDegActDiff | one-mode network | siena07 | ownx(primdegactdiff) |
| primDegActDiffSqrt | one-mode network | siena07 | ownx(primdegactdiffsqrt) |
| unspInt | one-mode network | siena07 | interact() |
| unspInt | two-mode network | siena07 | interact |
| unspInt | non-directed network | siena07 | symmetric + interact |

## G. Network effects of monadic covariates (constant covariate, coCovar)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altX | one-mode network + constant covariate | siena07 | altx() / nodeicov() |
| altSqX | one-mode network + constant covariate | siena07 | altsqx() |
| altLThresholdX | one-mode network + constant covariate | siena07 | altlthresholdx() |
| altRThresholdX | one-mode network + constant covariate | siena07 | altrthresholdx() |
| outXMore | one-mode network + constant covariate | siena07 | netx(outxmore:var) |
| egoX | one-mode network + constant covariate | siena07 | egox() / nodeocov() |
| egoSqX | one-mode network + constant covariate | siena07 | egosqx() |
| egoLThresholdX | one-mode network + constant covariate | siena07 | egolthresholdx() |
| egoRThresholdX | one-mode network + constant covariate | siena07 | egorthresholdx() |
| degAbsDiffX | one-mode network + constant covariate | siena07 | degabsdiffx() |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `degPosDiffX` | one-mode network + constant covariate | siena07 | `degposdiffx()` |
| `degNegDiffX` | one-mode network + constant covariate | siena07 | `degnegdiffx()` |
| `diffX` | one-mode network + constant covariate | siena07 | `diffx()` |
| `absDiffX` | one-mode network + constant covariate | siena07 | `absdiffx()` |
| `diffSqX` | one-mode network + constant covariate | siena07 | `diffsqx()` |
| `egoDiffX` | one-mode network + constant covariate | siena07 | `egodiffx()` |
| `egoPlusAltX` | one-mode network + constant covariate | siena07 | `nodecov()` |
| `altHigherEgoX` | one-mode network + constant covariate | siena07 | `althigheregox()` |
| `higher` | one-mode network + constant covariate | siena07 | `higher()` |
| `egoPlusAltSqX` | one-mode network + constant covariate | siena07 | `egoplusaltsqx()` |
| `simX` | one-mode network + constant covariate | siena07 | `simx() / simcov()` |
| `simRecipX` | one-mode network + constant covariate | siena07 | `simrecipx()` |
| `simXTransTrip` | one-mode network + constant covariate | siena07 | `simxtranstrip()` |
| `sameX` | one-mode network + constant covariate | siena07 | `samex() / nodematch()` |
| `unequalX` | one-mode network + constant covariate | siena07 | `unequalx()` |
| `sameXRecip` | one-mode network + constant covariate | siena07 | `samexrecip()` |
| `inPopX` | one-mode network + constant covariate | siena07 | `inpopx()` |
| `outPopX` | one-mode network + constant covariate | siena07 | `outpopx()` |
| `inActX` | one-mode network + constant covariate | siena07 | `inactx()` |
| `outActX` | one-mode network + constant covariate | siena07 | `outactx()` |
| `sameXInPop` | one-mode network + constant covariate | siena07 | `samexinpop()` |
| `diffXInPop` | one-mode network + constant covariate | siena07 | `diffxinpop()` |
| `sameXOutAct` | one-mode network + constant covariate | siena07 | `samexoutact()` |
| `diffXOutAct` | one-mode network + constant covariate | siena07 | `diffxoutact()` |
| `crossXOutAct` | one-mode network + constant covariate | siena07 | `netx(crossxoutact:var)` |
| `homXOutAct` | one-mode network + constant covariate | siena07 | `homxoutact()` |
| `homXOutAct2` | one-mode network + constant covariate | siena07 | `netx(homxoutact2:var)` |
| `altXOutAct` | one-mode network + constant covariate | siena07 | `altxoutact()` |
| `sameXReciAct` | one-mode network + constant covariate | siena07 | `samexreciact()` |
| `diffXReciAct` | one-mode network + constant covariate | siena07 | `diffxreciact()` |
| `transTripX` | one-mode network + constant covariate | siena07 | `netx(transtripx:var)` |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameXTransTrip | one-mode network + constant covariate | siena07 | samextranstrip() |
| diffXTransTrip | one-mode network + constant covariate | siena07 | diffxtranstrip() |
| homXTransTrip | one-mode network + constant covariate | siena07 | homxtranstrip() |
| jumpXTransTrip | one-mode network + constant covariate | siena07 | netx(jumpxtranstrip:var) |
| homXTransRecTrip | one-mode network + constant covariate | siena07 | netx(homxtransrectrip:var) |
| sameXTransRecTrip | one-mode network + constant covariate | siena07 | netx(samextransrectrip:var) |
| diffXTransRecTrip | one-mode network + constant covariate | siena07 | netx(diffxtransrectrip:var) |
| sameXCycle4 | one-mode network + constant covariate | siena07 | netx(samexcycle4:var) |
| sameInXCycle4 | one-mode network + constant covariate | siena07 | netx(sameinxcycle4:var) |
| egoXaltX | one-mode network + constant covariate | siena07 | egoxaltx() |
| egoXaltXRecip | one-mode network + constant covariate | siena07 | netx(egoxaltxrecip:var) |
| IndTies | one-mode network + constant covariate | siena07 | netx(indties:var) |
| altDist2 | one-mode network + constant covariate | siena07 | netx(altdist2:var) |
| totDist2 | one-mode network + constant covariate | siena07 | netx(totdist2:var) |
| simDist2 | one-mode network + constant covariate | siena07 | netx(simdist2:var) |
| simEgoDist2 | one-mode network + constant covariate | siena07 | netx(simegodist2:var) |
| sameEgoDist2 | one-mode network + constant covariate | siena07 | netx(sameegodist2:var) |
| altInDist2 | one-mode network + constant covariate | siena07 | netx(altindist2:var) |
| totInDist2 | one-mode network + constant covariate | siena07 | netx(totindist2:var) |
| simEgoInDist2 | one-mode network + constant covariate | siena07 | netx(simegoindist2:var) |
| sameEgoInDist2 | one-mode network + constant covariate | siena07 | netx(sameegoindist2:var) |
| egoX | two-mode network + constant covariate | siena07 | nodeocov() / egox() |
| egoSqX | two-mode network + constant covariate | siena07 | egosqx(var) |
| egoLThresholdX | two-mode network + constant covariate | siena07 | egolthresholdx(varlist) |
| egoRThresholdX | two-mode network + constant covariate | siena07 | egorthresholdx(varlist) |
| degAbsDiffX | two-mode network + constant covariate | siena07 | degabsdiffx(varlist) |
| degPosDiffX | two-mode network + constant covariate | siena07 | degposdiffx(varlist) |
| degNegDiffX | two-mode network + constant covariate | siena07 | degnegdiffx(varlist) |
| altInDist2 | two-mode network + constant covariate | siena07 | netx(altindist2:var) |
| totInDist2 | two-mode network + constant covariate | siena07 | netx(totindist2:var) |
| simEgoInDist2 | two-mode network + constant covariate | siena07 | netx(simegoindist2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameEgoInDist2 | two-mode network + constant covariate | siena07 | netx(sameegoindist2:var) |
| inPopX | two-mode network + constant covariate | siena07 | inpopx(varlist) |
| sameXInPop | two-mode network + constant covariate | siena07 | samexinpop(varlist) |
| diffXInPop | two-mode network + constant covariate | siena07 | diffxinpop(varlist) |
| crossXOutAct | two-mode network + constant covariate | siena07 | netx(crossxoutact:var) |
| sameXCycle4 | two-mode network + constant covariate | siena07 | netx(samexcycle4:var) |
| altX | two-mode network + constant covariate | siena07 | nodeicov() / altx() |
| altSqX | two-mode network + constant covariate | siena07 | altsqx(var) |
| altLThresholdX | two-mode network + constant covariate | siena07 | altlthresholdx(varlist) |
| altRThresholdX | two-mode network + constant covariate | siena07 | altrthresholdx(varlist) |
| outActX | two-mode network + constant covariate | siena07 | outactx(varlist) |
| sameInXCycle4 | two-mode network + constant covariate | siena07 | netx(sameinxcycle4:var) |
| homXOutAct | two-mode network + constant covariate | siena07 | homxoutact(varlist) |
| homXOutAct2 | two-mode network + constant covariate | siena07 | netx(homxoutact2:var) xparam(3) |
| altXOutAct | two-mode network + constant covariate | siena07 | altxoutact(varlist) |
| altX | non-directed network + constant covariate | siena07 | altx() |
| altSqX | non-directed network + constant covariate | siena07 | altsqx() |
| altLThresholdX | non-directed network + constant covariate | siena07 | altlthresholdx() |
| altRThresholdX | non-directed network + constant covariate | siena07 | altrthresholdx() |
| outXMore | non-directed network + constant covariate | siena07 | netx(outxmore:var) |
| egoX | non-directed network + constant covariate | siena07 | egox() |
| egoSqX | non-directed network + constant covariate | siena07 | egosqx() |
| egoPlusAltX | non-directed network + constant covariate | siena07 | nodecov() |
| egoPlusAltSqX | non-directed network + constant covariate | siena07 | egoplusaltsqx() |
| egoLThresholdX | non-directed network + constant covariate | siena07 | egolthresholdx() |
| egoRThresholdX | non-directed network + constant covariate | siena07 | egorthresholdx() |
| degAbsDiffX | non-directed network + constant covariate | siena07 | degabsdiffx() |
| degPosDiffX | non-directed network + constant covariate | siena07 | degposdiffx() |
| degNegDiffX | non-directed network + constant covariate | siena07 | degnegdiffx() |
| diffX | non-directed network + constant covariate | siena07 | diffx() |
| absDiffX | non-directed network + constant covariate | siena07 | absdiffx() |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| diffSqX | non-directed network + constant covariate | siena07 | diffsqx() |
| egoDiffX | non-directed network + constant covariate | siena07 | egodiffx() |
| altHigherEgoX | non-directed network + constant covariate | siena07 | althigheregox() |
| higher | non-directed network + constant covariate | siena07 | higher() |
| simX | non-directed network + constant covariate | siena07 | simx() |
| inPopX | non-directed network + constant covariate | siena07 | inpopx() |
| outPopX | non-directed network + constant covariate | siena07 | outpopx() |
| inActX | non-directed network + constant covariate | siena07 | inactx() |
| outActX | non-directed network + constant covariate | siena07 | outactx() |
| sameX | non-directed network + constant covariate | siena07 | samex() |
| unequalX | non-directed network + constant covariate | siena07 | unequalx() |
| sameXInPop | non-directed network + constant covariate | siena07 | samexinpop() |
| diffXInPop | non-directed network + constant covariate | siena07 | diffxinpop() |
| sameXOutAct | non-directed network + constant covariate | siena07 | samexoutact() |
| diffXOutAct | non-directed network + constant covariate | siena07 | diffxoutact() |
| crossXOutAct | non-directed network + constant covariate | siena07 | netx(crossxoutact:var) |
| homXOutAct | non-directed network + constant covariate | siena07 | homxoutact() |
| homXOutAct2 | non-directed network + constant covariate | siena07 | netx(homxoutact2:var) |
| altXOutAct | non-directed network + constant covariate | siena07 | altxoutact() |
| transTripX | non-directed network + constant covariate | siena07 | netx(transtripx:var) |
| sameXTransTrip | non-directed network + constant covariate | siena07 | samextranstrip() |
| diffXTransTrip | non-directed network + constant covariate | siena07 | diffxtranstrip() |
| homXTransTrip | non-directed network + constant covariate | siena07 | homxtranstrip() |
| jumpXTransTrip | non-directed network + constant covariate | siena07 | netx(jumpxtranstrip:var) |
| sameXCycle4 | non-directed network + constant covariate | siena07 | netx(samexcycle4:var) |
| sameInXCycle4 | non-directed network + constant covariate | siena07 | netx(sameinxcycle4:var) |
| egoXaltX | non-directed network + constant covariate | siena07 | egoxaltx() |
| IndTies | non-directed network + constant covariate | siena07 | netx(indties:var) |
| altDist2 | non-directed network + constant covariate | siena07 | netx(altdist2:var) |
| totDist2 | non-directed network + constant covariate | siena07 | netx(totdist2:var) |
| simEgoDist2 | non-directed network + constant covariate | siena07 | netx(simegodist2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameEgoDist2 | non-directed network + constant covariate | siena07 | netx(sameegodist2:var) |
| simDist2 | non-directed network + constant covariate | siena07 | netx(simdist2:var) |
| altInDist2 | non-directed network + constant covariate | siena07 | netx(altindist2:var) |
| totInDist2 | non-directed network + constant covariate | siena07 | netx(totindist2:var) |
| simEgoInDist2 | non-directed network + constant covariate | siena07 | netx(simegoindist2:var) |

## H. Network effects of monadic covariates (changing covariate, varCovar)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altX | one-mode network + changing covariate | siena07 | altx() with a tvcov stem |
| altSqX | one-mode network + changing covariate | siena07 | altsqx() with a tvcov stem |
| altLThresholdX | one-mode network + changing covariate | siena07 | altlthresholdx() with a tvcov stem |
| altRThresholdX | one-mode network + changing covariate | siena07 | altrthresholdx() with a tvcov stem |
| outXMore | one-mode network + changing covariate | siena07 | netx(outxmore:var) |
| egoX | one-mode network + changing covariate | siena07 | egox() with a tvcov stem |
| egoSqX | one-mode network + changing covariate | siena07 | egosqx() with a tvcov stem |
| egoLThresholdX | one-mode network + changing covariate | siena07 | egolthresholdx() with a tvcov stem |
| egoRThresholdX | one-mode network + changing covariate | siena07 | egorthresholdx() with a tvcov stem |
| degAbsDiffX | one-mode network + changing covariate | siena07 | degabsdiffx() with a tvcov stem |
| degPosDiffX | one-mode network + changing covariate | siena07 | degposdiffx() with a tvcov stem |
| degNegDiffX | one-mode network + changing covariate | siena07 | degnegdiffx() with a tvcov stem |
| diffX | one-mode network + changing covariate | siena07 | diffx() with a tvcov stem |
| absDiffX | one-mode network + changing covariate | siena07 | absdiffx() with a tvcov stem |
| diffSqX | one-mode network + changing covariate | siena07 | diffsqx() with a tvcov stem |
| egoDiffX | one-mode network + changing covariate | siena07 | egodiffx() with a tvcov stem |
| egoPlusAltX | one-mode network + changing covariate | siena07 | nodecov() with a tvcov stem |
| altHigherEgoX | one-mode network + changing covariate | siena07 | althigheregox() with a tvcov stem |
| higher | one-mode network + changing covariate | siena07 | higher() with a tvcov stem |
| egoPlusAltSqX | one-mode network + changing covariate | siena07 | egoplusaltsqx() with a tvcov stem |
| simX | one-mode network + changing covariate | siena07 | simx() with a tvcov stem |
| simRecipX | one-mode network + changing covariate | siena07 | netx(simrecipx:var) / netbeh(simrecipx) |
| simXTransTrip | one-mode network + changing covariate | siena07 | netx(simxtranstrip:var) / netbeh(simxtranstrip) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameX | one-mode network + changing covariate | siena07 | samex() with a tvcov stem |
| unequalX | one-mode network + changing covariate | siena07 | unequalx() with a tvcov stem |
| sameXRecip | one-mode network + changing covariate | siena07 | samexrecip() with a tvcov stem |
| inPopX | one-mode network + changing covariate | siena07 | inpopx() with a tvcov stem |
| outPopX | one-mode network + changing covariate | siena07 | outpopx() with a tvcov stem |
| inActX | one-mode network + changing covariate | siena07 | inactx() with a tvcov stem |
| outActX | one-mode network + changing covariate | siena07 | outactx() with a tvcov stem |
| sameXInPop | one-mode network + changing covariate | siena07 | samexinpop() with a tvcov stem |
| diffXInPop | one-mode network + changing covariate | siena07 | diffxinpop() with a tvcov stem |
| sameXOutAct | one-mode network + changing covariate | siena07 | samexoutact() with a tvcov stem |
| diffXOutAct | one-mode network + changing covariate | siena07 | diffxoutact() with a tvcov stem |
| crossXOutAct | one-mode network + changing covariate | siena07 | netx(crossxoutact:var) |
| homXOutAct | one-mode network + changing covariate | siena07 | homxoutact() with a tvcov stem |
| homXOutAct2 | one-mode network + changing covariate | siena07 | netx(homxoutact2:var) |
| altXOutAct | one-mode network + changing covariate | siena07 | altxoutact() with a tvcov stem |
| sameXReciAct | one-mode network + changing covariate | siena07 | samexreciact() with a tvcov stem |
| diffXReciAct | one-mode network + changing covariate | siena07 | diffxreciact() with a tvcov stem |
| transTripX | one-mode network + changing covariate | siena07 | netx(transtripx:var) |
| sameXTransTrip | one-mode network + changing covariate | siena07 | samextranstrip() with a tvcov stem |
| diffXTransTrip | one-mode network + changing covariate | siena07 | diffxtranstrip() with a tvcov stem |
| homXTransTrip | one-mode network + changing covariate | siena07 | homxtranstrip() with a tvcov stem |
| jumpXTransTrip | one-mode network + changing covariate | siena07 | netx(jumpxtranstrip:var) |
| homXTransRecTrip | one-mode network + changing covariate | siena07 | netx(homxtransrectrip:var) |
| sameXTransRecTrip | one-mode network + changing covariate | siena07 | netx(samextransrectrip:var) |
| diffXTransRecTrip | one-mode network + changing covariate | siena07 | netx(diffxtransrectrip:var) |
| sameXCycle4 | one-mode network + changing covariate | siena07 | netx(samexcycle4:var) |
| sameInXCycle4 | one-mode network + changing covariate | siena07 | netx(sameinxcycle4:var) |
| egoXaltX | one-mode network + changing covariate | siena07 | egoxaltx() with a tvcov stem |
| egoXaltXRecip | one-mode network + changing covariate | siena07 | netx(egoxaltxrecip:var) |
| IndTies | one-mode network + changing covariate | siena07 | netx(indties:var) |
| altDist2 | one-mode network + changing covariate | siena07 | netx(altdist2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totDist2 | one-mode network + changing covariate | siena07 | netx(totdist2:var) |
| simDist2 | one-mode network + changing covariate | siena07 | netx(simdist2:var) |
| simEgoDist2 | one-mode network + changing covariate | siena07 | netx(simegodist2:var) |
| sameEgoDist2 | one-mode network + changing covariate | siena07 | netx(sameegodist2:var) |
| altInDist2 | one-mode network + changing covariate | siena07 | netx(altindist2:var) |
| totInDist2 | one-mode network + changing covariate | siena07 | netx(totindist2:var) |
| simEgoInDist2 | one-mode network + changing covariate | siena07 | netx(simegoindist2:var) |
| sameEgoInDist2 | one-mode network + changing covariate | siena07 | netx(sameegoindist2:var) |
| avGroupEgoX | one-mode network + changing covariate | siena07 | netx(avgroupegox:var) |
| totGroupEgoX | one-mode network + changing covariate | siena07 | netx(totgroupegox:var) |
| egoX | two-mode network + changing covariate | siena07 | nodeocov(tvstem) |
| egoSqX | two-mode network + changing covariate | siena07 | egosqx(tvstem) |
| egoLThresholdX | two-mode network + changing covariate | siena07 | egolthresholdx(varlist) |
| egoRThresholdX | two-mode network + changing covariate | siena07 | egorthresholdx(varlist) |
| degAbsDiffX | two-mode network + changing covariate | siena07 | degabsdiffx(varlist) |
| degPosDiffX | two-mode network + changing covariate | siena07 | degposdiffx(varlist) |
| degNegDiffX | two-mode network + changing covariate | siena07 | degnegdiffx(varlist) |
| altInDist2 | two-mode network + changing covariate | siena07 | netx(altindist2:tvstem) |
| totInDist2 | two-mode network + changing covariate | siena07 | netx(totindist2:tvstem) |
| simEgoInDist2 | two-mode network + changing covariate | siena07 | netx(simegoindist2:tvstem) |
| sameEgoInDist2 | two-mode network + changing covariate | siena07 | netx(sameegoindist2:tvstem) |
| inPopX | two-mode network + changing covariate | siena07 | inpopx(varlist) |
| sameXInPop | two-mode network + changing covariate | siena07 | samexinpop(varlist) |
| diffXInPop | two-mode network + changing covariate | siena07 | diffxinpop(varlist) |
| crossXOutAct | two-mode network + changing covariate | siena07 | netx(crossxoutact:tvstem) |
| sameXCycle4 | two-mode network + changing covariate | siena07 | netx(samexcycle4:tvstem) |
| avGroupEgoX | two-mode network + changing covariate | siena07 | netx(avgroupegox:tvstem) |
| totGroupEgoX | two-mode network + changing covariate | siena07 | netx(totgroupegox:tvstem) |
| altX | non-directed network + changing covariate | siena07 | altx() |
| altSqX | non-directed network + changing covariate | siena07 | altsqx() |
| altLThresholdX | non-directed network + changing covariate | siena07 | altlthresholdx() |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altRThresholdX | non-directed network + changing covariate | siena07 | altrthresholdx() |
| outXMore | non-directed network + changing covariate | siena07 | netx(outxmore:var) |
| egoX | non-directed network + changing covariate | siena07 | egox() |
| egoSqX | non-directed network + changing covariate | siena07 | egosqx() |
| egoPlusAltX | non-directed network + changing covariate | siena07 | tvcov() + nodecov(stem) |
| egoPlusAltSqX | non-directed network + changing covariate | siena07 | egoplusaltsqx() |
| egoLThresholdX | non-directed network + changing covariate | siena07 | egolthresholdx() |
| egoRThresholdX | non-directed network + changing covariate | siena07 | egorthresholdx() |
| degAbsDiffX | non-directed network + changing covariate | siena07 | degabsdiffx() |
| degPosDiffX | non-directed network + changing covariate | siena07 | degposdiffx() |
| degNegDiffX | non-directed network + changing covariate | siena07 | degnegdiffx() |
| diffX | non-directed network + changing covariate | siena07 | diffx() |
| absDiffX | non-directed network + changing covariate | siena07 | absdiffx() |
| diffSqX | non-directed network + changing covariate | siena07 | diffsqx() |
| egoDiffX | non-directed network + changing covariate | siena07 | egodiffx() |
| altHigherEgoX | non-directed network + changing covariate | siena07 | althigheregox() |
| higher | non-directed network + changing covariate | siena07 | symmetric higher |
| simX | non-directed network + changing covariate | siena07 | simx() |
| inPopX | non-directed network + changing covariate | siena07 | inpopx() |
| outPopX | non-directed network + changing covariate | siena07 | outpopx() |
| inActX | non-directed network + changing covariate | siena07 | inactx() |
| outActX | non-directed network + changing covariate | siena07 | outactx() |
| sameX | non-directed network + changing covariate | siena07 | samex() |
| unequalX | non-directed network + changing covariate | siena07 | unequalx() |
| sameXInPop | non-directed network + changing covariate | siena07 | samexinpop() |
| diffXInPop | non-directed network + changing covariate | siena07 | diffxinpop() |
| sameXOutAct | non-directed network + changing covariate | siena07 | samexoutact() |
| diffXOutAct | non-directed network + changing covariate | siena07 | diffxoutact() |
| crossXOutAct | non-directed network + changing covariate | siena07 | netx(crossxoutact:var) |
| homXOutAct | non-directed network + changing covariate | siena07 | homxoutact() |
| homXOutAct2 | non-directed network + changing covariate | siena07 | netx(homxoutact2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altXOutAct | non-directed network + changing covariate | siena07 | altxoutact() |
| transTripX | non-directed network + changing covariate | siena07 | netx(transtripx:var) |
| sameXTransTrip | non-directed network + changing covariate | siena07 | samextranstrip() |
| diffXTransTrip | non-directed network + changing covariate | siena07 | diffxtranstrip() |
| homXTransTrip | non-directed network + changing covariate | siena07 | homxtranstrip() |
| jumpXTransTrip | non-directed network + changing covariate | siena07 | netx(jumpxtranstrip:var) |
| sameXCycle4 | non-directed network + changing covariate | siena07 | netx(samexcycle4:var) |
| sameInXCycle4 | non-directed network + changing covariate | siena07 | netx(sameinxcycle4:var) |
| egoXaltX | non-directed network + changing covariate | siena07 | egoxaltx() |
| IndTies | non-directed network + changing covariate | siena07 | netx(indties:var) |
| altDist2 | non-directed network + changing covariate | siena07 | netx(altdist2:var) |
| totDist2 | non-directed network + changing covariate | siena07 | netx(totdist2:var) |
| simEgoDist2 | non-directed network + changing covariate | siena07 | netx(simegodist2:var) |
| sameEgoDist2 | non-directed network + changing covariate | siena07 | netx(sameegodist2:var) |
| simDist2 | non-directed network + changing covariate | siena07 | netx(simdist2:var) |
| altInDist2 | non-directed network + changing covariate | siena07 | netx(altindist2:var) |
| totInDist2 | non-directed network + changing covariate | siena07 | netx(totindist2:var) |
| simEgoInDist2 | non-directed network + changing covariate | siena07 | netx(simegoindist2:var) |
| avGroupEgoX | non-directed network + changing covariate | siena07 | netx(avgroupegox:var) |
| totGroupEgoX | non-directed network + changing covariate | siena07 | netx(totgroupegox:var) |

## I. Network effects of two covariates (double covariate effects)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameXV | one-mode network + constant covariate + constant covariate | siena07 | netx(samexv:var1:var2) |
| sameXVInPop | one-mode network + constant covariate + constant covariate | siena07 | netx(samexvinpop:var1:var2) |
| sameXVInPop2 | one-mode network + constant covariate + constant covariate | siena07 | netx(samexvinpop2:var1:var2) |

## J. Network effects of dyadic covariates (coDyadCovar, varDyadCovar)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| X | one-mode network + constant dyadic covariate | siena07 | dyadcov() |
| XRecip | one-mode network + constant dyadic covariate | siena07 | dyadcovrecip() |
| WWX | one-mode network + constant dyadic covariate | siena07 | dyadx(wwx:matrix) |
| cyWWX | one-mode network + constant dyadic covariate | siena07 | dyadx(cywwx:matrix) |
| InWWX | one-mode network + constant dyadic covariate | siena07 | dyadx(inwwx:matrix) |
| OutWWX | one-mode network + constant dyadic covariate | siena07 | dyadx(outwwx:matrix) |
| WXX | one-mode network + constant dyadic covariate | siena07 | dyadx(wxx:matrix) |
| XWX | one-mode network + constant dyadic covariate | siena07 | dyadx(xwx:matrix) |
| XWX1 | one-mode network + constant dyadic covariate | siena07 | dyadx(xwx1:matrix) |
| XWX2 | one-mode network + constant dyadic covariate | siena07 | dyadx(xwx2:matrix) |
| XXW | one-mode network + constant dyadic covariate | siena07 | dyadx(xxw:matrix) |
| X | one-mode network + changing dyadic covariate | siena07 | tvdyadcov(stem) |
| XRecip | one-mode network + changing dyadic covariate | siena07 | tvdyadcovrecip(stem) |
| WWX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(wwx:stem) |
| cyWWX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(cywwx:stem) |
| InWWX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(inwwx:stem) |
| OutWWX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(outwwx:stem) |
| WXX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(wxx:stem) |
| XWX | one-mode network + changing dyadic covariate | siena07 | tvdyadx(xwx:stem) |
| XWX1 | one-mode network + changing dyadic covariate | siena07 | tvdyadx(xwx1:stem) |
| XWX2 | one-mode network + changing dyadic covariate | siena07 | tvdyadx(xwx2:stem) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| XXW | one-mode network + changing dyadic covariate | siena07 | tvdyadx(xxw:stem) |
| X | two-mode network + constant dyadic covariate | siena07 | dyadcov |
| X | non-directed network + constant dyadic covariate | siena07 | dyadcov() |
| XRecip | non-directed network + constant dyadic covariate | siena07 | dyadcovrecip() |
| WWX | non-directed network + constant dyadic covariate | siena07 | dyadx(wwx:matrix) |
| cyWWX | non-directed network + constant dyadic covariate | siena07 | dyadx(cywwx:matrix) |
| InWWX | non-directed network + constant dyadic covariate | siena07 | dyadx(inwwx:matrix) |
| OutWWX | non-directed network + constant dyadic covariate | siena07 | dyadx(outwwx:matrix) |
| WXX | non-directed network + constant dyadic covariate | siena07 | dyadx(wxx:matrix) |
| XWX | non-directed network + constant dyadic covariate | siena07 | dyadx(xwx:matrix) |
| XWX1 | non-directed network + constant dyadic covariate | siena07 | dyadx(xwx1:matrix) |
| XWX2 | non-directed network + constant dyadic covariate | siena07 | dyadx(xwx2:matrix) |
| XXW | non-directed network + constant dyadic covariate | siena07 | dyadx(xxw:matrix) |
| X | non-directed network + changing dyadic covariate | siena07 | tvdyadcov(stem) |
| XRecip | non-directed network + changing dyadic covariate | siena07 | symmetric tvdyadcovrecip |
| WWX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(wwx:stem) |
| cyWWX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(cywwx:stem) |
| InWWX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(inwwx:stem) |
| OutWWX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(outwwx:stem) |
| WXX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(wxx:stem) |
| XWX | non-directed network + changing dyadic covariate | siena07 | tvdyadx(xwx:stem) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| XWX1 | non-directed network + changing dyadic covariate | siena07 | tvdyadx(xwx1:stem) |
| XWX2 | non-directed network + changing dyadic covariate | siena07 | tvdyadx(xwx2:stem) |
| XXW | non-directed network + changing dyadic covariate | siena07 | tvdyadx(xxw:stem) |

## K. Network effects of a dependent behavior variable (behavior as covariate: selection)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altX | one-mode network + a behavior variable | siena07 | behaltx |
| altSqX | one-mode network + a behavior variable | siena07 | netbeh(altsqx) |
| altLThresholdX | one-mode network + a behavior variable | siena07 | netbeh(altlthresholdx) |
| altRThresholdX | one-mode network + a behavior variable | siena07 | netbeh(altrthresholdx) |
| outXMore | one-mode network + a behavior variable | siena07 | netbeh(outxmore) |
| egoX | one-mode network + a behavior variable | siena07 | behegox |
| egoSqX | one-mode network + a behavior variable | siena07 | netbeh(egosqx) |
| egoLThresholdX | one-mode network + a behavior variable | siena07 | netbeh(egolthresholdx) |
| egoRThresholdX | one-mode network + a behavior variable | siena07 | netbeh(egorthresholdx) |
| degAbsDiffX | one-mode network + a behavior variable | siena07 | netbeh(degabsdiffx) |
| degPosDiffX | one-mode network + a behavior variable | siena07 | netbeh(degposdiffx) |
| degNegDiffX | one-mode network + a behavior variable | siena07 | netbeh(degnegdiffx) |
| diffX | one-mode network + a behavior variable | siena07 | netbeh(diffx) |
| absDiffX | one-mode network + a behavior variable | siena07 | netbeh(absdiffx) |
| diffSqX | one-mode network + a behavior variable | siena07 | netbeh(diffsqx) |
| egoDiffX | one-mode network + a behavior variable | siena07 | netbeh(egodiffx) |
| egoPlusAltX | one-mode network + a behavior variable | siena07 | behegoplusalt |
| altHigherEgoX | one-mode network + a behavior variable | siena07 | netbeh(althigheregox) |
| higher | one-mode network + a behavior variable | siena07 | netbeh(higher) |
| egoPlusAltSqX | one-mode network + a behavior variable | siena07 | netbeh(egoplusaltsqx) |
| simX | one-mode network + a behavior variable | siena07 | behsim |
| simRecipX | one-mode network + a behavior variable | siena07 | netx(simrecipx:var) / netbeh(simrecipx) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| simXTransTrip | one-mode network + a behavior variable | siena07 | netx(simxtranstrip:var) / netbeh(simxtranstrip) |
| sameX | one-mode network + a behavior variable | siena07 | netbeh(samex) |
| unequalX | one-mode network + a behavior variable | siena07 | netbeh(unequalx) |
| sameXRecip | one-mode network + a behavior variable | siena07 | netbeh(samexrecip) |
| inPopX | one-mode network + a behavior variable | siena07 | netbeh(inpopx) |
| outPopX | one-mode network + a behavior variable | siena07 | netbeh(outpopx) |
| inActX | one-mode network + a behavior variable | siena07 | netbeh(inactx) |
| outActX | one-mode network + a behavior variable | siena07 | netbeh(outactx) |
| sameXInPop | one-mode network + a behavior variable | siena07 | netbeh(samexinpop) |
| diffXInPop | one-mode network + a behavior variable | siena07 | netbeh(diffxinpop) |
| sameXOutAct | one-mode network + a behavior variable | siena07 | netbeh(samexoutact) |
| diffXOutAct | one-mode network + a behavior variable | siena07 | netbeh(diffxoutact) |
| crossXOutAct | one-mode network + a behavior variable | siena07 | netbeh(crossxoutact) |
| homXOutAct | one-mode network + a behavior variable | siena07 | netbeh(homxoutact) |
| altXOutAct | one-mode network + a behavior variable | siena07 | netbeh(altxoutact) |
| sameXReciAct | one-mode network + a behavior variable | siena07 | netbeh(samexreciact) |
| diffXReciAct | one-mode network + a behavior variable | siena07 | netbeh(diffxreciact) |
| transTripX | one-mode network + a behavior variable | siena07 | netbeh(transtripx) |
| sameXTransTrip | one-mode network + a behavior variable | siena07 | netbeh(samextranstrip) |
| diffXTransTrip | one-mode network + a behavior variable | siena07 | netbeh(diffxtranstrip) |
| homXTransTrip | one-mode network + a behavior variable | siena07 | netbeh(homxtranstrip) |
| jumpXTransTrip | one-mode network + a behavior variable | siena07 | netbeh(jumpxtranstrip) |
| homXTransRecTrip | one-mode network + a behavior variable | siena07 | netbeh(homxtransrectrip) |
| sameXTransRecTrip | one-mode network + a behavior variable | siena07 | netbeh(samextransrectrip) |
| diffXTransRecTrip | one-mode network + a behavior variable | siena07 | netbeh(diffxtransrectrip) |
| sameXCycle4 | one-mode network + a behavior variable | siena07 | netbeh(samexcycle4) |
| sameInXCycle4 | one-mode network + a behavior variable | siena07 | netbeh(sameinxcycle4) |
| egoXaltX | one-mode network + a behavior variable | siena07 | netbeh(egoxaltx) |
| egoXaltXRecip | one-mode network + a behavior variable | siena07 | netbeh(egoxaltxrecip) |
| IndTies | one-mode network + a behavior variable | siena07 | netbeh(indties) |
| altDist2 | one-mode network + a behavior variable | siena07 | netbeh(altdist2) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totDist2 | one-mode network + a behavior variable | siena07 | netbeh(totdist2) |
| simDist2 | one-mode network + a behavior variable | siena07 | netbeh(simdist2) |
| simEgoDist2 | one-mode network + a behavior variable | siena07 | netbeh(simegodist2) |
| sameEgoDist2 | one-mode network + a behavior variable | siena07 | netbeh(sameegodist2) |
| altInDist2 | one-mode network + a behavior variable | siena07 | netbeh(altindist2) |
| totInDist2 | one-mode network + a behavior variable | siena07 | netbeh(totindist2) |
| simEgoInDist2 | one-mode network + a behavior variable | siena07 | netbeh(simegoindist2) |
| sameEgoInDist2 | one-mode network + a behavior variable | siena07 | netbeh(sameegoindist2) |
| avGroupEgoX | one-mode network + a behavior variable | siena07 | netbeh(avgroupegox) |
| totGroupEgoX | one-mode network + a behavior variable | siena07 | netbeh(totgroupegox) |
| egoX | two-mode network + a behavior variable | siena07 | behavior() + behegox |
| egoSqX | two-mode network + a behavior variable | siena07 | behavior() + netbeh(egosqx) |
| egoLThresholdX | two-mode network + a behavior variable | siena07 | egolthresholdx(varlist) |
| egoRThresholdX | two-mode network + a behavior variable | siena07 | egorthresholdx(varlist) |
| degAbsDiffX | two-mode network + a behavior variable | siena07 | degabsdiffx(varlist) |
| degPosDiffX | two-mode network + a behavior variable | siena07 | degposdiffx(varlist) |
| degNegDiffX | two-mode network + a behavior variable | siena07 | degnegdiffx(varlist) |
| altInDist2 | two-mode network + a behavior variable | siena07 | behavior() + netbeh(altindist2) |
| totInDist2 | two-mode network + a behavior variable | siena07 | behavior() + netbeh(totindist2) |
| simEgoInDist2 | two-mode network + a behavior variable | siena07 | behavior() + netbeh(simegoindist2) |
| sameEgoInDist2 | two-mode network + a behavior variable | siena07 | behavior() + netbeh(sameegoindist2) |
| inPopX | two-mode network + a behavior variable | siena07 | inpopx(varlist) |
| sameXInPop | two-mode network + a behavior variable | siena07 | samexinpop(varlist) |
| diffXInPop | two-mode network + a behavior variable | siena07 | diffxinpop(varlist) |
| crossXOutAct | two-mode network + a behavior variable | siena07 | behavior |
| sameXCycle4 | two-mode network + a behavior variable | siena07 | behavior() + netbeh(samexcycle4) |
| avGroupEgoX | two-mode network + a behavior variable | siena07 | netbeh |
| totGroupEgoX | two-mode network + a behavior variable | siena07 | netbeh |
| altX | non-directed network + a behavior variable | siena07 | behaltx |
| altSqX | non-directed network + a behavior variable | siena07 | netbeh(altsqx) |
| altLThresholdX | non-directed network + a behavior variable | siena07 | netbeh(altlthresholdx) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| altRThresholdX | non-directed network + a behavior variable | siena07 | netbeh(altrthresholdx) |
| outXMore | non-directed network + a behavior variable | siena07 | netbeh(outxmore) |
| egoX | non-directed network + a behavior variable | siena07 | behegox |
| egoSqX | non-directed network + a behavior variable | siena07 | netbeh(egosqx) |
| egoPlusAltX | non-directed network + a behavior variable | siena07 | behegoplusalt |
| egoPlusAltSqX | non-directed network + a behavior variable | siena07 | netbeh(egoplusaltsqx) |
| egoLThresholdX | non-directed network + a behavior variable | siena07 | netbeh(egolthresholdx) |
| egoRThresholdX | non-directed network + a behavior variable | siena07 | netbeh(egorthresholdx) |
| degAbsDiffX | non-directed network + a behavior variable | siena07 | symmetric + behavior |
| degPosDiffX | non-directed network + a behavior variable | siena07 | netbeh |
| degNegDiffX | non-directed network + a behavior variable | siena07 | netbeh(degnegdiffx) |
| diffX | non-directed network + a behavior variable | siena07 | netbeh(diffx) |
| absDiffX | non-directed network + a behavior variable | siena07 | netbeh(absdiffx) |
| diffSqX | non-directed network + a behavior variable | siena07 | netbeh(diffsqx) |
| egoDiffX | non-directed network + a behavior variable | siena07 | netbeh(egodiffx) |
| altHigherEgoX | non-directed network + a behavior variable | siena07 | netbeh(althigheregox) |
| higher | non-directed network + a behavior variable | siena07 | netbeh |
| simX | non-directed network + a behavior variable | siena07 | behsim |
| inPopX | non-directed network + a behavior variable | siena07 | netbeh(inpopx) |
| outPopX | non-directed network + a behavior variable | siena07 | netbeh(outpopx) |
| inActX | non-directed network + a behavior variable | siena07 | netbeh(inactx) |
| outActX | non-directed network + a behavior variable | siena07 | netbeh(outactx) |
| sameX | non-directed network + a behavior variable | siena07 | netbeh(samex) |
| unequalX | non-directed network + a behavior variable | siena07 | netbeh(unequalx) |
| sameXInPop | non-directed network + a behavior variable | siena07 | netbeh(samexinpop) |
| diffXInPop | non-directed network + a behavior variable | siena07 | netbeh(diffxinpop) |
| sameXOutAct | non-directed network + a behavior variable | siena07 | netbeh(samexoutact) |
| diffXOutAct | non-directed network + a behavior variable | siena07 | netbeh(diffxoutact) |
| crossXOutAct | non-directed network + a behavior variable | siena07 | netbeh(crossxoutact) |
| homXOutAct | non-directed network + a behavior variable | siena07 | netbeh(homxoutact) |
| altXOutAct | non-directed network + a behavior variable | siena07 | netbeh(altxoutact) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| transTripX | non-directed network + a behavior variable | siena07 | netbeh(transtripx) |
| sameXTransTrip | non-directed network + a behavior variable | siena07 | netbeh(samextranstrip) |
| diffXTransTrip | non-directed network + a behavior variable | siena07 | netbeh(diffxtranstrip) |
| homXTransTrip | non-directed network + a behavior variable | siena07 | netbeh(homxtranstrip) |
| jumpXTransTrip | non-directed network + a behavior variable | siena07 | netbeh(jumpxtranstrip) |
| sameXCycle4 | non-directed network + a behavior variable | siena07 | netbeh(samexcycle4) |
| sameInXCycle4 | non-directed network + a behavior variable | siena07 | netbeh(sameinxcycle4) |
| egoXaltX | non-directed network + a behavior variable | siena07 | netbeh(egoxaltx) |
| IndTies | non-directed network + a behavior variable | siena07 | netbeh(indties) |
| altDist2 | non-directed network + a behavior variable | siena07 | netbeh(altdist2) |
| totDist2 | non-directed network + a behavior variable | siena07 | netbeh(totdist2) |
| simEgoDist2 | non-directed network + a behavior variable | siena07 | netbeh(simegodist2) |
| sameEgoDist2 | non-directed network + a behavior variable | siena07 | netbeh(sameegodist2) |
| simDist2 | non-directed network + a behavior variable | siena07 | netbeh(simdist2) |
| altInDist2 | non-directed network + a behavior variable | siena07 | netbeh(altindist2) |
| totInDist2 | non-directed network + a behavior variable | siena07 | netbeh(totindist2) |
| simEgoInDist2 | non-directed network + a behavior variable | siena07 | netbeh(simegoindist2) |
| avGroupEgoX | non-directed network + a behavior variable | siena07 | netbeh(avgroupegox) |
| totGroupEgoX | non-directed network + a behavior variable | siena07 | netbeh(totgroupegox) |

## L. Cross-network effects (several dependent networks)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| from.w.ind | one-mode network + another network + another network | siena07 | cross1 |
| crprod | one-mode network + another network | siena07 | nwsaom multinet, cross1(crprod) |
| crprodRecip | one-mode network + another network | siena07 | nwsaom multinet, cross1(crprodrecip) |
| crprodMutual | one-mode network + another network | siena07 | nwsaom multinet, cross1(crprodmutual) |
| inPopIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(inpopintn) |
| inActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(inactintn) |
| crprodInActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(crprodinactintn) |
| outPopIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(outpopintn) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| outActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(outactintn) |
| divOutEgoIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(divoutegointn) |
| divInEgoIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(divinegointn) |
| divOutAltIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(divoutaltintn) |
| divInAltIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(divinaltintn) |
| outOutActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(outoutactintn) |
| outOutAvIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(outoutavintn) |
| dist2OutInActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(dist2outinactintn) |
| outOutDist2ActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(outoutdist2actintn) |
| avAlt.2M.tot | one-mode network + another network | siena07 | nwsaom multinet, cross1(avalt2mtot) |
| avDegIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(avdegintn) |
| absOutDiffIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(absoutdiffintn) |
| both | one-mode network + another network | siena07 | nwsaom multinet, cross1(both) |
| doubleOutAct | one-mode network + another network | siena07 | nwsaom multinet, cross1(doubleoutact) |
| doubleInPop | one-mode network + another network | siena07 | nwsaom multinet, cross1(doubleinpop) |
| betweenPop | one-mode network + another network | siena07 | cross1 |
| from | one-mode network + another network | siena07 | nwsaom multinet, cross1(from) |
| fromMutual | one-mode network + another network | siena07 | nwsaom multinet, cross1(frommutual) |
| fromAny | one-mode network + another network | siena07 | nwsaom multinet, cross1(fromany) |
| to | one-mode network + another network | siena07 | nwsaom multinet, cross1(to) |
| toBack | one-mode network + another network | siena07 | nwsaom multinet, cross1(toback) |
| toRecip | one-mode network + another network | siena07 | nwsaom multinet, cross1(torecip) |
| toAny | one-mode network + another network | siena07 | cross1 |
| cl.XWX | one-mode network + another network | siena07 | nwsaom multinet, cross1(clxwx) |
| cl.XWX1 | one-mode network + another network | siena07 | nwsaom multinet, cross1(clxwx1) |
| cl.XWX2 | one-mode network + another network | siena07 | nwsaom multinet, cross1(clxwx2) |
| WWX.EE | one-mode network + another network | siena07 | nwsaom multinet, cross1(wwxee) |
| WWX.FR | one-mode network + another network | siena07 | nwsaom multinet, cross1(wwxfr) |
| WXX.FE | one-mode network + another network | siena07 | nwsaom multinet, cross1(wxxfe) |
| WXX.ER | one-mode network + another network | siena07 | nwsaom multinet, cross1(wxxer) |
| XWX.ER | one-mode network + another network | siena07 | nwsaom multinet, cross1(xwxer) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| XWX.FE | one-mode network + another network | siena07 | nwsaom multinet, cross1(xwxfe) |
| closure | one-mode network + another network | siena07 | nwsaom multinet, cross1(closure) |
| cyClosure | one-mode network + another network | siena07 | nwsaom multinet, cross1(cyclosure) |
| sharedIn | one-mode network + another network | siena07 | nwsaom multinet, cross1(sharedin) |
| mixedInXW | one-mode network + another network | siena07 | nwsaom multinet, cross1(mixedinxw) |
| sharedTo | one-mode network + another network | siena07 | nwsaom multinet, cross1(sharedto) |
| avAlt.2M.tie | one-mode network + another network | siena07 | nwsaom multinet, cross1(avalt2mtie) |
| inPopOutW | one-mode network + another network | siena07 | nwsaom multinet, cross1(inpopoutw) |
| nDist2ActIntn | one-mode network + another network | siena07 | nwsaom multinet, cross1(ndist2actintn) |
| JinMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(jinmix) |
| JoutMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(joutmix) |
| gwespFFMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(gwespffmix) |
| gwespBBMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(gwespbbmix) |
| gwespBFMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(gwespbfmix) |
| gwespFBMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(gwespfbmix) |
| gwespRRMix | one-mode network + another network | siena07 | nwsaom multinet, cross1(gwesprrmix) |
| gwespMix | one-mode network + another network | siena07 | nwsaom multinet cross1(gwespfbmix) |
| inActIntn | two-mode network + another network | siena07 | cross1 |
| outActIntn | two-mode network + another network | siena07 | cross1 |
| divOutEgoIntn | two-mode network + another network | siena07 | cross1 |
| divInEgoIntn | two-mode network + another network | siena07 | cross1 |
| divInAltIntn | two-mode network + another network | siena07 | cross1 |
| avDegIntn | two-mode network + another network | siena07 | cross1 |
| outOutActIntn | two-mode network + another network | siena07 | cross1 |
| outOutAvIntn | two-mode network + another network | siena07 | cross1 |
| dist2OutInActIntn | two-mode network + another network | siena07 | cross1 |
| outOutDist2ActIntn | two-mode network + another network | siena07 | cross1 |
| avAlt.2M.tot | two-mode network + another network | siena07 | cross1 |
| to | two-mode network + another network | siena07 | cross1 |
| toBack | two-mode network + another network | siena07 | cross1 |
| toRecip | two-mode network + another network | siena07 | cross1 |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sharedTo | two-mode network + another network | siena07 | cross1 |
| avAlt.2M.tie | two-mode network + another network | siena07 | cross1 |
| inPopOutW | two-mode network + another network | siena07 | cross1 |
| nDist2ActIntn | two-mode network + another network | siena07 | cross1 |
| toAny | two-mode network + another network | siena07 | cross1 |
| JoutMix | two-mode network + another network | siena07 | cross1 |
| gwespFBMix | two-mode network + another network | siena07 | cross1 |
| from.w.ind | non-directed network + another network + another network | siena07 | cross1 |
| outPopIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(outpopintn) |
| outActIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(outactintn) |
| divOutEgoIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(divoutegointn) |
| divInEgoIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(divinegointn) |
| avDegIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(avdegintn) |
| from | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(from) |
| fromAny | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(fromany) |
| to | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(to) |
| toAny | non-directed network + another network | siena07 | cross1 |
| cl.XWX | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(clxwx) |
| closure | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(closure) |
| mixedInXW | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(mixedinxw) |
| JoutMix | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(joutmix) |
| gwespMix | non-directed network + another network | siena07 | cross1 |
| crprod | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(crprod) |
| outOutActIntn | non-directed network + another network | siena07 | cross1 |
| outOutAvIntn | non-directed network + another network | siena07 | cross1 |
| dist2OutInActIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(dist2outinactintn) |
| outOutDist2ActIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(outoutdist2actintn) |
| outOutDist2AvIntn | non-directed network + another network | siena07 | NOT IMPLEMENTED |
| both | non-directed network + another network | siena07 | cross1 |
| absOutDiffIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(absoutdiffintn) |
| doubleOutAct | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(doubleoutact) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sharedTo | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(sharedto) |
| avAlt.2M.tie | non-directed network + another network | siena07 | cross1 |
| inPopOutW | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(inpopoutw) |
| nDist2ActIntn | non-directed network + another network | siena07 | nwsaom multinet sym1 cross1(ndist2actintn) |

## M. Cross-network effects combined with a covariate, a behavior or a dyadic covariate

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| sameWWClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samewwclosure:var) |
| diffWWClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(diffwwclosure:var) |
| jumpWWClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(jumpwwclosure:var) |
| jumpFrom | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(jumpfrom:var) |
| jumpSharedIn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(jumpsharedin:var) |
| sameWXClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samewxclosure:var) |
| diffWXClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(diffwxclosure:var) |
| jumpWXClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(jumpwxclosure:var) |
| homWXClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(homwxclosure:var) |
| sameXWClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samexwclosure:var) |
| diffXWClosure | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(diffxwclosure:var) |
| homCovNetNet | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(homcovnetnet:var) |
| contrastCovNetNet | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(contrastcovnetnet: var) |
| allDifCovNetNet | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(alldifcovnetnet:var) |
| homCovNetNetIn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(homcovnetnetin:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| contrastCovNetNetIn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(contrastcovnetnetin: var) |
| allDifCovNetNetIn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(alldifcovnetnetin: var) |
| simEgoDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(simegodist2w:var) |
| altInDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(altindist2w:var) |
| totInDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(totindist2w:var) |
| inActIntnX | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(inactintnx:var) |
| sameXInActIntn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samexinactintn:var) |
| covNetNet | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(covnetnet:var) |
| covNetNetIn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(covnetnetin:var) |
| inPopIntnX | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(inpopintnx:var) |
| sameXInPopIntn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samexinpopintn:var) |
| simEgoInDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(simegoindist2w:var) |
| outPopIntnX | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(outpopintnx:var) |
| outActIntnX | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(outactintnx:var) |
| sameXOutPopIntn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samexoutpopintn:var) |
| sameXOutActIntn | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(samexoutactintn:var) |
| altDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(altdist2w:var) |
| totDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(totdist2w:var) |
| simDist2W | one-mode network + another network + constant covariate | siena07 | nwsaom multinet cross1(simdist2w:var) |
| sameWWClosure | one-mode network + another network + a behavior variable | siena07 | cross1(samewwclosure:behavior) |
| diffWWClosure | one-mode network + another network + a behavior variable | siena07 | cross1(diffwwclosure:behavior) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| jumpWWClosure | one-mode network + another network + a behavior variable | siena07 | cross1(jumpwwclosure:behavior) |
| jumpFrom | one-mode network + another network + a behavior variable | siena07 | cross1(jumpfrom:behavior) |
| jumpSharedIn | one-mode network + another network + a behavior variable | siena07 | cross1(jumpsharedin:behavior) |
| sameWXClosure | one-mode network + another network + a behavior variable | siena07 | cross1(samewxclosure:behavior) |
| diffWXClosure | one-mode network + another network + a behavior variable | siena07 | cross1(diffwxclosure:behavior) |
| jumpWXClosure | one-mode network + another network + a behavior variable | siena07 | cross1(jumpwxclosure:behavior) |
| homWXClosure | one-mode network + another network + a behavior variable | siena07 | cross1(homwxclosure:behavior) |
| sameXWClosure | one-mode network + another network + a behavior variable | siena07 | cross1(samexwclosure:behavior) |
| diffXWClosure | one-mode network + another network + a behavior variable | siena07 | cross1(diffxwclosure:behavior) |
| homCovNetNet | one-mode network + another network + a behavior variable | siena07 | cross1(homcovnetnet:behavior) |
| contrastCovNetNet | one-mode network + another network + a behavior variable | siena07 | cross1(contrastcovnetnet: behavior) |
| allDifCovNetNet | one-mode network + another network + a behavior variable | siena07 | cross1(alldifcovnetnet: behavior) |
| homCovNetNetIn | one-mode network + another network + a behavior variable | siena07 | cross1(homcovnetnetin: behavior) |
| contrastCovNetNetIn | one-mode network + another network + a behavior variable | siena07 | cross1(contrastcovnetnetin: behavior) |
| allDifCovNetNetIn | one-mode network + another network + a behavior variable | siena07 | cross1(alldifcovnetnetin: behavior) |
| simEgoDist2W | one-mode network + another network + a behavior variable | siena07 | cross1(simegodist2w:behavior) |
| altInDist2W | one-mode network + another network + a behavior variable | siena07 | cross1(altindist2w:behavior) |
| totInDist2W | one-mode network + another network + a behavior variable | siena07 | cross1(totindist2w:behavior) |
| inActIntnX | one-mode network + another network + a behavior variable | siena07 | cross1(inactintnx:behavior) |
| sameXInActIntn | one-mode network + another network + a behavior variable | siena07 | cross1(samexinactintn: behavior) |
| covNetNet | one-mode network + another network + a behavior variable | siena07 | cross1(covnetnet:behavior) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `covNetNetIn` | one-mode network + another network + a behavior variable | siena07 | `cross1(covnetnetin:behavior)` |
| `inPopIntnX` | one-mode network + another network + a behavior variable | siena07 | `cross1(inpopintnx:behavior)` |
| `sameXInPopIntn` | one-mode network + another network + a behavior variable | siena07 | `cross1(samexinpopintn: behavior)` |
| `simEgoInDist2W` | one-mode network + another network + a behavior variable | siena07 | `cross1(simegoindist2w: behavior)` |
| `outPopIntnX` | one-mode network + another network + a behavior variable | siena07 | `cross1(outpopintnx:behavior)` |
| `outActIntnX` | one-mode network + another network + a behavior variable | siena07 | `cross1(outactintnx:behavior)` |
| `sameXOutPopIntn` | one-mode network + another network + a behavior variable | siena07 | `cross1(samexoutpopintn: behavior)` |
| `sameXOutActIntn` | one-mode network + another network + a behavior variable | siena07 | `cross1(samexoutactintn: behavior)` |
| `altDist2W` | one-mode network + another network + a behavior variable | siena07 | `cross1(altdist2w:behavior)` |
| `totDist2W` | one-mode network + another network + a behavior variable | siena07 | `cross1(totdist2w:behavior)` |
| `simDist2W` | one-mode network + another network + a behavior variable | siena07 | `cross1(simdist2w:behavior)` |
| `sameWXClosure` | two-mode network + another network + constant covariate | siena07 | `cross1` |
| `sameWXClosure` | two-mode network + another network + changing covariate | target only | `cross1(samewxclosure:var) with tvcov()` |
| `sharedToU` | two-mode network + another network + constant dyadic covari... | siena07 | `cross1` |
| `avAltU.2M.tie` | two-mode network + another network + constant dyadic covari... | siena07 | `cross1` |
| `toU` | one-mode network + another network + constant dyadic covari... | siena07 | `cross1` |

## P. Behavior: shape effects

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `linear` | behavior | siena07 | `linear` |
| `quad` | behavior | siena07 | `quadratic` |
| `quad_cc` | behavior | siena07 | `behshape(quad_cc)` |
| `quad_nc` | behavior | siena07 | `behshape(quad_nc)` |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| constant | behavior | siena07 | beheff1 |
| threshold | behavior | siena07 | behshape(threshold:#) |
| threshold2 | behavior | siena07 | behshape(threshold:#) |
| threshold3 | behavior | siena07 | behshape(threshold:#) |
| threshold4 | behavior | siena07 | behshape(threshold:#) |
| simAllNear | behavior | siena07 | behshape(simallnear:#) |
| simAllFar | behavior | siena07 | behshape(simallfar:#) |
| avGroup | behavior | siena07 | behshape(avgroup:#) |
| totGroup | behavior | siena07 | behshape(totgroup:#) |
| behUnspInt | behavior | siena07 | behint |

## Q. Behavior: effects of the network (influence, degree and exposure effects)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avSim | behavior + another network | siena07 | avsim |
| avAttHigher | behavior + another network | siena07 | beheff(avatthigher) |
| avAttLower | behavior + another network | siena07 | beheff(avattlower) |
| totSim | behavior + another network | siena07 | beheff(totsim) |
| totAttHigher | behavior + another network | siena07 | beheff(totatthigher) |
| totAttLower | behavior + another network | siena07 | beheff(totattlower) |
| avInSim | behavior + another network | siena07 | beheff(avinsim) |
| totInSim | behavior + another network | siena07 | beheff(totinsim) |
| indegAvGroup | behavior + another network | siena07 | behnet2(indegavgroup:#) |
| indegTotGroup | behavior + another network | siena07 | behnet2(indegtotgroup:#) |
| indeg | behavior + another network | siena07 | beheff(indeg) |
| outdeg | behavior + another network | siena07 | beheff(outdeg) |
| isolate | behavior + another network | siena07 | beheff(isolate) |
| outIsolate | behavior + another network | siena07 | beheff(outisolate) |
| avSimRecip | behavior + another network | siena07 | beheff(avsimrecip) |
| totSimRecip | behavior + another network | siena07 | beheff(totsimrecip) |
| avSimPopAlt | behavior + another network | siena07 | beheff(avsimpopalt) |
| totSimPopAlt | behavior + another network | siena07 | beheff(totsimpopalt) |
| avInSimPopAlt | behavior + another network | siena07 | beheff(avinsimpopalt) |
| totInSimPopAlt | behavior + another network | siena07 | beheff(totinsimpopalt) |
| popAlt | behavior + another network | siena07 | beheff(popalt) |
| totPopAlt | behavior + another network | siena07 | beheff(totpopalt) |
| avSimRecPop | behavior + another network | siena07 | beheff(avsimrecpop) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totSimRecPop | behavior + another network | siena07 | beheff(totsimrecpop) |
| avSimVarAlt | behavior + another network | siena07 | behnet2(avsimvaralt) |
| avAlt | behavior + another network | siena07 | avalt |
| avAlt_cc | behavior + another network | siena07 | behnet2(avaltcc) |
| totAlt | behavior + another network | siena07 | beheff(totalt) |
| totAlt_cc | behavior + another network | siena07 | behnet2(totaltcc) |
| avRecAlt | behavior + another network | siena07 | beheff(avrecalt) |
| totRecAlt | behavior + another network | siena07 | beheff(totrecalt) |
| avAltPop | behavior + another network | siena07 | beheff(avaltpop) |
| totAltPop | behavior + another network | siena07 | beheff(totaltpop) |
| avInAlt | behavior + another network | siena07 | beheff(avinalt) |
| totInAlt | behavior + another network | siena07 | beheff(totinalt) |
| avAltDist2 | behavior + another network | siena07 | beheff(avaltdist2) |
| totAltDist2 | behavior + another network | siena07 | beheff(totaltdist2) |
| avTAltDist2 | behavior + another network | siena07 | beheff(avtaltdist2) |
| totAAltDist2 | behavior + another network | siena07 | beheff(totaaltdist2) |
| avInAltDist2 | behavior + another network | siena07 | beheff(avinaltdist2) |
| totInAltDist2 | behavior + another network | siena07 | beheff(totinaltdist2) |
| avTInAltDist2 | behavior + another network | siena07 | beheff(avtinaltdist2) |
| avInAltDist2_nc | behavior + another network | siena07 | behnet2(avinaltdist2nc) |
| totInAltDist2_nc | behavior + another network | siena07 | behnet2(totinaltdist2nc) |
| avTInAltDist2_nc | behavior + another network | siena07 | behnet2(avtinaltdist2nc) |
| totAInAltDist2_nc | behavior + another network | siena07 | behnet2(totainaltdist2nc) |
| avInSimDist2 | behavior + another network | siena07 | behnet2(avinsimdist2) |
| totInSimDist2 | behavior + another network | siena07 | behnet2(totinsimdist2) |
| totGwdspFFAlt | behavior + another network | siena07 | behnet2(totgwdspffalt:#) |
| totGwdspFBAlt | behavior + another network | siena07 | behnet2(totgwdspfbalt:#) |
| totGwdspFFAlt_nc | behavior + another network | siena07 | behnet2(totgwdspffaltnc:#) |
| totGwdspFBAlt_nc | behavior + another network | siena07 | behnet2(totgwdspfbaltnc:#) |
| maxAlt | behavior + another network | siena07 | beheff(maxalt) |
| minAlt | behavior + another network | siena07 | beheff(minalt) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totAInAltDist2 | behavior + another network | siena07 | beheff(totainaltdist2) |
| varAlt | behavior + another network | siena07 | behnet2(varalt) |
| behDenseTriads | behavior + another network | siena07 | behnet2(behdensetriads) |
| simDenseTriads | behavior + another network | siena07 | behnet2(simdensetriads) |
| recipDeg | behavior + another network | siena07 | beheff(recipdeg) |
| avSimPopEgo | behavior + another network | siena07 | beheff(avsimpopego) |

## R. Behavior: effects of two networks, and of a network with a dyadic covariate

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avAltW | behavior + another network + constant dyadic covariate | siena07 | behw(avaltw:matrix) |
| avInAltW | behavior + another network + constant dyadic covariate | siena07 | behw(avinaltw:matrix) |
| totAltW | behavior + another network + constant dyadic covariate | siena07 | behw(totaltw:matrix) |
| totInAltW | behavior + another network + constant dyadic covariate | siena07 | behw(totinaltw:matrix) |
| avSimW | behavior + another network + constant dyadic covariate | siena07 | behw(avsimw:matrix) |
| totSimW | behavior + another network + constant dyadic covariate | siena07 | behw(totsimw:matrix) |
| avWAlt | behavior + another network + constant dyadic covariate | siena07 | behw(avwalt:matrix) |
| avWInAlt | behavior + another network + constant dyadic covariate | siena07 | behw(avwinalt:matrix) |
| totWAlt | behavior + another network + constant dyadic covariate | siena07 | behw(totwalt:matrix) |
| totWInAlt | behavior + another network + constant dyadic covariate | siena07 | behw(totwinalt:matrix) |
| avAltW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(avaltw:stem) |
| avInAltW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(avinaltw:stem) |
| totAltW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(totaltw:stem) |
| totInAltW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(totinaltw:stem) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avSimW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(avsimw:stem) |
| totSimW | behavior + another network + changing dyadic covariate | siena07 | tvbehw(totsimw:stem) |
| avWAlt | behavior + another network + changing dyadic covariate | siena07 | tvbehw(avwalt:stem) |
| avWInAlt | behavior + another network + changing dyadic covariate | siena07 | tvbehw(avwinalt:stem) |
| totWAlt | behavior + another network + changing dyadic covariate | siena07 | tvbehw(totwalt:stem) |
| totWInAlt | behavior + another network + changing dyadic covariate | siena07 | tvbehw(totwinalt:stem) |
| FFDeg | behavior + another network + another network | siena07 | beheff12 |
| BBDeg | behavior + another network + another network | siena07 | beheff12 |
| FBDeg | behavior + another network + another network | siena07 | beheff12 |
| FRDeg | behavior + another network + another network | siena07 | beheff12 |
| BRDeg | behavior + another network + another network | siena07 | beheff12 |
| indegMixedPop | behavior + another network + another network | siena07 | beheff12 |
| outdegMixedPop | behavior + another network + another network | siena07 | beheff12 |
| BFDeg | behavior (non-directed network) + another network + another... | siena07 | beheff12 |

## S. Behavior: effects of covariates and of other behaviors

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avGroupEgoX | behavior + a behavior variable | siena07 | this row is the network effect avGroupEgoX with the dependen |
| totGroupEgoX | behavior + a behavior variable | siena07 | eff2 |
| effFrom | behavior + constant covariate | siena07 | behfrom() |
| totSimAltX | behavior + constant covariate + another network | siena07 | behx(totsimaltx:var) |
| avSimAltX | behavior + constant covariate + another network | siena07 | behx(avsimaltx:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avAltAltX | behavior + constant covariate + another network | siena07 | behx(avaltaltx:var) |
| avXAlt | behavior + constant covariate + another network | siena07 | behavxalt() |
| totXAlt | behavior + constant covariate + another network | siena07 | behtotxalt() |
| minXAlt | behavior + constant covariate + another network | siena07 | behx(minxalt:var) |
| maxXAlt | behavior + constant covariate + another network | siena07 | behx(maxxalt:var) |
| avXInAlt | behavior + constant covariate + another network | siena07 | behx(avxinalt:var) |
| totXInAlt | behavior + constant covariate + another network | siena07 | behx(totxinalt:var) |
| avXRecAlt | behavior + constant covariate + another network | siena07 | behx(avxrecalt:var) |
| totXRecAlt | behavior + constant covariate + another network | siena07 | behx(totxrecalt:var) |
| avSameXRecAlt | behavior + constant covariate + another network | siena07 | behx(avsamexrecalt:var) |
| totSameXRecAlt | behavior + constant covariate + another network | siena07 | behx(totsamexrecalt:var) |
| avSameXAlt | behavior + constant covariate + another network | siena07 | behavsamexalt() |
| totSameXAlt | behavior + constant covariate + another network | siena07 | behtotsamexalt() |
| avSameXInAlt | behavior + constant covariate + another network | siena07 | behx(avsamexinalt:var) |
| totSameXInAlt | behavior + constant covariate + another network | siena07 | behx(totsamexinalt:var) |
| avAltSameX | behavior + constant covariate + another network | siena07 | behx(avaltsamex:var) |
| totAltSameX | behavior + constant covariate + another network | siena07 | behx(totaltsamex:var) |
| avRecAltSameX | behavior + constant covariate + another network | siena07 | behx(avrecaltsamex:var) |
| totRecAltSameX | behavior + constant covariate + another network | siena07 | behx(totrecaltsamex:var) |
| avXAltDist2 | behavior + constant covariate + another network | siena07 | behx(avxaltdist2:var) |
| totXAltDist2 | behavior + constant covariate + another network | siena07 | behx(totxaltdist2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avTXAltDist2 | behavior + constant covariate + another network | siena07 | behx(avtxaltdist2:var) |
| totAXAltDist2 | behavior + constant covariate + another network | siena07 | behx(totaxaltdist2:var) |
| avXInAltDist2 | behavior + constant covariate + another network | siena07 | behx(avxinaltdist2:var) |
| totXInAltDist2 | behavior + constant covariate + another network | siena07 | behx(totxinaltdist2:var) |
| avTXInAltDist2 | behavior + constant covariate + another network | siena07 | behx(avtxinaltdist2:var) |
| totAXInAltDist2 | behavior + constant covariate + another network | siena07 | behx(totaxinaltdist2:var) |
| degAbsContrX | behavior + constant covariate + another network | siena07 | behx(degabscontrx:var) |
| degPosContrX | behavior + constant covariate + another network | siena07 | behx(degposcontrx:var) |
| degNegContrX | behavior + constant covariate + another network | siena07 | behx(degnegcontrx:var) |
| avSimEgoX | behavior + constant covariate + another network | siena07 | behx(avsimegox:var) |
| totSimEgoX | behavior + constant covariate + another network | siena07 | behx(totsimegox:var) |
| avAltEgoX | behavior + constant covariate + another network | siena07 | behx(avaltegox:var) |
| totAltEgoX | behavior + constant covariate + another network | siena07 | behx(totaltegox:var) |
| effFrom | behavior + changing covariate | siena07 | behfrom(stem) |
| totSimAltX | behavior + changing covariate + another network | siena07 | behx(totsimaltx:var) |
| avSimAltX | behavior + changing covariate + another network | siena07 | behx(avsimaltx:var) |
| avAltAltX | behavior + changing covariate + another network | siena07 | behx(avaltaltx:var) |
| avXAlt | behavior + changing covariate + another network | siena07 | behavxalt(stem) |
| totXAlt | behavior + changing covariate + another network | siena07 | behtotxalt(stem) |
| minXAlt | behavior + changing covariate + another network | siena07 | behx(minxalt:var) |
| maxXAlt | behavior + changing covariate + another network | siena07 | behx(maxxalt:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avXInAlt | behavior + changing covariate + another network | siena07 | behx(avxinalt:var) |
| totXInAlt | behavior + changing covariate + another network | siena07 | behx(totxinalt:var) |
| avXRecAlt | behavior + changing covariate + another network | siena07 | behx(avxrecalt:var) |
| totXRecAlt | behavior + changing covariate + another network | siena07 | behx(totxrecalt:var) |
| avSameXRecAlt | behavior + changing covariate + another network | siena07 | behx(avsamexrecalt:var) |
| totSameXRecAlt | behavior + changing covariate + another network | siena07 | behx(totsamexrecalt:var) |
| avSameXAlt | behavior + changing covariate + another network | siena07 | behavsamexalt(stem) |
| totSameXAlt | behavior + changing covariate + another network | siena07 | behtotsamexalt(stem) |
| avSameXInAlt | behavior + changing covariate + another network | siena07 | behx(avsamexinalt:var) |
| totSameXInAlt | behavior + changing covariate + another network | siena07 | behx(totsamexinalt:var) |
| avAltSameX | behavior + changing covariate + another network | siena07 | behx(avaltsamex:var) |
| totAltSameX | behavior + changing covariate + another network | siena07 | behx(totaltsamex:var) |
| avRecAltSameX | behavior + changing covariate + another network | siena07 | behx(avrecaltsamex:var) |
| totRecAltSameX | behavior + changing covariate + another network | siena07 | behx(totrecaltsamex:var) |
| avXAltDist2 | behavior + changing covariate + another network | siena07 | behx(avxaltdist2:var) |
| totXAltDist2 | behavior + changing covariate + another network | siena07 | behx(totxaltdist2:var) |
| avTXAltDist2 | behavior + changing covariate + another network | siena07 | behx(avtxaltdist2:var) |
| totAXAltDist2 | behavior + changing covariate + another network | siena07 | behx(totaxaltdist2:var) |
| avXInAltDist2 | behavior + changing covariate + another network | siena07 | behx(avxinaltdist2:var) |
| totXInAltDist2 | behavior + changing covariate + another network | siena07 | behx(totxinaltdist2:var) |
| avTXInAltDist2 | behavior + changing covariate + another network | siena07 | behx(avtxinaltdist2:var) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totAXInAltDist2 | behavior + changing covariate + another network | siena07 | behx(totaxinaltdist2:var) |
| degAbsContrX | behavior + changing covariate + another network | siena07 | behx(degabscontrx:var) |
| degPosContrX | behavior + changing covariate + another network | siena07 | behx(degposcontrx:var) |
| degNegContrX | behavior + changing covariate + another network | siena07 | behx(degnegcontrx:var) |
| avSimEgoX | behavior + changing covariate + another network | siena07 | behx(avsimegox:var) |
| totSimEgoX | behavior + changing covariate + another network | siena07 | behx(totsimegox:var) |
| avAltEgoX | behavior + changing covariate + another network | siena07 | behx(avaltegox:var) |
| totAltEgoX | behavior + changing covariate + another network | siena07 | behx(totaltegox:var) |
| effFrom | behavior + a behavior variable | siena07 | behfrom(beh2) |
| totSimAltX | behavior + a behavior variable + another network | siena07 | behx(totsimaltx:beh2) |
| avSimAltX | behavior + a behavior variable + another network | siena07 | behx(avsimaltx:beh2) |
| avAltAltX | behavior + a behavior variable + another network | siena07 | behx(avaltaltx:beh2) |
| avXAlt | behavior + a behavior variable + another network | siena07 | behavxalt(beh2) |
| totXAlt | behavior + a behavior variable + another network | siena07 | behtotxalt(beh2) |
| minXAlt | behavior + a behavior variable + another network | siena07 | behx(minxalt:beh2) |
| maxXAlt | behavior + a behavior variable + another network | siena07 | behx(maxxalt:beh2) |
| avXInAlt | behavior + a behavior variable + another network | siena07 | behx(avxinalt:beh2) |
| totXInAlt | behavior + a behavior variable + another network | siena07 | behx(totxinalt:beh2) |
| avXRecAlt | behavior + a behavior variable + another network | siena07 | behx(avxrecalt:beh2) |
| totXRecAlt | behavior + a behavior variable + another network | siena07 | behx(totxrecalt:beh2) |
| avSameXRecAlt | behavior + a behavior variable + another network | siena07 | behx(avsamexrecalt:beh2) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totSameXRecAlt | behavior + a behavior variable + another network | siena07 | behx(totsamexrecalt:beh2) |
| avSameXAlt | behavior + a behavior variable + another network | siena07 | behavsamexalt(beh2) |
| totSameXAlt | behavior + a behavior variable + another network | siena07 | behtotsamexalt(beh2) |
| avSameXInAlt | behavior + a behavior variable + another network | siena07 | behx(avsamexinalt:beh2) |
| totSameXInAlt | behavior + a behavior variable + another network | siena07 | behx(totsamexinalt:beh2) |
| avAltSameX | behavior + a behavior variable + another network | siena07 | behx(avaltsamex:beh2) |
| totAltSameX | behavior + a behavior variable + another network | siena07 | behx(totaltsamex:beh2) |
| avRecAltSameX | behavior + a behavior variable + another network | siena07 | behx(avrecaltsamex:beh2) |
| totRecAltSameX | behavior + a behavior variable + another network | siena07 | behx(totrecaltsamex:beh2) |
| avXAltDist2 | behavior + a behavior variable + another network | siena07 | behx(avxaltdist2:beh2) |
| totXAltDist2 | behavior + a behavior variable + another network | siena07 | behx(totxaltdist2:beh2) |
| avTXAltDist2 | behavior + a behavior variable + another network | siena07 | behx(avtxaltdist2:beh2) |
| totAXAltDist2 | behavior + a behavior variable + another network | siena07 | behx(totaxaltdist2:beh2) |
| avXInAltDist2 | behavior + a behavior variable + another network | siena07 | behx(avxinaltdist2:beh2) |
| totXInAltDist2 | behavior + a behavior variable + another network | siena07 | behx(totxinaltdist2:beh2) |
| avTXInAltDist2 | behavior + a behavior variable + another network | siena07 | behx(avtxinaltdist2:beh2) |
| totAXInAltDist2 | behavior + a behavior variable + another network | siena07 | behx(totaxinaltdist2:beh2) |
| degAbsContrX | behavior + a behavior variable + another network | siena07 | behx(degabscontrx:beh2) |
| degPosContrX | behavior + a behavior variable + another network | siena07 | behx(degposcontrx:beh2) |
| degNegContrX | behavior + a behavior variable + another network | siena07 | behx(degnegcontrx:beh2) |
| avSimEgoX | behavior + a behavior variable + another network | siena07 | behx(avsimegox:beh2) |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| totSimEgoX | behavior + a behavior variable + another network | siena07 | behx(totsimegox:beh2) |
| avAltEgoX | behavior + a behavior variable + another network | siena07 | behx(avaltegox:beh2) |
| totAltEgoX | behavior + a behavior variable + another network | siena07 | behx(totaltegox:beh2) |

## T. Rate effects (network and behavior)

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| Rate | rate (one-mode network) | siena07 | automatic rate |
| outRate | rate (one-mode network) | siena07 | outrate |
| inRate | rate (one-mode network) | siena07 | inrate |
| recipRate | rate (one-mode network) | siena07 | reciprate |
| outRateInv | rate (one-mode network) | siena07 | outrateinv |
| outRateLog | rate (one-mode network) | siena07 | outratelog |
| inRateInv | rate (one-mode network) | siena07 | inrateinv |
| inRateLog | rate (one-mode network) | siena07 | inratelog |
| recipRateInv | rate (one-mode network) | siena07 | reciprateinv |
| recipRateLog | rate (one-mode network) | siena07 | recipratelog |
| RateX | rate (one-mode network) + constant covariate | siena07 | ratecov() |
| RateX | rate (one-mode network) + changing covariate | siena07 | tvcov |
| RateX | rate (one-mode network) + a behavior variable | siena07 | ratecov |
| Rate | rate (behavior) | siena07 | automatic rate |
| outRate | rate (behavior) + another network | siena07 | behrate |
| inRate | rate (behavior) + another network | siena07 | behrate |
| recipRate | rate (behavior) + another network | siena07 | behrate |
| avExposure | rate (behavior) + another network | siena07 | behrate |
| susceptAvIn | rate (behavior) + another network | siena07 | behrate |
| totExposure | rate (behavior) + another network | siena07 | behrate |
| infectIn | rate (behavior) + another network | siena07 | behrate |
| infectOut | rate (behavior) + another network | siena07 | behrate |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| avTinExposureDist2 | rate (behavior) + another network | siena07 | behrate |
| totInExposureDist2 | rate (behavior) + another network | siena07 | behrate |
| totAInExposureDist2 | rate (behavior) + another network | siena07 | behrate |
| anyInExposureDist2 | rate (behavior) + another network | siena07 | behrate |
| RateX | rate (behavior) + constant covariate | siena07 | behratecov |
| susceptAvCovar | rate (behavior) + another network + constant covariate | siena07 | behrate |
| infectCovar | rate (behavior) + another network + constant covariate | siena07 | behrate |
| RateX | rate (behavior) + a behavior variable | siena07 | behratecov |
| RateX | rate (behavior) + changing covariate | siena07 | tvcov |
| susceptAvCovar | rate (behavior) + another network + changing covariate | siena07 | tvcov |
| infectCovar | rate (behavior) + another network + changing covariate | siena07 | tvcov |
| Rate | rate (two-mode network) | siena07 | automatic rate |
| outRate | rate (two-mode network) | siena07 | outrate |
| outRateInv | rate (two-mode network) | siena07 | outrateinv |
| outRateLog | rate (two-mode network) | siena07 | outratelog |
| RateX | rate (two-mode network) + constant covariate | siena07 | ratecov |
| RateX | rate (two-mode network) + changing covariate | siena07 | tvcov |
| RateX | rate (two-mode network) + a behavior variable | siena07 | ratecov |
| Rate | rate (non-directed network) | siena07 | automatic rate |
| outRate | rate (non-directed network) | siena07 | outrate |
| outRateInv | rate (non-directed network) | siena07 | outrateinv |
| outRateLog | rate (non-directed network) | siena07 | outratelog |
| RateX | rate (non-directed network) + constant covariate | siena07 | ratecov() |
| RateX | rate (non-directed network) + changing covariate | siena07 | tvcov |
| RateX | rate (non-directed network) + a behavior variable | siena07 | ratecov |

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `infectDeg` | rate (behavior (non-directed network)) + another network | siena07 | `behrate` |

## V. Placeholder rows and GMoM statistics

| RSiena name | Network / context | Validated | nwsaom option |
|---|---|---|---|
| `avXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | siena07 | `behavxalt(var)` |
| `totXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | siena07 | `behtotxalt(var)` |
| `avSameXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | siena07 | `behavsamexalt(var)` |
| `totSameXAlt` | non-directed network + ?yyyyyy + ?zzzzzz | siena07 | `behtotsamexalt(var)` |

# Appendix B

# RSiena definitions reproduced on purpose

`nwsaom` is built to give the same numbers as RSiena. Where RSiena's definition of an effect has a feature that a reader might not expect, for example where a tie enters a statistic twice or a covariate is read at the index of another node, the definition is reproduced on purpose, because a different definition would give statistics and estimates that no longer match `siena07`. This appendix lists the cases that matter to a user who compares output of the two programs or reads the effect definitions closely. Each was found by comparing the observed statistics (the targets) with those of `siena07`, and each is validated that way.

## Conventions that change the numbers

- **Centring.** Constant actor covariates are centred by their mean (RSiena's `coCovar`), changing covariates by their overall mean over all waves. The degree-weighted and distance-two covariate families depend on this: with the raw covariate their statistics are off by the mean.
- **Similarity.** The similarity covariate is $(x - \bar{x})/\text{range}$ with the similarity mean as the centring constant. In the similarity effects of a behavior, equal alters join the *higher* popularity count.
- **Behavior variance.** The variance used for behavior effects is the pooled population variance of waves $1, \ldots, W - 1$, and the behavior mean is taken over all waves.
- **Start network.** Statistics of behavior effects that involve the network use the network at the start of the period.
- **Scaling of `gwdspfb`.** The decay used by the effect is 0.01 times RSiena's parameter, so RSiena's 69 corresponds to a decay of 0.69.

## Single effects

- **`reciact` (root version).** When an actor with no mutual tie withdraws a tie, RSiena evaluates the square-root table at $-1$, which reads the memory just before the table and yields about 0. `nwsaom` takes the value to be 0.
- **`allDifCovNetNetIn` and `contrastCovNetNetIn`** are the same function in RSiena and give the same statistic.
- **Covariate stars.** `DifferentCovariateOutStar` tests the tie from the alter to $k$, whereas `SameCovariateOutStar` tests the tie from $k$ to the alter. Both are reproduced as they are.

- **Average versions of the same-covariate effects** divide by the number of actors with the ego's covariate value, not by the number of alters.
- **`avdeg` in a two-mode network** divides by the number of *actors*, not by the number of nodes.

## Two-mode networks

- `crossXOutAct` reads the actor covariate at the index of the *event*, as RSiena does.
- `avGroupEgoX` averages over the first $m$ actors and divides by $m$.
- `homXOutAct2` has the default parameter 3 in a bipartite network.
- `outInAss`/`outOutAss` and their relatives use the square roots that RSiena uses as the two-mode default (parameter 2).
- In `nwsaom multinet`, `divInAltIntn`, `JoutMix` and `gwespFBMix` of a two-mode dependent network, and `outOutActIntn`/`outOutAvIntn` of a one-mode network that depends on a two-mode one, index the other network by the node number of the *event*. `nwsaom` copies this and refuses these effects when the events outnumber the actors.
- `netbeh(totgroupegox)`, `netbeh(avgroupegox)` and `netbeh(crossxoutact)` in a two-mode network use the behavior of the actors for the events as well (the events mirror the actors' behavior), which needs at least as many actors as events.

## Other effects

- **`toU`.** In RSiena 1.6.12 and later `toU` counts, for the ego $i$ and the alter $j$, the actors $q$ with a tie $i \to q$ in the *other* network and a tie $q \to j$ in the *own* network, each weighted by the dyadic covariate value $D(i, q)$. `sharedToU` and `avAltU` are read in the same way.
- **`higher` in a non-directed network** equals the density statistic. The effect is not identified; RSiena returns a standard error of NA and `nwsaom` stops with an error message.
- **The shape effect `constant`** of a behavior is confounded with the behavior rate (RSiena gives a standard error of NA), so a fit that includes it stops.
- **The five double-degree behavior effects** (`FFDeg`, `BBDeg`, `FBDeg`, `FRDeg`, `BRDeg`) together are not identified, in RSiena as well.
- **`netbeh(indties)`** differs from `siena07` and is refused in `nwsaom multinet`.
- **Internal parameters.** In the current RSiena `outPop.c`, `outPopSqrt`, `inAct.c`, `inActSqrt`, `outPopMore` and relatives have the internal parameter 1 by default; `nwsaom` follows.

## Where `nwsaom` deliberately differs

- **Refusal where RSiena gives no usable result.** Some combinations are not reproduced because RSiena gives no usable result for them: `sameXOutAct` and `diffXOutAct` in a two-mode network abort `siena07`; `inPop_ego` in a two-mode network throws an exception in RSiena and `homXOutAct2` with a behavior is not implemented there; the non-directed `higher` effect is collinear with density. `nwsaom` refuses these with an error message instead (Section 13.3.2).

# Appendix C

# Effects with a Mata fallback

`nwsaom` simulates the model in a compiled plugin. The first effects were also written in Mata, and for them the program keeps working when no plugin is available for the platform (or the plugin is too old). The Mata simulator is, however, far too slow for the long simulations that estimation needs: a short two-wave fit that takes well under a second with the plugin takes about 30 seconds in Mata, and the larger models of this manual would take hours. For this reason the effects that were added later exist only in the plugin; writing a second, slower version of each of them would add no capability that a user could actually use. If you ask for such an effect and the plugin cannot be used, `nwsaom` stops with a message that says so. After a fit, `e(engine)` tells which simulator ran.

## C.1 Effects that run in Mata as well

**Network effects, structural** `outdegree`, `reciprocity`, `transtrip`, `transmedtrip`, `transties`, `transrectrip`, `cycle3`, `cycle4`, `balance`, `between`, `indegpopularity`, `outpopularity`, `outactivity`, `inactivity`, `outactsqrt`, `inpop`, `outpop`, `inact`, `isolatepop`, the shared-partner family `gwesp`, `gwespfb`, `gwespbf`, `gwespbb`, `gwesprr`, `gwdspff`, `gwdspfb`, `nbrdist2`, `dblnbrdist2`, `sqrtcycle4`, the degree-shape effects `outtrunc()`, `outmore()`, `outthreshold()`, `outinv()`, `outsqinv()`, the reciprocal-degree effects `recipop`, `recipopsqrt`, `reciact`, `reciactsqrt`, the assortativity effects `outoutass`, `ininass`, `inoutass`, `outinass`, and the isolate effects `antiiso`, `antiiniso`, `antiiniso2`, `outiso`, `inplus3`, `isolatenet`.

**Network effects of covariates** `nodematch()` (`samex`), `nodecov()`, `nodeicov()`, `nodeocov()` (`altx`, `egox`), `simcov()` (`simx`), `egoxaltx`, `samexrecip`, `simrecipx`, `higher`, `absdiffx`, dyadic covariates `dyadcov()`, `dyadcovrecip()`, and interactions `interact()`.

**Endowment and creation** `endow()` and `creation()` of the main network effects.

**Behavior effects** `linear`, `quadratic`, `avalt`, `avsim` (with their endowment and creation versions), the first set of `beheff()` tokens (degree and alter effects of the network on the behavior, `totalt`, `maxalt`, `minalt` and the similarity family), and `behsim`, `behegox`, `behaltx`, `behegoplusalt` for the network effects of the behavior itself.

**Models and data options** `ratecov()`, `tvcov()`, `present()`, `missnet()`, `missbeh()`, `structural()`, several behaviors in `behavior()`, `waves()`, `unconditional`, and `estat gof`.

## C.2 Effects that need the plugin

The following need the plugin, and `nwsaom` stops with an error if it cannot be used:

- the covariate-by-triad, popularity and activity effects `samextranstrip()`, `diffxtranstrip()`, `simxtranstrip()`, `homxtranstrip()`, `samexoutact()`, `diffxoutact()`, `samexreciact()`, `diffxreciact()`, `homxoutact()`, `altxoutact()`, `inpopx()`, `outpopx()`, `inactx()`, `outactx()`, `samexinpop()`, `diffxinpop()`;
- the centred and combined degree effects `inpopc`, `outpopc`, `outactc`, `inactc`, `degplus`, `degplusc`, `avdeg()`, `transtriads`;
- the threshold and contrast effects of covariates `egoplusaltsqx()`, `altsqx()`, `diffsqx()`, `egodiffx()`, `althigheregox()`, `unequalx()`, `egolthresholdx()`, `egorthresholdx()`, `altlthresholdx()`, `altrthresholdx()`, `degabsdiffx()`, `degposdiffx()`, `degnegdiffx()`, `covthreshold()`;
- the structural rate effects `outrate`, `inrate`, `reciprate` and their inverse and logarithmic versions;
- the option families `netx()`, `ownx()`, `dyadx()`, `tvdyadcov()`, `tvdyadcovrecip()`, `tvdyadx()`, `netbeh()`, `behx()`, `behw()`, `tvbehw()`, `behnet2()`, `behshape()`, `behfrom()`, `behavxalt()`, `behavsamexalt()`, `behtotxalt()`, `behtotsamexalt()` and the later `beheff()` tokens;
- co-evolution of a non-directed network with a behavior, two-mode networks, and `nwsaom multinet`;
- `estat importance`, `estat simulate` and `nwsaom simulate` use the plugin for the models they simulate or evaluate that need it.

The help file marks the options of the second group with *native plugin only*. The list above describes the situation when the manual was written; a new version of `nwsaom` adds new effects to the second group only.

# Appendix D

# Comparison with RSiena: the same estimator, and where the speed comes from

`nwsaom` fits stochastic actor-oriented models faster than RSiena, a median factor of 3.8 over 27 models on one thread. A faster program may be suspected of doing less: running fewer simulations, stopping the estimation earlier, or approximating. This appendix sets out what was measured to rule that out. It answers four questions: whether both programs run the same algorithm, whether they simulate the same model, whether they give the same answers, and where the difference in time comes from.

Every number below comes from a script in the replication files of the Stata Journal article on `nwsaom`, the folder `replication/` of the repository (https://github.com/thomas-u-grund/nwsaom/tree/main/replication). The script is named next to the number, relative to that folder, and can be rerun; the README of the folder lists every script with its output. The runs were made with `nwsaom` 1.5.7 in Stata 18 BE and RSiena 1.6.14 in R 4.6.0, on an Apple M5 Pro (18 cores), macOS 26.5.1, on 8 and 9 November 2026. Timings depend on the machine; the ratios between the two programs are more stable than the seconds.

## D.1 The same algorithm

Both programs estimate by the method of moments with the three-phase Robbins–Monro algorithm of Snijders (2001), and `nwsaom` takes RSiena's defaults. The settings were checked against the source of RSiena 1.6.14 (the functions `AnnouncePhase`, `proc2subphase`, `doIterations` and `CalculateDerivative` of its R code); in `nwsaom` they are in the Mata source of the estimator. Here $p$ is the number of estimated parameters.

| setting | `nwsaom` | RSiena |
|---|---|---|
| method | method of moments | method of moments (`maxlike = FALSE`) |
| rates, one network | conditional estimation; unconditional with `unconditional`, with composition change and in co-evolution models | conditional (`cond = NA`, the default); unconditional in the same cases |
| Phase 1 | $\max(50, 7 + 3p)$ simulations; derivatives by the score-function estimator, by finite differences if a derivative is not positive | the same |

| setting | `nwsaom` | RSiena |
|---|---|---|
| Phase 2 | 4 subphases, first gain 0.2, halved in each subphase; each step with the Dolby correction estimated in Phase 1 | 4 subphases, `firstg = 0.2`, `reduceg = 0.5`, `dolby = TRUE` |
| length of a subphase | at least $\lfloor 2.52 \max(5, 7+p) \rfloor$ iterations, multiplied by 2.52 in each further subphase, at most that plus 200; ends early once the minimum is reached and the largest autocorrelation of the deviations is negative; stopped after 50 iterations and repeated, up to four times, if an autocorrelation falls below $-0.8$ | the same rule (`maxrepeatsubphase`) |
| Phase 3 | 1,000 simulations (`k3()`) | 1,000 simulations (`n3`) |
| ministeps | one actor chosen in proportion to the rates, one tie change (or no change) chosen by the multinomial logit of the objective function; behavior: a step up, down or none | the same |
| convergence | all $t$-ratios below 0.1 and the overall maximum convergence ratio below 0.25 | the same |

**What is computed differently.** Some quantities are computed in a different way from RSiena without changing what is estimated. The linear shape statistic of a behavior is the sum of the raw values in `nwsaom` and of the values centred at the overall mean in RSiena; the two differ by a constant, which cancels in the deviations of the simulated from the observed statistics. (The quadratic shape statistic, where centring does matter, is centred as in RSiena.) The change statistics of a ministep are computed effect by effect over all alternatives of the acting actor at once rather than alternative by alternative (Section D.4.4 below); the values are the same. The random numbers differ: each simulation draws from its own stream, derived from the seed and the position of the simulation in the algorithm, so the estimates of the two programs, or of `nwsaom`'s Mata engine and its plugin, are not equal digit for digit, but they are identical in `nwsaom` for every number of threads. The conventions in which RSiena's definitions are reproduced on purpose are listed in Appendix B.

**Comparing the traces.** With `global NWSAOM_TRACE 1` before the command, `nwsaom` prints the length of Phase 1 and the diagonal of the derivative matrix, every 25th iteration of each Phase 2 subphase with the current parameters, and the length of every repeated subphase. RSiena writes the same information to its output file (the `projname` of `sienaAlgorithmCreate()`): the Phase 1 derivatives, the iterations of each subphase and the Phase 3 summary. The two traces can be compared line by line; they agree in the lengths of Phases 1 and 3 and in the rules that end a subphase, and differ in the Phase 2 counts only by simulation noise (Section D.4.1).

## D.2 The same simulation

If both programs simulate the same model, they must reproduce the same statistics at the same parameters. `R/15_fixed_sim.R` estimates models N1, N2 and R1 of the benchmark with RSiena (seed 1); then both programs simulate each model 1,000 times at these estimates, five times with seeds 1 to 5, without estimating (RSiena: `simOnly = TRUE`; `nwsaom`: `theta0()` with `fix(_all)`, `stata/15_fixed_sim.do`). For every statistic the table gives the $t$-ratio of the deviations from

the observed value, as a mean over the five runs. If the two programs simulate the same model, the two columns differ only by sampling error, with a standard deviation of about $\sqrt{2/5000} = 0.02$.

| model | statistic (RSiena name) | RSiena | `nwsaom` |
|---|---|---|---|
| N1 | `density` | −0.043 | −0.032 |
| | `recip` | −0.035 | −0.013 |
| N2 | `density` | −0.003 | −0.016 |
| | `recip` | 0.006 | −0.003 |
| | `gwespFF` | 0.017 | 0.007 |
| R1 | `density` | 0.044 | 0.006 |
| | `recip` | 0.043 | 0.013 |
| | `gwespFF` | 0.037 | −0.002 |
| | `transRecTrip` | 0.038 | 0.006 |
| | `inPopSqrt` | 0.039 | −0.004 |
| | `outPopSqrt` | 0.045 | 0.004 |
| | `sameX` | 0.068 | 0.009 |
| | `egoX` | −0.029 | 0.013 |
| | `altX` | −0.014 | 0.031 |
| | `simX` | −0.030 | −0.045 |

All $t$-ratios are close to zero in both programs, as they must be at estimates that satisfy the moment equations, and the two columns differ by at most 0.06, within three standard deviations of the sampling error. RSiena's estimates are reproduced by `nwsaom`'s simulation, including those of the effects that `nwsaom` computes by a different route (Section D.4.4).

## D.3 The same answers

`nwsaom` and RSiena were run on 27 models with each program's default settings. The data of the first 20 models are the 50-actor Glasgow panel (RSiena's `s50`); the larger networks are 2, 4 or 10 disconnected copies of it ($n$ = 100, 200, 500). Network-only models use conditional estimation, except N7 (unconditional). Models R1–R5 are applied-analysis specifications: R1 combines `outdegree`, `reciprocity`, `gwesp()`, transitive reciprocated triplets, square-root indegree and outdegree popularity, `samex(smoke1)` and `egox()`, `altx()`, `simx()` on alcohol use; R2 adds alcohol as co-evolving behavior; R3 is undirected; R4 and R5 repeat R1 and R2 on 200 actors. The 7 further models cover two-mode networks (X1, X2), several networks with and without a behavior and with a non-directed network (X3–X6), and newer single-network effects (X7). Each model was fitted with seeds 1–7 in both programs (seeds 1–5 for fits above one minute, 1–3 for fits above three minutes), on one thread (`stata/07_bench_one.do`, `R/07_bench_one.R`; the tables are built from one row per fit by `tools/build_tables.py`).

Table D.2: `nwsaom` and RSiena 1.6.14 on the same models and data, both single-threaded: largest difference in a mean estimate (in RSiena standard errors) and median seconds per fit

| | Specification | $n$ | Waves | Max diff | `nwsaom` | RSiena | Speed-up |
|---|---|---|---|---|---|---|---|
| *Network* | | | | | | | |
| N1 | `outdegree`, `reciprocity` | 50 | 2 | 0.06 | 0.22 | 0.84 | 3.8 |
| N2 | + `gwesp` | 50 | 3 | 0.02 | 0.52 | 1.81 | 3.5 |
| N4 | + `gwesp`, `samex()` | 50 | 3 | 0.04 | 0.60 | 2.24 | 3.8 |
| N5 | + `gwesp`, `indegpopularity`, `outactivity` | 50 | 3 | 0.08 | 0.77 | 3.02 | 3.9 |
| N6 | + `gwesp`, `egox()`, `altx()`, `simx()` | 50 | 3 | 0.04 | 0.82 | 3.57 | 4.3 |

*continued on the next page*

*Table D.2 continued*

| | Specification | $n$ | Waves | Max diff | `nwsaom` | RSiena | Speed-up |
|---|---|---|---|---|---|---|---|
| N7 | N2, unconditional estimation | 50 | 3 | 0.02 | 0.63 | 2.48 | 3.9 |
| *Undirected* | | | | | | | |
| U1 | `outdegree`, default model (forcing) | 50 | 2 | 0.02 | 0.09 | 0.33 | 3.5 |
| U2 | `outdegree`, `symtype(joint)` | 50 | 2 | 0.06 | 0.09 | 1.03 | 11.5 |
| *Co-evolution* | | | | | | | |
| C1 | N2 + alcohol: `linear`, `quadratic`, `avalt` | 50 | 3 | 0.03 | 0.98 | 4.66 | 4.7 |
| C2 | N2 + smoking: `linear`, `quadratic`, `avsim` | 50 | 3 | 0.04 | 1.05 | 5.52 | 5.2 |
| C3 | N1 + alcohol: `linear`, `avalt` | 50 | 2 | 0.03 | 0.45 | 1.84 | 4.1 |
| *Realistic* | | | | | | | |
| R1 | full network model (see text) | 50 | 3 | 0.06 | 1.33 | 5.29 | 4.0 |
| R2 | R1 network part + alcohol co-evolution | 50 | 3 | 0.03 | 1.64 | 7.35 | 4.5 |
| R3 | undirected: `gwesp`, `samex()`, `simx()` | 50 | 2 | 0.01 | 0.24 | 0.74 | 3.1 |
| R4 | R1 | 200 | 3 | 0.06 | 22.2 | 90.1 | 4.1 |
| R5 | R2 | 200 | 3 | 0.05 | 23.6 | 109 | 4.6 |
| *Larger networks* | | | | | | | |
| S100 | N2 | 100 | 3 | 0.05 | 1.97 | 6.82 | 3.5 |
| S200 | N2 | 200 | 3 | 0.02 | 8.30 | 31.9 | 3.8 |
| S500 | N2 | 500 | 3 | 0.03 | 64.5 | 231 | 3.6 |
| T200 | C1 | 200 | 3 | 0.08 | 13.8 | 66.5 | 4.8 |
| *Two-mode* | | | | | | | |
| X1 | `outdegree`, `indegpopularity`, `cycle4` | 40×20 | 3 | 0.10 | 0.85 | 2.75 | 3.3 |
| X2 | `outdegree`, `indegpopularity`, `outactivity`, `cycle4`, `outiso`, `antiiniso2` | 40×30 | 3 | 0.03 | 0.25 | 0.61 | 2.4 |
| *Two networks* | | | | | | | |
| X3 | both: `outdegree`, `reciprocity`; `crprod` | 50 | 3 | 0.07 | 1.41 | 3.75 | 2.7 |
| X4 | X3 without `crprod` + smoking: `linear`, `quadratic`, `avalt` (both networks) | 50 | 3 | 0.03 | 1.17 | 8.32 | 7.1 |
| X5 | X4 with `avalt` on network 1 only; `behegox`, `behaltx`, `behsim`, `samewwclosure:behavior` | 50 | 3 | 0.06 | 3.91 | 11.5 | 2.9 |
| X6 | undirected network 1 (`sym1`) + directed network 2; `crprod` | 50 | 3 | 0.03 | 1.00 | 3.08 | 3.1 |
| *Single network* | | | | | | | |
| X7 | `outdegree`, `reciprocity`, `gwesp`, `samextranstrip()`, `altxoutact()`, `egoplusaltsqx()` | 50 | 3 | 0.02 | 2.22 | 4.26 | 1.9 |

Medians over seven fits (seeds 1–7; five above one minute, three above three minutes) per program (RSiena 1.6.14: 8 November 2026; `nwsaom` 1.5.7: 9 November 2026; same seeds). Max diff includes the rate parameters and uses the fits that converged (overall convergence ratio $< 0.25$) in both programs. N7, X1, X2: unconditional estimation in both programs (`unconditional`; `cond = FALSE`); X3–X6: `nwsaom multinet` with `k0(100) k3(1000)` against RSiena `n3 = 1000, nsub = 4`. Fits that converged, RSiena / `nwsaom`, for the models with at least one that did not: C2 6/6, 6/7; R2 6/7, 7/7; R4 4/5, 7/7; R5 4/5, 6/7; X1 1/7, 7/7; X3 7/7, 5/7; X4 6/7, 6/7; X5 6/7, 7/7.

Over all 221 parameters the mean estimates over seeds differ by at most 0.10 RSiena standard errors (on average 0.02), which is the size of RSiena's own variation between seeds (median 0.03

standard errors, at most 0.11). The standard errors, themselves simulation estimates, agree (as medians over seeds) within 7% for the network models; over all models the ratio of the two programs' standard errors is between 0.89 and 1.12.

Of the 187 `nwsaom` fits, 182 converged by RSiena's criterion (overall maximum convergence ratio below 0.25), and 167 of RSiena's 178. Fits that did not converge are not used in the comparison of the estimates. `nwsaom` converged less often than RSiena only in X3 (5 of 7 fits against 7 of 7). This is chance: with seeds 1 to 20 the overall maximum convergence ratio of X3 had a median of 0.12 in `nwsaom` and 0.14 in RSiena, and it was above 0.25 for two seeds in `nwsaom` (3 and 6, the two of the benchmark) and for one in RSiena; the estimates of the two fits that did not converge agree with RSiena's as well.

**Limits of the comparison.** One data set (Glasgow and disconnected copies of it) for the one-mode models and one specification per family for the further models; agreement may differ for other effects and for sparser or denser networks. Both programs ran at their default settings, apart from unconditional estimation where RSiena used `cond = FALSE` and the phase lengths of X3–X6 given in the note of the table.

## D.4 Where the speed comes from

### D.4.1 The amount of work

The lengths of Phase 2 depend on the run, because a subphase ends when its autocorrelation criterion is met. `stata/11_phase_counts.do` and `R/11_phase_counts.R` count the simulations of each phase for three benchmark models, with seeds 1 to 10 in each program. For RSiena, Phase 2 is the total count of the fit less Phases 1 and 3, and a trace on `doIterations` records every run of a subphase. `nwsaom`'s trace fixes its Phase 2 count to within 24 iterations per subphase; the table uses the middle of that range.

| model | Phase 2, mean (range) over 10 seeds `nwsaom` | RSiena | repeated subphases `nwsaom` / RSiena | all phases, RSiena / `nwsaom` |
|---|---|---|---|---|
| N1 | 958 (748–1,148) | 1,016 (762–1,162) | none / none | 1.03 |
| N2 | 813 (648–1,073) | 801 (657–912) | none / none | 0.99 |
| R1 | 1,540 (1,348–1,773) | 1,390 (1,098–1,881) | 9 in 6 fits / 10 in 5 fits | 0.94 |

Phases 1 (50 simulations) and 3 (1,000) have the same length in both programs. In Phase 2 the differences, divided by their standard errors over the ten seeds, are 1.0 (N1), −0.2 (N2) and −1.9 (R1). The number of simulations thus accounts for a factor of at most 1.03 of the speed-ups of 3.8 (N1), 3.5 (N2) and 4.0 (R1); on R1 `nwsaom` is faster although it runs more simulations.

### D.4.2 Time for a fixed amount of work

`R/16_fixed_workload.R` and `stata/16_fixed_workload.do` time the same amount of work in both programs, without any estimation: each program simulates N1, N2 and R1 at RSiena's estimates 1,000 and 5,000 times (RSiena: `simOnly = TRUE`; `nwsaom`: `fix(_all)`), five times each, alternating, on one thread and after a warm-up. The difference between the medians is the time of 4,000 simulations.

| model | RSiena | `nwsaom` | ratio |
|---|---|---|---|
| | microseconds per simulation | | |
| N1 | 387 | 83 | 4.7 |
| N2 | 965 | 239 | 4.0 |
| R1 | 2,208 | 454 | 4.9 |

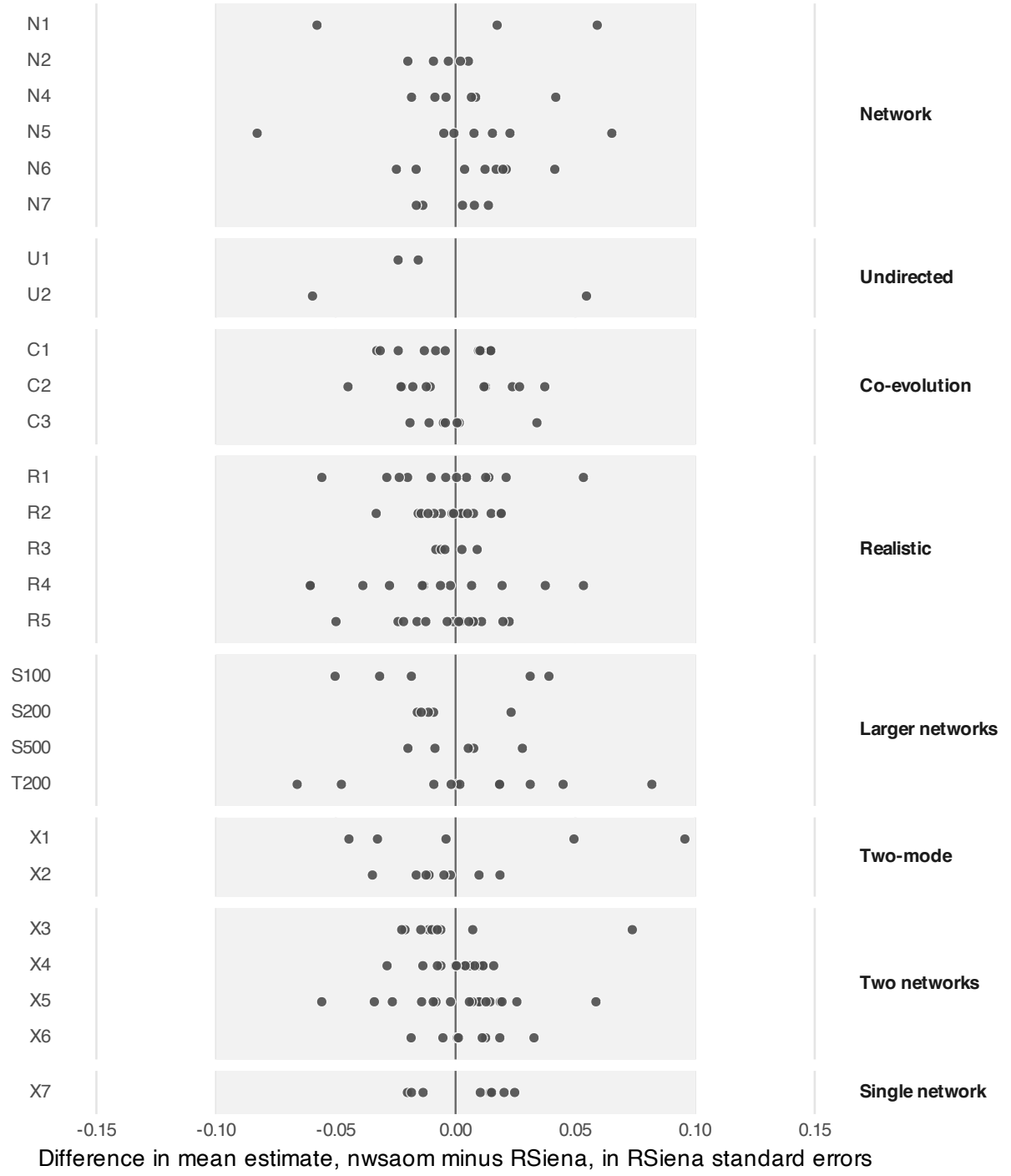



Figure D.1: Difference between the mean estimates of `nwsaom` and RSiena for every parameter of every model, in RSiena standard errors; the band marks $\pm 0.1$.

With the work held fixed, `nwsaom` is 4.0 to 4.9 times as fast. These ratios are of the same size as those of whole fits (3.8, 3.5 and 4.0 for the three models), so the speed-up of a fit is the speed of a simulation, not a shorter estimation. The ratios of whole fits are slightly lower because a fit also contains work that is not simulation, such as the set-up and `nwsaom`'s Phase 2 steps, which are not batched.

### D.4.3 The cost of a Phase 2 step

RSiena runs the loop over a fit's simulations in R and calls its compiled C++ code once per simulation. R's sampling profiler (`R/09_profile.R`, ten warm fits each of N1 and N2, three runs) puts the R code at 17 to 22 percent of the time for N1 and 8 to 12 percent for N2, or 69 to 89 and 78 to 117 microseconds per simulation. The first fit in a new R session spends 63 to 77 percent of its time in R code; the benchmark timed warm fits only.

`nwsaom` passes the data to its plugin once per fit, and in Phases 1 and 3, about half of a fit's simulations, the plugin runs the simulations as one batch. Phase 2 cannot be batched, because each step starts from the parameters of the step before; there Mata calls the plugin once per step. `stata/14_phase_cost.do` measures the cost: with the same seed, Phases 1 and 2 are the same whatever the length of Phase 3, so the difference between fits with 1,000 and 5,000 Phase 3 simulations gives the time of a batched simulation, and the rest of the fit the time of a Phase 2 step. A batched simulation took 82 microseconds on N1 and 231 on N2, and a Phase 2 step at most 131 and 310 (seed 1, one thread, medians of five fits; the time charged to Phase 2 includes the set-up of the fit, so these are upper bounds). The overhead of a Phase 2 step is thus at most 49 microseconds on N1 and 79 on N2, against RSiena's 69 to 89 and 78 to 117, and it does not arise in Phases 1 and 3. Removing all of RSiena's R code would account for a factor of 1.2 to 1.3 on N1 and about 1.1 on N2.

### D.4.4 The simulation itself

The rest of the speed-up comes from the time a simulation takes in compiled code. In both programs a ministep evaluates, for the acting actor, the change statistics of every effect for every possible tie change. RSiena's C++ code loops over the alternatives and, for each, over the effects, calling each effect on its own. The plugin turns the loops around: for each effect it passes once over all alternatives of the actor in tight loops over arrays, and it computes quantities that several effects share once per actor. For example, the two-path counts between the actor and every other actor are computed once and serve `gwesp()` and its variants, `gwdspff()`, `transrectrip`, `transties` and `balance`; the geometrically weighted terms come from a table computed once per fit; each network is kept as a dense adjacency matrix together with neighbor lists. These are choices about how to compute the same change statistics, as the fixed-parameter check of Section D.2 shows. RSiena's C++ code was not instrumented, so how much of the remaining factor each of these choices contributes was not measured.

The implementation can matter more than the algorithm. `nwsaom` contains a second implementation of the simulation in Mata, which runs the same algorithm with the same phases (`stata/09_mata_engine.do`, N1, seeds 1 to 3). It took a median of 51 seconds per fit (40 to 53) against 0.22 seconds for the plugin, with estimates that agree within simulation noise (outdegree −2.242, −2.245, −2.239 against −2.264, −2.248, −2.254): a factor of about 200 from the implementation alone.

### D.4.5 Several cores

The comparison above is on one thread: RSiena's default, and `nwsaom` with `cores(1)`. The table below gives `nwsaom` with several threads against RSiena's option for several processes (`useCluster = TRUE`); with several cores in both programs, `nwsaom` was 2.0 to 4.2 times as fast as RSiena's best time. `nwsaom`'s estimates are identical for every number of threads.

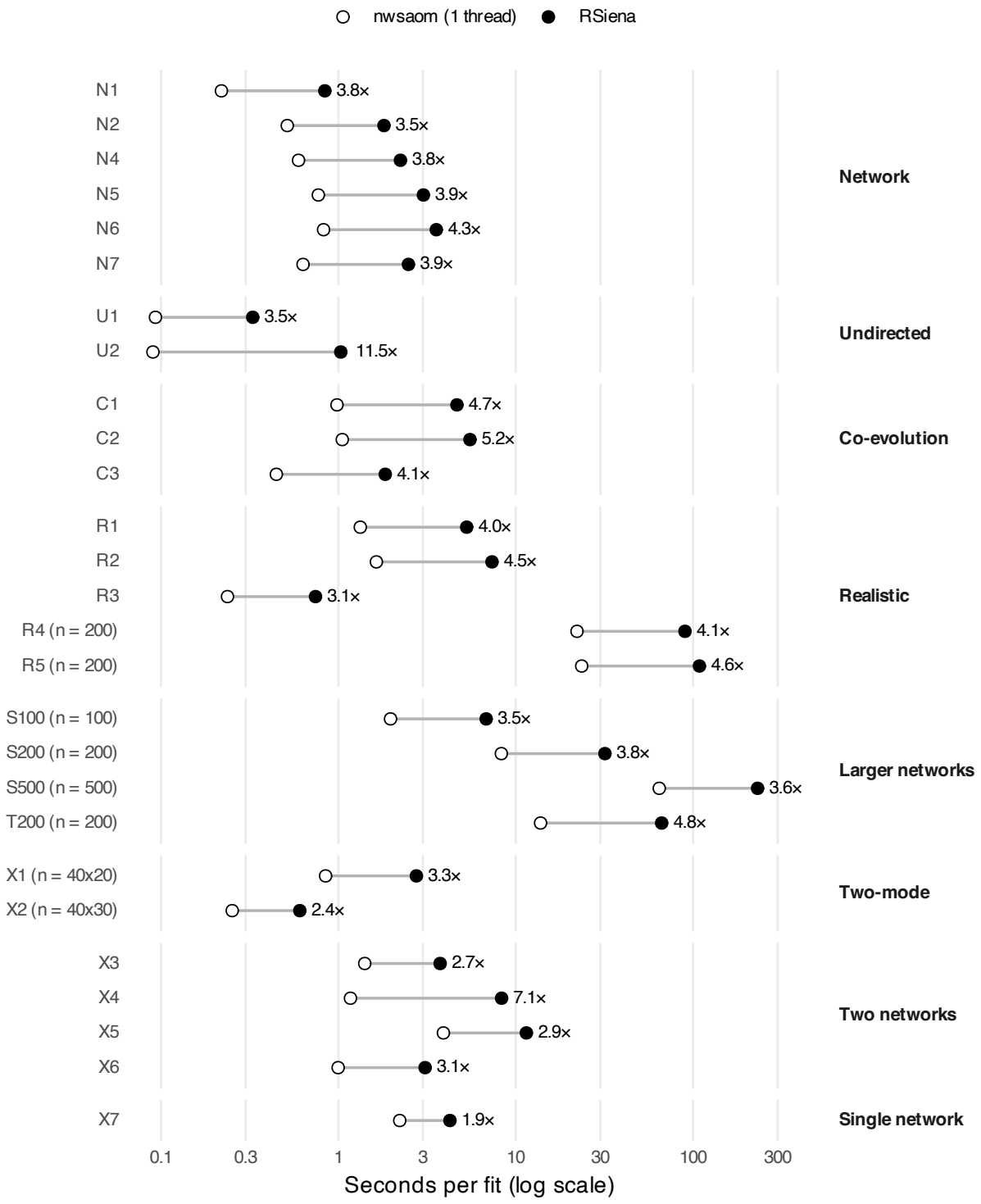



Figure D.2: Median seconds per fit of `nwsaom` and RSiena for every model of the benchmark, both on one thread.

Table D.3: `nwsaom` with 1 to 10 threads (in parentheses, speed-up over one thread) and RSiena with 1, 5 and 10 R processes: median seconds per fit over three seeds. The estimates of `nwsaom` are identical for every number of threads.

| | | `nwsaom` | | | | RSiena, processes | | |
|---|---|---|---|---|---|---|---|---|
| Model | $n$ | `cores(1)` | `cores(2)` | `cores(5)` | `cores(10)` | 1 | 5 | 10 |
| R1 | 50 | 1.34 (1.0) | 1.11 (1.2) | 0.96 (1.4) | 0.92 (1.5) | 4.74 | 3.53 | 3.97 |
| R4 | 200 | 22.0 (1.0) | 18.1 (1.2) | 15.4 (1.4) | 14.7 (1.5) | 89.5 | 34.4 | 29.7 |
| R5 | 200 | 23.6 (1.0) | 14.0 (1.7) | 11.7 (2.0) | 10.9 (2.2) | 111 | 46.0 | 49.1 |

### D.4.6 What was not measured

RSiena's C++ code was not profiled internally, so the split of the remaining factor among the design choices above is an inference from the source code. `nwsaom`'s Phase 2 counts are known to within 24 iterations per subphase. The timings come from one machine that also ran other jobs; the one-minute load average was recorded before every fit, at a median of 2.8 for the `nwsaom` fits and 2.3 for RSiena's, on 18 cores with single-threaded fits.

# Bibliography


[1] Snijders, T.A.B., van de Bunt, G.G., Steglich, C.E.G. (2010). Introduction to stochastic actor-based models for network dynamics. *Social Networks* 32, 44–60.

[2] Ripley, R.M., Snijders, T.A.B., Bóda, Z., Vörös, A., Preciado, P. RSiena: Simulation Investigation for Empirical Network Analysis. R package and manual.

[3] Snijders, T.A.B. (1996). Stochastic actor-oriented models for network change. *Journal of Mathematical Sociology* 21, 149–172.

[4] Snijders, T.A.B. (2001). The statistical evaluation of social network dynamics. *Sociological Methodology* 31, 361–395.

[5] Snijders, T.A.B., Koskinen, J.H., Ripley, R.M. multiSiena 1.2.40: multilevel and Bayesian extensions of RSiena. R package and manual.

[6] Koskinen, J.H., Snijders, T.A.B. (2007). Bayesian inference for dynamic social network data. *Journal of Statistical Planning and Inference* 137, 3930–3938.

[7] Steglich, C.E.G., Snijders, T.A.B., Pearson, M. (2010). Dynamic networks and behavior: separating selection from influence. *Sociological Methodology* 40, 329–393.

[8] Lospinoso, J., Snijders, T.A.B. (2019). Goodness of fit for stochastic actor-oriented models. *Methodology* 15.

[9] Grund, T.U. (2027). *Network Analysis with Stata: Concepts, Methods, and Applications.* College Station, TX: Stata Press (forthcoming).